

- An RCH-RCD pair
- One or more CXL Root Complex Integrated Endpoints, all of which are part of the Root Complex and appear at the same bus number

Enumeration of PCIe Host Bridges and PCIe hierarchy underneath them is governed by PCIe Base Specification. Enumeration of CXL Host Bridges is described below.

In an ACPI-compliant system, CXL Host Bridges are identified with an ACPI Hardware ID (HID) of "ACPI0016". CXL Early Discovery Table (CEDT) may be used to differentiate between the three software concepts listed above. RCD enumeration is described in [Section 9.11](#).

9.12.1 CXL Root Ports

Each CXL Host Bridge is associated with a Base Bus Number. If the Host Bridge is not associated with RCDs or CXL RCiEPs, that bus number shall contain one or more CXL Root Ports. These Root Ports appear in PCIe configuration space with a Type 1 header, and the Device/Port Type field in the PCIe Capabilities Register shall identify these as standard PCIe Root Ports. Unless specified otherwise, CXL Root Ports may implement all Capabilities that are defined in PCIe Base Specification as legal for PCIe Root Ports. These Root Ports can be in one of four states:

- Disconnected
- Connected to an eRCD
- Connected to CXL Device that is not an eRCD, or connected to a CXL Switch
- Connected to a PCIe Device/Switch

[Section 9.12.3](#) describes how software can determine the current state of a CXL Root Port and the corresponding enumeration algorithm.

9.12.2 CXL Virtual Hierarchy

CXL Root Ports may be directly connected to a CXL device that is not an eRCD, or a CXL Switch. These Root Ports spawn a CXL Virtual Hierarchy (VH). Enumeration within a CXL VH is described below.

These CXL devices appear as a standard PCIe Endpoints with a Type 0 Header. The CXL device's primary function (Device 0, Function 0) shall carry one instance of CXL DVSEC ID 0 with Revision 1 or greater. Software may use this DVSEC instance to distinguish a CXL device from an ordinary PCIe device. Unless specified otherwise, CXL devices may implement all Capabilities that are defined in PCIe Base Specification as legal for PCIe devices.

A CXL VH may include zero or more CXL switches. Specific configuration constraints are documented in [Chapter 7.0](#). From an enumeration software perspective, each CXL Switch consists of one Upstream Switch Port and one or more Downstream Switch Ports.

The configuration space of the Upstream Switch Port of a CXL Switch has a Type 1 header and the Device/Port Type field in the PCIe Capabilities Register shall identify it as an Upstream Port of a PCIe Switch. The configuration space carries one instance of the CXL DVSEC ID 3 and one instance of DVSEC ID 7. The DVSEC Flex Bus Port Status register in CXL DVSEC ID 7 structure of the peer Port shall indicate that CXL VH operation with 68B Flit mode was negotiated with the Upstream Switch Port during link training. Unless specified otherwise, CXL Upstream Switch Ports may implement all Capabilities that are defined in PCIe Base Specification as legal for PCIe Upstream Switch Ports.

The configuration space of a Downstream Switch Port of CXL Switch also has a Type 1 header, but the Device/Port Type field in the PCIe Capabilities Register shall identify these as a Downstream Port of a PCIe Switch. Unless specified otherwise, CXL Downstream Switch Ports may implement all Capabilities that are defined in PCIe Base Specification as legal for PCIe Downstream Switch Ports. All these Ports are CXL capable and can be in one of four states, just like the CXL Root Ports:

- Disconnected
- Connected to an eRCD
- Connected to CXL Device that is not an eRCD, or connected to a CXL Switch
- Connected to a PCIe Device/Switch

Section 9.12.3 describes how software can determine the current state of a CXL Downstream Switch Port and the corresponding enumeration algorithm.

A CXL Downstream Switch Port may be connected to another CXL Switch or a CXL device. The rules for enumerating CXL switches and CXL devices are already covered earlier in this section.

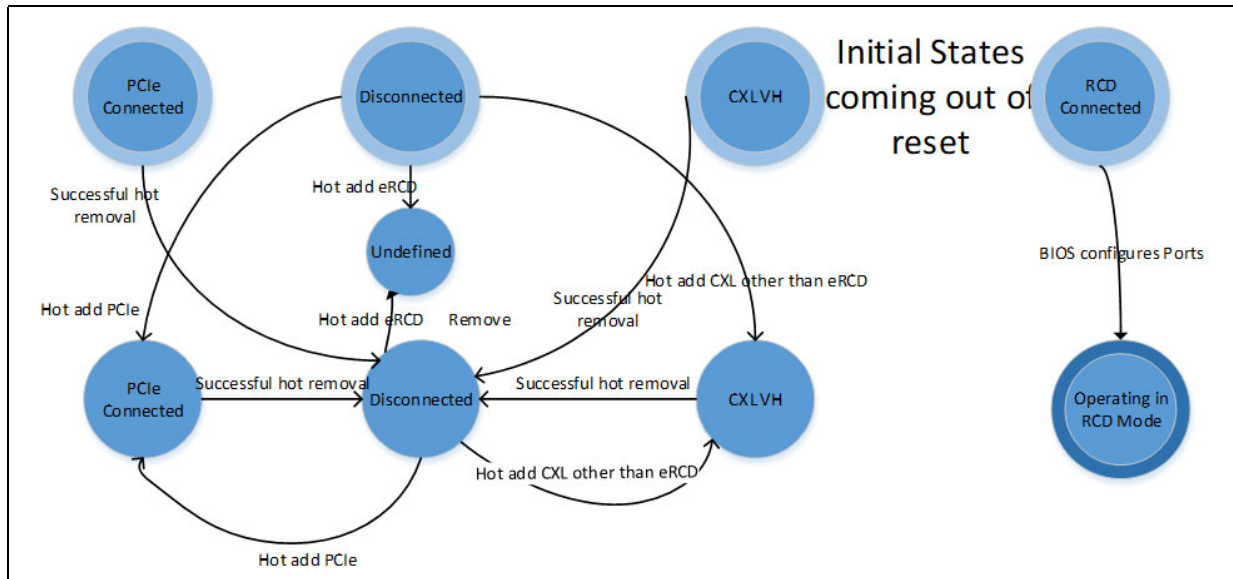
9.12.3 Enumerating CXL RPs and DSPs

Software may use the combination of the Link Status registers and the CXL DVSEC ID 7 capability in root port or DSP configuration space to determine which state a CXL Downstream Port is in, as follows:

1. CXL root port or DSP is in the Disconnected state when they do not have an active link. The status of the link can be detected by following PCIe Base Specification. If the link is not up, software shall ignore the CXL DVSEC ID 3 and the CXL DVSEC ID 7 capability structures. A Hot-Add event may transition a Disconnected Port to a CXL Connected state or a PCIe Connected state. Hot-adding an eRCD adapter will transition the Port to an Undefined state.
2. CXL root port or DSP connected to a CXL device that is not an RCD or connected to a CXL switch shall expose one instance of the CXL DVSEC ID 3 and one instance of the CXL DVSEC ID 7 capability structures. The DVSEC Flex Bus Port Status register in the CXL DVSEC ID 7 structure shall indicate that CXL VH operation with 68B Flit mode was successfully negotiated during link training. System Firmware may leave the Unmask SBR and the Unmask Link Disable bits in the Port Control register of the Downstream Port at the default (0) values to prevent CXL-unaware PCIe software from resetting the device and the link, respectively.
3. CXL root port or DSP connected to an eRCD shall expose one instance of the CXL DVSEC ID 3 and one instance of the CXL DVSEC ID 7 capability structures. The DVSEC Flex Bus Port Status register in the CXL DVSEC ID 7 structure shall indicate that CXL VH operation with 68B Flit mode was not negotiated, but that either the CXL.cache protocol or the CXL.mem protocol was negotiated during link training. There are two possible substates:
 - a. Not Operating with RCH Downstream Port addressing - Immediately after the link negotiation, the Port registers appear in the PCIe configuration space with a Type 1 header.
 - b. Operating with RCH Downstream Port addressing - System Firmware may program the RCRB Base register in the Port's CXL DVSEC ID 3 capability structure to transition the Port to this mode. Once the Port is in this mode, it can only transition out of the mode after a reset. A Downstream Port operating in this mode shall ignore hot reset requests received from the Upstream Port.
4. CXL root port or DSP connected to a PCIe device/switch may or may not expose the CXL DVSEC ID 3 and the CXL DVSEC ID 7 capability structures.

- a. If the PCIe root port configuration space contains an instance of the CXL DVSEC ID 3 structure, it shall also contain an instance of the CXL DVSEC ID 7 structure.
- b. If the PCIe root port configuration space contains an instance of the CXL DVSEC ID 7 structure, the DVSEC Flex Bus Port Status register shall indicate that this Port did not train up in CXL mode. Software shall ignore the contents of the CXL DVSEC ID 3 structure for such a Port.

Figure 9-9. CXL Root Port/DSP State Diagram



If the Port is in the disconnected state, the branch does not need further enumeration.

If the Port is connected to a CXL device other than an eRCD or connected to a CXL switch, the software follows [Section 9.12.2](#) for further enumeration until it reaches the leaf endpoint.

If the Port is connected to an RCD, the software follows [Section 9.12.4](#) to enumerate the device.

If the Port is connected to a PCIe device/switch, the enumeration flow is governed by PCIe Base Specification.

9.12.4 eRCD Connected to a CXL RP or DSP

An eRCD may be connected to a CXL Root Port or a CXL Downstream Switch Port. Each RCD Function must report itself as an RCiEP and therefore cannot appear, to software, to be below a PCIe-enumerable Downstream Port. System Firmware is responsible for detecting such a case and reconfiguring the CXL Ports in the path so that the RCD appears to software to be directly connected to an RCH Downstream Port and not in a CXL VH.

9.12.4.1 Boot time Reconfiguration of CXL RP or DSP to Enable an eRCD

1. At reset, the Downstream Port registers are visible in the PCIe configuration space with a Type 1 header. During enumeration, System Firmware shall identify all the Downstream Ports that are connected to the eRCD by reading the DVSEC ID 7 register instead of the Link status registers.

- If the link training was successful, the DVSEC Flex Bus Port Status register in the CXL DVSEC ID 7 structure shall indicate that CXL VH operation with 68B Flit mode was not negotiated, but shall indicate that either the CXL.cache protocol or the CXL.mem protocol was negotiated during link training.
 - If the link training was unsuccessful, the DVSEC Flex Bus Port Received Modified TS Data Phase1 Register in the CXL DVSEC ID 7 structure shall indicate that the device is CXL capable but not CXL VH capable. A DSP shall not report link-up status in the PCIe Link Status register when the DSP detects an eRCD on the other end to prevent the CXL-unaware software from discovering the eRCD.
2. System Firmware identifies MMIO and bus resource needs for all RCDs below a CXL root port. System Firmware adds MMIO resources needed for the RCH Downstream Port RCRB and RCD Upstream Port RCRB (8-KB MMIO per link) and CXL Component registers (128-KB MMIO per link).
 3. System Firmware assigns MMIO and bus resources and programs the Alternate MMIO Base/Limit and Alternate Bus Base/Limit registers in all the Root Ports and the Switch Ports in the path and the eRCD BARs except the Downstream Ports that are directly connected to eRCDs. These Alternate decoders are described in [Section 8.1.5](#).
 4. System Firmware sets the Alt BME and Alt Memory and ID Space Enable bits in all the Root Ports and the Switch Ports in the path of every eRCD.
 5. For each Downstream Port that is connected to an eRCD, System Firmware programs the CXL RCRB Base Address. System Firmware then writes 1 to the CXL RCRB Enable bit, which transitions the port addressing to RCH addressing. The Downstream Port registers now appear in MMIO space at CXL RCRB Base and not in configuration space. System Firmware issues a read to the address CXL RCRB Base + 4 KB. The RCD Upstream Port captures its RCRB Base as described in [Section 8.1.5](#). System Firmware configures Upstream Port and Downstream Port registers, as necessary. If this is a DSP, the Downstream Port shall ignore any hot reset requests received from the Upstream Port.
 6. System Firmware configures the eRCD, using the algorithm described in [Section 9.11.6](#).

The System Firmware shall report each RCD under a separate Host Bridge and not as part of the CXL VH.

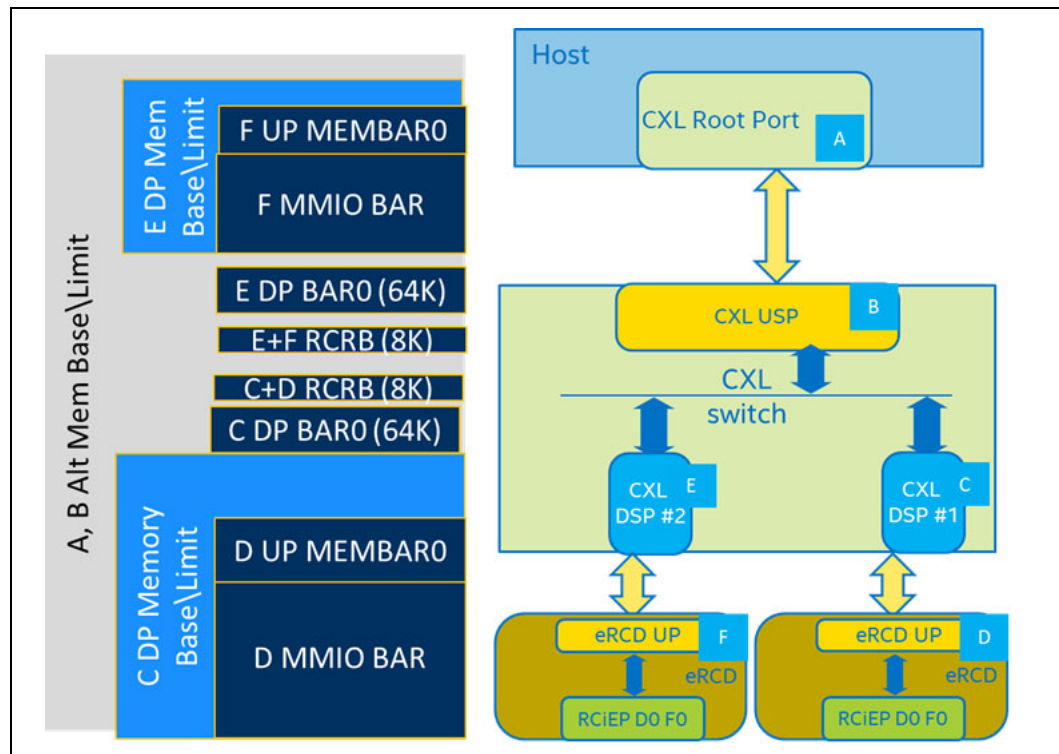
The Switch shall ensure that there is always a DSP visible at Device 0, Function 0.

These concepts are illustrated by the configuration shown in [Figure 9-10](#). In this configuration, eRCD F and D are attached to a CXL Switch. The Switch DSPs are labeled E and C. The Switch USP and the CXL Root Port are labeled B and A, respectively. The left half of [Figure 9-10](#) represents the address map and how the normal decoders and the Alt Mem decoders of A, B, C, and E are configured.

If the host accesses an MMIO address belonging to D, the access flows through A, B, and C as follows:

1. Host issues a read.
2. A Alt Decoder positively decodes the access and sends to B because A's Alt MSE=1.
3. B Alt Decoder positively decodes the access because B's Alt MSE=1.
4. C normal decoder positively decodes the access and forwards it to D because C MSE=1.
5. D positively decodes and responds because D MSE=1.

Figure 9-10. eRCD MMIO Address Decode - Example



The left half of Figure 9-11 represents the configuration space map for the same configuration as in Figure 9-10 and how the bus decoders and the Alt Mem decoders of A, B, C, and E are configured.

If the host accesses configuration space of F, the access flows through A, B, and E as follows:

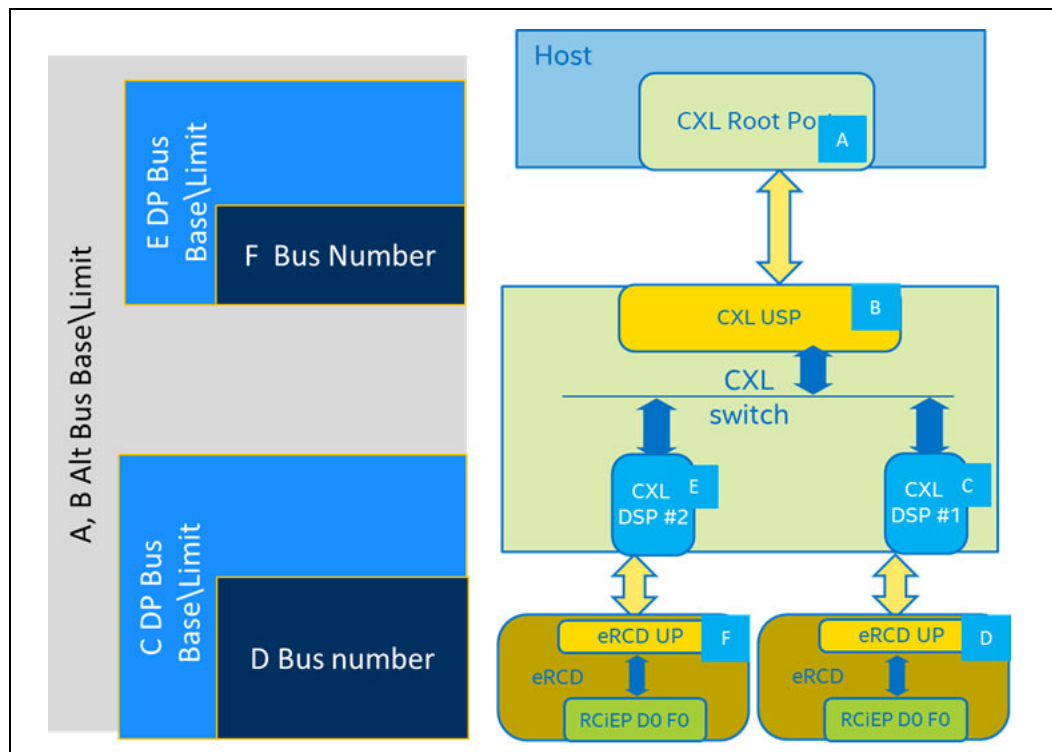
1. Host issues configuration read to F's configuration space
2. A's Alt Decoder positively decodes, forwards to B as Type 1
3. B's Alt Decoder positively decodes, forwards down as Type 1
4. E's RCRB regular decoder positively decodes, forwards to F as Type 0 because the bus number matches E's RCRB Secondary Bus number
5. F positively decodes and responds

If D detects a protocol or link error, the error signal will flow to the system via the following path:

1. D issues ERR_ message with the Requester ID of D.
2. C shall not expose DPC capability.
3. C forwards ERR_ message to B.
4. B forwards the message to A.
5. A forwards the message to RCEC in the Root Complex because the requester's bus number hits Alt Bus Decoder.
6. RCEC generates MSI if enabled.

7. Root Complex Event Collector Endpoint Association Extended Capability of RCEC describes that it can handle errors from bus range = Alt Bus Decoder in RP.
8. A shall not trigger DPC upon ERR_ message. Because the requester's bus number hits Alt Bus Decoder, it is treated differently than a normal ERR_ message.

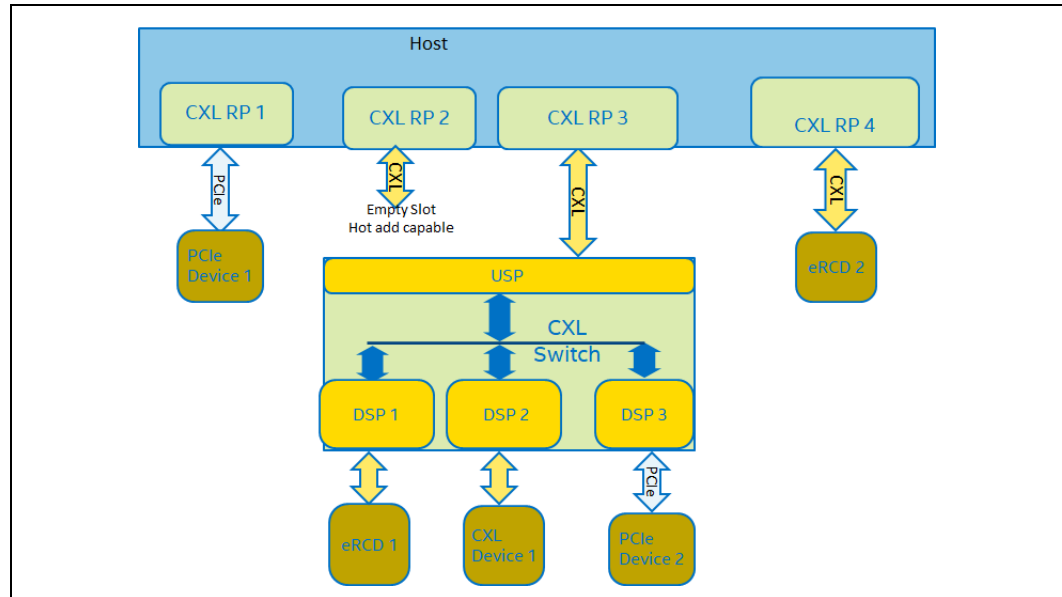
Figure 9-11. eRCD Configuration Space Decode - Example



9.12.5 CXL eRCD below a CXL RP and DSP - Example

Figure 9-12 represents the physical connectivity of a host with four Root Ports, one Switch, and 5 devices. The corresponding software view is shown in Figure 9-13. Note that the numbers (e.g., the “1” in PCIe Device 1) in this diagram do not represent the device number or the function number.

Figure 9-12. Physical Topology - Example



As shown in Figure 9-12, the Switch makes eRCD 1, below its DSP (DSP 1), appear as an RCIEP under an RCH. eRCD 1 is exposed as a separate Host Bridge to the Operating System. The device hosts a CXL DVSEC ID 0 instance in Device 0, Function 0 Configuration Space. The RCH Downstream Port registers and the RCD Upstream Port registers appear in MMIO space as expected.

When a CXL Root Port detects a PCIe device (PCIe Device 1), the Root Port trains up in PCIe mode. The Root Port configuration space (Type 1) may include the CXL DVSEC ID 3 and the CXL DVSEC ID 7. If present, the DVSEC ID 7 instance will indicate that the link trained up in PCIe mode. Other CXL DVSEC ID structures may be present as well.

If a CXL Root Port (RP 2) is connected to an empty slot, its configuration space (Type 1) hosts the CXL DVSEC ID 3 and the CXL DVSEC ID 7, but the DVSEC ID 7 shall indicate no CXL connectivity and the PCIe Link status register shall indicate that there is no PCIe connectivity. Other CXL DVSEC ID structures may be present as well. The user can hot-add a CXL device other than eRCD, a CXL Switch, or a PCIe device in this slot.

A CXL Root Port (RP 3) connected to a CXL Switch spawns a CXL VH. The Root Port as well as the Upstream Switch Port configuration space (Type 1) each host an instance of CXL DVSEC ID 3 and an instance of CXL DVSEC ID 7, but the DVSEC ID 7 instance will indicate that these Ports are operating in CXL VH operation with 68B Flit mode. Other CXL DVSEC ID structures may be present as well.

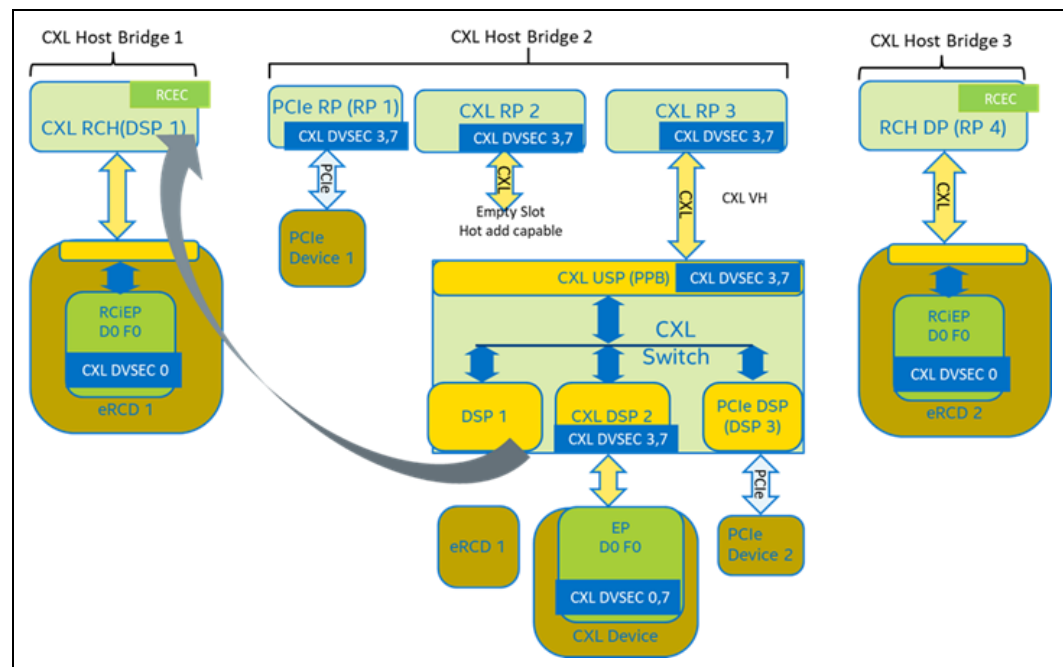
If a CXL Downstream Switch Port (DSP 2) is connected to a CXL device that is not an eRCD, DSP 2's configuration space (Type 1) hosts an instance of CXL DVSEC ID 3 and an instance of CXL DVSEC ID 7, but the DVSEC ID 7 instance will indicate that this Port is connected to a CXL device and is part of a CXL VH. Other CXL DVSEC ID structures may be present as well.

In this example, CXL Downstream Switch Port (DSP 3) is connected to a PCIe device and its configuration space (Type 1) does not host an instance of CXL DVSEC ID 7. Absence of a CXL DVSEC ID 7 indicates that this Port is not operating in the CXL mode. Note that it is legal for DSP 3 to host a DVSEC ID 7 instance as long as the DVSEC Flex Bus Port Status Register in the DVSEC ID 7 structure reports that the link is not operating in CXL mode.

If a CXL Root Port (RP 4) is connected to an eRCD, the Root Port operates as an RCH Downstream Port. eRCD 2 appears as an RCIEP under its own Host Bridge. This device hosts an instance of the CXL DVSEC ID 0 in Device 0, Function 0 Configuration Space. The RCH Downstream Port registers and the RCD Upstream Port registers appear in MMIO space as expected.

If the Switch is Hot-Pluggable, System Firmware may instantiate a _DEP object in the ACPI namespace to indicate that Device 1 is dependent on the CXL USP. A legacy PCIe bus driver interprets that to mean that the Switch hot removal has a dependency on eRCD 1, even though the ACPI/PCIe hierarchy does not show such a dependency.

Figure 9-13. Software View



9.12.6 Mapping of Link and Protocol Registers in CXL VH

In the presence of an eRCD, the link and protocol registers appear in MMIO space (RCRB and Component registers in the Downstream Port and the Upstream Port). See [Figure 9-7](#) and [Figure 9-8](#).

Because a CXL Virtual Hierarchy appears as a true PCIe hierarchy, the Component Register block is mapped using a standard BAR of CXL components.

Each CXL Host Bridge that is not an RCH includes CHBCR, which includes the registers that are common to all Root Ports under that Host Bridge. In an ACPI-compliant system, the base address of this register block is discovered via ACPI CEDT or the _CBR method. The CHBCR includes the HDM Decoder registers.

Each CXL Root Port carries a single BAR that maps the associated Component Register block. The offset within that BAR is discovered via the CXL DVSEC ID 8. See [Section 8.1.9](#). The layout of the Component Register Block is shown in [Section 8.2.3](#).

Each CXL device that is not an RCD can map its Component Register Block to any of its 6 BARs and a 64-KB-aligned offset within that BAR. The BAR number and the offset are discovered via CXL DVSEC ID 8. A Type 3 device Component Register Block includes HDM Decoder registers.

Each CXL USP carries a single BAR that maps the associated Component Register block. The offset within that BAR is discovered via CXL DVSEC ID 8. The Upstream Switch Port Component Register Block contains the registers that are not associated with a particular Downstream Port, such as HDM Decoder registers.

Each CXL DSP carries a single BAR that points to the associated CHBCR, the format of which closely mirrors that of a Root Port. The offset within that BAR is discovered via CXL DVSEC ID 8.

Figure 9-14. CXL Link/Protocol Register Mapping in a CXL VH

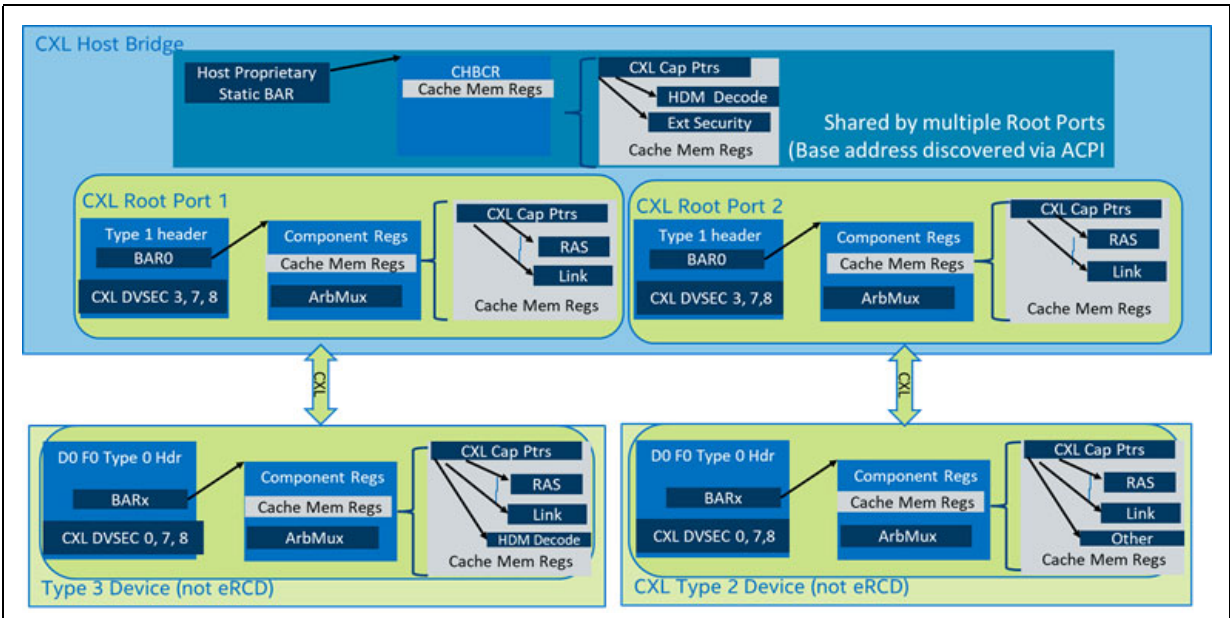
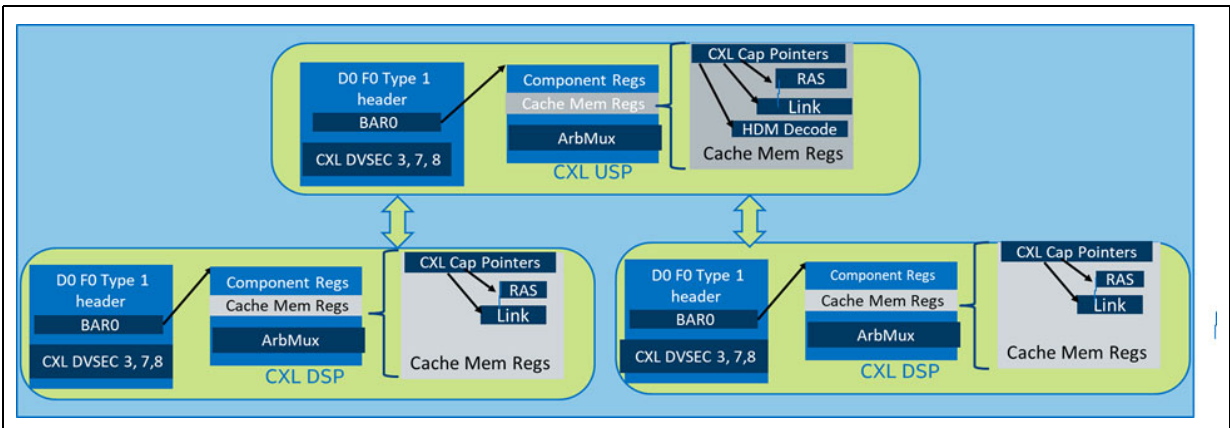


Figure 9-15. CXL Link/Protocol Registers in a CXL Switch



9.13 Software View of HDM

HDM is exposed to the OS/VMM as normal memory. However, HDM likely has different performance/latency attributes compared to host-attached memory. Therefore, a system with CXL.mem devices can be considered as a heterogeneous memory system.

ACPI HMAT was introduced for such systems and can report memory latency and bandwidth characteristics associated with different memory ranges. ACPI Specification version 6.2 and later carry the definition of revision 1 of HMAT. As of August 2018, ACPI WG has decided to deprecate revision 1 of HMAT because it had a number of shortcomings. As a result, the subsequent discussion refers to revision 2 of HMAT. In addition, ACPI has introduced a new type of Affinity structure called Generic Affinity (GI) Structure. GI structure is useful for describing execution engines such as accelerators that are not processors. CXL.mem-capable accelerators will result in two SRAT entries - One GI entry to represent the accelerator cores and one memory entry to represent the attached HDM. GI entry is especially useful when describing the CXL.cache accelerator. Previous to the introduction of GI, the CXL.cache accelerator could not be described as a separate entity in SRAT/HMAT and had to be combined with the attached CPU. With this specification change, the CXL.cache accelerator can be described as a separate proximity domain. `_PXM` method can be used to identify the proximity domain associated with the PCI device. Since Legacy OSs do not understand GI, System Firmware is required to return the processor domain that is most closely associated with the I/O device when running such an OS. ASL code can use bit 17 of Platform-Wide `_OSC` Capabilities DWORD 2 to detect whether the OS supports GI.

System Firmware must construct and report SRAT and HMAT to the OS in systems with CXL.cache devices and CXL.mem devices. Since System Firmware is not aware of HDM properties, that information must come from the CXL device in the form of CDAT. A device may export CDAT via Table Access DOE or via a UEFI driver.

System Firmware combines the information that it has about the host and CXL connectivity with CDAT content obtained from various CXL components during construction of SRAT and HMAT.

9.13.1 Memory Interleaving

Memory interleaving allows consecutive memory addresses to be mapped to different CXL devices at a uniform interval. eRCDs may support a limited form of interleaving as described in [Section 9.11.7.1](#), whereby memory is interleaved across the two links between a CPU and a dual-headed device.

The CXL 2.0 specification introduced a mechanism for interleaving across different devices. The set of devices that are interleaved together is known as the Interleave Set.

An Interleave Set is identified by the following:

- Base HPA - Multiple of 256 MB
- Size - Also a Multiple of 256 MB
- Interleave Way
- Interleave Granularity
- Targets (applicable only to Root Ports and Upstream Switch Ports)

These terms are described below.

Interleave Way: A CXL Interleave Set may contain either 1, 2, 3, 4, 6, 8, 12, or 16 CXL devices. 1-way Interleave is equivalent to no interleaving. The number of devices in an Interleave set is known as Interleave Ways (IW).

Interleave Granularity: Each device in an Interleave Set decodes a specific number of consecutive bytes, called Chunk, in HPA Space. The size of Chunk is known as Interleave Granularity (IG). The starting address of each Chunk is a multiple of IG.

- CXL Host Bridges (except RCH) and CXL switches must support the following IG values:
 - 256 Bytes (Interleaving on HPA[8])
 - 512 Bytes (Interleaving on HPA[9])
 - 1024 Bytes (Interleaving on HPA[10])
 - 2048 Bytes (Interleaving on HPA[11])
 - 4096 Bytes (Interleaving on HPA[12])
 - 8192 Bytes (Interleaving on HPA[13])
 - 16384 Bytes (Interleaving on HPA[14])
- CXL memory devices must support at least one of the two IG groups as reported via the CXL HDM Decoder Capability register (see [Section 8.2.4.20.1](#)):
 - Group 1: Interleaving on HPA[8], HPA[9], HPA[10], and HPA[11]
 - Group 2: Interleaving on HPA[12], HPA[13], and HPA[14]

Target: The HDM Decoders in the CXL Host Bridge are responsible for looking up the incoming HPA in a CXL.mem transaction and forwarding the HPA to the appropriate Root Port Target. The HDM Decoders in the CXL Upstream Switch Port are responsible for looking up the incoming HPA in a CXL.mem transaction and forwarding the HPA to the appropriate Downstream Switch Port Target.

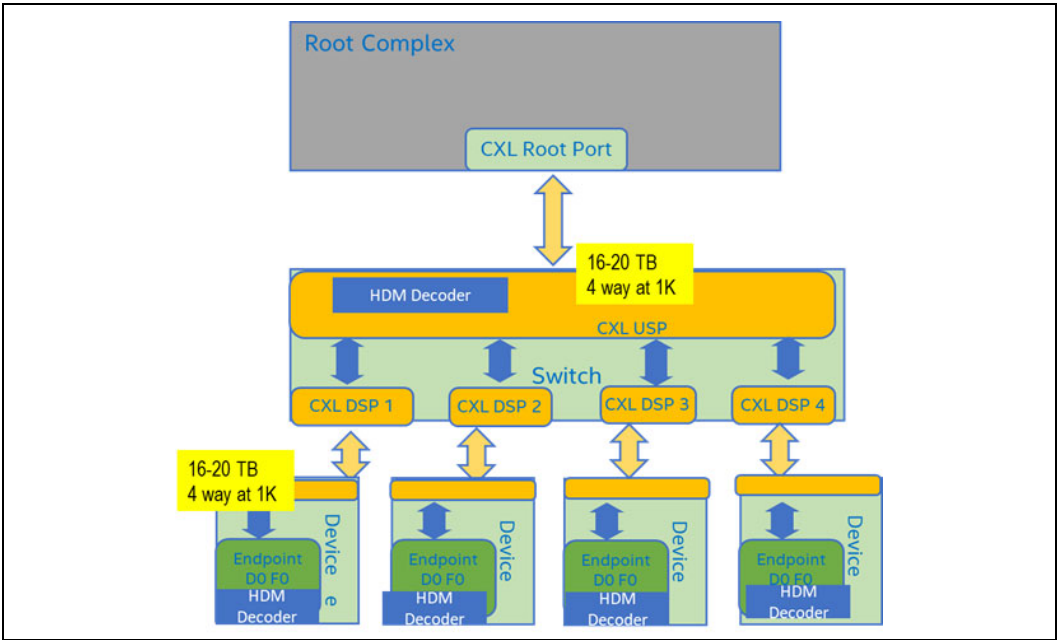
An HDM Decoder in a device is responsible for converting HPA into DPA by stripping off specific address bits. These flows are described in [Section 8.2.4.20.13](#).

An Interleave Set is established by programming an HDM Decoder and committing it (see [Section 8.2.4.20.12](#)). The number of decoders implemented by a component are enumerated via the CXL HDM Decoder Capability register (see [Section 8.2.4.20.1](#)). HDM Decoders within a component must be configured in a congruent manner and the Decoder Commit flow performs certain self-consistency checks to assist with correct programming.

Software is responsible for ensuring that HDM Decoders located inside the components along the path of a transaction must be configured in a consistent manner.

[Figure 9-16](#) illustrates a simple memory fan-out topology with 4 memory devices behind a CXL Switch. A single HDM Decoder in each Device as well as the Upstream Switch Port is configured to decode the HPA range 16 to 20 TB, at 1-KB granularity. The leftmost Device receives 1-KB ranges starting with HPAs 16 TB, 16 TB + 4 KB, 16 TB+8KB, ..., 20 TB - 4 KB (every 4th Chunk). The Root Complex does not participate in the interleaving process.

Figure 9-16. One-level Interleaving at Switch - Example



Multiple-level interleaving is supported as long as all the levels use different, but consecutive, HPA bits to select the target and no Interleave Set has more than 8 devices. This is illustrated in Figure 9-17 and Figure 9-18.

Figure 9-17. Two-level Interleaving

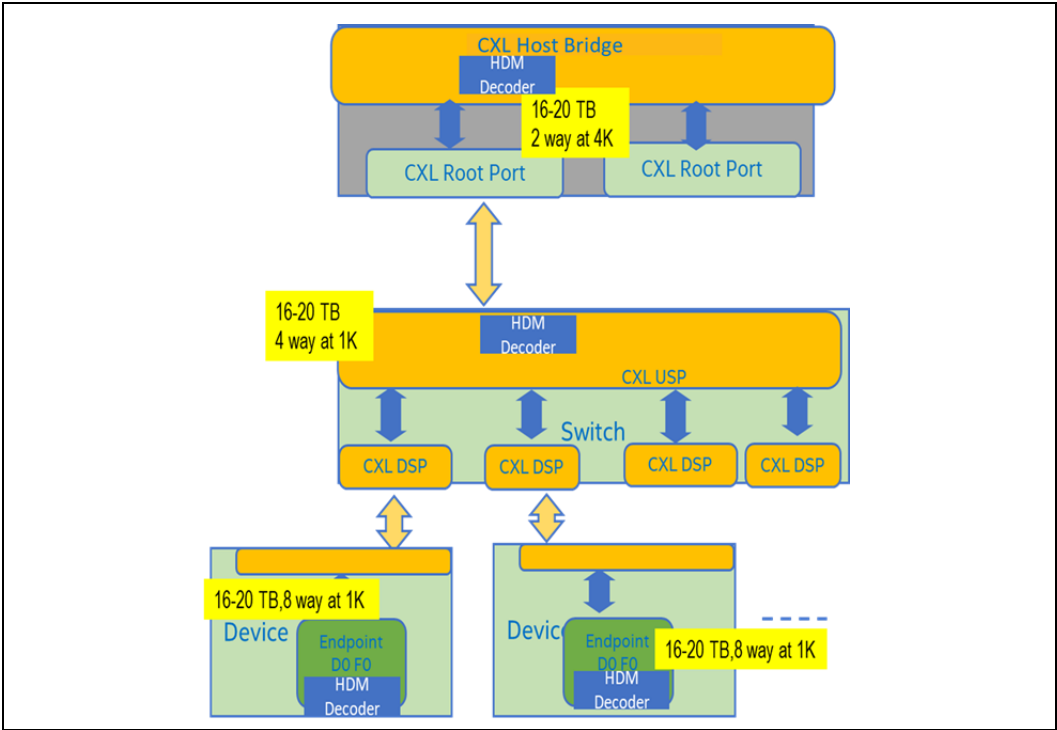
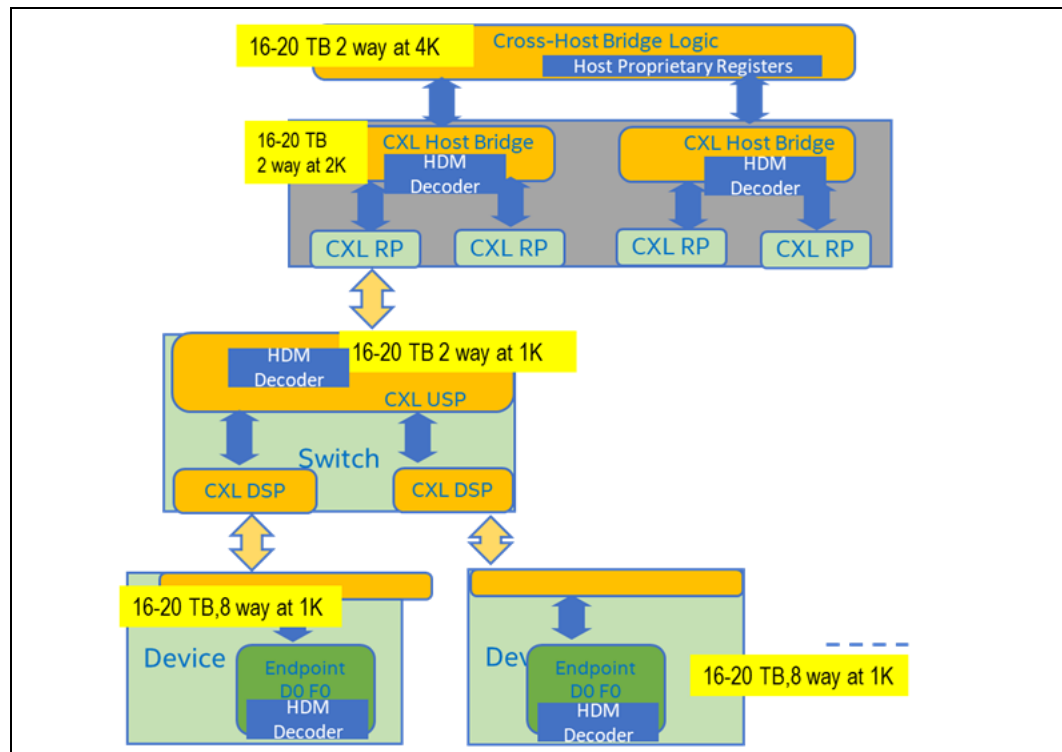


Figure 9-17 illustrates a two-level Interleave scheme where the Host Bridge as well as the switch participates in the interleaving process. This topology has 4 memory devices behind each CXL Switch. One HDM Decoder in each of the 8 devices, both Upstream Switch Ports and the Host Bridge are configured to decode the HPA range 16 to 20 TB. The Host Bridge partitions the address range in two halves at 4-KB granularity (based on HPA[12]), with each half directed to a Root Port. Each Upstream Switch Port further splits each half into 4 subranges at 1-KB granularity (based on HPA[11:10]). To each device, it appears as though the HPA range 16-20 TB is 8-way interleaved at 1-KB granularity based on HPA[12:10]. The leftmost Device receives 1-KB ranges starting with HPAs 16 TB, 16 TB+8KB, 16 TB+16KB, ..., 20 TB-8KB.

Figure 9-18 illustrates a three-level Interleave scheme where the cross-host Bridge logic, the Host Bridge, and the switch participate in the interleaving process. The cross-host Bridge logic is configured to interleave the address range in two halves, using host proprietary registers at 4-KB granularity. One HDM Decoder in 8 devices, 4 Upstream Switch Ports, and 2 Host Bridges are configured to decode the HPA range 16 to 20 TB. The Host Bridge further subdivides the address range in two at 2-KB granularity (using HPA[11]). The Upstream Switch Port in every switch further splits HPA space into 2 subranges at 1-KB granularity (using HPA[10]). To each device, it appears as though the HPA range 16-20 TB is 8-way interleaved at 1-KB granularity based on HPA[12:10]. Similar to Figure 9-17, the leftmost Device receives 1-KB ranges starting with HPAs 16 TB, 16 TB+8KB, 16 TB+16KB, ..., 20 TB-8KB.

Figure 9-18. Three-level Interleaving Example



9.13.1.1 Legal Interleaving Configurations: 12-way, 6-way, and 3-way

This section describes the legal 12-way, 6-way, and 3-way interleaving configurations. The term IGB represents the interleave granularity in number of bytes. The cross-host Bridge Interleaving logic selects the target Host Bridge according to the configurations specified in [Table 9-6](#), [Table 9-7](#), and [Table 9-8](#), respectively. The Root Complex and the switch select the target port as described in [Section 9.18.1](#).

Table 9-6. 12-Way Device-level Interleave at IGB

Cross-host Bridge Logic Interleaving	CXL Root Complex-level Interleaving	CXL Switch-level Interleaving
12 way at IGB	No interleaving	No interleaving/Absent
6 way at 2*IGB	2 way at IGB	No interleaving/Absent
6 way at 2*IGB	No interleaving	2 way at IGB
3 way at 4*IGB	4 way at IGB	No interleaving
3 way at 4*IGB	No interleaving	4 way at IGB
3 way at 4*IGB	2 way at IGB	2 way at 2*IGB
3 way at 4*IGB	2 way at 2*IGB	2 way at IGB

Table 9-7. 6-Way Device-level Interleave at IGB

Cross-host Bridge Logic Interleaving	CXL Host Bridge-level Interleaving	CXL Switch-level Interleaving
6 way at IGB	No interleaving	No interleaving/Absent
3 way at 2*IGB	2 way at IGB	No interleaving
3 way at 2*IGB	No interleaving	2 way at IGB

Table 9-8. 3-Way Device-level Interleave at IGB

Cross-host Bridge Logic Interleaving	CXL Host Bridge-level Interleaving	CXL Switch-level Interleaving
3 way at IGB	No interleaving	No interleaving/Absent

9.13.2 CXL Memory Device Label Storage Area

CXL memory devices that provide volatile memory, such as DRAM, may be exposed with different interleave geometries each time the system is booted. This can happen due to the addition or removal of other devices or changes to the platform's default interleave policies. For volatile memory, these changes to the interleave usually do not impact host software since there's generally no expectation that volatile memory contents are preserved across reboots. However, with persistent memory, the exact preservation of the interleave geometry is critical so that the persistent memory contents are presented to host software the same way each time the system is booted.

Similar to the interleaving configuration, persistent memory devices may be partitioned into *namespaces*, which define volumes of persistent memory. These namespaces must also be reassembled the same way each time the system is booted to prevent data loss.

[Section 8.2.9](#) defines mailbox operations for reading and writing the *Label Storage Area* (LSA) on CXL memory devices: Get LSA and Set LSA. In addition, the Identify Memory Device mailbox command exposes the size of the LSA for a given CXL memory device. The LSA allows both interleave and namespace configuration details to be stored persistently on all the devices involved, so that the configuration data "follows the

device” if the device is moved to a different socket or machine. The use of an LSA is analogous to how disk RAID arrays write configuration information to a reserved area of each disk in the array so that the geometry is preserved across configuration changes.

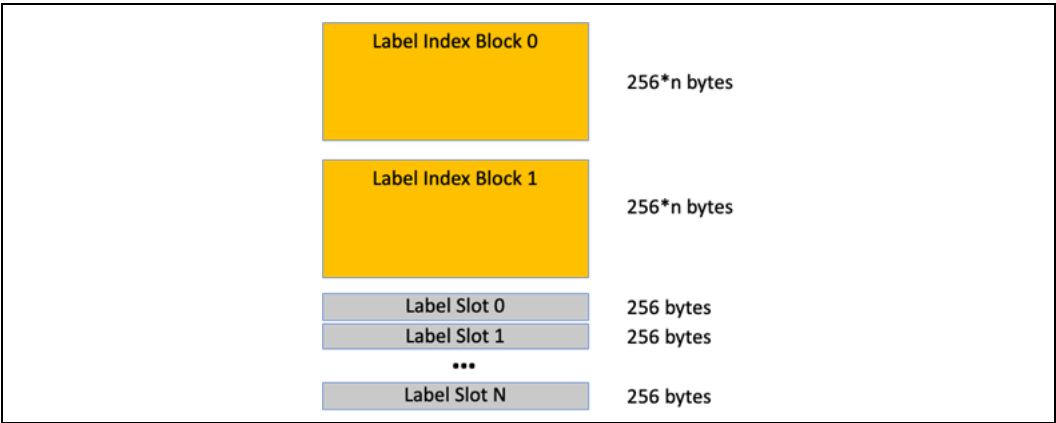
A CXL memory device may contribute to multiple persistent memory interleave sets, limited by interleave resources such as HDM decoders or other platform resources. Each persistent memory Interleave Set may be partitioned into multiple namespaces, limited by resources such as label storage space and supported platform configurations.

The LSA format and the rules for updating and interpreting the LSA are specified in this section. CXL memory devices do not directly interpret the LSA, they just provide the storage area and mailbox commands for accessing it. Software configuring Interleave Sets and namespaces, such as pre-boot firmware or host operating systems shall follow the LSA rules specified here to correctly inter-operate with CXL-compliant memory devices.

9.13.2.1 Overall LSA Layout

The LSA consists of two Label Index Blocks followed by an array of label slots. As shown in Figure 9-19, the Label Index Blocks are always a multiple of 256 bytes in size, and each label slot is exactly 256 bytes in size.

Figure 9-19. Overall LSA Layout



The LSA size is implementation dependent and software must discover the size using the Identify Memory Device mailbox command. The minimum allowed size is two index blocks, 256-bytes each in length, two label slots (providing space for a minimal one region label and one namespace label), and one free slot to allow for updates. This makes the total minimum LSA size 1280 bytes. It is recommended (but not required) that a device provides for configuration flexibility by implementing an LSA large enough for two region labels per device and one namespace label per 8 GB of persistent memory capacity available on the device.

All updates to the LSA shall follow the update rules laid out in this section, which guarantee that the LSA remains consistent in the face of interruptions such as power loss or software crashes. There are no atomicity requirements on the Set LSA mailbox operation – it simply updates the range of bytes provided by the caller. Atomicity and consistency of the LSA is achieved using checksums and the principle that only free slots (currently unused) are written to – in-use data structures are never written, avoiding the situation where an interrupted update to an in-use data structure makes it

inconsistent. Instead, all updates are made by writing to a free slot and then following the rules laid out in this section to atomically swap the in-use data structure with the newly written copy.

The LSA layout uses *Fletcher64* checksums. Figure 9-20 shows a Fletcher64 checksum implementation that produces the correct result for the data structures in this specification when run on a 64-bit system. When performing a checksum on a structure, any multi-byte integer fields shall be in little-endian byte order. If the structure contains its own checksum, as is commonly the case, that field shall contain 0 when this checksum routine is called.

Figure 9-20. Fletcher64 Checksum Algorithm in C

```
/*
 * checksum -- compute a Fletcher64 checksum
 */
uint64_t
checksum(void *addr, size_t len)
{
    uint32_t *p32 = addr;
    uint32_t *p32end = addr + len;
    uint32_t lo32 = 0;
    uint32_t hi32 = 0;

    while (p32 < p32end) {
        lo32 += *p32++;
        hi32 += lo32;
    }

    return (uint64_t)hi32 << 32 | lo32;
}
```

The algorithm for updating the LSA is single-threaded. Software is responsible for protecting a device's LSA so that only a single thread is updating the LSA at any time. This is typically done with a common mutex lock.

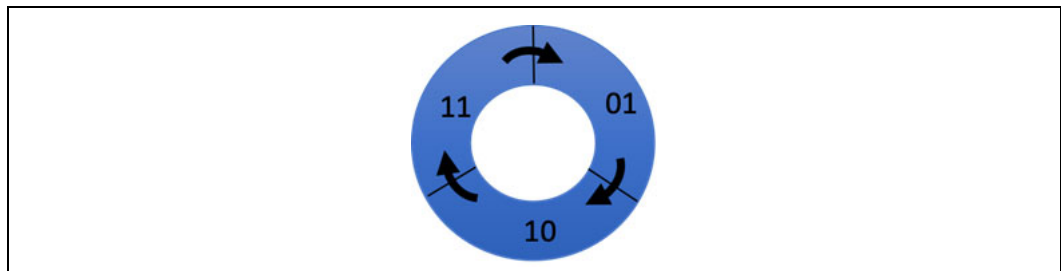
9.13.2.2 Label Index Blocks

Table 9-9 shows the layout of a Label Index Block.

Table 9-9. Label Index Block Layout

Field	Byte Offset	Length in Bytes	Description
Sig	00h	10h	Signature indicating a Label Index Block. Shall be set to "NAMESPACE_INDEX\0".
Flags	10h	3	No flags defined yet, shall be 0.
LabelSize	13h	1	Shall be 1. This indicates the size of labels in this LSA in multiples of 256 bytes (e.g., 1 for 256, 2 for 512, etc.).
Seq	14h	4	Sequence number. Only the two least significant bits of this field are used and shown in Figure 9-21 . All other bits shall be 0.
MyOff	18h	8	Offset of this index block in the LSA. Label Index Block 0 shall have 0 in this field, Label Index Block 1 shall have the size of the index block as its offset.
MySize	20h	8	Size of an index block in bytes. Shall be a multiple of 256.
OtherOff	28h	8	Offset of the other index block paired with this one.
LabelOff	30h	8	Offset of the first slot where labels are stored.
NSlot	38h	4	Total number of label slots.
Major	3Ch	2	The major version number of this layout. Shall be 2.
Minor	3Eh	2	The minor version number of this layout. Shall be 1.
Checksum	40h	8	Fletcher64 checksum of all fields in this Label Index Block. This field is assumed to be 0 when the checksum is calculated.
Free	48h	Varies	NSlot bits, padded with 0s to align index block to 256 bytes.

When reading Label Index Blocks, software shall consider index blocks to be valid only when their Sig, MyOff, OtherOff, and Checksum fields are correct. In addition, any blocks with Seq cleared to 0 are discarded as invalid. Finally, if more than 1 Label Index Block is found to be valid, the one with the older sequence number (immediately counterclockwise to the other, according to [Figure 9-21](#)) is discarded. If all checks pass and the sequence numbers match, the index block at the higher offset shall be considered the valid block. If no valid Label Index Blocks are found, the entire LSA is considered uninitialized.

Figure 9-21. Sequence Numbers in Label Index Blocks

When updating the Label Index Block, the current valid block, according to the rules above, is never directly written to. Instead, the alternate block is updated with the appropriate fields and a sequence number that is immediately clockwise as shown in [Figure 9-21](#). It is the appearance of a new block that passes all the checks and has a higher sequence number that makes this update atomic in the face of interruption.

Using this method of atomic update, software can allocate and deallocate label slots, even multiple slots, in a single, atomic operation. This is done by setting the Free bits to indicate which slots are free and which are in-use, and then updating the Label Index Block atomically as described above. To ensure that it is always possible to update a label atomically, there must always be at least one free label slot. That way, any used

label slots can be updated by writing the new contents to the free slot and using the Label Index Block update algorithm to mark the new version and in-use and the old version and free in one atomic operation. For this reason, software must report a “label storage area full” error when a caller tries to use the last label slot.

The Free field contains an array of NSlot bits, indicating which label slots are currently free. The Label Index Block is then padded with 0 bits until the total size is a multiple of 256 bytes. This means that up to 1472 label slots are supported by Label Index Blocks that are 256 bytes in length. For 1473 to 3520 label slots, the Label Index Block size must be 512 bytes in length, and so on.

9.13.2.3 Common Label Properties

Three types of labels may occupy the label slots in the LSA: Region Labels, Namespace Labels, and Vendor Specific Labels. The first two are identified by type fields containing UUIDs as specified in the following sections. Vendor Specific Labels contain a type UUID determined by the vendor per IETF RFC 4122. Software shall ignore any labels with unknown types. In this way, the Type field in the labels provides a *major version number*, where software can assume that a UUID that it expects to find indicates a label that it understands, since only backward-compatible changes are allowed to the label layout from the point where that UUID first appears in a published CXL specification.

Region Labels and Namespace Labels contain a 4-byte Flags field, used to indicate the existence of new features. Since those features must be backward compatible, software may ignore unexpected flags encountered in this field (no error generated). Software should always write 0s for Flags bits that were not defined at the time of implementation. In this way, the Flags field provide a *minor version number* for the label.

It is sometimes necessary to update labels atomically across multiple CXL devices. For example, when a Region or Namespace is being defined, the labels are written to every device that contributes to it. Region Labels and Namespace Labels define a flag, UPDATING, that indicates a multi-device update is in-progress. Software shall follow this flow when creating or updates a set of labels across devices:

1. Write each label across all devices with the UPDATING flag set.
2. Update each label, using the update algorithm described in the previous section, clearing the UPDATING flag.

Any time software encounters a set of labels with any UPDATING flags, it shall execute these rules:

- If there are missing labels (some components with the expected UUID are missing), then the entire set of labels is rolled-back due to the update operation being interrupted before all labels are written. The roll-back means marking each label in the set as free, following the update algorithm described in the previous section.
- If there are no missing labels, then the entire set of labels is rolled-forward, completing the interrupted update operation by removing the UPDATING flag from all labels in the set, following the update algorithm described in the previous section.

When sets of Region Labels or Namespace Labels are found to have missing components, software shall consider them invalid and not attempt to configure the regions or surface the namespaces with these errors. Exactly how these errors are reported and how users recover from them is implementation-specific, but it is recommended that software first report the missing components, providing the opportunity to correct the misconfiguration, before deleting the erroneous regions or namespaces.

9.13.2.4 Region Labels

Region labels describe the geometry of a persistent memory Interleave Set (the term “region” is synonymous with “Interleave Set” in this section). Once software has configured a functional Interleave Set for a set of CXL memory devices, region labels are added to the LSA for each device that contributes capacity to it. Table 9-10 shows the layout of a Region Label.

Table 9-10. Region Label Layout

Field	Byte Offset	Length in Bytes	Description
Type	00h	16	Shall contain this UUID: 529d7c61-da07-47c4-a93f-ecdf2c06f444. In the future, if a new, incompatible Region Label is defined, it shall be assigned a new UUID in the CXL specification defining it.
UUID	10h	16	UUID of this region per RFC 4122. This field is used to match up labels on separate devices that together describe a region.
Flags	20h	4	Boolean attributes of the region: 0000 0008h = UPDATING The UPDATING flag is used to coordinate Region Label updates across multiple CXL devices, as described in Section 9.13.2.3. All bits below 0000 0008h are reserved and shall be written as 0 and ignored when read. All bits above 0000 0008h are currently unused and shall be written as 0. The intention is to indicate the existence of backward-compatible features added in the future, so any unexpected 1s in this area shall be ignored (i.e., not treated as an error).
NLabel	24h	2	Total number of devices in this Interleave Set (interleave ways).
Position	26h	2	Position of this device in the Interleave Set, starting with the first device in position 0 and counting up from there.
DPA	28h	8	The DPA where the region begins on this device.
RawSize	30h	8	The capacity this device contributes to the Interleave Set (bytes).
HPA	38h	8	If nonzero, this region needs to be mapped at this HPA. This field is for platforms that need to restore an Interleave Set to the same location in the system memory map each time. A platform that does not support this shall report an error when a nonzero HPA field is encountered.
Slot	40h	4	Slot index of this label in the LSA.
Interleave Granularity	44h	4	The number of consecutive bytes that are assigned to this device: 0 = 256 Bytes 1 = 512 Bytes 2 = 1024 Bytes (1 KB) 3 = 2048 Bytes (2 KB) 4 = 4096 Bytes (4 KB) 5 = 8192 Bytes (8 KB) 6 = 16384 Bytes (16 KB) - All other encodings are reserved
Alignment	48h	4	The desired region alignment in multiples of 256 MB: 0 = No desired alignment 1 = 256-MB desired alignment 2 = 512-MB desired alignment - etc.
Reserved	4Ch	ACh	Shall be 0.
Checksum	F8h	8	Fletcher64 checksum of all fields in this Region Label. This field is assumed to be 0 when the checksum is calculated.

9.13.2.5 Namespace Labels

Namespace labels describe partitions of persistent memory that are exposed as volumes to software, analogous to NVMe* namespaces or SCSI logical unit numbers (LUNs). Exactly how an operating system uses these volumes is beyond the scope of this specification – namespaces may be exposed to applications directly, exposed via file systems, or used internally by the operating system. [Table 9-11](#) shows the layout of a Namespace Label.

Table 9-11. Namespace Label Layout

Field	Byte Offset	Length in Bytes	Description
Type	00h	10h	Shall contain this UUID: 68bb2c0a-5a77-4937-9f85-3caf41a0f93c. In the future, if a new, incompatible Namespace Label is defined, it shall be assigned a new UUID in the CXL specification defining it.
UUID	10h	10h	UUID of this namespace per IETF RFC 4122. All labels for this namespace shall contain matching UUIDs.
Name	20h	40h	“Friendly name” for the namespace, null-terminated UTF-8 characters. This field may be cleared to all 0s if no name is desired.
Flags	60h	4	Boolean attributes of the region: 0000 0008h = UPDATING The UPDATING flag is used to coordinate Namespace Label updates across multiple CXL devices, as described in Section 9.13.2.3 . All bits below 0000 0008h are reserved and shall be written as 0 and ignored when read. All bits above 0000 0008h are currently unused and shall be written as 0. The intention is to indicate the existence of backward-compatible features added in the future, so any unexpected 1s in this area shall be ignored (i.e., not treated as an error).
NRange	64h	2	Number of discontinuous ranges that this device contributes to namespace, used when the capacity contributed by this device is not contiguous. Each contiguous range will be described by a label and NRange described how many labels were required.
Position	66h	2	Position of this device in the range set, starting with zero for the first label and counting up from there.
DPA	68h	8	The DPA where the namespace begins on this device.
RawSize	70h	8	The capacity this range contributes to the namespace (bytes).
Slot	78h	4	Slot index of this label in the LSA.
Alignment	7Ch	4	The desired region alignment in multiples of 256 MB: 0 = No desired alignment 1 = 256-MB desired alignment 2 = 512-MB desired alignment - etc.
RegionUUID	80h	10h	UUID of the region that contains this namespace. If a valid region does not exist with this UUID, then this namespace is also considered unusable.
AddressAbstractionUUID	90h	10h	If nonzero, the address abstraction used by this namespace. Software defines the UUIDs used in this field and their meaning in software-specific and beyond the scope of this specification.
LBASize	A0h	2	If nonzero, logical block size of this namespace.
Reserved	A2h	56h	Shall be 0.
Checksum	F8h	8	Fletcher64 checksum of all fields in this Namespace Label. This field is assumed to be 0 when the checksum is calculated.

9.13.2.6 Vendor-specific Labels

Table 9-12 shows the layout of a Vendor-specific Label. Other than the Type field and the Checksum field, the vendor is free to store anything in the remaining 232 (E8h) bytes of the label.

Table 9-12. Vendor Specific Label Layout

Field	Byte Offset	Length in Bytes	Description
Type	00h	10h	Vendor-specific UUID.
	10h	E8h	Vendor-specific content.
Checksum	F8h	8	Fletcher64 checksum of all fields in this Vendor-specific Label. This field is assumed to be 0 when the checksum is calculated.

9.13.3 Dynamic Capacity Device (DCD)

Dynamic Capacity is a feature of a CXL memory device that allows memory capacity to change dynamically without the need for resetting the device. A DCD is a CXL memory device that implements Dynamic Capacity. Unlike a traditional DPA range that a CXL memory device might support, a Dynamic Capacity DPA range is subdivided into 1 to 8 DC Regions, each of which is subdivided by the DCD into a number of fixed-size blocks, referred to as DC blocks. The host software is expected to program the maximum potential capacity utilizing one or more HDM decoders to span the entire DPA range of all configured regions. The DCD controls the allocation of these DC blocks to the host and utilizes events to signal the host when changes to the allocation of these DC blocks occurs. The DCD communicates the state of these DC blocks through an Extent List that describes the starting DPA and length of all DC blocks the host can access. The Extent List does not contain extents that are still pending acceptance from the host via the Add Dynamic Capacity Response command (see [Section 8.2.9.9.3](#)). Similarly, the Extent List does contain extents that are still pending release acceptance from the host via the Release Dynamic Capacity command (see [Section 8.2.9.9.4](#)). [Figure 9-22](#) illustrates a typical Extent List. [Figure 9-23](#) illustrates an Extent List in which the DC blocks are shared by multiple hosts. Adding and releasing capacity utilizes the Extent List to control the host's access to portions of the memory without the need to alter the HDM programming of the total potential Dynamic Capacity.

Figure 9-22. Extent List Example (No Sharing)

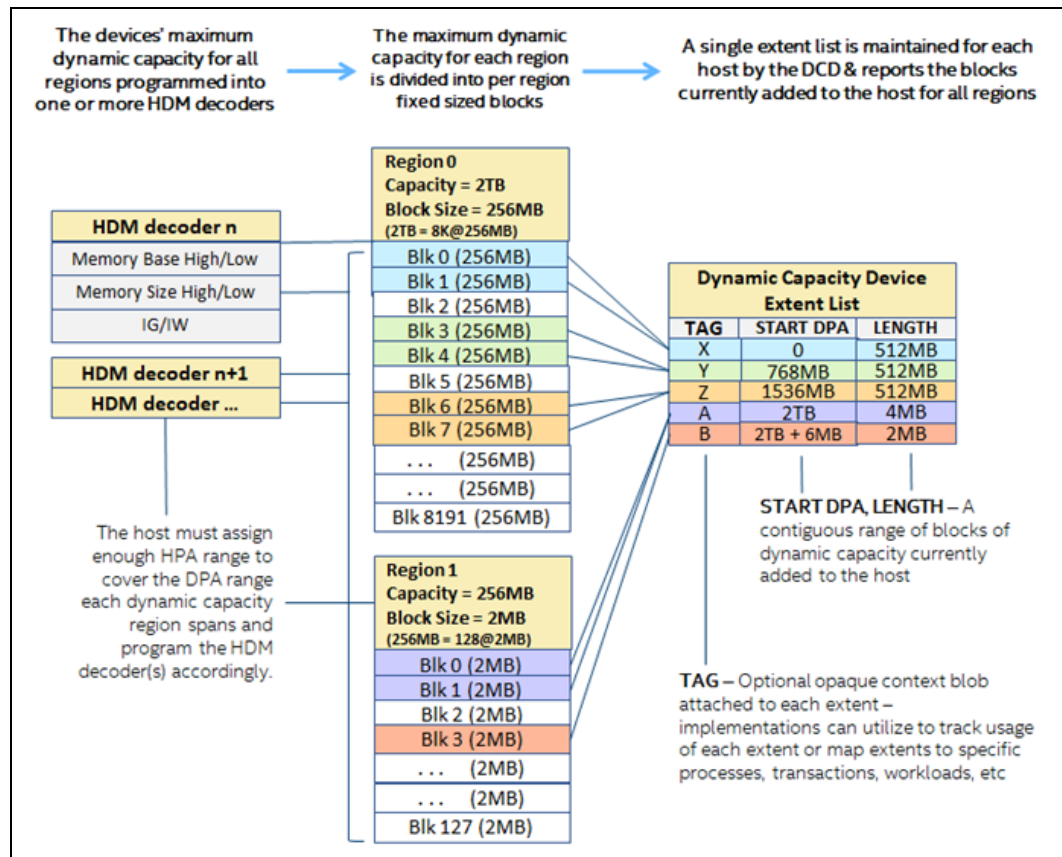
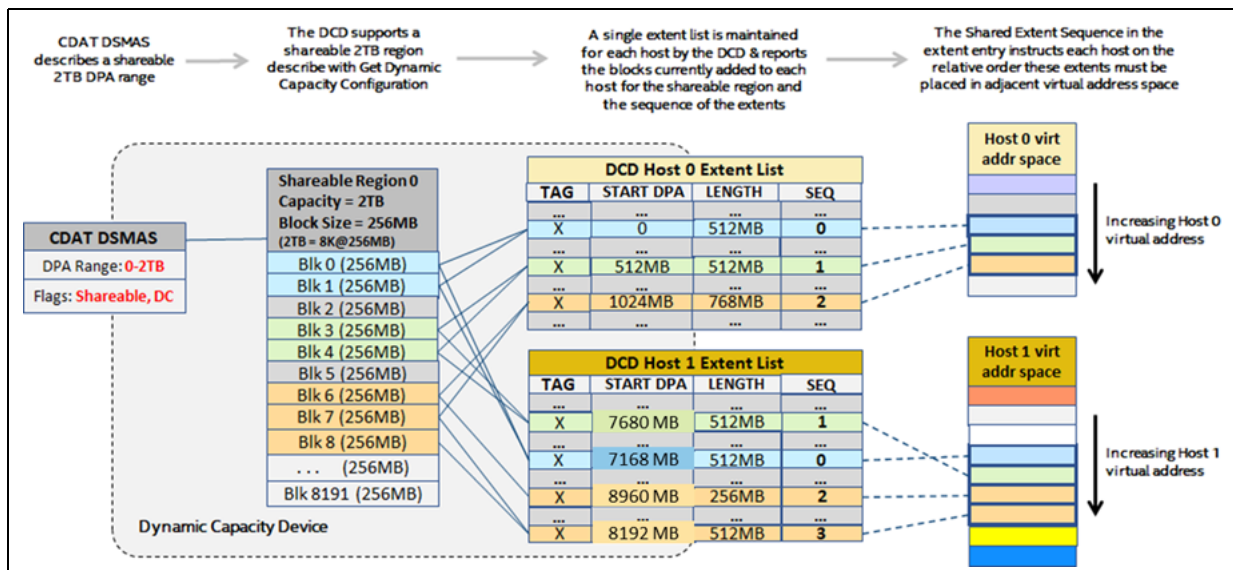


Figure 9-23. Shared Extent List Example

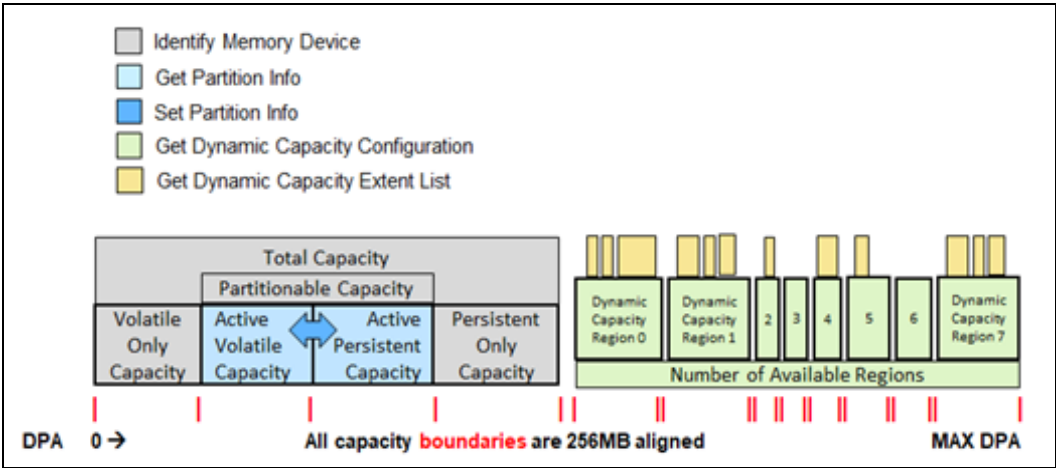


Dynamic Capacity is organized into 1 to 8 DC Regions as defined by the device. [Figure 9-24](#) illustrates this. Each DC Region has a unique maximum potential capacity, supported block size, and memory attributes. While the beginning and end of each region is 256 MB aligned, the start of the first block of data within each region is controlled by the DCD and aligned to the Dynamic Capacity block size configured for that region. Because the Extent List is DPA based, a single list can describe the extents in all regions. When the host fetches the current Extent List using the Get Dynamic Capacity Extent List mailbox command, the returned Extent List contains the device-assigned starting DPA and length for each extent that is assigned to the host. Regions are used in increasing-DPA order, with Region 0 being used for the lowest DPA of Dynamic Capacity and Region 7 for the highest DPA.

The DCD controls which DPA range it assigns to each region for each host. The DPA ranges exposed by the device to each host are independent of one another.

If the host issues a read to a DPA that is not allocated to the host, the device behavior is specified in [Table 8-27](#). If the host issues a write to a DPA that is not allocated to the host, the device shall drop the write and send an NDR (see [Section 3.3.9](#)) as a response. If the host issues a write to any DPA in a read-only DC Region, the device shall drop the write and send an NDR (see [Section 3.3.9](#)) as a response.

Figure 9-24. DCD DPA Space Example



The attributes associated with each region are described in the device's CDAT. The device associates each supported region with a specific DSMAS instance so the host can determine the memory attributes associated with each given region. A device that supports Dynamic Capacity shall report its configured regions in one or more CDAT DSMAS structures and shall set the Dynamic Capacity DSMAS Flag in each structure to indicate a Dynamic Capacity supported range. When reporting the region configuration, the DCD shall supply the DSMAD Handle with which each region is associated.

Devices that instantiate multiple LDs, including MLDs and Multi-Headed devices, share certain region configuration parameters, as defined in [Table 7-67](#), across all LDs in that device.

The basic sequence to utilize Dynamic Capacity include:

- Utilize Get Supported Logs sub-list (see [Section 8.2.9.5.6](#)) or Get Supported Logs (see [Section 8.2.9.5.1](#)) and Get Log (see [Section 8.2.9.5.2](#)) to retrieve the Command Effects Log (CEL). Verify that the necessary Dynamic Capacity commands are returned in the CEL, indicating Dynamic Capacity is supported by the device.

- Issue Get Dynamic Capacity Configuration command: The device reports its number of available regions and each region's base address, length, block size, and DSMAD Handle (see [Section 8.2.9.9.9.1](#)).
- Program the HDM decoders appropriately for each region's base and length from Get Dynamic Capacity Configuration data. The host may utilize one or more HDM decoders to span the current configuration of Dynamic Capacity reported by the device. It is strongly recommended that the host provide adequate decoder size to cover all of the regions that are enabled. If not, the host may not be able to accept some of the Add Dynamic Capacity offers from the DCD.
- Retrieve the initial Extent List with one or more calls to Get Dynamic Capacity Extent List (see [Section 8.2.9.9.9.2](#)). If the list contains extents, then that memory can be utilized immediately.

The basic sequence to add Dynamic Capacity to a host:

- The DCD adds an Add Capacity Event Record (see [Section 8.2.9.2.1.6](#)) to the device's Dynamic Capacity Event Log containing the extent of the capacity being added, sets the Dynamic Capacity Event Log bit in the Event Status register and, if enabled, generates an interrupt to alert the host to the new event record. The DCD does this for each extent in the Add Capacity operation being performed, using the More flag as necessary (see [Table 8-50](#)), avoiding overflow, and allowing the host to consume the events as necessary to complete the operation. If the Dynamic Capacity Event Log overflows at any point, the host shall utilize Get Dynamic Capacity Extent List to retrieve the current list of host accessible DC blocks.
- When the host software retrieves the Add Capacity event record containing the extent of the capacity to be added, it responds back to the device with the updated extent for the exact capacity it added with a single call to Add Dynamic Capacity Response (see [Section 8.2.9.9.9.3](#)). This allows the host to control exactly how much of the added capacity it wishes to utilize, which may be less than the amount of capacity sent in the add capacity event, or even 0.
- If supported by the device, the host may utilize Get Poison List or Scan Media with the Starting DPA and Length of the added capacity extent to check for poisoned addresses.

The basic sequence to release Dynamic Capacity from a host:

- The DCD adds a Release Capacity Event Record to the device's Dynamic Capacity Event Log (see [Section 8.2.9.2.1.6](#)) containing the extent of the capacity it is requesting to be released, sets the Dynamic Capacity Event Log bit in the Event Status register and, if enabled, generates an interrupt to alert the host to the new event record. The DCD does this for each extent in the Release Capacity operation being performed, using the More flag as necessary (see [Table 8-50](#)), avoiding overflow, and allowing the host to consume the events as necessary to complete the operation. If the Dynamic Capacity Event Log overflows at any point, the host shall utilize Get Dynamic Capacity Extent List to retrieve the current list of host accessible DC blocks.
- When the host software retrieves the Release Capacity event record containing the extent of the capacity to be released, the host software releases some or all of the capacity from use and responds back to the device with the updated Extent List for the exact capacity it released using the Release Dynamic Capacity command (see [Section 8.2.9.9.9.4](#)). If desired, the host may choose to make unavailable the contents of the capacity being released by whatever means it chooses, including but not limited to issuing the Sanitize or Secure Erase commands, if supported by the device, before the Release Dynamic Capacity command. The host may call Release Dynamic Capacity multiple times, returning different portions of the total capacity over time, in response to the Release Capacity event record. This allows the host to control exactly how much of the released capacity it wishes to release and when it is released.

Prior to issuing Release Dynamic Capacity command, the host software is required to off-line the capacity and complete the necessary coherence management actions.

The basic sequence to release Dynamic Capacity asynchronously from a host (not associated with an event from the device):

- The host may release Dynamic Capacity back to the device, at any time, without receiving a Release Capacity Event Record by calling Release Dynamic Capacity (see [Section 8.2.9.9.4](#)) with an Extent List containing specific released capacity.

Devices may forcefully release Dynamic Capacity from a host:

- Host access to the released capacity may be immediately disabled and the DCD behaves as if the capacity is no longer allocated to the host. The DCD adds a Forced Capacity Release Event Record to the device's Dynamic Capacity Event Log containing the extent of the capacity being released, sets the Dynamic Capacity Event Log bit in the Event Status Register and, if enabled, generates an interrupt to alert the host to the new event record. If the Dynamic Capacity Event Log overflows at any point, the forced removal still occurs and the host shall utilize Get Dynamic Capacity Extent List to retrieve a new list of host accessible DC blocks.

LD-FAM based DCD shall forcefully release any shared Dynamic Capacity associated with an LD upon a Conventional Reset or a CXL Reset of that LD. MH-SLD or MH-MLD based DCD shall forcefully release shared Dynamic Capacity associated with all associated hosts upon a Conventional Reset of a head. LD-FAM based DCD shall forcefully release shared Dynamic Capacity associated with all associated hosts upon a Conventional Reset of the entire DCD. No Forced Capacity Release Event Record is created when capacity is released as a result of a reset.

The host retrieves the Release Capacity event record containing the extent of the capacity that has been released. The host may respond back to the device with the updated Extent List for the released capacity using the Release Dynamic Capacity command. The host may call Release Dynamic Capacity multiple times, returning different portions of the total capacity over time. Host responses to this event are optional and shall not influence the device's release of the capacity.

9.13.3.1 DCD Management By FM

LD-FAM DCDs implement multiple LDs to support multiple host interfaces and can dynamically assign and reassign memory capacity among those LDs. All G-FAM Devices (GFDs) are DCDs since GFDs exclusively use Dynamic Capacity mechanisms for their capacity management.

The FM is responsible for discovering a DCD's capabilities and for configuring memory assignment.

1. The FM issues Get DCD Info (see [Section 7.6.7.6.1](#)) to discover the number of supported hosts, supported features, and dynamic memory capacity. The current assignment of capacity to a specific host is queried with Get Host DC Region Configuration and Get DC Region Extent Lists (see [Section 7.6.7.6.2](#) and [Section 7.6.7.6.4](#), respectively). See [Section 8.2.9.9.10](#) for the equivalent GFD commands.
2. Resources are assigned to each host using Initiate Dynamic Capacity Add and Initiate Dynamic Capacity Release (see [Section 7.6.7.6.5](#) and [Section 7.6.7.6.6](#), respectively). The device generates a Dynamic Capacity Event Record (see [Section 8.2.9.9.4](#)) to notify the FM of any host responses. See [Section 7.7.2](#) and [Section 7.7.14](#) for the equivalent GFD commands and policies.

9.13.3.2 Setting up Memory Sharing

The FM may use the following sequence to set up sharing between hosts, where all hosts are able to read and write to the shared capacity:

1. Issue Initiate Dynamic Capacity Add Request with the Selection Policy set to Free or Contiguous or Prescriptive with the Host ID associated with the first host. The region number must correspond to a region that is advertised as sharable.
2. If the above request is successful as indicated by a new Add Capacity Response event in the Dynamic Capacity Event record, issue Initiate Dynamic Capacity Add Request with Selection Policy=Enable Shared access with the Host ID associated with the second host. The Tag field must match the Tag value used in step 1.
3. Repeat step 2 for any other hosts that need to share this memory range.

The FM may use the following example sequence to allocate a set of tagged capacity and allow it to be initialized by a host and then shared with one or more hosts as read-only.

1. Issue Initiate Dynamic Capacity Add Request with the Selection Policy set to Free or Contiguous or Prescriptive with the Host ID associated with the first host. The region number must correspond to a region that is advertised as writable and sharable.
2. If the above request is successful, the tagged shared capacity can be initialized by the first host.
3. Issue a Dynamic Capacity Add Reference Request for the tag associated with the capacity. Holding this Reference prevents the tagged capacity from being freed and sanitized in step 4.
4. After the first host has initialized the tagged shared capacity, issue an Initiate Dynamic Capacity Release Request for the tag associated with the capacity, and then await completion.
5. If the request in step 4 is successful as indicated by a new Release Capacity Response event in the Dynamic Capacity Event record, the capacity associated with the Tag is preserved but not mapped to any hosts.
6. Issue an Initiate Dynamic Capacity Add Request with Selection Policy=Enable Shared Access with the Host ID associated with the second host, specifying a Region that is Sharable and read-only. The Tag field must match the Tag value used in step 1.
7. Repeat step 5 for any other hosts that need to share the tagged capacity.
8. Issue a Dynamic Capacity Remove Reference Request to remove the FM reference to the tagged capacity.
9. To withdraw the shared capacity, issue a Initiate Dynamic Capacity Release command for each host.
10. When the tagged capacity has been released from all hosts, if the FM does not hold a reference, the tagged capacity will be sanitized (if appropriate) and freed, at which point the tag no longer exists and the capacity is available for future use.

9.13.3.3 Extent List Tracking

The storage of extent list information, including individual extents and their associated tags, consumes resources in a DCD. As such, DCDs are permitted to limit the number of extents and number of tags of which they are capable of tracking. This capability is reported in a DCD's **Get Host DC Region Configuration** and **Get Dynamic Capacity Configuration** responses.

A DCD is responsible for tracking all extents and tags that comprise extent lists in the following states:

- **Pending:** Defining capacity specified in an **Initiate Dynamic Capacity Add** request that has not been responded to by a host
- **Added:** Defining capacity that has been accepted by a host as part of an **Add Dynamic Capacity** request and is present in the extent list returned to the host in the response to a **Get Dynamic Capacity Extent List** request
- **FM-referenced:** Defining capacity to which an FM reference has been added, as reported by the **FM Holds Reference** bit in the response to **Dynamic Capacity List Tags**

A DCD reports its **Number of Available Extents** and **Number of Available Tags** as its total capacity minus all extents and tags tracked for capacity in the Pending, Added, and FM-referenced states, respectively.

9.13.4 Capacity or Performance Degradation

A CXL device may detect an unrecoverable error during its initialization and may be able to operate with a reduced capacity or reduced performance. If this failure results in capacity degradation and it is detected prior to `Memory_Info_Valid=1`, the device shall update the `Memory_Size` fields in the corresponding DVSEC CXL Range Size registers (see [Section 8.1.3.8.1](#), [Section 8.1.3.8.2](#), [Section 8.1.3.8.5](#), and [Section 8.1.3.8.6](#)), CDAT DSMAS structures, response to Identify Memory Device command, and response to Get Partition Info command to report the reduced size. It is recommended that the device also set the Memory Capacity Degraded flag in the Health Status field (see [Table 8-133](#)).

If the failure results in performance degradation and it is detected prior to `Memory_Info_Valid=1`, the CDAT DSLBIS structure shall be updated and the Performance Degraded flag in the Health Status field (see [Table 8-133](#)) should be set. If `Mem_HwInit_Mode=1`, `Memory_Active` bit(s) shall be set when the memory range is fully initialized and available for software use.

If this failure is detected after the `Memory_Info_Valid` bit is set, but before the `Memory_Active` bit is set, the device shall not set the `Memory_Active` bit. The device updates the CDAT in the following manner:

- CDAT sequence number shall be incremented to indicate to SW that CDAT content has changed.
- If the failure results in capacity degradation, the CDAT DSEMTS entries shall mark the bad memory as "EFIUnusableMemory" indicating to the SW that it shall not use the associated DPA range on this device. The Memory Capacity Degraded flag in the Health Status field (see [Table 8-133](#)) shall be set.
- If the failure results in performance degradation, the CDAT DSLBIS structure shall be updated and the Performance Degraded flag in the Health Status field (see [Table 8-133](#)) shall be set.

If `Mem_HwInit_Mode=1`, `Memory_Active_Degraded` shall be set when the reduced capacity is fully initialized and available for software use.

The device capacity reported by Identify Memory Device (see [Section 8.2.9.9.1.1](#)) and Get Partition Info (see [Section 8.2.9.9.2.1](#)) commands shall be consistent with capacity advertised by CDAT that is not marked as EFIUnusableMemory.

9.14 Back-Invalidate Configuration

This section describes how System Software may discover whether a component supports Back-Invalidate and how BI-IDs are assigned.

9.14.1 Discovery

Back-Invalidate (BI) messages require the link to operate in 256B Flit mode. Alternate Protocol Negotiation flow establishes the optimal Flit mode and PCIe DVSEC for Flex Bus Port registers (see [Section 8.2.1.3](#)) identifies the negotiated Flit mode. The presence of the CXL BI Decoder Capability Structure indicates that the component is capable of supporting BI.

9.14.2 Configuration

Before enabling a device to issue BI requests, System Software must ensure that the device, the host, and any switch(es) in the path are capable of BI and that the link(s) between the device and the host are operating in 256B Flit mode.

BI-capable Downstream Ports and devices advertise the CXL BI Decoder Capability Structure (see [Section 8.2.4.27](#)). System Software configures them to enable BI functionality. The BI-ID of a device must be unique within a VH. This is ensured by using the device's Bus Number as the BI-ID. The Downstream Port decode functionality is described in [Table 9-13](#) and [Table 9-14](#).

Table 9-13. Downstream Port Handling of BISnp

BI Enable Value	BI Forward Value	Behavior
0	0	Discard
0	1	Forward upstream as is
1	0	Perform the following checks: - Locate the HDM decoder in the USP or RC that decodes the BISnp address. - Verify that the BI bit in that HDM decoder is set. - Optionally, verify that the Target Port that corresponds to the BISnp address matches the port that generated the BISnp request. If this is a DSP: - If above checks pass, Set BI-ID= Secondary Bus Number and forward upstream; otherwise, discard. If this is a root port: - If above checks pass, forward upstream; otherwise, discard. Root port may use host proprietary mechanisms to initialize BI-ID and route the associated BIRsp messages.
1	1	Discard (Invalid setting)

Table 9-14. Downstream Port Handling of BIRsp

BI Enable Value	BI Forward Value	Behavior
0	0	Discard
0	1	Forward downstream as is
1	0	If this is a DSP: - If BI-ID=Secondary Bus Number, forward downstream; otherwise, discard. If this is a root port: - Use host-specific checks to ensure correct routing of the BISnp response. Forward downstream if these checks pass; otherwise, discard.
1	1	Discard (Invalid setting)

The USP in a BI-capable Switch may advertise the CXL BI Route Table capability Structure (see [Section 8.2.4.26](#)). If a USP receives an M2S BIRsp message, the USP shall look up the Port Number associated with the Bus Number that is carried in the message's BI-ID field, and then forward the message to that Port. The BI-ID is guaranteed to correspond to a valid BI-capable device, specifically the one that generated the BISnp request. If the Port Number does not match any DSP due to incorrect programming, the BIRsp message shall be dropped.

If a USP receives an S2M BISnp message, the USP may look up the Port Number associated with the Bus Number that is carried in the message's BI-ID field, and then verify that the Port Number matches the Port Number of the originating DSP before forwarding the BISnp message upstream. If the Port Number derived from this structure does not match the DSP's Port Number, the BISnp message may be dropped.

IMPLEMENTATION NOTE

System software may use the following sequence to configure a BI-capable Device D below a Switch S as follows:

1. Verify that all the CXL link(s) between Device D and the host are operating in 256B Flit mode.
2. Ensure the device has been assigned a valid Bus number.
3. Enable BI on the DSP of Switch S that is directly connected to Device D:
 - a. BI Forward=0.
 - b. BI Enable=1.
4. If the DSP's BI Decoder Capability register indicates Explicit BI Decoder Commit Required=1, commit the BI-ID changes via the following sequence:
 - a. BI Decoder Commit=0 to rearm.
 - b. BI Decoder Commit=1.
 - c. Poll bits 0 and 1 of the BI Decoder Status register until timeout or one of them is set. The timeout value is reported in the BI Decoder Status register.
 - d. If BI Decoder Committed=1, the changes were committed. Proceed to step 5.
 - e. If BI Decoder Error Not Committed=1, the changes were not committed. Software should treat this as an error condition.
 - f. If neither bit is set and the timeout is reached, Software should treat this as an error condition.
5. If the USP implements CXL BI Route Table Capability Structure and Explicit BI RT Commit Required=1, commit the BI-ID changes as follows:
 - a. BI RT Decoder Commit=0 to rearm.
 - b. BI RT Decoder Commit=1.
 - c. Poll bits 0 and 1 of the BI RT Status register until timeout or one of them is set. The timeout value is reported in the BI RT Status register.
 - d. If BI RT Error Not Committed=1, the changes were not committed. Software should treat this as an error condition.
 - e. If BI RT Committed=1, the changes were committed. Proceed to step 6.
 - f. If neither bit is set and the timeout is reached, Software should treat this as an error condition.
6. If the previous steps were successful, configure the Root Port that is directly connected to Switch S to forward BI messages if it isn't already set up that way:
 - a. If BI Forward=0, set BI Forward=1.
 - b. Ensure BI Enable=0.
7. If the previous steps were successful, configure Device D to enable BI:
 - a. BI Enable=1.
8. If the previous steps were successful, inform the device driver that Device D may now issue BI requests.

IMPLEMENTATION NOTE

System software may use the following sequence to deallocate the BI-ID B that was previously assigned to Device D below Switch S as follows:

1. Notify Device D's device driver that Device D is no longer allowed to issue BI requests and then wait for acknowledgment.
2. Configure Device D to disable BI:
 - a. BI Enable=0.
3. Configure the DSP of Switch S that is directly connected to Device D to unassign BI-ID B as follows:
 - a. BI Forward=0.
 - b. BI Enable=0.
4. If the DSP's CXL BI Decoder Capability register indicates Explicit BI Decoder Commit Required=1, commit the BI-ID changes as follows:
 - a. BI Decoder Commit=0 to rearm.
 - b. BI Decoder Commit=1.
 - c. Poll bits 0 and 1 of the BI Decoder Status register until timeout or one of them is set. The timeout value is reported in the BI Decoder Status register.
 - d. If BI Decoder Error Not Committed=1, the changes were not committed. Software should treat this as an error condition.
 - e. If BI Decoder Committed=1, the changes were committed. Proceed to step 5.
 - f. If neither bit is set and the timeout is reached, Software should treat this as an error condition.
5. If the USP implements CXL BI Route Table Capability Structure and Explicit BI RT Commit Required=1, commit the BI-ID changes as follows:
 - a. BI RT Commit=0 to rearm.
 - b. BI RT Commit=1.
 - c. Poll bits 0 and 1 of the BI RT Status register until timeout or one of them is set. The timeout value is reported in the BI RT Status register.
 - d. If BI RT Error Not Committed=1, the changes were not committed. Software should treat this as an error condition.
 - e. If BI RT Committed=1, the changes were committed. Proceed to step 6.
 - f. If neither bit is set and the timeout is reached, Software should treat this as an error condition.
6. If the previous steps were successful, and no other devices in this VCS have been assigned a BI-ID, configure the Root Port that is directly connected to Switch S to stop forwarding BI messages as follows:
 - a. BI Forward=0.

Ensure BI Enable=0.

9.14.3**Mixed Configurations**

This section describes scenarios where a BI-capable device is plugged into a system that does not support BI.

9.14.3.1 BI-capable Type 2 Device

If a BI-capable Type 2 device is connected to a Downstream Port that does not support 256B Flit mode, the device is able to detect this condition during the Hardware Autonomous Mode Negotiation (see [Section 6.4.1.1](#)) and fall back to another mode (e.g., Type 2 HDM-D mode or PCIe mode) based on the device vendor's policy.

If a BI-capable Type 2 device is connected to a switch that supports BI, but the host does not support BI, the device cannot be operated in BI mode. In this case, the System Software or the System Firmware may choose to reconfigure the Type 2 device to operate in a fallback mode.

It is legal for BI-capable Type 2 devices to not support HDM-D flow; however, such a device must support fallback to either operate as a PCIe device, Type 1 device, or a Type 3 device. These flows are described in [Section 9.14.3.2](#).

If a Type 2 device advertises support for HDM-D flow via the BI Decoder Capability register (see [Section 8.2.4.27.1](#)), the device is operated in that mode as long as the number of Type 2 devices using HDM-D flow does not exceed the host's capabilities and the CXL specification restrictions. A CXL Type 2 device that supports HDM-D flow may be unable to operate in that mode due to system configuration restrictions. In many scenarios, the device may be unable to make that determination on its own and may require assistance from System Software or System Firmware. See [Section 9.14.3.2](#).

9.14.3.2 Type 2 Device Fallback Modes

[Table 9-15](#) describes the actions that System Software or System Firmware may take when a Type 2 device cannot be operated in either HDM-DB mode or in HDM-D mode, based on the Fallback Capability field value in the DVSEC CXL Capability2 register (see [Section 8.1.3.7](#)).

Table 9-15. CXL Type 2 Device Behavior in Fallback Operation Mode

Register Value ¹	Behavior
00b	The device can be operated as an RCD. If the device does not support HDM-DB flow, it supports HDM-D flow. If the device supports HDM-DB flow, it also supports HDM-D flow and must return HDM-D Capable=1 (see Section 8.2.4.27.1). If the device cannot be operated as a Type 2 device, it must be disabled.
01b	The device supports either HDM-DB flow or HDM-D flow or both. In addition, it can operate as a PCIe device. If the device cannot be operated in either HDM-DB mode or in HDM-D mode, System Firmware or System Software may disable Alternate Protocol Negotiation by programming the DSP registers and retraining the link so that the link comes up in PCIe mode.
10b	The device supports either HDM-DB flow or HDM-D flow or both. In addition, it can operate as a CXL Type 1 device. If the device cannot be operated in either HDM-DB mode or in HDM-D mode, System Firmware or System Software may reconfigure the DVSEC Flex Bus Port Control register (see Section 8.2.1.3.2) in the Downstream Port above the device to not advertise CXL.mem and then retrain the link, thereby bringing up the device as a CXL Type 1 device.
11b	The device supports either HDM-DB flow or HDM-D flow or both. In addition, it can operate as a CXL Type 3 device. If the device cannot be operated in either HDM-DB mode or in HDM-D mode, System Firmware or System Software may reconfigure the Flex Bus Port Control register (see Section 8.2.1.3.2) in the Downstream Port above the device to not advertise CXL.cache and then retrain the link, thereby bringing up the device as a CXL Type 3 device.

1. Fallback Capability field values in the DVSEC CXL Capability2 register (see [Section 8.1.3.7](#)).

More-complex policies, such as configuring the Device to operate in CXL.io only mode or another mode based on peer devices, are possible; however, those policies are beyond the scope of this specification.

9.14.3.3 BI-capable Type 3 Device

A BI-capable Type 3 device is required to operate correctly when System Software has not enabled BI. In this case, the device functionality that is dependent on BI will not be available.

If a BI-capable Type 3 device is connected to a Downstream Port that does not support 256B Flit mode, the device may continue to advertise BI capability via the CXL BI Decoder Capability Structure (see [Section 8.2.4.27](#)). The System Software shall ensure that the BI bit in none of the HDM decoders in the device, the switch, or the host that spans the device's HDM is set. If a BI-capable Type 3 device is present in a system where the host does not support BI, the System Software shall ensure that the BI bit in none of the HDM decoders in the device, the switch, or the host that spans the device's HDM is set. In both cases, the System Software is responsible for ensuring that the BI bit in the CXL BI Decoder Control register (see [Section 8.2.4.27.2](#)) in the device, as well as the Downstream Port it is connected to, is programmed to 0.

9.15 Cache ID Configuration and Routing

The CXL 3.0 specification introduces protocol enhancements that allow for more than one active CXL.cache agent per VCS. The identity of the CXL.cache agent is carried via the CacheID field in the CXL.cache messages. If the CXL link is operating in 256B Flit mode, the CXL.cache messages can carry 4 CacheID bits. Before enabling more than one CXL.cache device per VCS, Software must ensure that the host and any switch(es) in the path advertise the CXL Cache ID Decoder Capability Structure, and that all the link(s) between the lowest-level switch and the host are operating in 256B Flit mode.

Downstream Ports advertise the CXL Cache ID Decoder Capability structure to indicate that the Downstream Ports can assign and decode the CacheID field in CXL.cache messages (see [Section 8.2.4.29](#)). Software configures the Downstream Ports to enable CacheID forwarding functionality and assign a CacheID to the device. The CacheID must be unique within a VH and must account for the constraints placed by the Flit mode and the host capabilities.

Any CXL.cache device can operate correctly in a system that is capable of supporting more than one active CXL.cache agent per VCS; however, System Firmware or System Software that is aware of this new capability and capable of correctly configuring the switch and/or host is required to take advantage of this capability.

9.15.1 Host Capabilities

The host requires dedicated resources to track each CacheID source. As such, it is necessary to account for host constraints when assigning CacheID. The host constraints are expressed in terms of the total number of CacheIDs that the host can track per CXL Host Bridge. This information is conveyed via the Cache ID Target Count field in the CXL Cache ID Route Table Capability register (see [Section 8.2.4.28.1](#)) associated with the Host Bridge.

9.15.2 Downstream Port Decode Functionality

Downstream Port decode functionality is described in [Table 9-16](#) and [Table 9-17](#). The associated registers are defined in [Section 8.2.4.14](#).

Table 9-16. Downstream Port Handling of D2H Request Messages

Assign Cache ID Value	Forward Cache ID Value	Behavior
0	0	Discard
0	1	Forward upstream. If the message was received over a link operating in 68B Flit mode, the request is processed as if CacheID field is 0.
1	0	Set CacheID=Local Cache ID and forward upstream. The link between the device and the Downstream Port may be operating in 68B Flit mode, in which case the D2H request message received by the Downstream Port does not contain the CacheID field.
1	1	Discard (Invalid setting)

In addition to the checks documented in [Table 9-16](#), the root port shall implement the following steps before forwarding the message upstream:

- If HDM-D Type 2 Device Present=1, compare CacheID with the HDM-D Type 2 Device Cache ID field. If there is a match, identify this device as a Type 2 device that is using HDM-D flows. The host shall follow the HDM-D flows when responding to this device, which includes enforcing the setting in the CXL.cache Trust Level field of the Root Port Security Policy register (see [Table 8-29](#)).
- If the Requester is using HDM-DB flows, abort the request if Block CXL.cache HDM-DB=1.

Table 9-17. Downstream Port Handling of H2D Response Message and H2D Request Message

Assign Cache ID Value	Forward Cache ID Value	Behavior
0	0	Discard
0	1	Forward downstream as is
1	0	If CacheID=Local CacheID, forward downstream; otherwise, discard. The link between the device and the Downstream Port may be operating in 68B Flit mode, in which case the H2D message received by the device does not contain the CacheID field. The device shall ignore the CacheID field in H2D messages, if present.
1	1	Discard (Invalid setting)

D2H response messages and D2H data messages do not carry CacheID and are always routed back to the host.

9.15.3 Upstream Switch Port Routing Functionality

When a USP receives a D2H request message from a DSP, the USP shall forward the message upstream. A USP may look up the Port Number associated with the CacheID field in the message from the CXL Cache ID Route Table and may compare that to the Port Number of the DSP that the message came from before forwarding the message.

When a USP receives an H2D request message, H2D data message or an H2D response message, the USP shall use the message's CacheID field to look up the corresponding CXL Cache ID Target N register (see [Section 8.2.4.28.4](#)). If the Valid bit in the Cache ID Target register is 0, the H2D message shall be discarded without a response. If the Valid bit is 1, the message shall be forwarded to the local DSP based on the Port Number field that is programmed in the CXL Cache ID Target N register.

D2H response messages and D2H data messages do not carry CacheID and are always routed back to the host.

If a USP receives CXL.cache message over a link operating in 68B Flit mode, it shall process the request as if the CacheID field is 0. A switch that is not capable of decoding CacheID field must be configured such that no more than one DSP is enabled for CXL.cache traffic (indicated by Cache_Enable=1 in the DVSEC Flex Bus Port Status register; see [Section 8.2.1.3.3](#)). The USP shall direct all H2D traffic to that DSP.

9.15.4 Host Bridge Routing Functionality

When the Host Bridge receives the equivalent of an H2D request or an H2D response message from the host, the Host Bridge logic shall use the CacheID field to look up the corresponding CXL Cache ID Target N register (see [Section 8.2.4.28.4](#)). If the Valid bit is 0, the H2D message is discarded. If the Valid bit is 1, the message is forwarded to the local root port based on the Port Number field that is programmed in the CXL Cache ID Target N register.

When the Host Bridge receives a D2H request message from the root port, the Host Bridge shall forward the message to the host, using host-specific mechanisms. The Host Bridge may optionally look up the root port that is associated with the CacheID and discard the message if the message was received from a different root port.

IMPLEMENTATION NOTE

System Software may use the following sequence to allocate a Cache ID to a BI-capable CXL.cache Device D below a Switch S and enable the Device to generate CXL.cache transactions that target any memory:

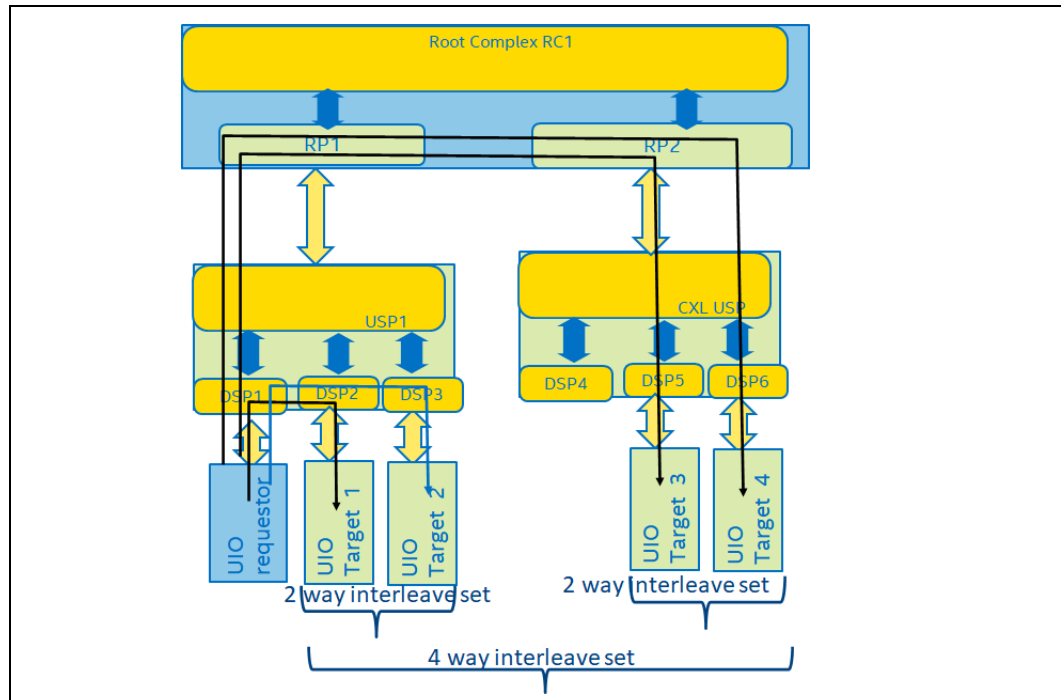
1. Verify that the CXL link between Switch S and the host is operating in 256B Flit mode.
2. Identify an unused and legal CacheID value, c, and allocate it to Device D. Software must take into account the current Flit mode, as well as the Cache ID Target Count fields, while assigning Cache IDs to devices.
3. Configure the DSP of Switch S that is directly connected to Device D to assign Cache ID=c to Device D:
 - a. Forward Cache ID=0.
 - b. Local Cache ID=c.
 - c. Assign Cache ID=1.
4. If the above DSP of Switch S reports Explicit Cache ID Decoder Commit Required=1, commit the Cache ID changes as follows:
 - a. Cache ID Decoder Commit=0 to rearm.
 - b. Cache ID Decoder Commit=1.
 - c. Poll bits 0 and 1 of the Cache ID Decoder Status register until timeout or one of them is set. The timeout value is reported in the Cache ID Decoder Status register.
 - d. If Cache ID Decoder Error Not Committed=1, the changes were not committed. Software should treat this as an error condition.
 - e. If Cache ID Decoder Committed=1, the changes were committed. Proceed to Step 5.
 - f. If neither bit is set and the timeout is reached, software should treat this as an error condition.
5. Configure the USP of Switch S to route Cache ID c:
 - a. Route Table[c]= Port Number register of the DSP that is connected directly to Device D.
6. If the USP reports Explicit Cache ID RT Commit Required=1, commit the Cache ID changes as follows:
 - a. Cache ID RT Commit=0 to rearm.
 - b. Cache ID RT Commit=1.
 - c. Poll bits 0 and 1 of the Cache ID RT Status register until timeout or one of them is set. The timeout value is reported in the Cache ID RT Status register.
 - d. If Cache ID RT Error Not Committed=1, the changes were not committed. Software should treat this as an error condition.
 - e. If Cache ID RT Committed=1, the changes were committed. Proceed to Step 7.
 - f. If neither bit is set and the timeout is reached, software should treat this as an error condition.
7. Configure the Root Port, R, that is directly connected to Switch S to decode the CXL.cache messages from Device D:
 - a. If Forward Cache ID=0, set Forward Cache ID=1.
 - b. Ensure Assign Cache ID=0.
8. If the previous steps were successful, configure the CXL Cache ID Route Table (see [Section 8.2.4.28.1](#)) in the Host Bridge:
 - a. Route Table[c].Port Number=Port Number register of Root Port R.
9. If the previous steps were successful, inform the device driver that Device D may now issue CXL.cache requests.

9.16 UIO Direct P2P to HDM

CXL.mem devices that can complete UIO requests that target its HDM, advertise the capability via the UIO Capable bit in the CXL HDM Decoder Capability register (see [Section 8.2.4.20.1](#)). CXL switches may allow routing of UIO accesses to HDM in the same VH as the UIO requester and advertise this capability via the same bit. CXL Host Bridges may allow routing of UIO accesses to host memory or HDM below another root ports in the same Host Bridge and advertise this capability via this bit. Prior to setting up a UIO path from a UIO requester to an HDM or to host memory, the Software must consult the capabilities of the target device and any switches or Host Bridges in the path.

[Figure 9-25](#) shows a configuration with four CXL.mem devices that form three separate interleave sets and how a UIO requester is able to access the HDM range. UIO accesses to UIO Target 1 and UIO Target 2 are directly routed by the switch, whereas UIO accesses to UIO Target 3 and UIO Target 4 are routed through the host. As shown, UIO Target 1 and UIO Target 2 participate in a 2-way interleave set. The UIO requester can efficiently access this interleave set without going through the host.

Figure 9-25. UIO Direct P2P to Interleaved HDM



The HDM that is a target of P2P UIO accesses must be part of either a 1-way, 2-way, 4-way, 8-way, or 16-way interleave set. Any HDM that is part of a 3-way, 6-way, or 12-way interleave arrangement cannot be a P2P UIO target. The HDM address must be carved out of a CFMWS entry with Interleave Arithmetic=Standard Modulo arithmetic (see [Table 9-22](#)). In addition, P2P UIO traffic is not protected by Selective IDE Streams.

In addition, Software must configure the switch and Host Bridge HDM decoders with additional information regarding any HDM interleaving calculations that are performed upstream to it before setting the UIO bit in that HDM decoder. The UIG, UIW, and ISP fields allow the switch and the Host Bridge to determine whether the UIO target address belongs to itself or to a peer component. The rules regarding the processing of UIO Direct P2P to HDM requests are described in [Table 9-18](#). The ISP field in the target CXL.mem device allow the device to determine how it should respond.

These requirements are in addition to the UIO related requirements that are defined in PCIe Base Specification.

9.16.1 Processing of UIO Direct P2P to HDM Messages

This section describes how CXL components handle UIO Direct P2P accesses to HDM.

UIO To HDM Enable bit is defined in [Section 8.1.5.2](#) and allows System Software to control whether a requester below a switch can use UIO to access HDM.

Table 9-18. Handling of UIO Accesses

Received by	UIO Address	Behavior
CXL.mem device that reports UIO Capable=1 (see Section 8.2.4.20.1)	Complete match with an HDM decoder with UIO=1	Respond to the UIO request per PCIe Base Specification
	Complete match with an HDM decoder with UIO=0	Return Completer Abort, do not commit data if it is a UIO write
	Partial match with an HDM decoder, irrespective of the UIO bit	Return Completer Abort, do not commit data if it is a UIO write
	Mismatch	Handle per PCIe Base Specification
USP ingress of a CXL Switch that reports UIO Capable=1 (see Section 8.2.4.20.1)	Either Partial or Complete match with an HDM decoder, irrespective of the UIO bit	Identify the port number of the target DSP and forward
	Mismatch	Handle per PCIe Base Specification
DSP ingress of a CXL Switch that reports UIO Capable=1 (see Section 8.2.4.20.1)	Complete match with an HDM decoder with UIO=1 ¹ and UIO To HDM Enable=1	Identify the port number of the target DSP and forward to that peer port regardless of ACS configuration including egress control vector
	Complete match with an HDM decoder with UIO=0 and UIO To HDM Enable=1	Forward toward the host regardless of ACS configuration including egress control vector
	Partial match with an HDM decoder and UIO To HDM Enable=1	Forward toward the host regardless of ACS configuration including egress control vector
	Complete or Partial match, and UIO To HDM Enable=0	Return Completer Abort
	Mismatch	Handle per PCIe Base Specification
RP ingress of a Host Bridge that reports UIO Capable=1 (see Section 8.2.4.20.1)	Complete match with an HDM decoder with UIO=1	Identify the port number of the target RP and forward to that peer port, subject to host-specific access controls
	Complete match with an HDM decoder with UIO=0	Handle via host-specific mechanisms
	Partial match with an HDM decoder	Handle via host-specific mechanisms
	Mismatch	Handle via host-specific mechanisms

1. Because the DSP does not take length into account during this check, transactions that cross an interleave boundary get forwarded to the device that owns the starting address. They are aborted by the device because the device checks the length field. If the UIO traffic is encrypted using Stream IDE, some of the address bits may be encrypted and the switch may unknowingly forward these to the wrong device, which will issue a Completer Abort.

9.16.1.1 UIO Address Match (DSP and Root Port)

For a DSP or a root port, UIO address is considered a complete match if there exists an HDM Decoder[n] (see [Section 8.2.4.20](#) and [Section 8.2.4.30](#)) for which the following conditions are true:

1. AT field in the UIO request indicates that it is carrying a translated address.
2. $\text{UIO.Address}[63:2] \geq \text{Decoder}[n].\text{Base}[63:2]$.

3. $\text{UIO.Address}[63:2] + \text{UIO.Length}[63:2] \leq \text{Decoder}[n].\text{Base}[63:2] + \text{Decoder}[n].\text{Size}[63:2]$.
4. Either of these sub-conditions are true:
 - a. $\text{Decoder}[n].\text{UIW} = 0$
 - b. $\text{UIO.Address}[\text{Decoder}[n].\text{UIW} + \text{Decoder}[n].\text{UIG} + 7 : \text{Decoder}[n].\text{UIG} + 8] = \text{ISP}$

where $\text{UIO.Address}[63:2]$ is derived from the Address field in the UIO TLP request, and $\text{UIO.Length}[63:2]$ is derived from the Length field in the UIO TLP request.

DSP calculations use the HDM decoders in the corresponding USP. The root port calculations make use of the HDM decoders in the associated Host Bridge.

The first condition is in place because HDM decoder operates on translated address. The second and the third condition ensures that all addresses fall within one of the HDM decoders. The fourth condition ensures that the interleave set positions match (i.e., a CXL.mem request from the host to the start address would ordinarily be decoded by this component). 4a is the trivial case where the memory is not interleaved.

If the first three conditions are met but the fourth condition is not met, it is considered a partial match. If the first three conditions are not met, it is considered a mismatch.

9.16.1.2 UIO Address Match (CXL.mem Device)

For a CXL.mem device, UIO address is considered a complete match if there exists an HDM Decoder[n] (see [Section 8.2.4.20](#) and [Section 8.2.4.30](#)) for which the following conditions are true:

1. AT field in the UIO request indicates it is carrying a translated address.
2. $\text{UIO.Address}[63:2] \geq \text{Decoder}[n].\text{Base}[63:2]$.
3. $\text{UIO.Address}[63:2] + \text{UIO.Length}[63:2] \leq \text{Decoder}[n].\text{Base}[63:2] + \text{Decoder}[n].\text{Size}[63:2]$.
4. Either of these sub-conditions are true:
 - a. $\text{Decoder}[n].\text{UIW} = 0$
 - b. $\text{UIO.Address}[\text{Decoder}[n].\text{IW} + \text{Decoder}[n].\text{IG} + 7 : \text{Decoder}[n].\text{IG} + 8] = \text{ISP}$
5. $\text{UIO.Address}[\text{Decoder}[n].\text{IG} + 7 : 2] + \text{UIO.Length}[\text{Decoder}[n].\text{IG} + 7 : 2] \leq (2^{**} \text{IG} + 8)$.

The first three conditions are identical to the DSP case. The terms involved in the fourth check are different, but it serves the same purpose (i.e., ensures that a CXL.mem request from the host to the start address would ordinarily be decoded by this component). The fifth condition ensures that the access does not cross an interleave boundary, thus ensuring that all the addresses that are referenced by the request are owned by the device.

If the first three conditions are met but either of the other two conditions are not met, it is considered a partial match. If the first three conditions are not met, it is considered a mismatch.

9.17 Direct P2P CXL.mem for Accelerators

The Direct P2P CXL.mem feature enables accelerators to use .mem semantics to access peer Type 3 devices. This feature is supported only by PBR Fabrics, and each accelerator and peer Type 3 device must be attached directly to an Edge Port. Configuration of the Fabric and Edge Ports is performed by the host and FM.

Through mechanisms beyond the scope of this specification, the FM is preconfigured or informed of which Type 3 device(s) (i.e., SLD, MLD, or GFD) are to be configured for Direct P2P CXL.mem access by a given accelerator.

9.17.1 Peer SLD Configuration

Host software and the FM may use the following high-level flow to configure Direct P2P CXL.mem communication between an accelerator and a peer Type 3 SLD:

1. The FM binds the SLD's Edge Port to the host VH of the accelerator, setting the vPPB.root.PID field to the PBR ID (PID) of the accelerator's Edge Port. This enables the host to configure the SLD, but the accelerator to carry out CXL.mem transactions with the SLD.
2. Using the **Set LDST Segment Entries** command (see [Section 7.7.13.16](#)), the host configures the LDST in the accelerator's Edge Port with one or more LDST Segments for the HPA range of the SLD, specifying the vPPB of the SLD's Edge Port.
3. Host software configures the SLD, notably its HDM Decoders, on behalf of the accelerator. HDM addresses in the SLD are HPAs.

9.17.2 Peer MLD Configuration

Host software and the FM may use the following high-level flow to configure Direct P2P CXL.mem communication between one or more accelerators that belong to the host and a peer Type 3 MLD:

1. The FM binds a vPPB in the MLD's Edge Port to the host VH of its accelerator(s) and an additional vPPB for each accelerator under that host that will be accessing the MLD. Each of these will have a distinct LD-ID. For each vPPB assigned to an accelerator, the vPPB.root.PID field is set to the PID of the accelerator's Edge Port.
2. Using the **Set LDST Segment Entries** command (see [Section 7.7.13.16](#)), the host configures the LDST in each accelerator's Edge Port with one or more LDST Segments for the HPA range of the accelerator's LD, specifying the accelerator's vPPB in the MLD's Edge Port.
3. Host software configures its LDs in the MLD, notably their HDM Decoders, on behalf of itself and its accelerator(s). HDM addresses in the LD of the host and the LD(s) of the accelerator(s) are HPAs.

9.17.3 Peer GFD Configuration

Host software and the FM may use the following high-level flow to configure Direct P2P CXL.mem communication between one or more accelerators that belong to a host and a peer Type 3 GFD:

1. The FM configures the GFD for host access normally, while configuring each of the host's accelerators as an additional RPID within the GFD.
2. Using the **Set FAST Segment Entries** command (see [Section 7.7.14.7](#)), the host configures the FAST decoder in its Edge Port as well as each accelerator's Edge Port with one or more FAST Segments for the HPA range, specifying the GFD's PID.

9.18 CXL OS Firmware Interface Extensions

9.18.1 CXL Early Discovery Table (CEDT)

CXL Early Discovery Table enables OSs to locate CXL Host Bridges and the location of Host Bridge registers early during the boot (i.e., prior to parsing of ACPI namespace). The information in this table may be used by early boot code to perform pre-initialization of CXL hosts, such as configuration of CXL.cache and CXL.mem.

9.18.1.1 CEDT Header

The pointer to CEDT is found in RSDT or XSDT, as described in ACPI Specification. An ACPI specification-compliant CXL system shall support CEDT and shall include a CHBS entry for every CXL host bridge that is present at boot.

CEDT begins with the following header.

Table 9-19. CEDT Header

Field	Length in Bytes	Byte Offset	Description
Header:			
Signature	4	00h	'CEDT'. Signature for the CXL Early Discovery Table.
Length	4	04h	Length, in bytes, of the entire CEDT.
Revision	1	08h	Value is 2.
Checksum	1	09h	Entire table must sum to 0.
OEM ID	6	0Ah	OEM ID
OEM Table ID	8	10h	Manufacturer Model ID
OEM Revision	4	18h	OEM Revision
Creator ID	4	1Ch	Vendor ID of the utility that created the table.
Creator Revision	4	20h	Revision of the utility that created the table.
CEDT Structure[n]	Varies	24h	A list of CEDT structures for this implementation.

Table 9-20. CEDT Structure Types

Value	Description
0	CXL Host Bridge Structure (CHBS)
1	CXL Fixed Memory Window Structure (CFMWS)
2	CXL XOR Interleave Math Structure (CXIMS)
3	RCEC Downstream Port Association Structure (RDPAS)
4	CXL System Description Structure (CSDS) ¹
5-255	Reserved

1. Introduced in Revision 2 of CEDT.

9.18.1.2 CXL Host Bridge Structure (CHBS)

The CHBS structure describes a CXL Host Bridge.

In an ACPI-compliant system, there shall be one instance of the CXL Host Bridge Device object in ACPI namespace (HID="ACPI0016") for every CHBS entry. The _UID object under a CXL Host Bridge object, when evaluated, shall match the UID field in the associated CHBS entry.

Table 9-21. CHBS Structure

Field	Length in Bytes	Byte Offset	Description
Type	1	00h	=0 to indicate that this is a CHBS entry
Reserved	1	01h	Reserved
Record Length	2	02h	Length of this record (20h).
UID	4	04h	CXL Host Bridge Unique ID. Used to associate a CHBS instance with a CXL Host Bridge instance. The value of this field shall match the output of _UID under the associated CXL Host Bridge in ACPI namespace.
CXL Version	4	08h	0000 0000h: RCH 0000 0001h: Host Bridge that is associated with one or more CXL root ports
Reserved	4	0Ch	Reserved
Base	8	10h	- If CXL Version = 0000 0000h, this represents the base address of the RCH Downstream Port RCRB - If CXL Version = 0000 0001h, this represents the base address of the CHBCR See Table 8-17 for more details.
Length	8	18h	- If CXL Version = 0000 0000h, this field must be set to 8 KB (2000h) - If CXL Version = 0000 0001h, this field must be set to 64 KB (1 0000h)

9.18.1.3 CXL Fixed Memory Window Structure (CFMWS)

The CFMWS structure describes zero or more Host Physical Address (HPA) windows that are associated with each CXL Host Bridge. Each window represents a contiguous HPA range that may be interleaved across one or more targets, some of which are CXL Host Bridges. Associated with each window are a set of restrictions that govern its usage. It is the OSPM's responsibility to utilize each window for the specified use.

The HPA ranges described by CFMWS may include addresses that are currently assigned to CXL.mem devices. Before assigning HPAs from a fixed-memory window, the OSPM must check the current assignments and avoid any conflicts.

For any given HPA, it shall not be described by more than one CFMWS entry.

Table 9-22. CFMWS Structure (Sheet 1 of 4)

Field	Length in Bytes	Byte Offset	Description
Type	1	00h	1 = indicates this is a CFMWS entry
Reserved	1	01h	Reserved
Record Length	2	02h	Length of this record = 024h + 4 * NIW. NIW is the raw count of Interleave ways whereas ENIW is the encoded value: - If ENIW < 8, NIW = 2 ** ENIW - If ENIW ≥ 8, NIW = 3 * 2 ** (ENIW - 8)
Reserved	4	04h	Reserved
Base HPA	8	08h	Base of this HPA range. This value shall be a 256-MB-aligned address.
Window Size	8	10h	The total number of consecutive bytes of HPA this window represents. This value shall be a multiple of NIW*256 MB.

Table 9-22. CFMWS Structure (Sheet 2 of 4)

Field	Length in Bytes	Byte Offset	Description
Encoded Number of Interleave Ways (ENIW)	1	18h	The encoded number of targets with which this window is interleaved. The valid encoded values are specified in the Interleave Ways field of the CXL HDM Decoder n Control register (see Section 8.2.4.20.7). This field determines the number of entries in the Interleave Target List, starting at Offset 24h.
Interleave Arithmetic	1	19h	This field defines the arithmetic used for mapping HPA to an interleave target in the Interleave Target List: 00h = Standard Modulo arithmetic 01h = Modulo arithmetic combined with XOR - All other encodings are reserved
Reserved	2	1Ah	Reserved
Host Bridge Interleave Granularity (HBIG)	4	1Ch	The number of consecutive bytes within the interleave that are decoded by each target in the Interleave Target List represented in an encoded format. The valid values are specified in the Interleave Granularity field of the CXL HDM Decoder n Control register (see Section 8.2.4.20.7).
Window Restrictions	2	20h	A bitmap describing the restrictions being placed on the OSPM's use of the window. It is the OSPM's responsibility to adhere to these restrictions. Failure to adhere to these restrictions results in undefined behavior. More than one bit within this field may be set: - Bit[0]: Device Coherent: Formerly known as CXL Type 2 Memory: 1 = Window is configured to expose device-coherent memory (HDM-D if Bit[5]=0 ; HDM-DB if Bit[5]=1). - Bit[1]: Host-only Coherent: Formerly known as CXL Type 3 Memory: 1 = Window is configured to expose host-only coherent memory (HDM-H). If an HDM decoder that is mapped to this windows has the BI bit set, it will result in undefined behavior. - Bit[2]: Volatile: 1 = Window is configured for use with volatile memory. - Bit[3]: Persistent: 1 = Window is configured for use with persistent memory. - Bit[4]: Fixed Device Configuration: 1 = Any device ranges that have been assigned an HPA from this window must not be reassigned. - Bit[5]: BI: 1 = Window is configured for use with Back-Invalidate flows. - Bits[15:6]: Reserved
QTG ID	2	22h	The ID of the QoS Throttling Group associated with this window. The _DSM for retrieving QTG ID is utilized by the OSPM to determine to which QTG a device HDM range should be assigned. This field must not exceed the Max Supported QTG ID returned by the _DSM for retrieving QTG.

Table 9-22. CFMWS Structure (Sheet 3 of 4)

Field	Length in Bytes	Byte Offset	Description
Interleave Target List	4*NIW	24h	A list of all the Interleave Targets. The number of entries in this list shall match the Number of Interleave Ways (NIW). The order of the targets reported in this List indicates the order in the Interleave Set. For Interleave Sets that only span CXL Host Bridges, this is a list of CXL Host Bridge _UIDs that are part of the Interleave Set. In this case, for each _UID value in this list, there must exist a corresponding CHBS structure. If the Interleave Set spans non-CXL domains, this list may contain values that do not match _UID field in any CHBS structures. These entries represent Interleave Targets that are not CXL Host Bridges. The set of HPAs decoded by Entry N in the Interleave Target List shall satisfy the following equations: - Base HPA $\leq$ HPA < Base HPA + Windows Size: where the Base HPA and Window size shall be multiple of NIW. If (Interleave Arithmetic==0): - If ENIW=0 N=0 - If ENIW=1 N = HPA[8+HBIG] - If ENIW<8 AND ENIW>1 N = HPA[7+HBIG+ENIW:8+HBIG] - If NIW = 8 // 3 way N = HPA[51:8+HBIG] MOD 3 - If NIW=9 // 6 way N = HPA[8+HBIG] + 2* HPA[51:9+HBIG] MOD 3 - If NIW=10 //12 way N = HPA[9+HBIG:8+HBIG] + 4* HPA[51:10+HBIG] MOD 3 - If (Interleave Arithmetic==1): - If NIW=0 //1 way N=0 - If NIW =1 // 2 way N = XORALLBITS (HPA AND XORMAP[0]) If NIW=2 // 4 way N = XORALLBITS (HPA AND XORMAP[0]) + 2* XORALLBITS (HPA AND XORMAP[1])

Table 9-22. CFMWS Structure (Sheet 4 of 4)

Field	Length in Bytes	Byte Offset	Description
Interleave Target List	4*NIW	24h	c. If NIW=3 // 8 way $N = \text{XORALLBITS}(\text{HPA AND XORMAP}[0]) +$ $* \text{XORALLBITS}(\text{HPA AND XORMAP}[1]) +$ $* \text{XORALLBITS}(\text{HPA AND XORMAP}[2])$ d. If NIW=4 // 16 way $N = \text{XORALLBITS}(\text{HPA AND XORMAP}[0]) +$ $2 * \text{XORALLBITS}(\text{HPA AND XORMAP}[1]) +$ $4 * \text{XORALLBITS}(\text{HPA AND XORMAP}[2]) +$ $8 * \text{XORALLBITS}(\text{HPA AND XORMAP}[3])$ e. If NIW =8 // 3 way, same as Interleave Arithmetic=0 $N = \text{HPA}[51:8+\text{HBIG}] \text{ MOD } 3$ f. If NIW =9 // 6 way $N = \text{XORALLBITS}(\text{HPA AND XORMAP}[0])$ $+ 2 * \text{HPA}[51:9+\text{HBIG}] \text{ MOD } 3$ g. If NIW=10 // 12 way $N = \text{XORALLBITS}(\text{HPA AND XORMAP}[0])$ $+ 2 * \text{XORALLBITS}(\text{HPA AND XORMAP}[1])$ $+ 4 * \text{HPA}[51:10+\text{HBIG}] \text{ MOD } 3$ N is 0 based ($0 \leq N < \text{NIW}$). Where XORALLBITS is an operation that outputs a single bit by XORing all the bits in the input. AND is a standard bitwise AND operation and XORMAP[m] is the mth element (m is 0 based) in the XORMAP array that is part of the CXIMS entry with the matching HBIG value.

9.18.1.4 CXL XOR Interleave Math Structure (CXIMS)

If a CFMWS entry reports Interleave Arithmetic=1, there must be one CXIMS entry associated with the HBIG value in the CFMWS. CXIMS carries an array of bitmaps. Each bitmap represents the bits that are XORed together to calculate the individual bits of the Interleave Way as described in the definition of the Interleave Target List field in CFMWS. The host implementation is responsible for selecting an XOR function that generates even distribution of addresses and does not lead to address aliasing.

Table 9-23. CXIMS Structure

Field	Length in Bytes	Byte Offset	Description
Type	1	00h	2 = Indicates that this is a CXIMS entry
Reserved	1	01h	Reserved
Record Length	2	02h	Length of this record = 8 + 8 * NIB.
Reserved	2	04h	Reserved
HBIG	1	06h	Host Bridge Interleave Granularity to which this CXIMS instance corresponds. See Table 9-22 for the definition of the term HBIG.
Number of Bitmap Entries (NIB)	1	07h	The number of entries in the XORMAP list.
XORMAP List	8 * NIB	08h	A list of Bitmaps. XORMAP[0] is the first entry.

9.18.1.5 RCEC Downstream Port Association Structure (RDPAS)

RDPAS structure enables error handler to locate the Downstream Port(s) that report errors to a given RCEC. For every RCEC, zero or more entries of this type are permitted.

Table 9-24. RDPAS Structure

Field	Length in Bytes	Byte Offset	Description
Type	1	00h	3 = Indicates that this is an RDPAS entry
Reserved	1	01h	Reserved
Record Length	2	02h	Length of this record = 10h
RCEC Segment Number	2	04h	The PCIe segment number associated with this RCEC
RCEC BDF	2	06h	- Bits[2:0]: RCEC Function Number - Bits[7:3]: RCEC Device Number - Bits[15:8]: RCEC Bus Number
Protocol Type	1	07h	00h = The error source is CXL.io 01h = The error source is CXL.cachemem
Base Address	8	08h	If Protocol Type = CXL.io, this field shall be the RCRB base associated with the Downstream Port. If Protocol Type = CXL.cachemem, this will be the Component Base Register Base associated with the Downstream Port.

IMPLEMENTATION NOTE

CXL-aware software may take the following steps upon observing an Uncorrected Internal Error or an Corrected Internal Error being logged in an RCEC at Segment Number S and BDF=B.

If the CEDT contains RDPAS structures:

- For all RDPAS structures where RCEC Segment Number=S and RCEC BDF= B:
 - If Protocol Type=CXL.io, read the Base Address field and use that information to access the RCRB AER registers and determine whether any errors are logged there
 - If Protocol Type=CXL.cachemem, read the Base Address field and use that information to access the Component Register RAS Capability registers (see [Section 8.2.4.17](#)) and determine whether any errors are logged there

Else:

- Probe all CXL Downstream Ports and determine whether they have logged an error in the CXL.io or CXL.cachemem status registers

9.18.1.6 CXL System Description Structure (CSDS)

The CSDS describes CXL System Wide Description and Configuration.

In a system, there shall be only one instance of the CSDS in the CEDT.

Table 9-25. CSDS Structure

Field	Length in Bytes	Byte Offset	Description
Type	1	00h	4 = Indicates that this is a CSDS entry
Reserved	1	01h	Reserved
Record Length	2	02h	Length of this record = 08h
System Capabilities	2	04h	A bitmap that describes system-wide capabilities. More than one bit within this field is permitted to be set. - Bit[0]: Cmp-M: 1 = System is configured for use with devices that return modified data using the Cmp-M response. - Bit[1]: No Clean Writeback: Specifies the clean writeback behavior of the host. 0 = The host may or may not generate clean writebacks 1 = The host guarantees to never generate clean writeback transactions at the host's cacheline granularity - Bits[15:2]: Reserved
Reserved	2	06h	Reserved

9.18.2 CXL _OSC

According to ACPI Specification, _OSC (Operating System Capabilities) is a control method that is used by OSs to communicate to the System Firmware the capabilities supported by the OS and to negotiate ownership of specific capabilities.

The _OSC interface defined in this section applies only to “Host Bridge” ACPI devices that originate CXL hierarchies. As specified in [Section 9.12](#), these ACPI devices must have an _HID of (or a _CID that includes) EISAID(“ACPI0016”). CXL _OSC is required for a CXL VH. CXL _OSC is optional for an RCD. A CXL Host Bridge also originates a PCIe hierarchy and will have a _CID of EISAID(“PNP0A08”). As such, a CXL Host Bridge device may expose both CXL _OSC and PCIe _OSC.

The _OSC interface for a CXL Host Bridge is identified by the Universal Unique Identifier (UUID) 68f2d50b-c469-4d8a-bd3d-941a103fd3fc.

A revision ID of 1 encompasses fields defined within this section, composed of 5 DWORDs, as listed in [Table 9-26](#).

Table 9-26. _OSC Capabilities Buffer DWORDs (Sheet 1 of 2)

_OSC Capabilities Buffer DWORD #	Description
1	Contains bits that are generic to _OSC and defined by ACPI. These include status and error information.
2	PCIe Support Field as defined by PCI Firmware Specification.

Table 9-26. _OSC Capabilities Buffer DWORDs (Sheet 2 of 2)

_OSC Capabilities Buffer DWORD #	Description
3	PCIe Control Field as defined by PCI Firmware Specification.
4	CXL Support Field: Bits defined in the CXL Support Field provide information regarding CXL features supported by the OS. Just like the PCIe Support field, contents in the CXL Support Field are passed in a single direction; the OS will disregard any changes to this field when returned.
5	CXL Control Field: Just like the PCIe Control Field, bits defined in the CXL Control Field are used to submit OS requests for control/handling of the associated feature, typically including but not limited to features that utilize native interrupts or events that are handled by an OS-level driver. If any bits in the CXL Control Field are returned cleared (i.e., masked to 0) by the _OSC control method, the respective feature is designated as unsupported by the platform and must not be enabled by the OS. Some of these features may be controlled by System Firmware prior to OS boot or during runtime for an OS that is unaware of these features, while others may be disabled/inoperative until native OS support for such features is available. If the CXL _OSC control method is absent from the scope of a Host Bridge device, then the OS must not enable or attempt to use any features defined in this section for the hierarchy originated by the Host Bridge. Doing so could conflict with System Firmware operations, or produce undesired results. It is recommended that a machine with multiple Host Bridge devices should report the same capabilities for all Host Bridges, and also negotiate control of the features described in the CXL Control Field in the same way for all Host Bridges.

Table 9-27. Interpretation of CXL _OSC Support Field

Support Field Bit Offset	Interpretation
0	RCD and RCH Port Register Access Supported The OS sets this bit to 1 if it supports access to RCD and RCH Port registers as defined in Section 9.11 . Otherwise, the OS clears this bit to 0.
1	CXL VH Register Access Supported The OS sets this bit to 1 if it supports access to CXL VH component registers as defined in Section 9.12 . If this bit is 1, bit 0 must also be 1. Otherwise, the OS clears this bit to 0.
2	CXL Protocol Error Reporting Supported The OS sets this bit to 1 if it supports handling of CXL Protocol Errors. Otherwise, the OS clears this bit to 0. If the OS sets this bit, it must also set either bit 0 or bit 1 above. Note: Firmware may retain control of AER if the OS does not support CXL Protocol Error reporting because the owner of AER owns CXL Protocol error management.
3	CXL Native Hot-Plug Supported The OS sets this bit to 1 if it supports CXL hot-add and managed CXL Hot-Remove without firmware assistance. Otherwise, the OS clears this bit to 0. If the OS sets this bit, it must request PCIe Native Hot-Plug control. If PCIe Native Hot-Plug control is granted to the OS, such an OS must natively handle CXL Hot-Plug as well. If the OS sets this bit, it must also set bit 1 above.
4-31	Reserved

Table 9-28. Interpretation of CXL _OSC Control Field, Passed in via Arg3

Control Field Bit Offset	Interpretation
0	CXL Memory Error Reporting Control The OS sets this bit to 1 to request control over CXL Memory Error Reporting. If the OS successfully receives control of this feature, it must handle memory errors from all CXL.mem-capable devices that support this capability, as described in Section 12.2.3.2 . If the OS sets this bit, the OS must also set either bit 0 or bit 1 in the CXL _OSC Support Field (see Table 9-26).
1-31	Reserved

Table 9-29. Interpretation of CXL _OSC Control Field, Returned Value

Control Field Bit Offset	Interpretation
0	CXL Memory Error Reporting Control The firmware sets this bit to 1 to grant control over CXL Memory Expander Error Reporting. If firmware grants control of this feature, firmware must ensure that memory expanders are not configured in Firmware First error reporting mode. If control of this feature was requested and denied or was not requested, firmware returns this bit cleared to 0.
1-31	Reserved

9.18.2.1 Rules for Evaluating _OSC

This section defines when and how the OS must evaluate _OSC, as well as restrictions on firmware implementations.

9.18.2.1.1 Query Support Flag

If the Query Support Flag (_OSC Capabilities Buffer DWORD 1, bit 0) is set by the OS while evaluating _OSC, hardware settings are not permitted to be changed by firmware in the context of the _OSC call. It is strongly recommended that the OS evaluate _OSC with the Query Support Flag set until _OSC returns the Capabilities Masked bit cleared to negotiate the set of features to be granted to the OS for native support. A platform may require a specific combination of features to be natively supported by an OS before granting native control of a given feature.

9.18.2.1.2 Evaluation Conditions

The OS must evaluate _OSC under the following conditions:

- During initialization of any driver that provides native support for features described in the section above. These features may be supported by one or many drivers, but should be evaluated only by the main bus driver for that hierarchy. Secondary drivers must coordinate with the bus driver to install support for these features. Drivers shall not relinquish control of previously obtained features. That is, bits set in _OSC Capabilities Buffer DWORD 3 and DWORD 5 after the negotiation process must be set on all subsequent negotiation attempts.
- When a Notify(<device>, 8) is delivered to the CXL Host Bridge device.
- Upon resume from S4, System Firmware will handle context restoration when resuming from S1 through S3.

If a CXL Host Bridge device exposes CXL _OSC, CXL-aware OSPM shall evaluate CXL _OSC and not evaluate PCIe _OSC.

9.18.2.1.3 Sequence of _OSC Calls

The following rules govern sequences of calls to _OSC that are issued to the same Host Bridge and occur within the same boot:

- The OS is permitted to evaluate _OSC an arbitrary number of times.
- If the OS declares support of a feature in the Status Field in one call to _OSC, then it must preserve the set state of that bit (and thereby declare support for that feature) in all subsequent calls.
- If the OS is granted control of a feature in the Control Field in one call to _OSC, then it must preserve the set state of that bit (requesting that feature) in all subsequent calls.

- Firmware shall not reject control of any feature it has previously granted control to.
- There is no mechanism for the OS to relinquish control of a previously requested and granted feature.

9.18.2.1.4 ASL Example

```
Device(CXL0)
{
    Name(_HID, EISAID("ACPI0016")) // CXL Host Bridge
    Name(_CID, Package(2) {

        EISAID("PNP0A03"), // PCI Compatible Host Bridge

        EISAID("PNP0A08") // PCI Express Compatible Host Bridge

    })

    Name(SUPP, 0) // PCI _OSC Support Field value
    Name(CTRL, 0) // PCI _OSC Control Field value
    Name(SUPC, 0) // CXL _OSC Support Field value
    Name(CTRC, 0) // CXL _OSC Control Field value

    Method( _OSC, 4)
    {
        // Check for proper UUID
        If(LEqual(Arg0, ToUUID("68f2d50b-c469-4d8a-bd3d-941a103fd3fc ")))
        {
            // Create DWord-addressable fields from the Capabilities Buffer
            CreatedWordField(Arg3, 0, CDW1)
            CreatedWordField(Arg3, 4, CDW2)
            CreatedWordField(Arg3, 8, CDW3)
            CreatedWordField(Arg3, 12, CDW4)
            CreatedWordField(Arg3, 16, CDW5)
            // Save Capabilities DWord2, 3. 4. 5
            Store(CDW2, SUPP)
            Store(CDW3, CTRL)

            Store(CDW4, SUPC)
            Store(CDW4, CTRC)
            ..
        }
        Else {
            Or(CDW1, 4, CDW1) // Unrecognized UUID
            Return(Arg3)
        }
    } // End _OSC
// Other methods such as _BBN, _CRS, PCIe _OSC
} //End CXL0
```

9.18.3 CXL Root Device Specific Methods (_DSM)

_DSM is a control method that enables devices to provide device-specific functions for the benefit of the device driver. See ACPI Specification for details. [Table 9-30](#) lists the _DSM Functions that are associated with the CXL Root Device (HID="ACPI0017").

Table 9-30. _DSM Definitions for CXL Root Device

UUID	Revision	Function	Description
F365F9A6-A7DE-4071-A66A-B40C0B4F8E52	1	1	Retrieve QTG ID (see Section 9.18.3.1)
	-	All other	Reserved

All other Function values are reserved. The Revision field represents the version of the individual _DSM Function. The Revision associated with a _DSM Function is incremented whenever that _DSM Function is extended to add more functionality. Backward compatibility shall be maintained during this process. Specifically, for all values of n, a _DSM Function with Revision n+1 may extend Revision ID n by assigning meaning to the fields that are marked as reserved in Revision n but must not redefine the meaning of existing fields and must not change the size or type of I/O parameters.

Software that was written for a lower Revision may continue to operate on _DSM Functions with a higher Revision but will not be able to take advantage of new functionality. It is legal for software to invoke a _DSM Function and pass in any nonzero Revision ID value that does not exceed the Revision ID defined in this specification for that _DSM Function.

For example, if the most-current version of this specification defines Revision ID=4 for _DSM Function Index f, software is permitted to invoke the _DSM Function with Function Index f with a Revision ID value that belongs to the set {1, 2, 3, 4}.

9.18.3.1 _DSM Function for Retrieving QTG ID

This section describes how the OSPM can request the firmware to determine the optimum QoS Throttling Group (QTG) to which a device HDM range should be assigned, based on its performance characteristics. It is strongly recommended that OSPM evaluate this _DSM Function to retrieve QTG recommendations and map the device HDM range to an HPA range that is described by a CFMWS entry that follows the platform recommendations.

For each Device Scoped Memory Affinity Structure (DSMAS) in the Device CDAT, the OSPM should calculate the Read Latency, Write Latency, Read Bandwidth, and Write Bandwidth from the CXL Root Port within the same VCS. The term DSMAS is defined in Coherent Device Attribute Table Specification. This calculation must consider the latency and bandwidth contribution of any intermediate switches. The OSPM should call this _DSM with the performance characteristics for the Device HDM range thus calculated, utilize the return ID value(s) to pick an appropriate CFMWS, and then map the DSMAS DPA range to HPAs that are covered by that CFMWS. This process may be repeated for each DSMAS memory range that the OSPM wishes to utilize from the device.

Location:

This object shall be a child of the CXL Root Device (HID="ACPI0017").

Arguments:

Arg0: UUID: f365f9a6-a7de-4071-a66a-b40c0b4f8e52

Arg1: Revision ID: 1

Arg2: Function Index: 01h

Arg3: A package of memory device performance characteristic. The package consists of 4 DWORDs.

Package {

Read Latency

Write Latency

Read Bandwidth

Write Bandwidth

}

Return:

A package containing two elements - a WORD that returns the maximum throttling group that the platform supports, and a package containing the QTG ID(s) that the platform recommends.

```
Package {
    Max Supported QTG ID
    Package {QTG Recommendations}
}
```

Table 9-31. _DSM for Retrieving QTG, Inputs, and Outputs

Field	Size	Description
Input Package:		
Read Latency	DWORD	The best-case read latency as measured from the CXL root port within the same VCS, expressed in picoseconds.
Write Latency	DWORD	The best-case write latency as measured from the CXL root port within the same VCS, expressed in picoseconds.
Read Bandwidth	DWORD	The best-case read bandwidth as measured from the CXL root port within the same VCS, expressed in MB/s.
Write Bandwidth	DWORD	The best-case write bandwidth as measured from the CXL root port within the same VCS, expressed in MB/s.
Return Package:		
Max Supported QTG ID	WORD	The highest QTG ID supported by the platform. The platform must be capable of supporting all QTGs whose ID, Q, satisfies the following equation: $0 > Q \geq \text{Max Supported QTG ID}$ For every value of Q, there may be zero or more CFMWS entries.
QTG Recommendations	Package	A package that consists of 0 or more WORD elements. It is a prioritized list of QTG IDs that are considered acceptable by the platform for the specified performance characteristics. If the package contains more than one element, element[n] is preferred by the platform over element[n+1]. If the package is empty, the platform is unable to find any suitable QTGs for this set of input values. If the OSPM does not follow platform QTG recommendations, it may result in severe performance degradation. Every element in this package must be no greater than the Max Supported QTG ID above. For example, if QTG Recommendations = Package () {2, 1}, the OSPM should first attempt to assign from QTG ID 2, and then attempt to assign QTG ID 1 if an assignment cannot be found in QTG ID 2.

9.19 Manageability Model for CXL Devices

Manageability is the set of capabilities that a managed entity exposes to a management entity. In the context of CXL, a CXL device is the managed entity. These capabilities are generally classified in sensors and effectors. An Event Log is an example of a sensor, whereas the ability to update the device firmware is an example of an effector. Sensors and effectors can either be accessed in-band (i.e., by OS/VMM resident software), or out-of-band (i.e., by firmware running on a management controller that is OS independent).

In-band software can access a CXL device's manageability capabilities by issuing PCIe configuration read/write or MMIO read/write transactions to its Mailbox registers. These accesses are generally mediated by the CXL device driver. This is consistent with how PCIe adapters are managed.

Out-of-band manageability in S0 state can leverage transports for which an MCTP binding specification has been defined. This assumes that the CXL.io path will decode and forward MCTP over PCIe VDMs in both directions. Form factors, such as PCIe CEM Specification, provision two SMBUS pins (clock and data). The SMBUS path can be used for out-of-band manageability in Sx state or in the Link Down case. This is consistent with PCIe adapters. CXL components may also support additional management capabilities defined in other specifications, such as Platform-Level Data Model (PLDM).

9.20 Component Command Interface

Runtime management of CXL components is facilitated by a Component Command Interface (CCI). A CCI represents a command target that is used to process management and configuration commands that are issued to the component. [Table 8-37](#), [Table 8-126](#), and [Table 8-215](#) define the commands that a CCI can support.

A component can implement multiple CCIs of varying types that operate independently of one another and that have a uniquely defined list of supported commands. There are 2 types of CCIs:

- Mailbox Registers: A component can expose up to 2 mailboxes through its Mailbox registers, as defined in [Section 8.2.8.4](#). Each mailbox represents a unique CCI instance.
- MCTP-based CCIs: Components with MCTP-capable interconnects can expose up to 1 CCI per interconnect. There is a 1:1 relationship between the component's MCTP-based CCIs and MCTP-capable interconnects. Transfer of commands via MCTP uses the transport protocol defined in [Section 7.6.3](#).

All CCIs shall comply with the properties described in [Section 9.20.1](#).

9.20.1 CCI Properties

Components that implement more than one CCI shall process commands from those CCIs in a manner that avoids starvation so that commands submitted to one CCI do not prevent commands from other CCIs from being handled. The exact algorithm for accepting commands from multiple CCIs is implementation specific. Each CCI within a component reports its supported command list through the Command Effects Log (CEL), as described in [Section 8.2.9.5.2.1](#).

Interface-specific properties of commands, background operation, and timeouts are defined in [Section 8.2.8.4](#) for mailbox CCIs and in [Section 9.20.2](#) for MCTP-based CCIs. Each CCI can support the execution of only one background command at a time.

When a command is successfully started as a background operation, the component shall return the Background Command Started return code defined in [Section 8.2.8.4.5.1](#). While the command is executing in the background, the component should update the percentage complete at least once per second.

A component may return the Busy return code if a command is sent to initiate a Background Operation while a Background Operation is already running on any other CCI.

An ongoing background command may be aborted by issuing a Request Abort Background Operation command on the same CCI (see [Section 8.2.9.1.5](#)).

Each CCI within a component shall maintain a unique context with respect to the following capabilities:

- Events, including reading contents, clearing entries, and configuring interrupt settings, and
- CEL content

IMPLEMENTATION NOTE

It is recommended that components with multiple CCIs that support commands that run as Background Operations only advertise support for those commands on one CCI.

Coordination between management entities attempting concurrent commands over separate CCIs that have component-level impact (e.g., FW update, etc.) is beyond the scope of this specification.

9.20.2 MCTP-based CCI Properties

The CCI command timeout is 2 seconds, measured from when the command has been received by the component to when the component has started to transmit its response. Components should respond within this time limit; otherwise, requesters may timeout. Requesters must account for round-trip transmission time in addition to the command timeout.

IMPLEMENTATION NOTE

MCTP-based CCIs are intended to provide a dedicated management interface that operates independently from the state of any of the component's CXL interfaces; it is strongly recommended, but not required, that commands initiated on MCTP-based CCIs are not interrupted by Conventional Resets or any other changes of state of a component's CXL interface(s).

MCTP-based CCIs report background operation status using the Background Operation Status command as defined in [Section 8.2.9.1.2](#).

In the event of a command timeout, the requester may retransmit the request. New Message Tags shall be used every time that a request is retransmitted. Requesters may discard responses that arrive after the command timeout period has lapsed.

Commands sent to MCTP-based CCIs on MLD components are processed by the FM-owned LD.

§ §

10.0 Power Management

10.1 Statement of Requirements

All CXL implementations are required to support Physical Layer Power management as defined in this chapter. CXL Power management is divided into protocol-specific Link Power management and CXL Physical Layer power management. The ARB/MUX Layer is also responsible for managing protocol-specific Link Power Management between the Protocols on both sides of the links. The ARB/MUX coordinates the Power Management states between Multiple Protocols on both sides of the links, consolidates the Power states, and drives the Physical Layer Power Management.

10.2 Policy-based Runtime Control - Idle Power - Protocol Flow

10.2.1 General

For CXL-connected devices, there is a need to optimize power management of the entire system, with the device included.

As such, a hierarchical power-management architecture scheme is defined, where the discrete device is viewed as a single autonomous entity, with thermal and power management executed locally, but in coordination with the processor. Vendor-defined Messages (VDMs) over CXL are used to coordinate state transitions with the processor. The coordination between the primary power management controller on the host and the device is best accomplished via PM2IP and IP2PM messages that are encoded as VDMs.

Since native support of PCIe* is also required, support of more-simplified protocols is also possible. [Table 10-1](#) highlights the required and recommended handling method for Idle transitions.

Table 10-1. Runtime-Control - CXL vs. PCIe Control Methodologies

Case	PCIe	CXL ¹
Pkg-C Entry/Exit	Devices that do not share coherency with CPU can work with the PCIe profile: - LTR-notifications from Device - Allow-L1 signaling from CPU on Pkg_C entry	Optimized handshake protocol, for all non-PCIe CXL profiles: - LTR-notifications from Device - PMreq/Rsp (VDM) signaling between CPU and device on Pkg_C entry and exit²

1. All CXL components support PM VDMs and use PM Controller - PM Controller sequences where possible.

2. PM2IP: VDMs are associated with different Reset/PM flows.

10.2.2 Package-level Idle (C-state) Entry and Exit Coordination

At a high level, a discrete CXL device that is coherent with the processor is treated like another processor package. The expectation is that there is coordination and agreement between the processor and the discrete device before the platform can enter idle power state. Neither the device nor the processor can individually enter a low-power state as long as its memory resources are needed by the other components.

For example, in a case where the device may contain shared High-Bandwidth memory (HBM), while the processor controls the system's DDR, if the device wants to be able to enter a low-power state, the Device must take into account the processor's need for accessing the HBM. Likewise, if the processor wants to enter a low-power state, the processor must take into account, among other things, the need for the device to access DDR. These requirements are encapsulated in the LTR requirements that are provided by entities that need QoS for memory access. In this case, we would have a notion of LTR for DDR access and LTR for HBM access. We would expect the device to inform the processor about its LTR with regard to DDR, and the processor to inform the device about its LTR with regard to HBM.

Latency requirements can be managed by using either of the following two methods:

- CXL devices that do not share coherency with the CPU (i.e., either a shared coherent memory or a coherent cache), can notify the processor of changes in its latency tolerance via the PMReq() and PMRsp() messages. When appropriate latency is supported and the processor execution has stopped, the processor will enter an Idle state and proceed to transition the Link to L1 (see Link-Layer [Section 10.3](#)).
- CXL devices that include a coherent cache or memory device are required to coordinate their state transitions using the CXL-optimized, VDM-based protocol, which includes the ResetPrep(), PMReq(), PMRsp(), and PMGo() messages, to prevent memory coherency loss.

10.2.2.1 PMReq Message Generation and Processing Rules

The rules associated with generation and processing of PMReq Req, PMReq Rsp, and PMReq Go messages are as follows:

- A CXL device communicates its latency tolerance via a PMReq Req message. A host communicates its latency tolerance either via a PMReq Rsp message or a PMReq Go message.
- A CXL device is permitted to unilaterally generate a PMReq Req message as long as the Device has the necessary credits. A host shall not generate a PMReq Req message.
- A CXL device shall not generate a PMReq Rsp message. A host is permitted to unilaterally generate a PMReq Rsp message as long as the Host has the necessary credits, even if the Host has never received a PMReq Req message. A CXL device must process a PMReq Rsp message normally, even if that CXL device has never previously issued a PMReq Req message.
- A CXL device is not permitted to generate a PMReq Go message. A host is permitted to unilaterally generate a PMReq Go message as long as the Host has the necessary credits, even if the Host has never received a PMReq Req message. A CXL device must process a PMReq Go message normally, even if that CXL device has never:
 - Previously issued a PMReq Req message.
 - Received a PMReq Rsp message.
- A CXL device must continue to operate correctly, even if the device never receives a PMReq Rsp in response to the device generating a PMReq Req.
- A CXL device must continue to operate correctly, even if the device never receives a PMReq Go in response to the device generating PMReq Req.
- The Requirement bit associated with the non-snoop Latency Tolerance field in the PMReq messages must be cleared to 0 by all non-eRCD components.

[Section 10.2.3](#) and [Section 10.2.4](#) include example flows that illustrate these rules.

Table 10-2. PMReq(), PMRsp(), and PMGo() Encoding

Message	PM Logical Opcode[7:0]	Parameter[15:0]
PMReq.Req, abbreviated as PMReq	04h	0001h
PMReq.Rsp, abbreviated as PMRsp	04h	0000h
PMReq.Go, abbreviated as PMGo	04h	0004h or 0005h

10.2.3 PkgC Entry Flows

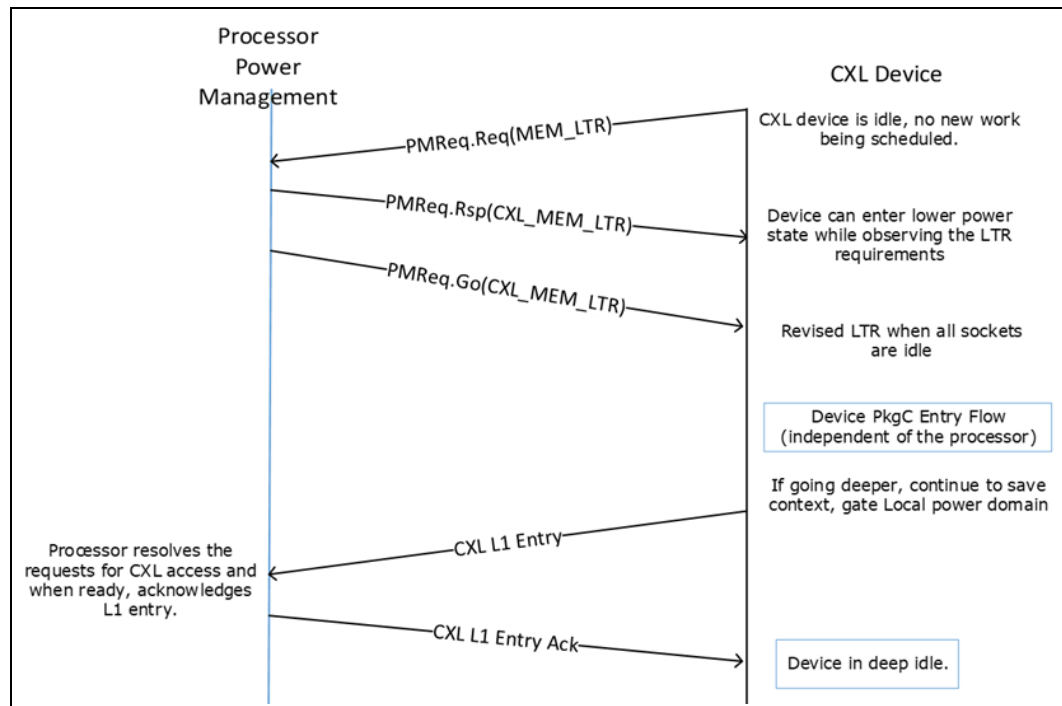
Figure 10-1. PkgC Entry Flow Initiated by Device - Example

Figure 10-1 illustrates the PkgC entry flow. When a Device needs to enter a higher-latency Idle state, in which the CPU is not active, the Device will issue a PMReq.Req with the LTR field marking the memory-access tolerance of the entity. As specified in [Section 10.2.2.1](#), a device may unilaterally generate PMReq.Req to communicate any changes to its latency, without any dependency on receipt of a prior PMReq.Rsp or PMReq.Go. Specifically, a device may transmit two PMReq.Req messages without an intervening PMReq.Rsp from the host. The LTR value communicated by the device is labeled MEM_LTR, and represents the Device's latency tolerance regarding CXL.cache accesses and it could be different from what is communicated via LTR messages over CXL.io.

If Idle state is allowed, the processor will respond with a matching PMReq.Rsp message, with the negotiated allowable latency-tolerance LTR (labeled CXL_MEM_LTR). Both entities can independently enter an Idle state without coordination as long as the shared resources remain accessible.

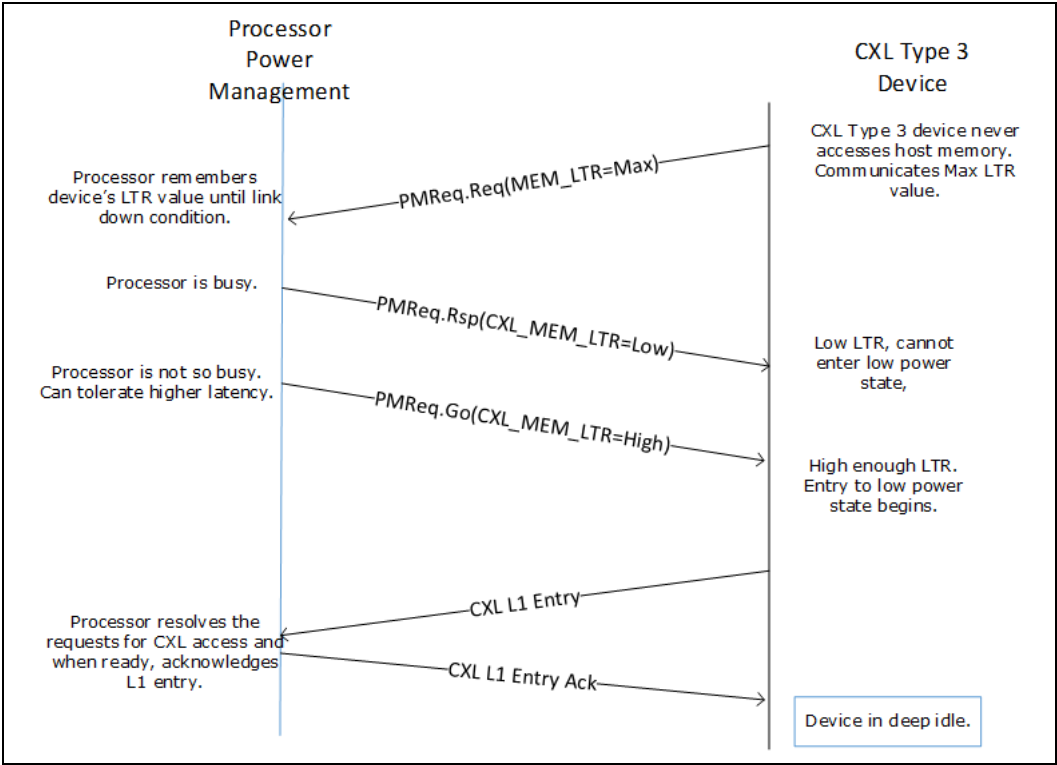
For a full PkgC entry, both entities need to negotiate as to the depth/latency tolerance by responding with a PMReq.Rsp message that includes the agreeable latency tolerance. After the master power management agent has coordinated LTR across all

the agents within the system, the agent will send a `PMReq.Go()` with the correct Latency field set (labeled `CXL_MEM_LTR`), indicating that local idle power actions can be taken subject to the communicated latency-tolerance value.

In case of a transition into deep-idle states, mostly typical of client systems, the device will initiate a CXL transition into L1.

These diagrams represent sequences, but do not imply any timing requirements. A host may respond much later with a `PMReq.Rsp` to a `PMReq.Reg` from a device when the Host is ready to enter a low-power state, or the Host may not respond at all. A device, having sent a `PMReq.Reg`, shall not implement a timeout to wait for `PMReq.Rsp` or `PMReq.Go`. Similarly, a device is not required to reissue `PMReq.Reg` if the Device's latency-tolerance requirements have not changed since previous communication and the link has remained up. As shown in [Figure 10-2](#), a CXL Type 3 device may issue `PMReq.Reg` after the link is up to indicate to the host that the Device either has no latency requirements or has a high latency tolerance. The host may communicate any changes to its latency expectations to such a device. Such a device may initiate low-power entry based only on the latency-tolerance value that the Device receives from the host, as shown in [Figure 10-2](#). When the host communicates a sufficiently high latency-tolerance value to the device, the device may enter a low-power state. A CXL Type 3 device may enter and exit a low-power state based only on the `PMReq.Go` message that the Device received from the host, without dependency on a prior `PMReq.Rsp`.

Figure 10-2. PkgC Entry Flows for CXL Type 3 Device - Example



10.2.4 PkgC Exit Flows

Figure 10-3. PkgC Exit Flows - Triggered by Device Access to System Memory

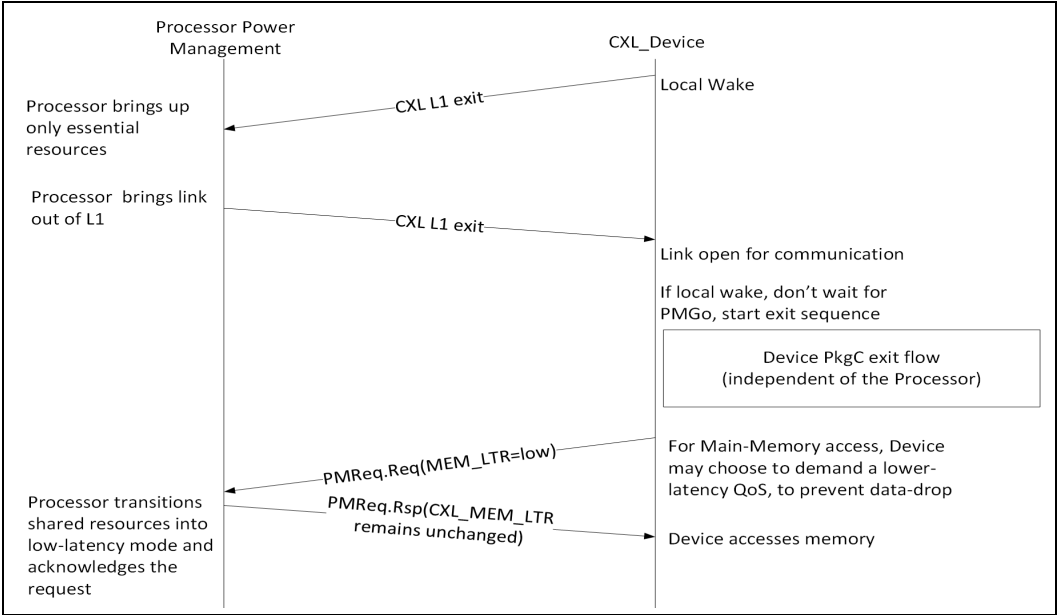


Figure 10-3 illustrates the PkgC exit flow initiated by the device. Link state during Idle may be in one of the select L1.x states, during Deep-Idle (as depicted here). In-band wake signaling will be used to transition the link back to L0. For more details, see [Section 10.3](#).

After the CXL link exits L1, signaling can be used to transfer the device into a PkgC state, in which shared resources are available across CXL. The device requests a low-latency tolerance value to the processor. Based on that value, the processor will bring the shared resources out of Idle and communicate its latest latency requirements with a `PMReq.Rsp()`.

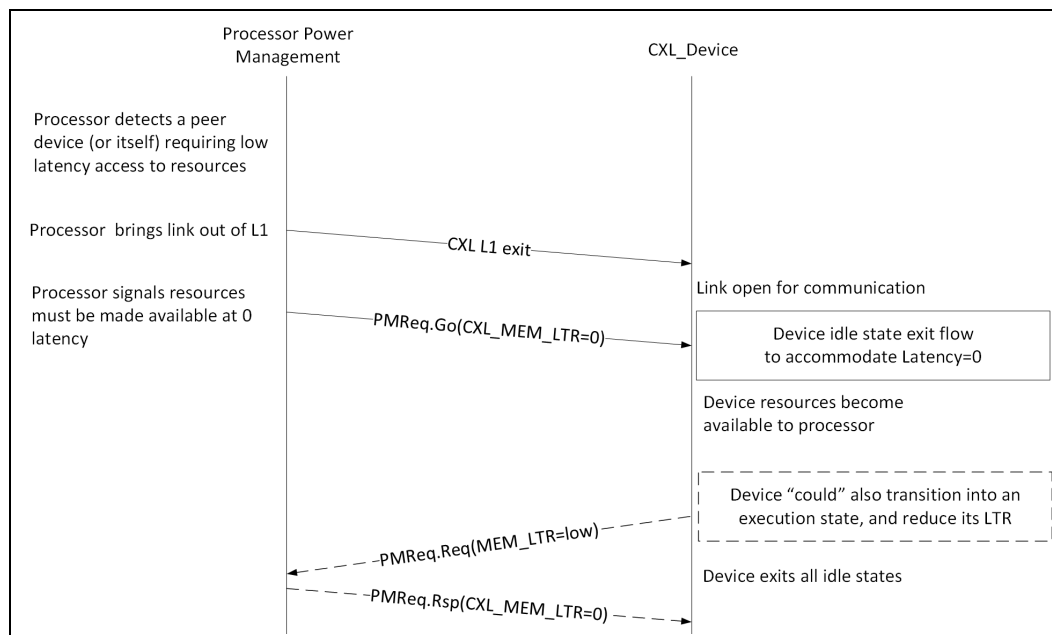
Figure 10-4. PkgC Exit Flows - Execution Required by Processor

Figure 10-4 illustrates the PkgC exit flow initiated by the processor. In the case where the processor, or one of the peer devices connected to the processor, must have coherent low-latency access to system memory, the processor will initiate a Link L1 exit toward the device.

After the link is running, the processor will follow with a PMGo(Latency=0), indicating some device in the platform requires low-latency access to coherent memory and resources. A device that receives PMReq.Go with Latency=0 must ensure that further low-power actions that might impede memory access are not taken.

10.2.5 CXL Physical Layer Power Management States

CXL Physical layer supports L1 and L2 states as defined in PCIe Base Specification. CXL Physical Layer does not support L0s. The entry and exit conditions from these states are also as defined in PCIe Base Specification. The notable difference is that for CXL Physical Layer, entry and exit from Physical Layer Power Management states is directed by the CXL ARB/MUX.

10.3 CXL Power Management

CXL Link Power Management supports Active Link State Power Management (ASPM), and L1 and L2 are the only 2 Power states supported. For 256B Flit mode, L0p negotiation is also supported. The PM Entry/Exit process is further divided into 3 phases as described below.

For 68B Flit mode, if the LTSSM goes through Recovery before the ARB/MUX vLSM moves to PM state, then the PM Entry process must restart from Phase 1, if the conditions for PM entry are still met after exit from Recovery and ARB/MUX Status Synchronization Protocol. For 256B Flit mode, the PM entry handshakes are not impacted by Link Recovery transitions because Link Recovery is not forwarded to the ARB/MUX vLSMs.

10.3.1 CXL PM Entry Phase 1

CXL PM Entry Phase 1 involves protocol-specific mechanisms to negotiate entry into a supported PM state. As shown in Figure 10-5, in 256B Flit mode, this transition does not require any synchronization between the ARB/MUX instances on the two ends. 68B Flit mode, however, does require such synchronization (see Figure 10-6). After the conditions to enter the PM state as defined in Section 10.2 are satisfied, the Transaction Layer is ready for Phase 2 entry and directs the ARB/MUX to enter the PM State.

Figure 10-5. CXL Link PM Phase 1 for 256B Flit Mode

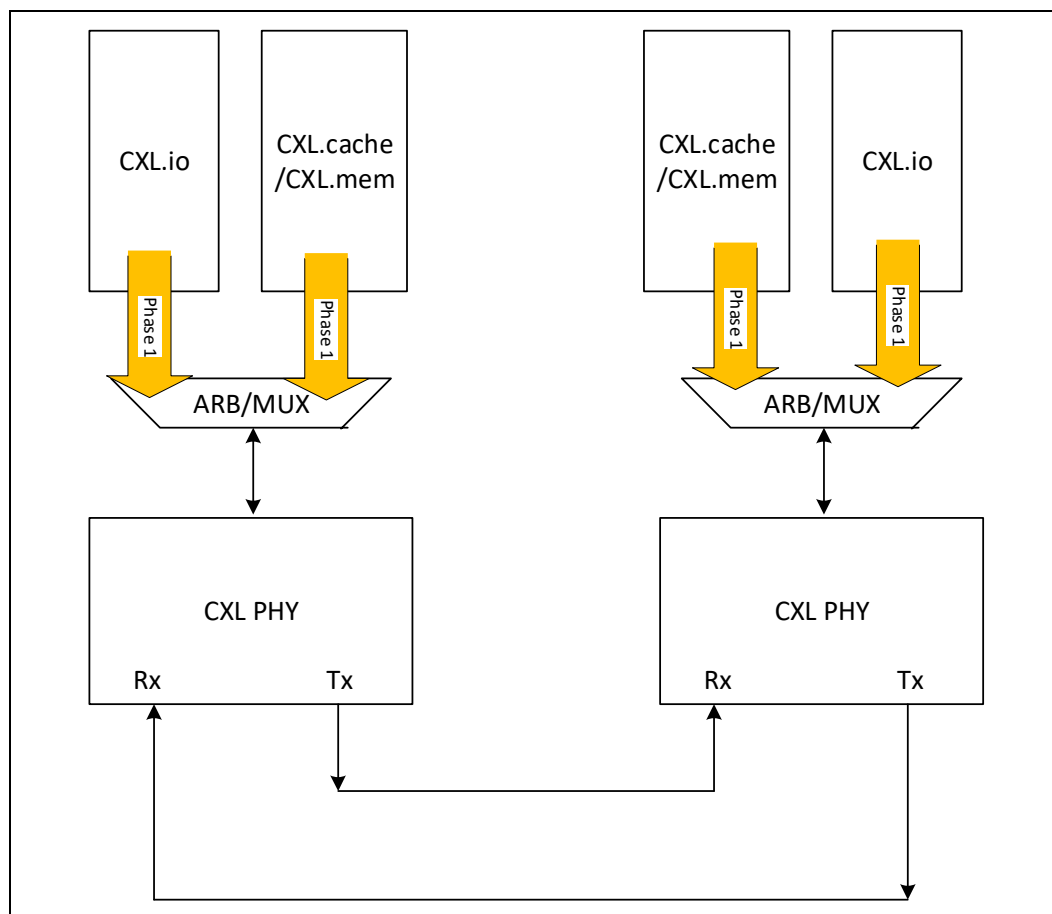
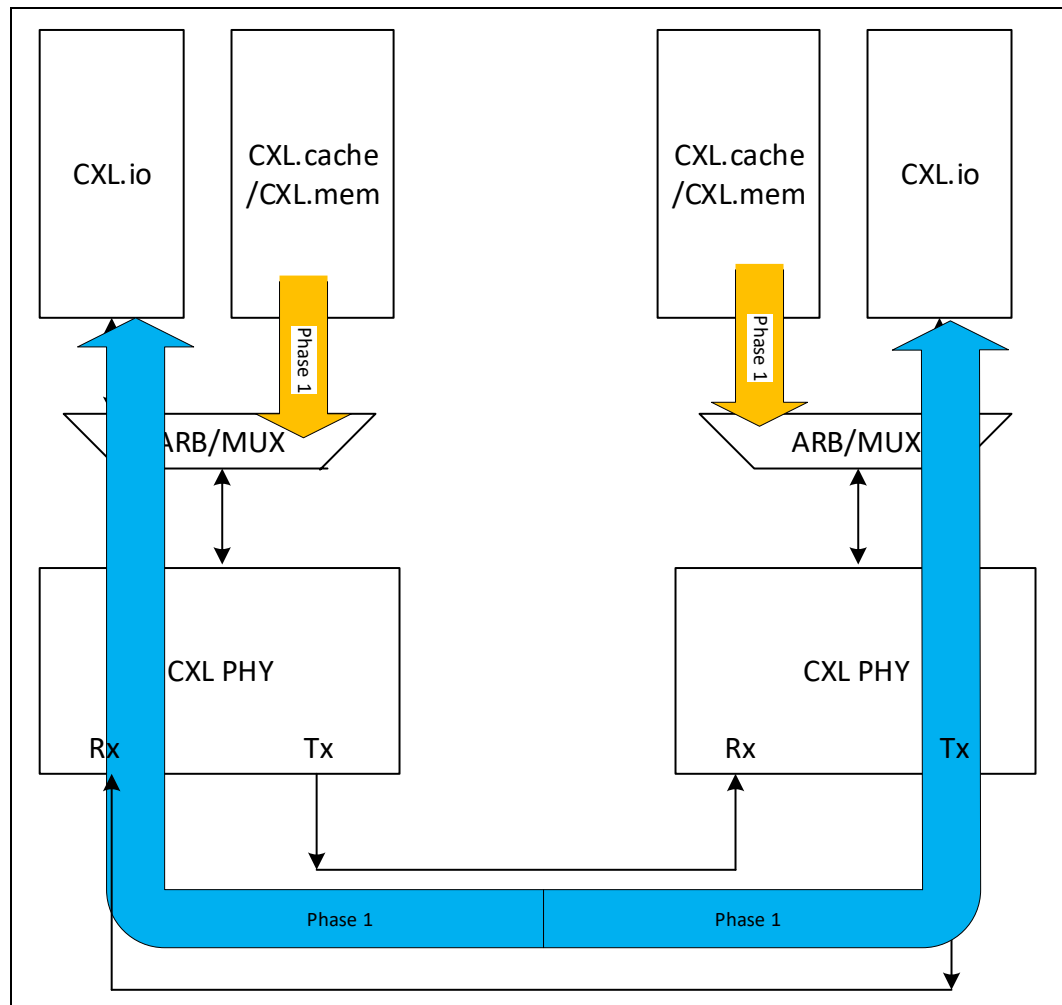


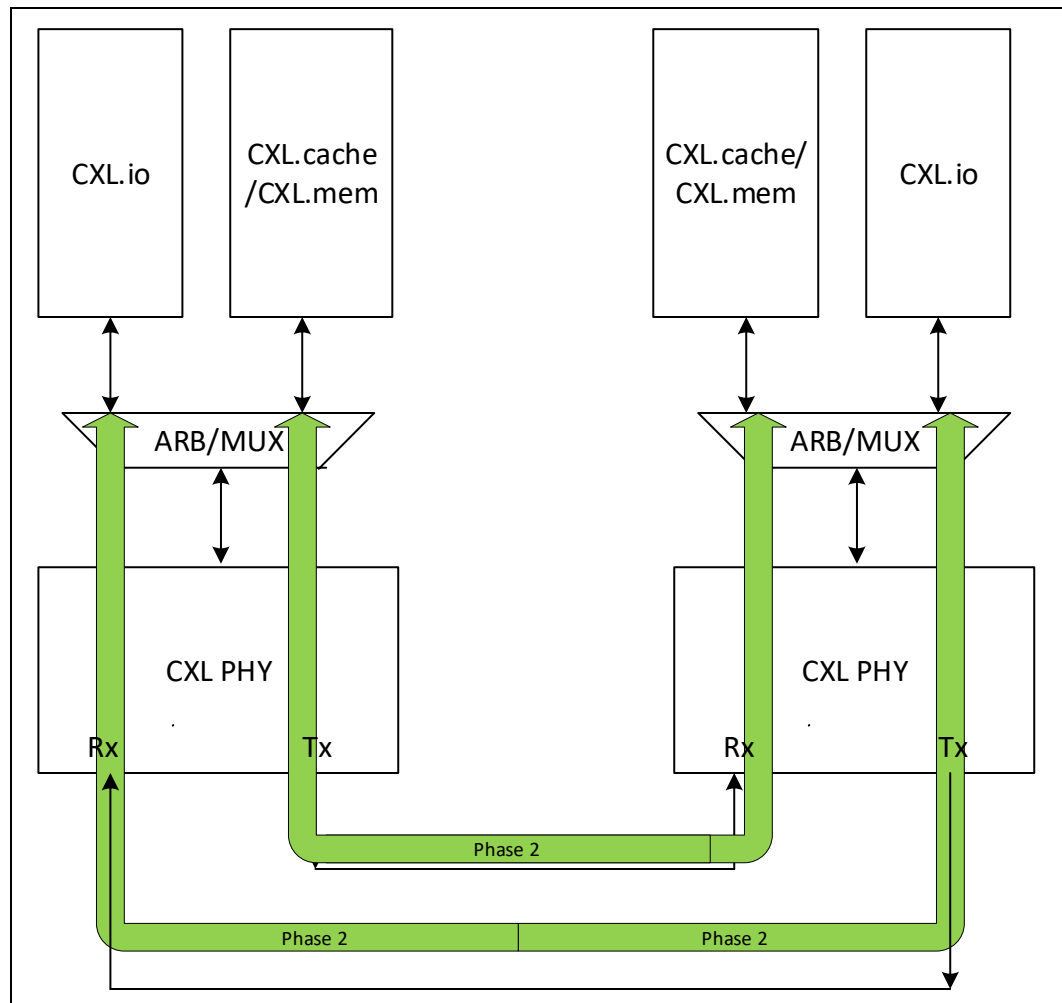
Figure 10-6. CXL Link PM Phase 1 for 68B Flit Mode



10.3.2 CXL PM Entry Phase 2

When directed by the Transaction Layer to enter PM, the ARB/MUX initiates the CXL PM Entry Phase 2 process. Phase 2 consists of bringing the ARB/MUX interface of both sides of the Link into a supported PM state. ALMPs are used to coordinate PM state entry as described below. Phase 2 entry is independently managed for each protocol. The Physical Layer continues to be in L0 until all the Transaction Layers enter Phase 2 state.

Figure 10-7. CXL Link PM Phase 2



Rules for the Phase 2 entry into ASPM are as follows (summarized in [Figure 10-7](#)):

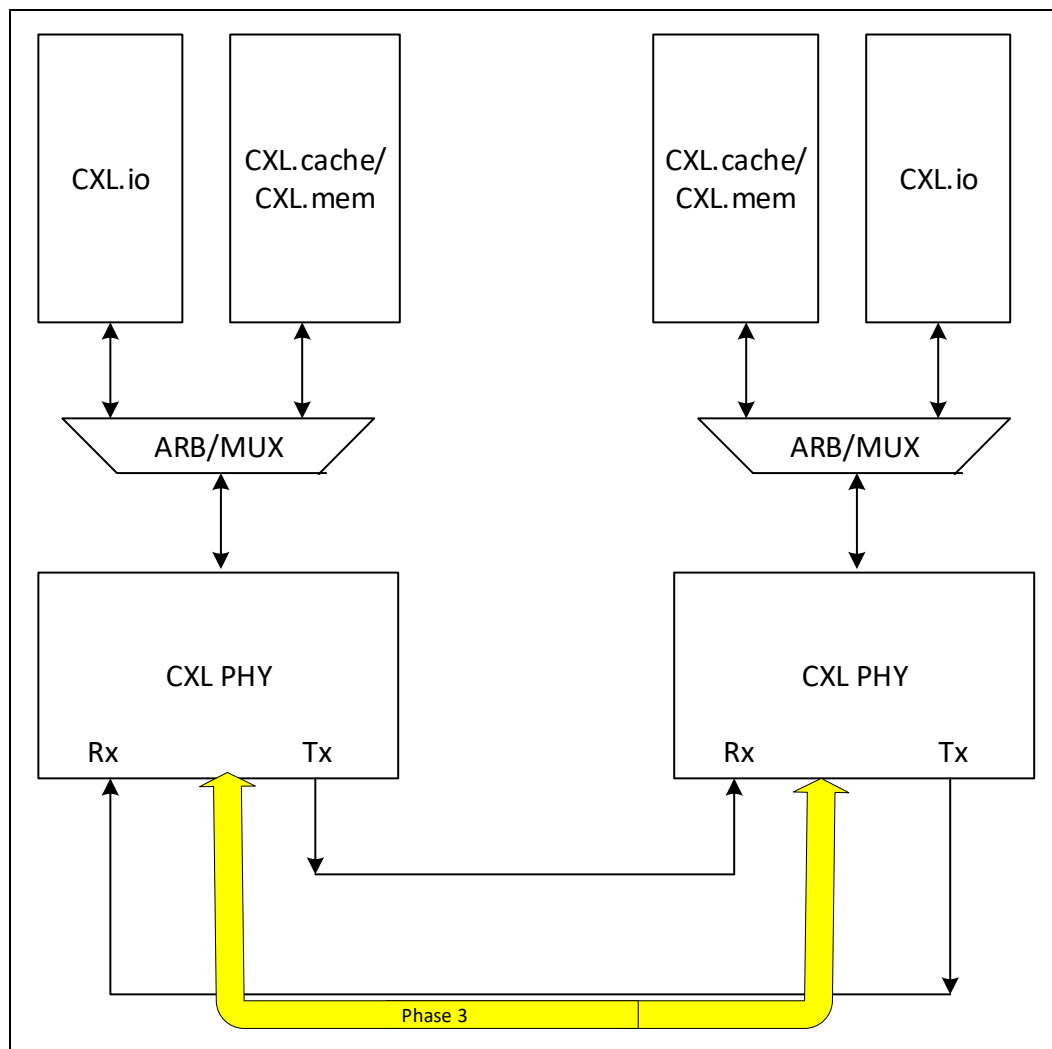
1. Phase 2 Entry into the supported PM State is always initiated by the ARB/MUX on the Downstream Component.
2. When directed by the Transaction Layer, the ARB/MUX on the Downstream Component must transmit an ALMP request to enter vLSM state PM.
3. When the ARB/MUX on the Upstream Component is directed to enter L1 and receives an ALMP request from the Downstream Component, the Upstream Component responds with an ALMP response indicating acceptance of entry into L1 state. The Transaction Layer on the Upstream Component must also be notified that the ARB/MUX port has accepted entry into the supported PM state.
4. The Upstream Component ARB/MUX port does not respond with an ALMP response if not directed by the upper layers to enter PM state.
5. When the ARB/MUX on the Downstream Component is directed to enter L1 and receives an ALMP response from the Upstream Component, the ARB/MUX notifies acceptance of entry into the PM state to the Transaction Layer on the Downstream Component.

6. The Downstream Component ARB/MUX port must wait for at least 1 ms (not including time spent in recovery states) for a response from the Upstream Component before retrying PM entry. The Downstream Component ARB/MUX is permitted to abort the PM entry before the 1-ms timeout by sending an Active Request ALMP for the corresponding vLSM.
7. L2 entry is an exception to Rule 6. Protocol must ensure that the Upstream Component is directed to enter L2 before setting up the conditions for the Downstream Component to request L2 state entry. This ensures that L2 abort or L2 Retry conditions do not exist. The Downstream Component may use indications such as the PME_Turn_Off message or a RESETPREP VDM to trigger L2 state entry.
8. The Transaction Layer on either side of the Link is permitted to directly exit from L1 state after the ARB/MUX interface enters L1 state.

10.3.3 CXL PM Entry Phase 3

CXL PM Entry Phase 3 is a conditional phase of PM entry and is executed only when all the Protocol interfaces of ARB/MUX have entered the same virtual PM state. The phase consists of bringing the Tx lanes to electrical idle and is always initiated by the Downstream Component. As shown in Figure 10-8, the PHY Layers on the two ends of the link communicate. If the link transitions to recovery during or after entry into electrical idle, the Downstream Component must wait for at least 1 us after entering L0 before re-initiating entry into electrical idle. This is to allow sufficient time for an Active State Request ALMP transfer to occur in case either side wants to initiate a PM exit (and to provide sufficient time for the remote ARB/MUX to stop requesting PM entry to LogPHY). The electrical idle entry flow is defined in the “Power Management” chapter of PCIe Base Specification.

Figure 10-8. CXL PM Phase 3



10.3.4 CXL Exit from ASPM L1

Components on either end of the Link may initiate exit from the L1 Link State. The ASPM L1 exit depends on whether the exit is from Phase 3 or Phase 2 of L1. The exit is hierarchical and Phase 3 must exit before Phase 2.

Phase 3 exit is initiated when directed by the ARB/MUX from either end of the link. The ARB/MUX Layer initiates exit from Phase 3 when there is an exit requested on any one of its primary protocol interfaces. The Phase 3 ASPM L1 exit is the same as exit from L1 state as defined in PCIe Base Specification. The steps are followed until the LTSSM enters L0 state. Protocol-level information is not permitted to be exchanged until the vLSM on the ARB/MUX interface has exited L1 state.

Phase 2 exit involves bringing the protocol interface independently out of L1 state at the ARB/MUX. The Transaction Layer directs the ARB/MUX state to exit vLSM state. If the PHY is in Phase 3 L1, then the ARB/MUX waits for the PHY LTSSM to enter L0 state. After the PHY is in L0 state, the following rules apply:

1. The ARB/MUX on the protocol side that is triggering an exit transmits an ALMP requesting entry into Active state.
2. Any ARB/MUX interface that receives the ALMP request to enter Active State must transmit an ALMP acknowledge response on behalf of that interface. The ALMP acknowledge response is an indication that the corresponding protocol side is ready to process received packets.
3. Any ARB/MUX interface that receives the ALMP request to enter Active State must also transmit an ALMP Active State request on behalf of that interface if not already sent.
4. Protocol-level transmission must be permitted by the ARB/MUX after an Active State Status ALMP is transmitted and received. This guarantees that the receiving protocol is ready to process packets.

10.3.5 L0p Negotiation for 256B Flit Mode

See [Chapter 5.0](#) for the L0p negotiation rules.

10.4 CXL.io Link Power Management

CXL.io Link Power Management is as defined in PCIe Base Specification with the following notable differences:

- RCD links support ASPM-directed L1 entry but do not support PCI-PM-directed L1 entry. An eRCD is not required to initiate entry into L1 state when software transitions the device into D3Hot or D1 device state. When a component is not operating in RCD mode, the component shall support ASPM L1 as well as PCI-PM. As such, a component not operating in RCD mode shall initiate CXL.io L1 entry when the device is placed in D3Hot or D1 device state.
- L0s state is not supported.

All CXL functions shall implement PCI* Power Management Capability Structure as defined in PCIe Base Specification and shall support D0 and D3 device states.

10.4.1 CXL.io ASPM Entry Phase 1 for 256B Flit Mode

There must not be any DLLP exchanges initiated for PM entry for 256B Flit mode. The Link Layer on each side independently requests its local ARB/MUX to enter a PM state. The ARB/MUX Layers on both sides of the Link coordinate entry into a PM state using ALMPs as part of Phase 2.

10.4.2 CXL.io ASPM L1 Entry Phase 1 for 68B Flit Mode

The first phase consists of completing the ASPM L1 negotiation rules as defined in PCIe Base Specification with the following notable exception for the rules in case of acceptance of ASPM L1 Entry:

- All rules up to the completion of the ASPM L1 handshake are maintained; however, the process of bringing the Transmit Lanes into Electrical Idle state are divided into 2 additional phases described in [Section 10.3](#).

See PCIe Base Specification for the PCIe ASPM L1 Entry flow.

10.4.3 CXL.io ASPM L1 Entry Phase 2

Phase 2 of L1 entry consists of bringing the CXL.io ARB/MUX interface of both sides of the Link into L1 state. ALMPs are used to coordinate L1 state entry. For 256B Flit mode, the ALMP exchange rules are the same for CXL.io and CXL.cachemem, and are defined in [Chapter 5.0](#).

The rules for Phase 2 entry into ASPM L1 for 68B Flit mode are as follows:

1. CXL.io on the Upstream Component must direct the ARB/MUX to be ready to enter L1 before returning the PM_Request_Ack DLLPs as shown above in Phase 1.
2. When the PM_Request_Ack DLLPs are successfully received by the CXL.io on the Downstream Component, the CXL.io must direct the ARB/MUX on the Downstream Component to transmit the ALMP request to enter vLSM state L1.
3. When the ARB/MUX on the Upstream Component is directed to enter L1 and receives an ALMP request from the Downstream Component, the ARB/MUX notifies the CXL.io that the interface has received an ALMP request to enter L1 state and has entered L1 state.
4. When the Upstream Component is notified of the vLSM state L1 entry, the Upstream Component ceases sending PM_Request_Ack DLLPs.
5. When the ARB/MUX on the Downstream Component is directed to enter L1 and receives ALMP Status from the Upstream Component, the ARB/MUX notifies the CXL.io that the interface has entered L1 state.

10.4.4 CXL.io ASPM Entry Phase 3

Phase 3 entry is dependent on the vLSM state of multiple protocols and is managed by the ARB/MUX as described in [Section 10.3.3](#).

10.5 CXL.cache + CXL.mem Link Power Management

CXL.cache and CXL.mem both support only ASPM. Unlike CXL.io, there is no PM Entry handshake defined between the Link Layers. Each side independently requests the ARB/MUX to enter L1. The ARB/MUX Layers on both sides of the Link coordinate the entry into a PM state using ALMPs. CXL.cache + CXL.mem Link Power Management follows the process for PM entry and exit as defined in [Section 10.3](#).

§ §

11.0 CXL Security

11.1 CXL IDE Overview

CXL Integrity and Data Encryption (CXL IDE) defines mechanisms for providing Confidentiality, Integrity, and Replay protection for data that traverses the CXL link. The cryptographic schemes are aligned with current industry best practices. CXL IDE supports a variety of usage models while providing for broad interoperability. CXL IDE can be used to secure traffic within a Trusted Execution Environment (TEE) that is composed of multiple components (see [Section 11.5](#)).

This chapter focuses on the changes for CXL.cache and CXL.mem traffic that traverses the link, and updates and constraints to PCIe* Base Specification that govern CXL.io traffic.

- CXL.io IDE definition including CXL.io IDE key establishment is based on PCIe IDE. Differences and constraints for CXL.io usage are identified in [Section 11.2](#).
- CXL.cachemem IDE may use the CXL.io-based mechanisms for discovery, negotiation, device attestation, and key negotiation using a standard mechanism as described in [Section 11.4](#).

In this specification, the term CXL IDE is used to refer to the scheme that protects CXL.io, CXL.cache, and CXL.mem traffic. The term CXL.cachemem IDE is used to refer to the protections associated with CXL.cache and CXL.mem traffic.

IMPLEMENTATION NOTE: SECURITY MODEL**Assets**

Assets that are in scope are as follows:

- Transactions (data + metadata communicated) between the two sides of the physical link. Only the definition for providing integrity, replay protection and encryption/decryption of traffic between the ports is included in this specification.

Notes:

- This threat model does not cover the security exposure due to inadequate Device implementation.
- Agents that are on each side of the physical link are within the trust boundary of the respective devices/hardware blocks in which they reside. These agents will need to provide implementation-specific mechanisms to protect data internal to the device and any external connections over which such data can be sent by the device. Mechanisms for such protection are beyond the scope of this definition.
- Symmetric cryptographic keys are used to provide confidentiality, integrity, and replay protection of data in transit between physically connected CXL ports. This specification does not define mechanisms for protecting these keys inside the host and the device.
- Certificates and asymmetric keys that are used for device authentication and key exchange are beyond the scope of this specification. The device attestation and key exchange mechanism determine the security model for those assets.

TCB

The TCB consists of the following:

- Functional blocks on each side of the link that implement the link encryption and integrity.
- Agents that are used to configure the cryptographic engines in the functional blocks on each side of link. For example, trusted firmware/software agent and/or security agent hardware and firmware that implement key exchange protocol or facilitate programming of the keys.
- Other hardware blocks in the device that may have direct or indirect access to the assets, including those that perform operations such as reset, debug, and link power management.

Adversaries and Threats

- Threats from physical attacks on links, including cases where an adversary can examine data intended to be confidential, modify data or protocol metadata, record and replay recorded transactions, reorder and/or delete data flits, inject transactions including requests/data or non-data responses, using lab equipment, purpose-built interposers, and/or malicious Extension Devices.
- Threats arising from physical replacement of a trusted device with an untrusted device, and/or removal of a trusted device and accessing the trusted device with a system that is under an adversary's control.
- CXL.cachemem IDE provides point-to-point protection. Any switches present in the path between the Host and the Endpoint, or between two Endpoints, must support this specification. In these cases, such switches will be in the TCB.

Denial of service attacks are beyond the scope of this specification.

11.2 CXL.io IDE

CXL.io IDE follows the PCIe IDE definition. This section covers the notable constraints and differences between the CXL.io IDE definition and the PCIe IDE definition.

Table 11-1. Mapping of PCIe IDE to CXL.io

PCIe IDE Definition	CXL.io Support	Notes
Link IDE stream	Supported	Required for CXL.cachemem IDE. CXL.cachemem will only use keys associated with Link IDE stream.
Selective IDE stream	Supported	Selective IDE stream applies only to CXL.io.
Aggregation	Supported	PCIe-defined aggregation levels apply only to CXL.io traffic.
Switches with flow-through selective IDE streams	Supported	CXL Switches need to support Link IDE streams. CXL Switches may either operate as a boundary for selective IDE streams or forward the IDE streams toward Endpoints.
PCRC mechanism	Supported	PCRC mechanism may be optionally enabled for the CXL.io ports.

One of the PCIe IDE reserved sub-stream encodings (1000b) is assigned for CXL.cachemem usage.

11.3 CXL.cachemem IDE

All protocol-level retryable flits are encrypted and integrity protected.

When operating in 68B Flit mode:

- Link Layer control flits and flit CRC are not encrypted or integrity protected. There is no confidentiality or integrity on these flits.
- Link CRC shall be calculated on encrypted flits. Link retries occur first and only flits that pass Link CRC will be decrypted and then integrity checked.

When operating in 256B Flit mode:

- Link Layer control information, flit header, and flit CRC/FEC is not encrypted or integrity protected. There is no confidentiality protection, integrity protection, or replay protection for this content.
- Link CRC shall be calculated on encrypted flits. Link retries occur first and only flits that pass Link CRC will be decrypted and then integrity checked.

Any integrity check failures shall result in all future secure traffic being dropped.

Multi-Data Header capability must be supported. This allows packing of multiple (up to 4) data headers into a single slot, followed immediately by 16 slots of all-data.

IDE will operate on a flit granularity for CXL.cache and CXL.mem protocols. IDE makes use of the Advanced Encryption Standard-Galois Counter Mode Advanced Encryption and Advanced Decryption Functions (referred to herein as AES-GCM), as defined in NIST* Special Publication 800-38D. AES-GCM with a 256-bit key size shall be used for confidentiality protection, integrity protection, and replay protection. The AES-GCM Functions take three inputs:

- additional authentication data (AAD; denoted as *A*)
- plaintext (denoted as *P*)
- initialization vector (denoted as *IV*)

Key refresh without any data loss must be supported. There are a number of scenarios where the keys need to be refreshed. Some examples include:

- An accelerator device that is migrated from one VM (or process) to a different VM (or process).
- Crypto considerations (concerns about key wear-out) for long-running devices or devices that are part of the platform.

Key refresh is not expected to occur frequently. It is acceptable to take a latency/bandwidth hit; however, there must not be any data loss.

Encrypted PCRC mechanism is supported to provide robustness against hard and soft faults that are internal to the encryption and decryption engines. Encrypted PCRC integrates into the standard MAC check mechanism, does not consume incremental link bandwidth, and can be implemented without adding significant incremental latency. PCRC is mandatory for CXL.cachemem IDE and is enabled by default.

11.3.1 CXL.cachemem IDE Architecture in 68B Flit Mode

IDE shall operate on a flit granularity for CXL.cachemem protocols. IDE makes use of the AES-GCM algorithm, and AES-GCM takes three inputs – A , P , and IV – as described earlier in [Section 11.3](#).

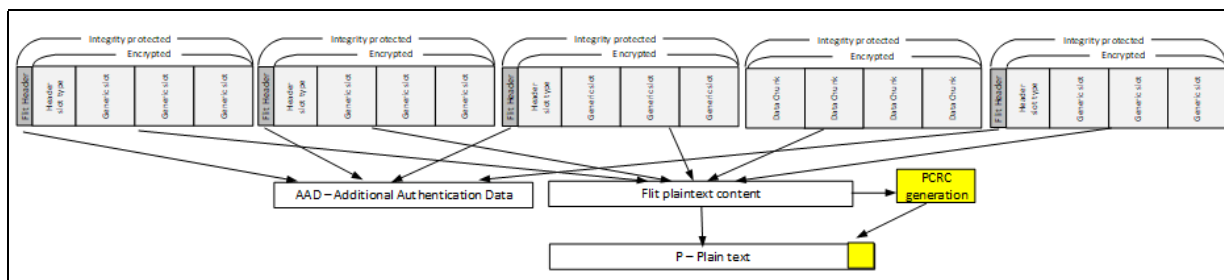
In the case of CXL.cachemem protocol header flits, the 32 bits of the flit header that are part of Slot 0 map to A – it is not encrypted, but it is integrity protected. The remainder of the Slot 0/1/2/3 contents maps to P , which is encrypted and integrity protected (see handling of MAC slot below). CXL.cachemem protocol also supports ADF. In the case of an ADF, all 4 slots in the flit map to P .

The link CRC is not encrypted or integrity protected. The CRC is calculated on the flit content after the flit has been encrypted.

As with other protocol flits, IDE flits shall be covered by link layer mechanisms for detecting and correcting errors. This process shall operate on flits after the flits are cryptographically processed by the transmitter and before the flits are submitted for cryptographic processing by the receiver.

AES-GCM is applied to an aggregation of multiple flits referred to as a MAC epoch. The number of flits in the aggregation is determined by the Aggregation Flit Count (see [Section 11.3.5](#) for details). If PCRC (see [Section 11.3.3](#)) is enabled in the CXL IDE Control register (see [Section 8.2.4.22.2](#)), the 32 bits of PCRC shall be appended to the end of the aggregated flit content to contribute to the final P value that is integrity protected. However, the 32 bits of PCRC are not transmitted across the link. [Figure 11-1](#) shows the mapping of the flit contents into A and P for the case of aggregation of MAC across 5 flits.

Figure 11-1. 68B Flit: CXL.cachemem IDE Showing Aggregation of 5 Flits



The Message Authentication Code (MAC), also referred to as the authentication tag in NIST Special Publication 800-38D, shall be 96 bits. The MAC must be transmitted in a Slot 0 header of type H6 (see Figure 4-12). Unlike other Slot 0 headers, the MAC itself is neither encrypted nor integrity protected. Figure 11-2 shows the mapping of flit contents to A and P for the case of aggregation of MAC across 5 flits with one of the flits carrying a MAC.

Figure 11-2. 68B Flit: CXL.cachemem IDE Showing Aggregation across 5 Flits where One Flit Contains MAC Header in Slot 0

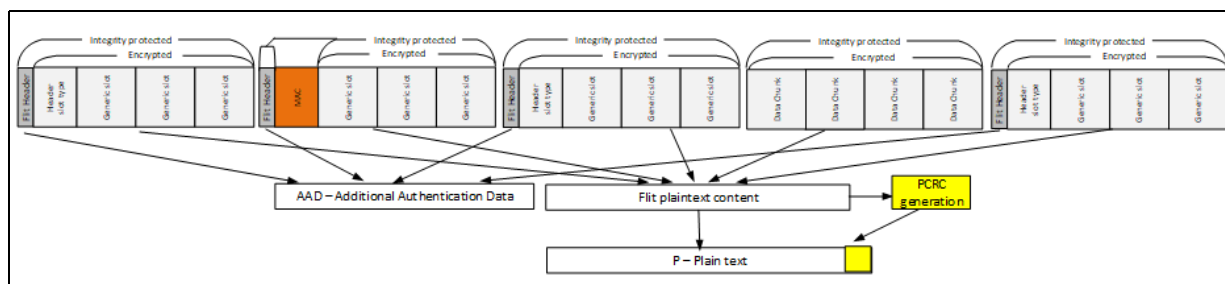


Figure 11-3 provides a more-detailed view of the 5-flit MAC epoch example. Flit0 shown on the top is the first flit to be transmitted in this MAC epoch. The figure shows the header fields that are only integrity protected, and plaintext content that is encrypted and integrity protected. Flit0 plaintext0 byte0 is the first byte of the plaintext. Flit1 plaintext0 byte0 shall immediately follow flit0 plaintext3 byte15.

Figure 11-3. 68B Flit: More-detailed View of a 5-Flit MAC Epoch Example

[illegible]

Figure 11-4 shows the mapping of the header bytes to AES-GCM AAD (A) for the example in Figure 11-3.

Figure 11-4. 68B Flit: Mapping of AAD Bytes for the Example Shown in Figure 11-3

AAD for current MAC Epoch																
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
1																
2																
3																
4																
5																
6																
7																

11.3.2 CXL.cachemem IDE Architecture in 256B Flit Mode

If the header slot is used for sending control messages other than IDE.MAC, the entire flit shall not carry any protocol traffic. This applies for other usages of IDE type (IDE.TMAC, IDE.Start, and IDE.Idle), In-band Error, and INIT.

The receiver uses 4 bits of the header slot that encode the slot type to determine whether the slot contains control or protocol information. If the header slot is carrying protocol information, then 4 bits of the header slot that encode the slot type will map to AES-GCM input A. Although the slot type will not be encrypted, it is integrity protected. If the header slot is carrying control information, then the entire slot is neither encrypted nor integrity protected.

In case the header slot is carrying protocol information, then the plaintext (*P*) starts at bit 20. To simplify implementation and align to the AES block size of 128 bits, 20 bits of 0s shall be padded in front of the header slot content and the padded 128 bits of information shall be mapped to AES-GCM input *P*.

- The padded header slot will be used when calculating PCRC (see [Section 11.3.3](#)).
- Encrypted pad will not be transmitted on the link. Receiver must reconstruct the ciphertext for the padded region when calculating the AES-GCM MAC.

Credit return (CRD) field does not carry any confidential data. The CRD field needs to be integrity protected, so the CRD field shall map to AES-GCM input A.

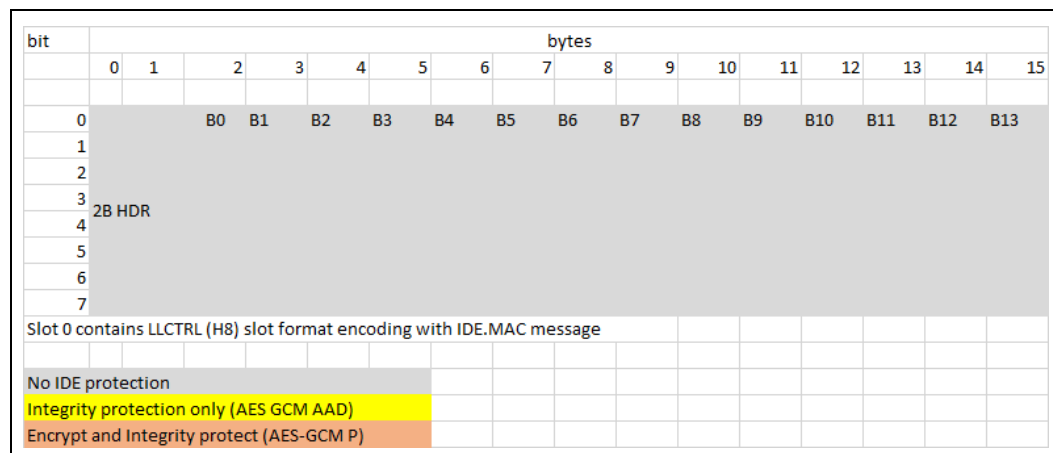
The rules for handling latency-optimized flits are as follows:

- Slot 7 bytes shall be packed together before mapping to AES-GCM input *P*.
- Slot 8 is only 12 bytes long. It shall be padded with 32 bits of 0 at the end of slot content. This will enable subsequent slots to be aligned on the 128-bit AES block boundary.
- Packed Slot 7 and padded Slot 8 should be used when calculating PCRC (see [Section 11.3.3](#)).
- Receiver must reconstruct the ciphertext for the padded region in Slot 8 when calculating AES-GCM MAC.
- AES-GCM input A for each flit shall be padded with 0s to align it to 32 bits.
- Header slot contains protocol header: `slot_type|CRD|012`.
- Header slot contains MAC: `CRD|016`.

In the case of 256B flits, only Slot 0 and Slot 15 contribute to the AAD. [Figure 11-5](#), [Figure 11-6](#), [Figure 11-7](#), [Figure 11-8](#), [Figure 11-9](#), and [Figure 11-10](#) depict handling of the AAD field.

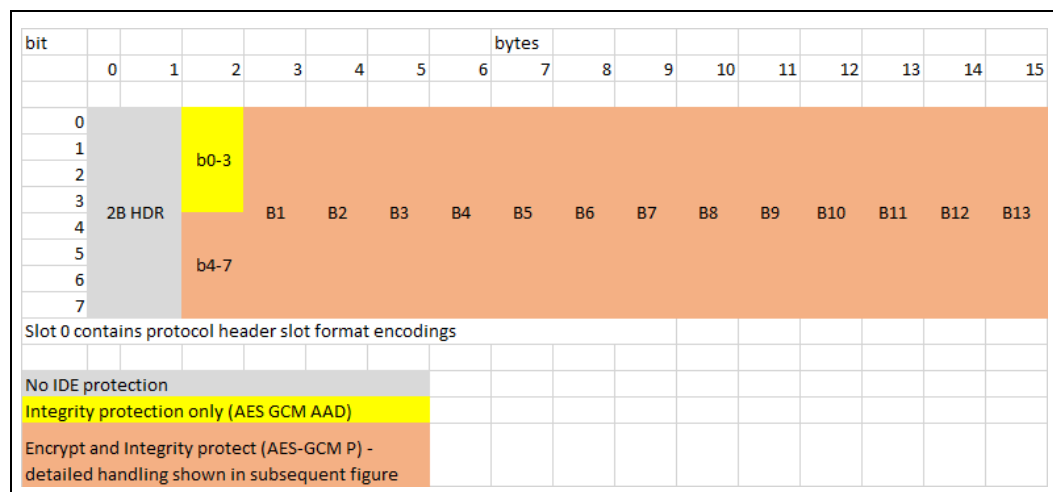
[Figure 11-5](#) depicts the case when Slot 0 contains LLCTRL (H8) slot format encoding with an IDE.MAC message. In this case, Slot 0 does not contain any bits that require integrity protection; therefore, Slot 0 is not IDE protected.

Figure 11-5. 256B Flit: Handling of Slot 0 when it Carries H8



[Figure 11-6](#) shows the case where Slot 0 contains protocol header slot format encoding (H0 - H7, H9 - H15). The first 2 bytes that contain the flit header are not IDE protected. Bits 0 - 3 of Slot 0 that carry the slot format encoding are not encrypted, but are integrity protected and therefore map to the AAD field. The remaining bits of the slot are encrypted and integrity protected (additional details regarding mapping to the P field are provided in [Figure 11-11](#)).

Figure 11-6. 256B Flit: Handling of Slot 0 when it Does Not Carry H8



Handling of Slot 15 is shown in [Figure 11-7](#). When the flit carries protocol information, the CRD field carried in Slot 15 needs to be integrity protected. In this case, the first two bytes of CRD information (Credit return byte 0 and Credit return byte 1) map to the AES-GCM AAD field.

Figure 11-7. 256B Flit: Handling of Slot 15

bytes															
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C0b0	C0b1	CRC (8B)								FEC (6B)					
No IDE protection															
Integrity protection only (AES GCM AAD)															
Encrypt and Integrity protect (AES-GCM P)															

Figure 11-8 shows how the bits that only need to be integrity protected are mapped to the AAD field when the first flit carries a protocol header in Slot 0 and the second flit carries IDE.MAC in Slot 0.

Figure 11-8. Mapping of Integrity-only Protected Bits to AAD - Case 1

bit	Flit0 H8 MAC				Flit1 protocol header			
	bytes				bytes			
	0	1	2	3	4	5	6	7
0	C0b0	C1b0	0	0	C0b4	C1b4	0	0
1	C0b1	C1b1	0	0	C0b5	C1b5	0	0
2	C0b2	C1b2	0	0	C0b6	C1b6	0	0
3	C0b3	C1b3	0	0	C0b7	C1b7	0	0
4	C0b4	C1b4	0	0	C0b0	C1b0	0	0
5	C0b5	C1b5	0	0	C0b1	C1b1	0	0
6	C0b6	C1b6	0	0	C0b2	C1b2	0	0
7	C0b7	C1b7	0	0	C0b3	C1b3	0	0
(b) First flit carries MAC in Slot 0								
No IDE protection								
Header bits for Integrity protection only (AES GCM AAD)								
Zero padding required by CXL.cachemem IDE. Include these bits in the computation of AAD length.								

Figure 11-9 shows how the bits that only need to be integrity protected are mapped to the AAD field when the first flit carries IDE.MAC in Slot 0 and the second flit carries a protocol header.

Figure 11-9. Mapping of Integrity-only Protected Bits to AAD - Case 2

bit	Flit1 H8 MAC				Flit1 protocol header			
	bytes				bytes			
	4	5	6	7	4	5	6	7
0	C0b0	C1b0	0	0	C0b4	C1b4	0	0
1	C0b1	C1b1	0	0	C0b5	C1b5	0	0
2	C0b2	C1b2	0	0	C0b6	C1b6	0	0
3	C0b3	C1b3	0	0	C0b7	C1b7	0	0
4	C0b4	C1b4	0	0	C0b0	C1b0	0	0
5	C0b5	C1b5	0	0	C0b1	C1b1	0	0
6	C0b6	C1b6	0	0	C0b2	C1b2	0	0
7	C0b7	C1b7	0	0	C0b3	C1b3	0	0
First flit carries MAC in Slot 0								
No IDE protection								
Header bits for Integrity protection only (AES GCM AAD)								
Zero padding required by CXL.cachemem IDE. Include these bits in the computation of AAD length.								

Figure 11-10 shows the third case of how the bits that only need to be integrity protected are mapped to the AAD field. In this case, both flits carry protocol headers in Slot 0. When the flit carries a protocol header, there are 20 bits that require integrity protection. These 20 bits are made up of 4 bits of Slot encoding (Slot 0) and 16 bits of CRD (Slot 15). These are padded with trailing 0s to create a 32-bit AAD input.

Figure 11-10. Mapping of Integrity-only Protected Bits to AAD - Case 3

bit	Flit0 protocol header				Flit1 protocol header			
	bytes				bytes			
	0	1	2	3	4	5	6	7
0	C0b4	C1b4		0	C0b4	C1b4		0
1	b0-3	C0b5	C1b5	0	b0-3	C0b5	C1b5	0
2		C0b6	C1b6	0		C0b6	C1b6	0
3		C0b7	C1b7	0		C0b7	C1b7	0
4	C0b0	C1b0		0	C0b0	C1b0		0
5	C0b1	C1b1		0	C0b1	C1b1		0
6	C0b2	C1b2		0	C0b2	C1b2		0
7	C0b3	C1b3		0	C0b3	C1b3		0
Both flits carry protocol header in Slot 0								
No IDE protection								
Header bits for Integrity protection only (AES GCM AAD)								
Zero padding required by CXL.cachemem IDE. Include these bits in the computation of AAD length.								

Because there can be only one IDE.MAC within any given MAC epoch, it is impossible for both flits to carry IDE.MAC. Such a case does not exist and hence not shown here.

Figure 11-11 shows the transmitter's handling of bits that require both encryption and integrity protection for the standard 256B flit when Slot 0 contains LLCTRL (H8) Slot format encoding with an IDE.MAC message. Slot 0 content is not IDE protected. Slots 1 - 14 are mapped to P.

Figure 11-11. Standard 256B Flit - Mapping to AAD and P bits when Slot 0 carries H8

	bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
slot0		2B HDR		B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13
slot1	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot2	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot3	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot4	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot5	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot6	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot7	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot8	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot9	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot10	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot11	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot12	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot13	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot14	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot15		COB0	COB1	CRC (8B)										FEC (6B)			
Slot 0 contains LLCTRL (H8) slot format encoding with IDE.MAC message																	
No IDE protection																	
Integrity protection only (AES GCMAAD)																	
Encrypt and Integrity protect (AES-GCMP)																	
Zero padding required by CXL.cachemem IDE. Encrypt and Integrity protect (AES-GCMP). Include these bits in the computation of P length.																	

Figure 11-12 shows the transmitter's handling of bits that require both encryption and integrity protection for the standard 256B flit when Slot 0 contains protocol header slot format encoding (H0 - H7, H9 - H 15). Slot 0 contains 108 bits, starting from bit 4 of the slot header. These bits are padded with leading 0s to align the content to a 128-bit boundary. The padded Slot 0 content, and Slots 1 - 14, are mapped to *P*.

Figure 11-12. Standard 256B Flit - Mapping to AAD and P bits when Slot 0 Does Not Carry H8

		bytes															
	bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
slot0	0	0	0	0	B1 B2 B3 B4 B5 B6 B7 B8 B9 B10 B11 B12 B13												
	1	0	0	0													
	2	0	0	0													
	3	0	0	0													
	4	0	0														
	5	0	0														
	6	0	0														
	7	0	0														
slot1	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot2	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot3	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot4	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot5	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot6	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot7	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot8	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot9	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot10	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot11	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot12	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot13	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot14	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot15		COB0	COB1	CRC (8B)										FEC (6B)			
Slot 0 contains protocol header slot format encoding (H0 – H7, H9 – H 15)																	
No IDE protection																	
Integrity protection only (AES GCMAAD)																	
Encrypt and Integrity protect (AES-GCM P)																	
Zero padding required by CXL.cachemem IDE. Encrypt and Integrity protect (AES-GCM P). Include these bits in the computation of P length.																	

Figure 11-13 shows the transmitter's handling of bits that require both encryption and integrity protection for the latency-optimized 256B flit when Slot 0 contains LLCTRL (H8) Slot format encoding with an IDE.MAC message. Slot 0 content is not IDE protected. Slots 1 - 14 are mapped to *P*.

Figure 11-13. Latency-Optimized 256B Flit - Mapping to AAD and P Bits when Slot 0 Carries H8

		bytes															
	bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
slot0		2B HDR		B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13
slot1	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot2	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot3	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot4	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot5	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot6	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot7	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot8	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	0	0	0	0
slot9	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot10	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot11	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot12	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot13	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot14	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot15		COB0	COB1	Slot8 B10, B11				CRC (8B)								FEC (6B)	
Slot 0 contains LLCTRL (H8) slot format encoding with IDE.MAC message																	
No IDE protection																	
Integrity protection only (AES GCM AAD)																	
Encrypt and Integrity protect (AES-GCMP)																	
Zero padding required by CXL.cachemem IDE. Encrypt and Integrity protect (AES-GCM P). Include these bits in the computation of P length.																	

Figure 11-14 shows the transmitter's handling of bits that require both encryption and integrity protection for the latency-optimized 256B flit when Slot 0 contains protocol header slot format encoding (H0 - H7, H9 - H15). Slot 0 contains 108 bits, starting from bit 4 of the slot header. These bits are padded with leading 0s to align the content to a 128-bit boundary. The padded Slot 0 content, and Slots 1 - 14, are mapped to P.

Figure 11-14. Latency-Optimized 256B Flit - Mapping to AAD and P Bits when Slot 0 Does Not Carry H8

		bytes															
	bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
slot0	0	0	0	0	b4-7												
	1	0	0	0													
	2	0	0	0													
	3	0	0	0													
	4	0	0														
	5	0	0														
	6	0	0														
	7	0	0														
slot1	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot2	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot3	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot4	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot5	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot6	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot7	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot8	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	0	0	0	0
slot9	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot10	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot11	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot12	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot13	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot14	0-7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
slot15		COB0	COB1	Slot8 B10, B11		CRC (8B)								FEC (6B)			
Slot 0 contains protocol header slot format encoding (H0 – H7, H9 – H15)																	
No IDE protection																	
Integrity protection only (AES GCM AAD)																	
Encrypt and Integrity protect (AES-GCMP)																	
Zero padding required by CXL.cachemem IDE. Encrypt and Integrity protect (AES-GCM P). Include these bits in the computation of P length.																	

When operating in Skid mode, implementations can choose to maximize the benefits of latency optimization by decrypting and processing Slot 8 bytes 0 - 9, and Slots 9 - 14 as soon as they are received. Only MAC computation and decryption of Slot 8 bytes 10 - 11 needs to wait until Slot 14 is received. In such cases, implementation-specific mechanisms should exist to unwind IDE processing if CRC/FEC checks fail.

11.3.3 Encrypted PCRC

A polynomial with the coefficients 1EDC 6F41h shall be used for PCRC calculation. PCRC calculation shall begin with an initial value of FFFF FFFFh. The PCRC shall be calculated across all the bytes of plaintext in the aggregated flits that are part of the given MAC epoch. PCRC calculation shall begin with bit0 byte0 of flit plaintext content and sequentially include bits 0 - 7 for each byte of the flit contents that are mapped to the plaintext. After accumulating the 32-bit value across the flit contents, the PCRC value shall be finalized by taking 1's complement of the bits of accumulated value to obtain PCRC[31:0].

On the transmitter side (see Figure 11-15), the PCRC value shall be appended to the end of the aggregated flit plaintext content, encrypted, and then included in the MAC calculation. The encrypted PCRC value is not transmitted across the link.

On the receiver side (see Figure 11-16), the PCRC value shall be recalculated based on the received, decrypted ciphertext. When the last flit of the current MAC epoch has been processed, the accumulated PCRC value shall be XORed (encrypted) with the AES keystream bits that immediately follow the values that are used for decrypting the received cipher flit. This encrypted PCRC value shall be appended to the end of the received ciphertext for the purposes of MAC computation.

Figure 11-15. Inclusion of the PCRC Mechanism in the AES-GCM Advanced Encryption Function

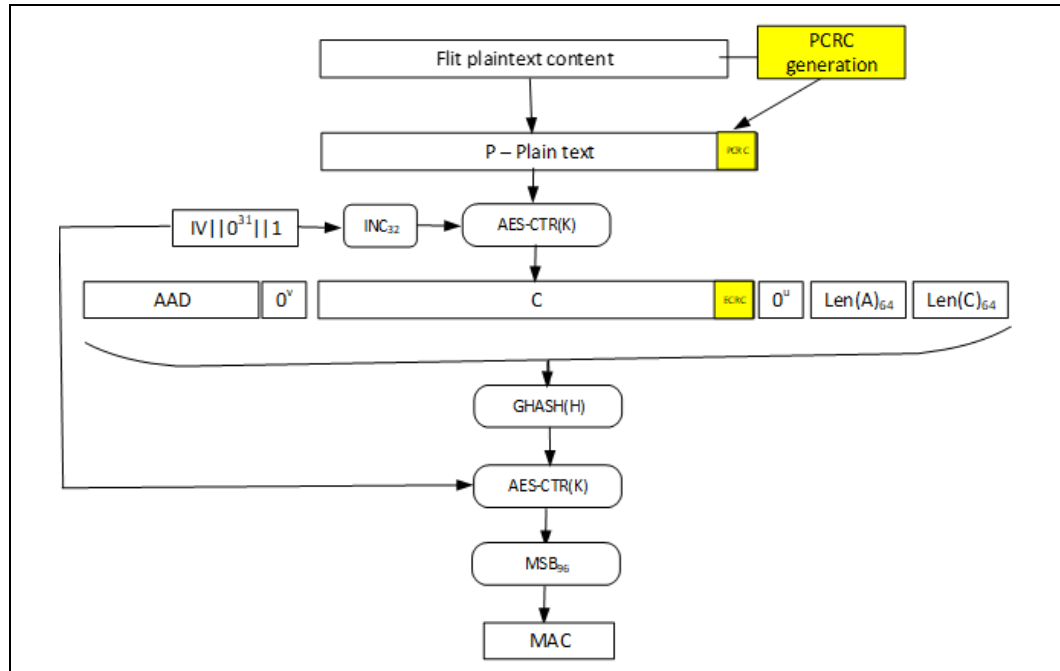
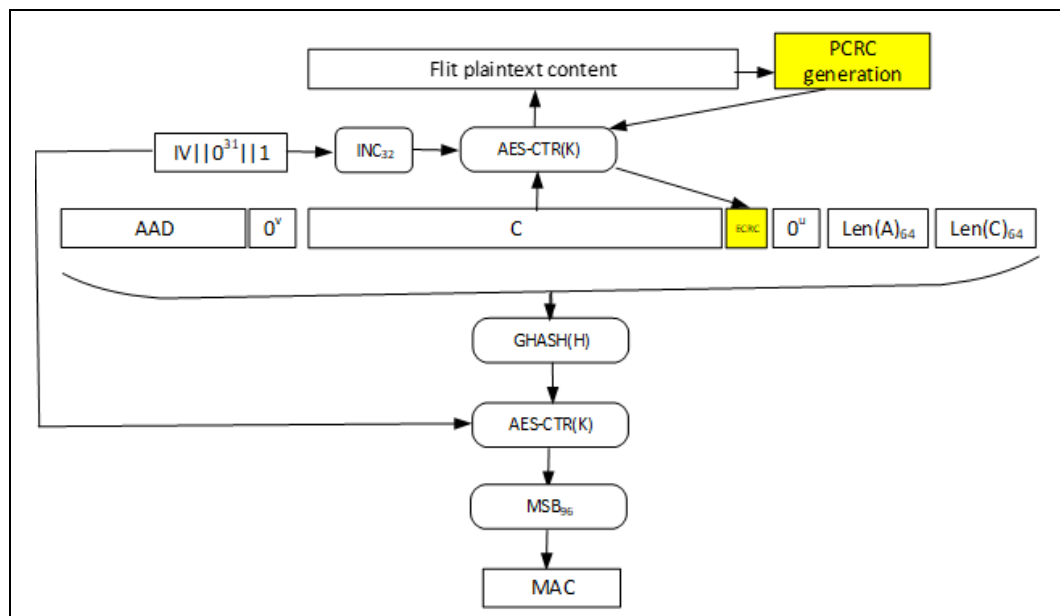


Figure 11-16. Inclusion of the PCRC Mechanism in the AES-GCM Advanced Decryption Function



11.3.4 Cryptographic Keys and IV

Initialization of a CXL.cachemem IDE Stream involves multiple steps. It is possible that some of these steps can be merged or performed in a different order. The first step is to establish the authenticity and identity of the components that contain the two ports that operate as endpoints for a CXL.cachemem IDE Stream. The second step is to establish the IDE Stream keys. In some cases, these two steps may be combined. Third, the IDE is configured. Finally, the establishment of the IDE Stream is triggered.

CXL.cachemem IDE may make use of CXL.io IDE mechanisms for device attestation and key exchange using a standard mechanism, as described in [Section 11.4](#).

IV construction of CXL.cachemem IDE is described below. A 96-bit IV of deterministic construction is used as per NIST Special Publication 800-38D for AES-GCM.

All ports shall support the Default IV Construction. The default IV construction is as follows:

- A fixed field is located at bits 95:64 of the IV, where bits 95:92 contain the sub-stream identifier, 1000b, and bits 91:64 are all 0s. The same sub-stream encoding is used for both transmitted and received flits; however, the keys that the port uses during transmit and receive flows must be distinct.
- Bits 63:0 of the IV are referred to as the invocation field. The invocation field contains a monotonically incrementing counter with rollover properties. The invocation field is initially set to the value 0000 0001h for each sub-stream upon establishment of the IDE Stream including a rekeying flow. If the CXL.cachemem IV Generation Capable bit in CXL_QUERY_RESP returns the value of 1, the port is capable of initially setting IV to a value other than what is generated via the Default IV Construction. See the CXL_KEY_PROG message definition (see [Section 11.4.5](#)) for details.

In either case, the invocation field is incremented every time an IV is consumed. Neither the transmitter nor the receiver are required to detect IV rollover¹ and are not required to take any special action when the IV rolls over.

11.3.5 CXL.cachemem IDE Modes

CXL.cachemem IDE supports two modes of operation:

- Containment mode: In Containment mode, the data is released for further processing only after the integrity check passes. This mode impacts both latency and bandwidth. The latency impact is due to the need to buffer several flits until the integrity value has been received and checked. The bandwidth impact comes from the fact that integrity value is sent quite frequently. If Containment mode is supported and enabled, the devices (and hosts) shall use an Aggregation Flit Count of 5 in 68B Flit mode and 2 in 256B Flit mode.
- Skid mode: Skid mode allows the data to be released for further processing without waiting for the integrity value to be received and checked. This allows for less-frequent transmission of the integrity value. Skid mode allows for near-zero latency overhead and low bandwidth overhead. In this mode, data modified by an adversary is potentially consumed by software; however, such an attack will subsequently be detected when the integrity value is received and checked. If Skid mode is supported and enabled, all devices (and hosts) shall use an Aggregation Flit Count of 128 in 68B Flit mode and of 32 in 256B Flit mode. When using this mode, the software and application stack must be capable of tolerating attacks within a narrow time window, or the result is undefined.

1. For a x16 link operating at 32 GT/s, a 64-bit IV will take longer than 1000 years to roll over.

11.3.5.1 Discovery of Integrity Modes and Settings

Each port shall enumerate the modes that the port supports and other capabilities via registers in the CXL IDE Capability Structure (see [Section 8.2.4.22](#)). All devices adherent to this specification shall support Containment mode.

11.3.5.2 Negotiation of Operating Mode and Settings

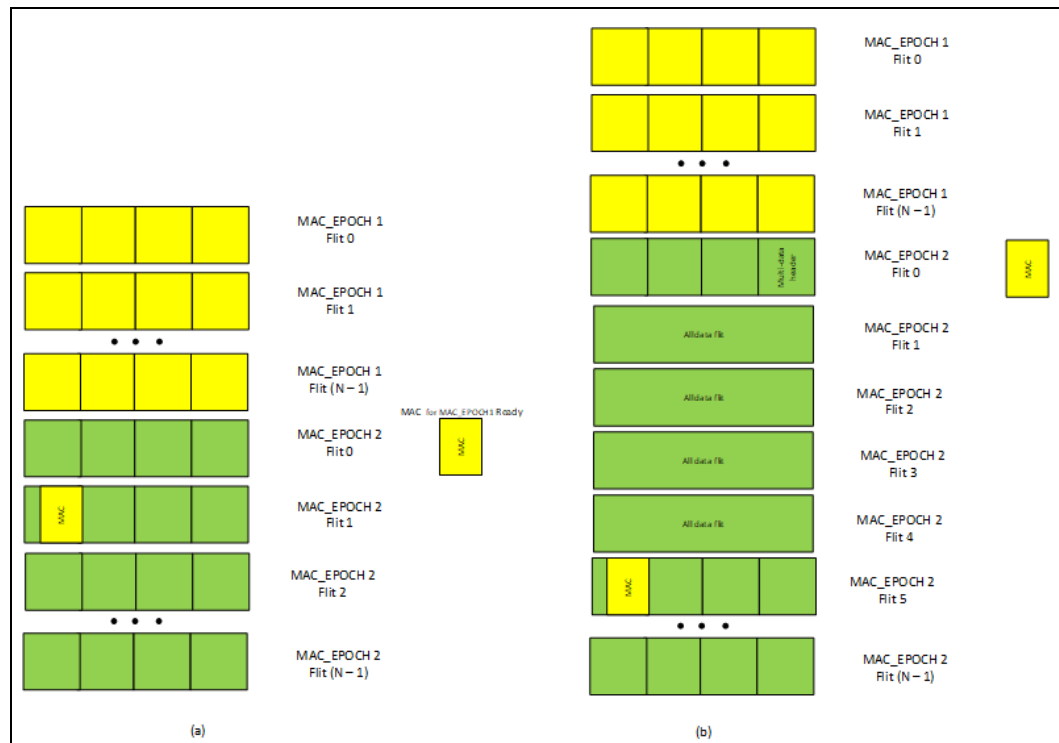
The operating mode and timing parameters are configured in the CXL IDE Capability Structure (see [Section 8.2.4.22](#)) prior to enabling of CXL.cachemem IDE.

11.3.5.3 Rules for MAC Aggregation

The rules for generation and transfer of MAC are as follows:

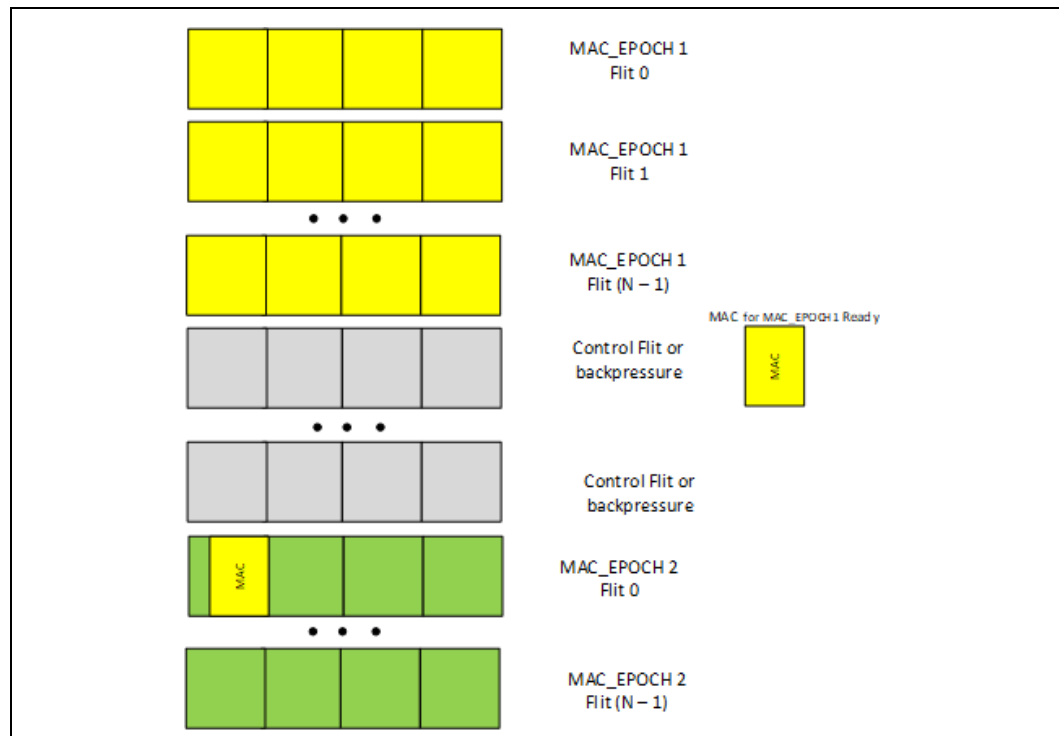
- MAC epoch: A MAC epoch is defined as the set of consecutive flits that are part of a given aggregation unit. The IDE mode (see [Section 11.3.5](#)) determines the number of flits in a standard MAC epoch. This number is known as Aggregation Flit Count (referred to as N below). Every MAC epoch with the exception of early MAC termination (see [Section 11.3.6](#)) carries N flits. A given MAC header shall contain the tag for exactly one MAC epoch. The transmitter shall accumulate the integrity value over flits in exactly one MAC epoch (that is at most N flits) prior to transmitting the MAC epoch.
- In all cases, the transmitter must send MACs in the same order as MAC epochs.
- [Figure 11-17](#) shows an example of MAC generation and transmission for one MAC epoch in the presence of back-to-back protocol traffic for the 68B flit format. [Figure 11-17](#) (a) shows that the earliest MAC may be transmitted, assuming that the transmitter completes MAC computation (and gets MAC ready) one cycle after the MAC epoch completes. The earliest flit to be transmitted or received is shown on the top of the figure. Thus, Flits 0 to N-1 (shown in yellow) belonging to MAC Epoch 1 are transmitted in that order. The MAC is calculated over Flits 0 to N-1.

Figure 11-17. MAC Epochs and MAC Transmission in Case of Back-to-Back Traffic
(a) Earliest MAC Header Transmit
(b) Latest MAC Header Transmit in the Presence of Multi-Data Header



- The transmitter shall send the MAC header that contains this integrity value at the earliest possible time. Protocol flits belonging to the next MAC epoch are permitted to be sent between the last flit of the current epoch and the transmission of the MAC header for the current epoch. This is needed to handle the transmission of all-data flits and is also useful for avoiding bandwidth bubbles due to MAC calculation latency. It is recommended that the transmitter send the MAC header on the first available Slot 0 header immediately after the MAC calculations are complete.
- On the receiver side, the receiver may expect the MAC header to come in on any protocol flit, from first to sixth protocol flits, after the last flit of the previous MAC epoch (see [Figure 11-17 \(b\)](#)).

Figure 11-18. Example of MAC Header Being Received in the First Flit of the Current MAC Epoch



- In Containment mode, the receiver must not release flits of a given MAC epoch for consumption until the MAC header that contains the integrity value for those flits has been received and the integrity check has passed. In 68B Flit mode, because the receiver can receive up to 5 protocol flits that belong to the current MAC epoch before receiving the MAC header for the previous MAC epoch, the receiver shall buffer the current MAC epoch's flits to ensure that there is no data loss. For example, referring to Figure 11-17 (b), both the yellow and green flits are buffered until MAC Epoch 1's MAC header is received and the integrity check passes. If the check passes, the yellow flits can be released for consumption. The green flits cannot, however, be released until the green MAC flit has been received and the integrity verified. Section 11.3.8 defines the receiver behavior upon integrity check failure.
- In Skid mode, the receiver may decrypt and release the flits for consumption as soon as they are received. The MAC value shall be accumulated as needed and then checked when the MAC header for the flits in the MAC epoch arrives. Again, referring to Figure 11-17 (b), both the yellow and green flits may be decrypted and released for consumption without waiting for the MAC header for MAC Epoch 1 to be received and verified. When MAC Epoch 1's MAC header is received, the header is verified. If the check passes, there is no action to be taken. If the MAC header is not received within 6 protocol flits after the end of the previous MAC epoch, the receiver shall treat the absence of MAC as an error. Section 11.3.8 defines the receiver behavior upon integrity check failure, a missing MAC header, or a delayed MAC header.
- In 68B Flit mode, in all cases (including the cases with multi-data headers), at most 5 protocol flits belonging to the current MAC epoch are allowed to be transmitted prior to the transmission of the MAC for the previous MAC epoch. If the MAC header is not received within 6 protocol flits after the end of the previous MAC epoch, the receiver shall treat the absence of MAC as an error.

- In 256B Flit mode, in all cases, at most 1 protocol flit that belongs to the current MAC epoch is allowed to be transmitted prior to the transmission of the MAC for the previous MAC epoch. If the MAC header is not received within 2 protocol flits after the end of the previous MAC epoch, the receiver shall treat the absence of MAC as an error.

IMPLEMENTATION NOTE

In Containment mode, the receiver must not release any decrypted flits for consumption unless their associated MAC check has been performed and passed. This complies with the algorithm for the AES-GCM Authenticated Decryption Function as defined in NIST Special Publication 800-38D.

In Skid mode, the receiver is permitted to release any decrypted flits for consumption without waiting for their associated MAC check to be performed. Unless there are additional device-specific mechanisms to prevent this consumption, the use of Skid mode will not meet the requirements of the above-mentioned algorithm.

Solution stack designers must carefully weigh the benefits vs. the constraints when choosing between Containment mode and Skid mode. Containment mode guarantees that potentially corrupted data will not be consumed. Skid mode provides data privacy and eventual detection of data integrity loss, with significantly less latency impact and link-bandwidth loss compared to Containment mode. However, the use of Skid mode may be more vulnerable to security attacks and will require additional device-specific mechanisms if it is necessary to prevent corrupt data from being consumed.

11.3.6 Early MAC Termination

A transmitter is permitted to terminate the MAC epoch early and transmit the MAC for the flits in a truncated MAC epoch when fewer than the Aggregation Flit Count of flits have been transmitted in the current MAC epoch. This can occur as part of link idle handling. The link may be ready to go idle after the transmission of a number of protocol flits, less than the Aggregation Flit Count, in the current MAC epoch.

The following rules shall apply to the early MAC epoch termination and the MAC transmission.

- The transmitter is permitted to terminate the MAC epoch early if and only if the number of protocol flits in the current MAC epoch is less than Aggregation Flit Count. The MAC for this truncated MAC epoch shall be transmitted by itself in the IDE.TMAC Link Layer Control flit (see [Table 4-10](#)). This subtype is referred to as a Truncated MAC flit within this specification.
- Any subsequent protocol flits would become part of a new MAC epoch and would be transmitted after the Truncated MAC flit.
- The MAC for the truncated MAC epoch is calculated identically to the MAC calculation for normal cases, except that it is accumulated over fewer flits.

[Figure 11-20](#) shows an example of truncating the current MAC epoch after 3 protocol flits. Flits in current MAC epoch can contain any valid protocol flit including a header flit that contains the MAC for the previous MAC epoch. The MAC for the current MAC epoch shall be sent using a Truncated MAC flit. The Truncated MAC flit will be transmitted following the three protocol flits of the current MAC epoch with no other intervening protocol flits from the next MAC epoch.

Figure 11-19. Early Termination and Transmission of Truncated MAC Flit

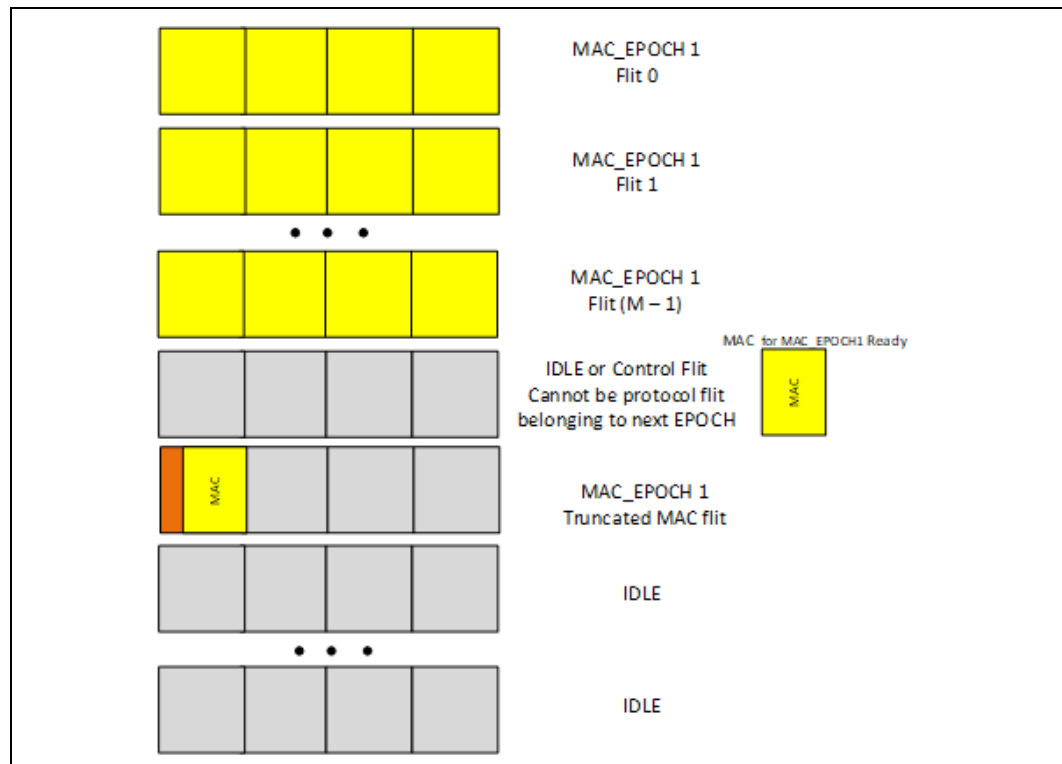
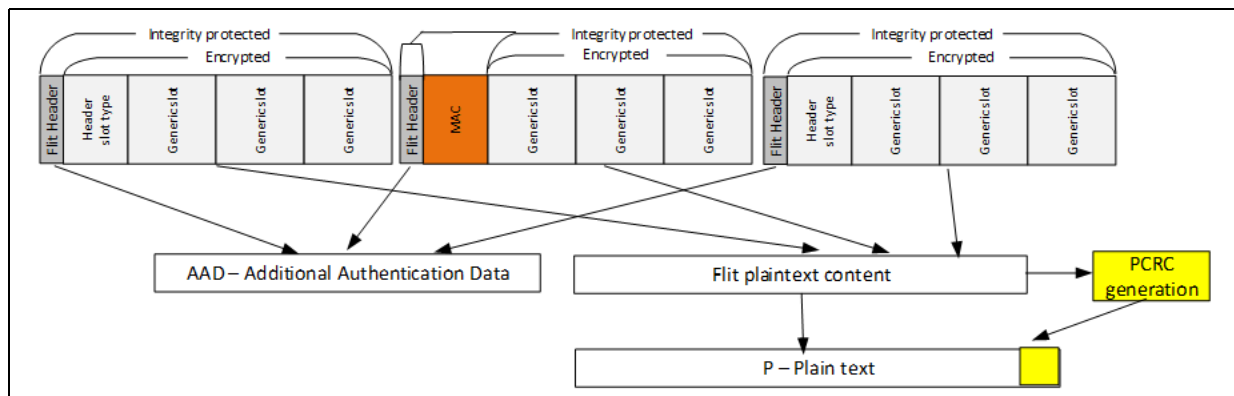
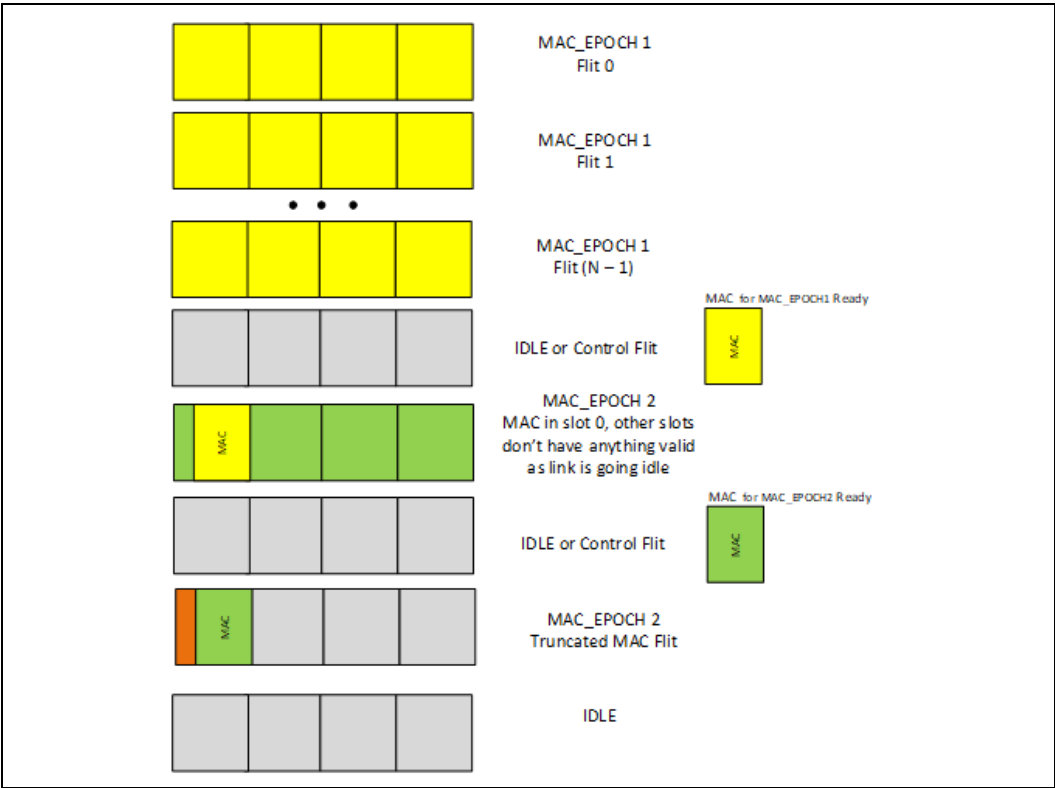


Figure 11-20. CXL.cachemem IDE Transmission with Truncated MAC Flit



In the case where the link goes idle after sending exactly the Aggregation Flit Count number of flits in the MAC epoch, then the Truncated MAC flit as defined above must not be used. The MAC header must be part of the next MAC epoch. This new MAC epoch is permitted to be terminated early using the Truncated MAC flit (see [Figure 11-21](#)).

Figure 11-21. Link Idle Case after Transmission of Aggregation Flit Count Number of Flits



After the transmitter sends out the MAC flit for all the previous flits that were in flight, the transmitter may go idle. The receiver is permitted to go idle after the MAC flit that corresponds to previously received flits has been received and verified. IDE.Idle control flits are retryable and may be resent as part of replay.

After early MAC termination and transmittal of the Truncated MAC, the transmitter must send at least TruncationDelay number of IDE.Idle flits before it can transmit any protocol flits. TruncationDelay is defined via the following equation:

Equation 11-1.

$$\text{TruncationDelay} = \text{Min}(\text{Remaining Flits}, \text{Tx Truncation Transmit Delay})$$

Tx Truncation Transmit Delay (see [Section 8.2.4.22.8](#)) is a configuration parameter to account for the potential discarding of any precalculated AES keystream values for the current MAC epoch that need to be discarded. Remaining Flits represent the number of flits remaining to complete the current MAC epoch and is calculated as follows:

Equation 11-2.

$$\text{Remaining Flits} = \text{Aggregation Flit Count} - \text{Number of protocol flits transmitted in current MAC epoch}$$

11.3.7 Handshake to Trigger the Use of Keys

Each port exposes a register interface that software can use to program the transmit and receive keys and their associated parameters. These programmed keys remain pending in registers until activation. While the keys are in the process of being exchanged and configured in the Upstream and Downstream Ports, the link may actively be using a previously configured key. The new keys shall not take effect until the actions described below are taken.

The mechanism described below is used to switch the backup keys to the active state. This is needed to ensure that the Transmitter and Receiver change to using the programmed keys in a coordinated manner.

After the keys are programmed into pending registers on both sides of the link, receipt of the CXL_K_SET_GO request shall cause each transmitter on each port to trigger the transmission of an IDE.Start Link Layer Control flit (see [Table 4-3](#)).

After the IDE.Start flit has been sent, all future protocol flits shall be protected by the new keys. To allow the receiver to prepare to receive the flits protected by the new key, the Transmitter is required to send IDE.Idle flits, as defined in [Table 4-10](#) for the number of flits specified by the Tx Key Refresh Time field in the Key Refresh Time Control register (see [Section 8.2.4.22.7](#)) prior to sending any protocol flits with the new key. These IDE.Idle flits are not encrypted or integrity protected. To prepare to use the new keys, the Tx Key Refresh Time in the transmitter must be configured to a value that is higher than the worst-case latency in the receiver, which is advertised by the receiver via Rx Min Key Refresh Time field in the Key Refresh Time Capability register (see [Section 8.2.4.22.5](#)) or Rx Min Key Refresh Time2 field in the Key Refresh Time Capability2 register (see [Section 8.2.4.22.9](#)), depending on the Flit mode.

After receiving the IDE.Start flit, the receiver must change to using the new keys if the transmitter has met the AES-GCM requirements. During key refresh, it is recommended that the transmitter send an IDE.TMAC before sending an IDE.Start.

It is also permissible for the transmitter to send an IDE.Start after the MAC epoch ends but before the corresponding MAC header is transmitted. In this scenario, the receiver must use the old keys to decrypt the message and to check the MAC.

The transmitter must not send an IDE.Start in the middle of a MAC epoch because doing so violates the fundamental AES-GCM requirement that a single key be used as the input. If the IDE.Start is received in the middle of a MAC epoch, then the receiver shall drop the IDE.Start. The receiver may also set the Rx Error Status field in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) to CXL.cachemem IDE Establishment Security error and may transition to Insecure State upon detecting this condition.

The IDE.Start flit shall be ordered with respect to the protocol flits. In case of link-level retries, the receiver shall complete retries of previously sent protocol flits before handling the IDE.Start flit and changing to the new key. Other events such as link retraining can occur in the middle of this flow as long as the ordering specified above is maintained.

11.3.8 Error Handling

CXL IDE does not impact or require any changes to the link CRC error handling and the link retry flow.

CXL.cachemem IDE error conditions are enumerated and logged in the Rx Error Status field, Tx Error Status or Unexpected IDE.Stop received fields in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)). When a CXL.cachemem IDE error is detected,

the appropriate bits in the Uncorrectable Error Status register (see [Section 8.2.4.17.1](#)) are also set and the error is signaled using the standard CXL.cachemem protocol error signaling mechanism.

Unless stated otherwise, errors logged in Rx Error Status field or Tx Error Status field cause the CXL.cachemem IDE stream to transition from Active State to Insecure State if it is Active at the time of the error. Note that some of the error conditions that are logged under CXL.cachemem IDE Establishment may not always result in termination of CXL.cachemem IDE stream.

Upon transition to Insecure state:

- Any buffered protocol flits are dropped and all subsequent protocol traffic is dropped until the link is reset.
- Components shall prevent any leakage of keys or user data. The component may need to implement mechanisms to clear the data/state or have access control to prevent leakage of secrets. Such mechanisms and actions are component specific and beyond the scope of this specification.

11.3.9 Switch Support

A CXL switch that supports CXL.cachemem IDE must support Link IDE for CXL.io traffic. A CXL switch may also optionally support Selective Stream IDE for CXL.io traffic, including flow-through Selective IDE Streams. A CXL switch may only support Selective Stream IDE in flow-through mode for CXL.io traffic. In this case, CXL.cachemem IDE cannot be enabled on the host side. In the case of multi-VCS capable switches, CXL IDE may be enabled on a per-root port basis. However, after any root port has enabled CXL IDE, the downstream link from the switch to the MLDs that support CXL IDE, must also have Link IDE enabled. Thus, the traffic from a root port which has not enabled CXL IDE that is targeting an MLD that has enabled CXL IDE would be encrypted and integrity protected between the switch and the device.

IMPLEMENTATION NOTE: IDE CONFIGURATION OF CXL SWITCHES

The following examples describe three different models for configuring the CXL.cachemem IDE and performing key exchanges with the CXL switches and the devices attached to them.

Model A

Host performs key exchange with the CXL switch and enables CXL IDE. The host will then enumerate the Downstream Ports in the CXL switch and perform key exchange with those downstream devices that support CXL IDE. The Host then programs the keys into the respective Downstream Ports of the switch and enables CXL IDE.

Model B

Host performs key exchange with the CXL switch and enables CXL IDE. In parallel, CXL switch will enumerate downstream devices and then perform key exchange with those downstream devices that support CXL IDE. The Switch then programs the keys into the respective Downstream Ports of the switch and enables CXL IDE. Host may obtain a report from the CXL switch regarding the enabling of CXL IDE for downstream devices, which includes information about the public key that was used to attest to the device EP. Host may directly obtain an attestation from the device Endpoint and confirm that the Endpoint in question has the same public key that the Switch used as part of the key exchange.

Model C

An out-of-band agent may configure keys into the host, switch, and devices via out-of-band means and then directly enable CXL IDE.

11.3.10 IDE Termination Handshake

This section describes a mechanism that disables IDE on both the transmitter and receiver. This is accomplished via IDE.Stop control flit (see [Table 4-20](#)). This optional capability for 256B Flit mode simplifies the software synchronization and quiescing requirements. This ensures that the transmitter and receiver disable CXL.cachemem IDE in a coordinated manner.

After IDE is enabled and functional, receipt of a CXL_K_SET_STOP request shall cause each transmitter on each IDE.Stop capable port to trigger the transmission of an IDE.Stop Link Layer Control flit (see [Table 4-20](#)) if enabled by programming CXL IDE Control register (see [Section 8.2.4.22.2](#)). The transmitter shall ensure that the currently active MAC epoch is terminated using an IDE.TMAC prior to sending an IDE.Stop message with no intervening protocol flits. IDE.TMAC sent before IDE.Stop shall follow the standard rules for early MAC termination defined in [Section 11.3.6](#). If a valid TMAC sequence is not received before IDE.Stop, the IDE.Stop shall be dropped and Unexpected IDE.Stop received bit in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) shall be set.

After the IDE.Stop is sent, all future protocol flits shall not be IDE protected. To allow the receiver to cleanly clear any pending IDE states, including precomputed information, the transmitter is required to send IDE.Idle flits, as defined in [Table 4-10](#), for the number of flits specified by the Tx Key Refresh Time field in the Key Refresh Time Control register (see [Section 8.2.4.22.7](#)) prior to sending any protocol flits without IDE protection.

After receiving an IDE.Stop flit, the receiver must complete all pending actions for the currently active MAC epoch prior to disabling IDE.

Any IDE.Stop message that is received prior to a successful CXL_K_SET_STOP shall be dropped and the Unexpected IDE.Stop received bit in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) shall be set.

If the IDE.Stop is received by a receiver that is IDE.Stop Capable but is not configured to process IDE.Stop, it shall drop the IDE.Stop flit and the Unexpected IDE.Stop received bit in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) shall be set. If the Rx port receives an IDE.Stop while the IDE stream is inactive, the Rx port shall drop the IDE.Stop flit and set the Unexpected IDE.Stop received bit in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)).

11.4 CXL.cachemem IDE Key Management (CXL_IDE_KM)

System software or system firmware may follow this specification to configure the ports at both ends of a CXL link that have matching CXL.cachemem IDE keys, Initial IV, and other settings, in an interoperable way. The software or firmware entity that performs this activity is referred to as CXL.cachemem IDE Key Management Agent (CIKMA).

The port pairs, also called the partner ports, may consist of the following:

- A CXL RP and a CXL USP
- A CXL RP and a CXL EP
- A CXL DSP and a CXL EP

CXL root port CXL.cachemem IDE key programming may be performed via host-specific method and may not use the programming steps described in this section.

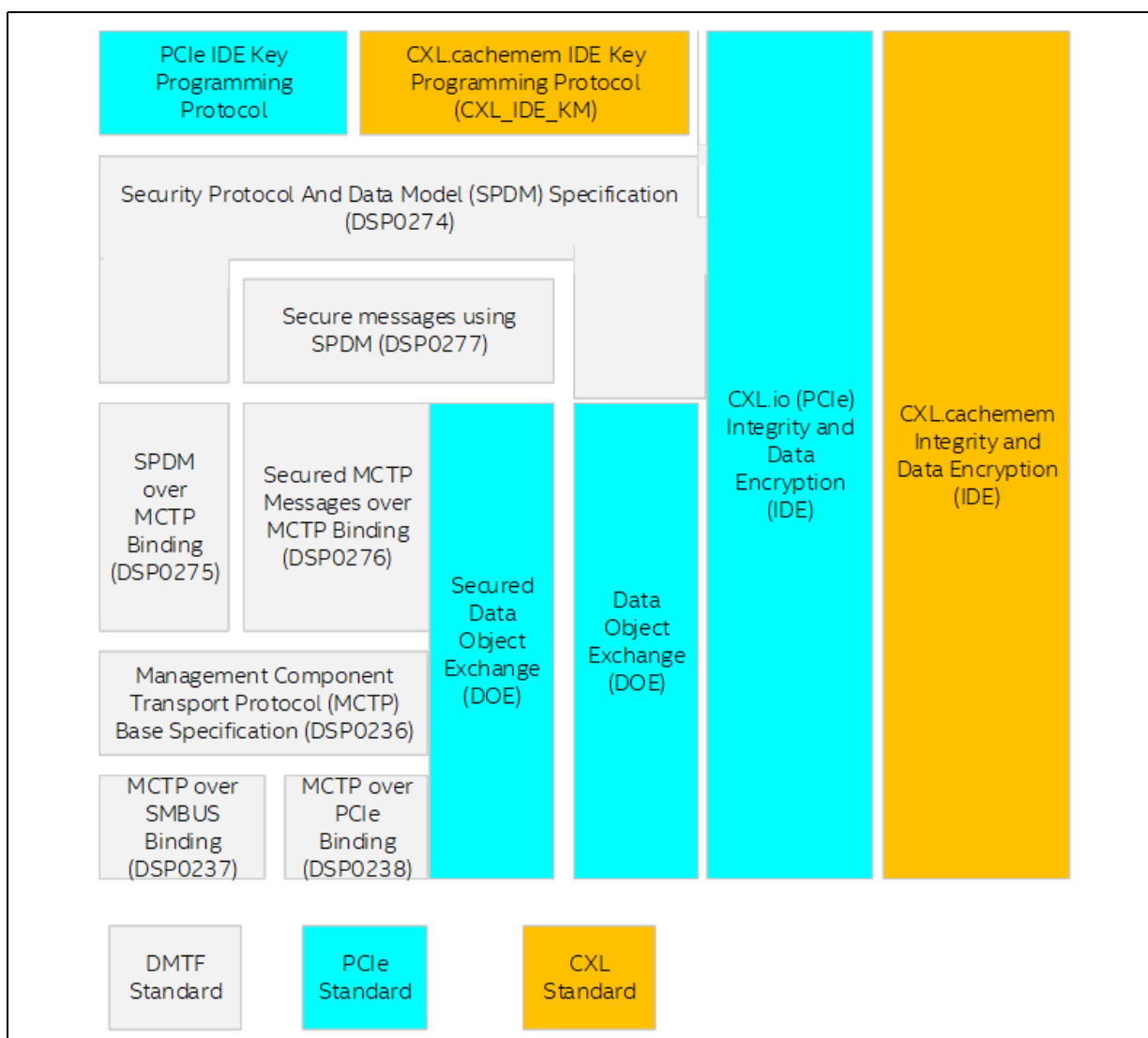
The CXL.cachemem IDE Establishment flow consists of three major steps:

1. CIKMA reads CXL IDE capability registers on both ends of the CXL link and configures various CXL.cachemem IDE control registers on both ends of the CXL

link. See [Section 8.2.4.21](#) for definition of these registers and the programming guidelines.

2. CIKMA sets up an SPDm secure session with each of the partner ports that are being set up. This is accomplished by issuing SPDm key exchange messages over transports such as PCIe DOE or MCTP. If one of the partner ports is an RP and the RP supports a proprietary IDE programming flow, an SPDm secure session with RP may not be needed.
3. CIKMA queries port capabilities, optionally obtains locally generated key and IV from each port if they are capable, configures CXL.cachemem IDE Rx/Tx keys/IV, and enables CXL.cachemem IDE using CXL_IDE_KM messages that are defined in [Section 11.4.1](#). These messages are secured using the SPDm session key that was established by CIKMA via the previous step.

Figure 11-22. Various Interface Standards that are Referenced by this Specification and their Lineage



11.4.1 CXL_IDE_KM Protocol Overview

CXL_IDE_KM Messages are constructed as SPDML vendor-defined requests and SPDML vendor-defined responses. All request messages begin with a standard Request Header (see [Table 11-2](#)) and all response messages carry a standard Response Header (see [Table 11-3](#)). For the definition of individual fields in the Request and Response Header, please refer to DSP0274. Unless specified otherwise, the behaviors specified in DSP0236, DSP0237, DSP0238, DSP0274, DSP0275, DSP0276, DSP0277, and PCIe Base Specification apply.

CXL_IDE_KM Messages shall be confidentiality and integrity protected in accordance with DSP0277. These secured messages may be sent over a variety of transports, including Secured CMA/SPDM Messages over DOE (see PCIe Base Specification) or Secured Messages over MCTP (see DSP0276).

All CXL.cachemem IDE-capable CXL Switches and Endpoints shall support CMA/SPDM and Secured CMA/SPDM Data Object types over PCIe DOE mailbox. The specific rules regarding the placement of the DOE mailbox are governed by PCIe Base Specification. These data object types are defined in PCIe Base Specification. All CXL.cachemem IDE-capable switches and devices shall support CXL_IDE_KM protocol and CXL_IDE_KM being sent as Secured CMA/SPDM Data Objects.

CXL.cachemem IDE-capable switches and devices may optionally support CXL_IDE_KM messages over MCTP.

The maximum amount of time that the Responder has to provide a response to a CXL_IDE_KM request is 1 second. The requester shall wait for 1 second plus the transport-specific, round-trip transport delay prior to concluding that the request resulted in an error.

11.4.2 Secure Messaging Layer Rules

CXL_IDE_KM messages shall not be issued before an SPDML secure session has been established between the two ports. Any CXL_IDE_KM messages that are not secured shall be silently dropped by the receiver. The first CXL_IDE_KM request message after the SPDML secure session setup shall be CXL_QUERY.

After a successful response to CXL_QUERY, this SPDML session may be used to establish a CXL.cachemem IDE Stream. While this SPDML Session is in progress, any CXL_IDE_KM messages received using a different session ID shall be silently dropped and shall not generate a CXL_IDE_KM response. Any CXL_IDE_KM messages that fail integrity check shall be silently dropped and shall not generate a CXL_IDE_KM response. The act of terminating this SPDML Session or establishment of a different SPDML Secure session by themselves shall not affect the state of the CXL.cachemem IDE stream.

If SPDML Session S1 is used to establish a CXL.cachemem IDE Stream I1, termination of SPDML Session S1 followed by receipt of any valid CXL_IDE_KM message with a new Session S2 shall transition CXL.cachemem IDE Stream I1 to Insecure State. The transition shall occur prior to processing the newly received CXL_IDE_KM message unless the receiver can ensure, via mechanisms not defined here, that S1 and S2 were set up by the same entity; otherwise, the receiver drops the CXL_IDE_KM message with a new Session S2. If the CXL.cachemem IDE stream enters Insecure State due to this condition, the receiver shall set the Rx Error Status field in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) to CXL.cachemem IDE Establishment Security error.

It is permitted for a single DOE mailbox instance be used to service CXL_IDE_KM messages as well as CXL.io IDE_KM messages. It is permitted for a single SPDML session be used to set up CXL.io IDE stream as well as CXL.cachemem IDE stream with

a component. The operation and the establishment of CXL.cachemem IDE stream is independent of the operation and establishment of CXL.io IDE stream. It is permitted for a component to support CXL.io IDE but not CXL.cachemem IDE, and vice versa. If a component supports both CXL.io IDE and CXL.cachemem IDE, it may be operated in a mode where only one of the two is active. It is permitted for CXL_IDE_KM messages to be interleaved with IDE_KM messages. CIKMA shall ensure there is at most one outstanding SPDM request of any kind at any time in accordance with DSP0274.

11.4.3 CXL_IDE_KM Common Data Structures

For consistency and reuse reasons, the names of the individual messages follow PCIe Base Specification except for the addition of the prefix CXL, and the message contents closely match PCIe Base Specification.

Unless specified otherwise, all fields are defined as little-endian.

Please refer to DSP0274 for definitions of the fields in the CXL_IDE_KM Request header and Response header.

Table 11-2. CXL_IDE_KM Request Header

Byte Offset	Length in Bytes	Description
0h	1	SPDMVersion
1h	1	RequestResponseCode: Value is 0FEh (VENDOR_DEFINED_REQUEST).
2h	1	Reserved
3h	1	Reserved
4h	2	StandardsID: Value is 03h (PCI-SIG*), indicating that the Vendor ID is assigned by the PCI-SIG.
6h	1	Length of Vendor ID: Value is 02h.
7h	2	Vendor ID: Value is 1E98h (CXL).
9h	2	Request Length: The number of bytes in the message that follow this field. Varies based on the operation that is being requested.

Table 11-3. CXL_IDE_KM Successful Response Header

Byte Offset	Length in Bytes	Description
0h	1	SPDMVersion
1h	1	RequestResponseCode: Value is 07Eh (VENDOR_DEFINED_RESPONSE).
2h	1	Reserved
3h	1	Reserved
4h	2	StandardsID: Value is 03h (PCI-SIG), indicating that the Vendor ID is assigned by the PCI-SIG.
6h	1	Length of Vendor ID: Value is 02h.
7h	2	Vendor ID: Value is 1E98h (CXL).
9h	2	Response Length: The number of bytes in the message that follow this field. Varies based on the operation that was requested.

Table 11-4 lists the various generic error conditions that a responder may encounter during the processing of CXL_IDE_KM messages and how the conditions are handled.

Table 11-4. CXL_IDE_KM Generic Error Conditions

Error Condition	Response	Effect on an Active CXL.cachemem IDE Stream
CXL_IDE_KM message carries an Object ID that is not defined in this specification	No response is generated. The request is silently dropped.	No change
Unrecognized SPDMA major version		

11.4.4 Discovery Messages

The CXL_QUERY request is used to discover the CXL.cachemem IDE capabilities and the current configuration of a port. The port supplies this information in the form of CXL_QUERY_RESP response. CIKMA shall not issue another type of CXL_IDE_KM request after CXL_QUERY until CIKMA has received a successful CXL_QUERY_RESP response. If CXL_QUERY request is not successful, CIKMA is permitted to retry it.

CIKMA may cross-check the CXL IDE Capability Structure contents that are returned by CXL_QUERY_RESP against the component's CXL IDE Capability Structure register values. CIKMA shall abort the CXL.cachemem IDE Establishment flow if CIKMA detects a mismatch.

Table 11-5. CXL_QUERY Request

Byte Offset	Length in Bytes	Description
0h	Bh	Standard Request Header: See Table 11-2.
Bh	1	Protocol ID: Value is 0.
Ch	1	Object ID: Value is 0, indicating CXL_QUERY request.
Dh	1	Reserved
Eh	1	PortIndex: See PCIe Base Specification.

Table 11-6 lists the various error conditions that a responder may encounter that are unique to CXL_QUERY and how the conditions are handled.

Table 11-6. CXL_QUERY Processing Errors

Error Condition	Response	Effect on an Active CXL.cachemem IDE Stream
Protocol ID is nonzero	No response is generated. The request is silently dropped.	No change
Invalid Request Length		
PortIndex does not correspond to a valid port		

Table 11-7. Successful CXL_QUERY_RESP Response (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
00h	Bh	Standard Response Header: See Table 11-3.
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 1, indicating CXL_QUERY response.
0Dh	1	Reserved

Table 11-7. Successful CXL_QUERY_RESP Response (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
0Eh	1	PortIndex: See PCIe Base Specification.
0Fh	1	Dev/Fun Number: See PCIe Base Specification.
10h	1	Bus Number: See PCIe Base Specification.
11h	1	Segment: See PCIe Base Specification.
12h	1	MaxPortIndex: See PCIe Base Specification.
13h	1	- Bits[3:0]: CXL IDE Capability Version: Must be set to 1. See Section 8.2.4.7. - Bit[4]: CXL.cachemem IV Generation Capable: 0 = Port is not capable of locally generating the 96-bit IV and shall always use the default IV construction. 1 = Port is capable of locally generating the 96-bit IV. If a CXL_KEY_PROG message indicates Use Default IV=0, the port shall use the Locally generated CXL.cachemem IV in the Tx path and shall use the IV value supplied as part of the CXL_KEY_PROG message in the Rx path. The port shall return the Locally generated CXL.cachemem IV as part of CXL_GETKEY_ACK response. - Bit[5]: CXL.cachemem IDE Key Generation Capable: 0 = Port is not capable of locally generating an IDE key. 1 = Port is capable of locally generating the IDE key, shall use that key in the Tx path, and then return that key as part of the CXL_GETKEY_ACK response. - Bit[6]: CXL_K_SET_STOP Capable: 0 = Port does not support CXL_K_SET_STOP. 1 = Port supports CXL_K_SET_STOP. - Bit[7]: Reserved
14h	Varies	CXL IDE Capability Structure: For CXL IDE Capability Version=1, the length shall be 20h. Carries the contents of the CXL IDE Capability Structure of this port (see Section 8.2.4.21).

11.4.5 Key Programming Messages

Each CXL.cachemem IDE-capable port shall be capable of storing four keys - Rx active, Tx active, Rx pending, and Tx pending. If CXL.cachemem IDE is active, the Tx active key is used to encrypt the flits and generate the MAC. If CXL.cachemem IDE is active, the Rx active key is used to decrypt the flits and verify the MAC in the Rx direction. This specification does not define a mechanism for directly updating the active keys. A Conventional Reset shall reset the active CXL.cachemem IDE Stream and transition the stream to Insecure State. A CXL reset shall reset the active CXL.cachemem IDE Stream and transition the stream to Insecure State. Transition of the CXL.cachemem IDE session to Insecure State shall clear all the keys, make the keys unreadable, and then mark the keys as invalid. An FLR shall not affect an active CXL.cachemem IDE Stream or the CXL.cachemem IDE keys.

The CXL_KEY_PROG request is used to supply the pending keys. Offset 11h, Bit 1, is used to select between the Rx and the Tx. If CXL.cachemem IV Generation Capable=1, the CXL_KEY_PROG request may also be used to establish the Initial CXL.cachemem IDE IV value to be used with the new IDE session including the rekeying flow.

If both ports (Port1 and Port2) return CXL.cachemem IV Generation Capable=1 in QUERY_RSP, it is recommended that CIKMA issue a CXL_GETKEY request to both ports and obtain Locally generated CXL.cachemem IV values. When issuing a CXL_KEY_PROG message to Port1 Rx and Port2 Tx, CIKMA should initialize the Initial CXL.cachemem IDE IV field (Offset 13h+KSIZE) to match the Port2 Locally generated CXL.cachemem IV and set Default IV=0. When issuing a CXL_KEY_PROG message to Port1 Tx and Port2 Rx, CIKMA should initialize the Initial CXL.cachemem IDE IV field (Offset 13h+KSIZE) to match the Port1 Locally generated CXL.cachemem IV and set Default IV=0.

If either port returns CXL.cachemem IV Generation Capable=0 in QUERY_RSP, CIKMA should set Use Default IV=1 in the CXL_KEY_PROG messages to both ports to indicate that the ports should use the default IV construction in Rx directions and Tx directions.

If Port1 and Port2 are partner ports and if Port1 returns CXL.cachemem IDE Key Generation Capable=1 in QUERY_RSP, it is recommended that CIKMA issue a CXL_GETKEY request to Port1 and obtain its Locally generated CXL.cachemem IDE Key. When issuing the CXL_KEY_PROG message to Port1 Tx and Port2 Rx, CIKMA should initialize the CXL.cachemem IDE Key field to match the Port1 Locally generated CXL.cachemem IDE Key. If Port2 returns CXL.cachemem IDE Key Generation Capable=1 in QUERY_RSP, it is recommended that CIKMA issue a CXL_GETKEY request to Port2 and obtain its Locally generated CXL.cachemem IDE Key. When issuing a CXL_KEY_PROG message to Port2 Tx and Port1 Rx, CIKMA should initialize the CXL.cachemem IDE Key field to match the Port2 Locally generated CXL.cachemem IDE Key. The port is expected to return a different IDE key during every CXL_GETKEY_ACK response. Therefore, CIKMA should ensure that the CXL.cachemem IDE Key supplied during a CXL_KEY_PROG request matches the locally generated CXL.cachemem IDE Key from the previous CXL_GETKEY_ACK responses from that port.

Table 11-8. CXL_KEY_PROG Request

Byte Offset	Length in Bytes	Description
00h	Bh	Standard Request Header: See Table 11-2.
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 2, indicating CXL_KEY_PROG request.
0Dh	2	Reserved
0Fh	1	Stream ID: Value is 0.
10h	1	Reserved
11h	1	- Bit[0]: Reserved - Bit[1]: RxTxB: 0 = Rx 1 = Tx - Bit[2]: Reserved - Bit[3]: Use Default IV: 0 = Port shall use the Initial IV specified at Offset 13h+KSIZE 1 = Port shall use the Default IV construction - Bits[7:4]: Key Sub-stream: Value is 1000b.
12h	1	PortIndex: See PCIe Base Specification.
13h	KSIZE	CXL.cachemem IDE Key: Program the Pending Key with this value. KSIZE must be 32 for this version of the specification. For layout, see PCIe Base Specification.
13h+KSIZE	12h	Initial CXL.cachemem IDE IV: Overwrites the Pending Initial IV. This field must be ignored if Use Default IV=1. Byte Offsets 16h+KSIZE:13h+KSIZE carry the IV DWORD, IV[95:64]. Byte Offsets 20h+KSIZE:17h+KSIZE carry the IV DWORD, IV[63:32]. Byte Offsets 24h+KSIZE:21h+KSIZE carry the IV DWORD, IV[31:0].

Table 11-9 lists the various error conditions that a responder may encounter that are unique to CXL_KEY_PROG and how the conditions are handled. When these conditions are detected, the responder shall respond with a CXL_KP_ACK and set the Status field to a nonzero value.

Table 11-9. CXL_KEY_PROG Processing Errors

Error Condition	Response	Effect on an Active CXL.cachemem IDE Stream
Invalid Request Length	Do not update the key and IV. Return CXL_KP_ACK with Status=01h.	No change
PortIndex does not correspond to a valid port		
Protocol ID is nonzero		
Stream ID is nonzero		
Key Sub-stream is not 1000b		
CXL_KEY_PROG received prior to CXL_QUERY		
Request to set the Tx Key, but the input Tx key is identical to the current Rx Pending Key. This check is optional.		
Request to set the Rx Key, but the input Rx key is identical to the current Tx Pending Key. This check is optional.		
Request to program the key failed because the pending key slot has a valid key		
Request to update Tx key, but the supplied key does not match the locally generated CXL.cachemem IDE key returned during the last CXL_GETKEY_ACK response.		
Request to update Tx IV, but the supplied IV does not match the Locally generated CXL.cachemem IV returned during the last CXL_GETKEY_ACK response. The port returned IV Generation Capable=0 in QUERY_RSP, but Use Default IV in CXL_KEY_PROG was not set.		

Upon successful processing of CXL_KEY_PROG, the responder shall acknowledge by sending the CXL_KP_ACK response with Status=0. The nonzero Status values not listed here are reserved by this specification but should be interpreted as an error condition by the requester.

Table 11-10. CXL_KP_ACK Response

Byte Offset	Length in Bytes	Description
00h	Bh	Standard Response Header: See Table 11-3.
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 3, indicating CXL_KP_ACK response.
0Dh	2	Reserved
0Fh	1	Stream ID: Value is 0.
10h	1	Status: See Table 11-9.
11h	1	- Bit[0]: Reserved - Bit[1]: RxTxB: See PCIe Base Specification - Bits[3:2]: Reserved - Bits[7:4]: Key Sub-stream: Value is 1000b
12h	1	PortIndex: See PCIe Base Specification.

11.4.6 Activation/Key Refresh Messages

The CXL_K_SET_GO request is used to prepare an Rx port for a CXL.cachemem IDE Stream. The port shall respond with a CXL_K_GOSTOP_ACK message to indicate that the port is ready.

The CXL_K_SET_GO request is also used to instruct a Tx port to generate an IDE.Start Link Layer Control flit and to start a CXL.cachemem IDE Stream that is protected with the pending Tx key as outlined in [Section 11.3.7](#). As part of successful CXL_K_SET_GO processing, the Tx port shall copy the pending key to be the active key and mark the pending key slot as invalid. If CXL.cachemem IV Generation Capable=1 and the last CXL_KEY_PROG request indicated Use Default IV=0, the Initial CXL.cachemem IDE IV shall also be re-initialized to the value supplied as part of the CXL_KEY_PROG request. If CXL.cachemem IV Generation Capable=0 or the last CXL_KEY_PROG request indicated Use Default IV=1, Default IV construction shall be used. All subsequent protocol flits shall be protected by the new active key until the port enters Insecure State.

Upon receipt of an IDE.Start Link Layer Control flit, the Rx port shall copy the pending key to the active key slot and then mark the pending key slot as invalid. If CXL.cachemem IV Generation Capable=1 and the last CXL_KEY_PROG request indicated Use Default IV=0, the Initial CXL.cachemem IDE IV shall also be re-initialized to the value supplied as part of the CXL_KEY_PROG request. If CXL.cachemem IV Generation Capable=0 or the last CXL_KEY_PROG request indicated Use Default IV=1, Default IV construction shall be used. All subsequent protocol flits shall be protected by the new active key until the port enters Insecure State.

If the Rx port receives an IDE.Start Link Layer Control flit prior to a successful CXL_KEY_PROG since the last Conventional Reset, the Rx port shall drop the IDE.Start flit and then optionally set the Rx Error Status field in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) to CXL.cachemem IDE Establishment Security error. If the Rx port receives an IDE.Start Link Layer Control flit while CXL.cachemem IDE is active, but prior to a successful CXL_KEY_PROG since the last IDE.Start, the Rx port shall either (1) drop the IDE.Start flit and then optionally program Rx Error Status=8h to CXL.cachemem IDE Establishment Security error or (2) set the Rx Error Status field in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) to CXL.cachemem IDE Establishment Security error and then transition to Insecure State.

If the Rx port receives an IDE.Start Link Layer Control flit prior to a successful CXL_K_SET_GO since the last Conventional Reset, the Rx port shall drop the IDE.Start flit and then optionally set the Rx Error Status field in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) to CXL.cachemem IDE Establishment Security error. If the Rx port receives an IDE.Start Link Layer Control flit while CXL.cachemem IDE is active, but prior to a successful CXL_K_SET_GO since the last IDE.Start, the Rx port shall either (1) drop the IDE.Start flit and then optionally set the Rx Error Status field in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) to CXL.cachemem IDE Establishment Security error or (2) program Rx Error Status=8h to CXL.cachemem IDE Establishment Security error and then transition to Insecure State.

Offset 11h, Bit 1, is used to select between the Rx and Tx. Offset 11h, Bit 3, controls whether the CXL.cachemem IDE operates in Skid mode or Containment mode.

CIKMA should issue a CXL_K_SET_GO request message to an Rx port and wait for success before issuing a CXL_K_SET_GO request message to the partner Tx port.

Table 11-11. CXL_K_SET_GO Request

Byte Offset	Length in Bytes	Description
00h	Bh	Standard Request Header: See Table 11-2.
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 4, indicating CXL_K_SET_GO structure.
0Dh	2	Reserved
0Fh	1	Stream ID: Value is 0.
10h	1	Reserved
11h	1	- Bit[0]: Reserved - Bit[1]: RxTxB: See PCIe Base Specification - Bit[2]: Reserved - Bit[3]: CXL IDE Mode: 0 = Skid mode 1 = Containment mode - Bits[7:4]: Key Sub-stream: Value is 1000b
12h	1	PortIndex: See PCIe Base Specification.

Table 11-12 lists the various error conditions that a responder may encounter that are unique to CXL_K_SET_GO and how the conditions are handled.

Table 11-12. CXL_K_SET_GO Error Conditions

Error Condition	Response	Effect on an Active CXL.cachemem IDE Stream
Port receives CXL_K_SET_GO request, and pending key is invalid	No response is generated. The request is silently dropped.	No change
If a port receives CXL_K_SET_GO request with an IDE mode that is not supported		
If a port receives CXL_K_SET_GO request while IDE is active and the current IDE mode does not match Byte Offset 11, bit 3, in the new CXL_K_SET_GO request		
Protocol ID is nonzero		
Stream ID is nonzero		
Key Sub-stream is not 1000b		
PortIndex does not correspond to a valid port		
Invalid Request Length		

When a port receives a valid CXL_K_SET_STOP request, the port shall clear the active and pending CXL.cachemem IDE keys and then transition to IDE Insecure State. No errors shall be logged in the IDE Status register when an IDE stream is terminated in response to CXL_K_SET_STOP because this is not an error condition. If both ports support the IDE.Stop message as advertised by the CXL IDE Capability register (see Section 8.2.4.22.1), CIKMA may enable IDE.Stop on both ends of the link by programming the CXL IDE Control register (see Section 8.2.4.22.2). If IDE.Stop is enabled on both ends, it is unnecessary to quiesce the CXL.cache and CXL.mem traffic prior to issuing the CXL_K_SET_STOP request. If IDE.Stop is enabled, CIKMA is required to issue a CXL_K_SET_STOP to the Rx and then wait for an acknowledgment of CXL_K_SET_STOP before issuing a CXL_K_SET_STOP to the Tx. If IDE.Stop is not enabled, the Software is expected to quiesce the CXL.cache and CXL.mem traffic prior

to issuing a CXL_K_SET_STOP request to a port that is actively participating in CXL.cachemem IDE to prevent spurious CXL.cachemem IDE errors. The port shall respond with a CXL_K_GOSTOP_ACK message after the port has successfully processed a CXL_K_SET_STOP request.

If the Rx port receives an IDE.Stop Link Layer Control flit while the CXL.cachemem IDE is active, but prior to a successful CXL_K_SET_STOP since the last IDE.Start or any other CXL IDE Key Programming message, the Rx port shall drop the IDE.Stop flit, set the Unexpected IDE.Stop received bit in the CXL IDE Error Status register (see [Section 8.2.4.22.4](#)) but not transition to Insecure State.

Table 11-13. CXL_K_SET_STOP Request

Byte Offset	Length in Bytes	Description
00h	8h	Standard Request Header: See Table 11-2 .
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 5, indicating CXL_K_SET_STOP structure.
0Dh	2	Reserved
0Fh	1	Stream ID: Value is 0.
10h	1	Reserved
11h	1	- Bit[0]: Reserved - Bit[1]: RxTxB: See PCIe Base Specification - Bits[3:2]: Reserved - Bits[7:4]: Key Sub-stream: Value is 1000b
12h	1	PortIndex: See PCIe Base Specification.

[Table 11-14](#) lists the various error conditions that a responder may encounter that are unique to CXL_K_SET_STOP and how the conditions are handled.

Table 11-14. CXL_K_SET_STOP Error Conditions

Error Condition	Response	Effect on an Active CXL.cachemem IDE Stream
Port does not support CXL_K_SET_STOP (CXL_K_SET_STOP Capable=0)	No response is generated. The request is silently dropped.	No change
Protocol ID is nonzero		
Stream ID is nonzero		
Key Sub-stream is not 1000b		
PortIndex does not correspond to a valid port		
Invalid Request Length		

Table 11-15. CXL_K_GOSTOP_ACK Response (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
00h	8h	Standard Response Header: See Table 11-3 .
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 6, indicating CXL_K_GOSTOP_ACK structure.
0Dh	2	Reserved
0Fh	1	Stream ID: Value is 0.

Table 11-15. CXL_K_GOSTOP_ACK Response (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
10h	1	Reserved
11h	1	- Bit[0]: Reserved - Bit[1]: RxTxB: See PCIe Base Specification - Bits[3:2]: Reserved - Bits[7:4]: Key Sub-stream: Value is 1000b
12h	1	PortIndex : See PCIe Base Specification.

11.4.7 Get Key Messages

If the QUERY_RSP response message from the port indicates CXL.cachemem IDE Key Generation Capable=1 or CXL.cachemem IV Generation Capable=1, the port shall support the CXL_GETKEY message.

The CXL_GETKEY message is used to get the Locally generated CXL.cachemem IDE Key from the port and Locally generated CXL.cachemem IV.

Table 11-16. CXL_GETKEY Request

Byte Offset	Length in Bytes	Description
00h	Bh	Standard Request Header : See Table 11-2.
0Bh	1	Protocol ID : Value is 0.
0Ch	1	Object ID : Value is 7, indicating CXL_GETKEY request.
0Dh	2	Reserved
0Fh	1	Stream ID : Value is 0.
10h	1	Reserved
11h	1	- Bits[3:0]: Reserved - Bits[7:4]: Key Sub-stream: Value is 1000b
12h	1	PortIndex : See PCIe Base Specification.

Table 11-17 lists the various error conditions that a responder may encounter that are unique to CXL_GETKEY and how the conditions are handled.

Table 11-17. CXL_GETKEY Processing Error

Error Description	Error Code	Effect on an Active CXL.cachemem IDE Stream
Invalid Request Length	No response is generated. The request is silently dropped.	No change
PortIndex does not correspond to a valid port		
Protocol ID is nonzero		
Stream ID is nonzero		
Key Sub-stream is not 1000b		
CXL_GETKEY received prior to CXL_QUERY		
Port does not support CXL_GETKEY		

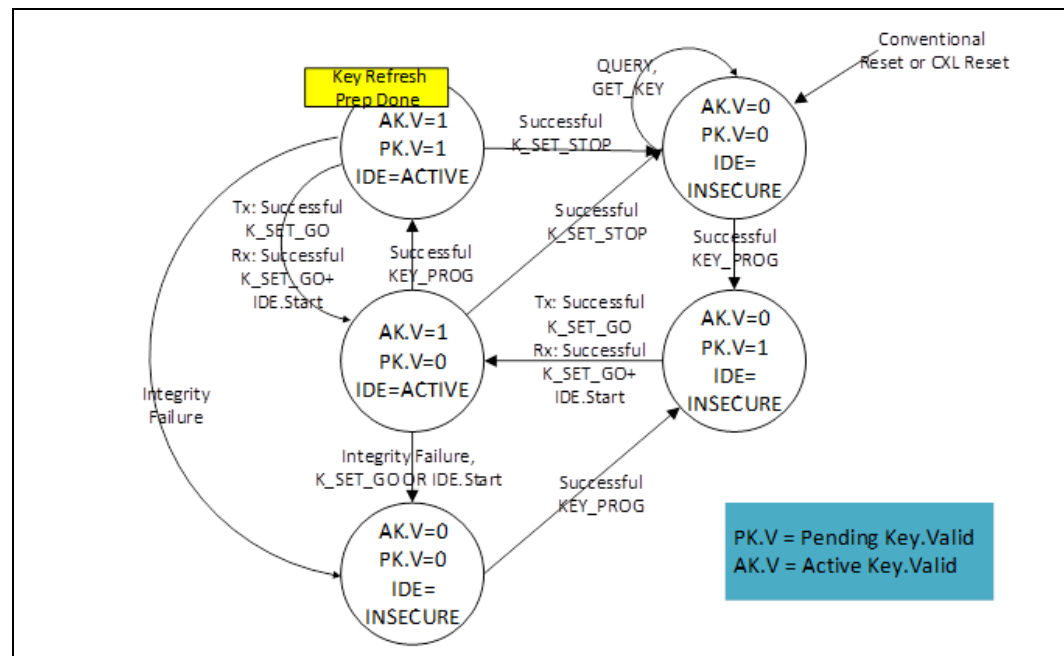
Upon successful processing of CXL_GETKEY, the responder shall acknowledge by sending the CXL_GETKEY_ACK response.

Table 11-18. CXL_GETKEY_ACK Response

Byte Offset	Length in Bytes	Description
00h	8h	Standard Response Header: See Table 11-3.
0Bh	1	Protocol ID: Value is 0.
0Ch	1	Object ID: Value is 8, indicating CXL_GETKEY_ACK response.
0Dh	2	Reserved
0Fh	1	Stream ID: Value is 0.
10h	1	Reserved
11h	1	- Bits[3:0]: Reserved - Bits[7:4]: Key Sub-stream: Value is 1000b
12h	1	PortIndex: See PCIe Base Specification.
13h	KSIZE	Locally Generated CXL.cachemem IDE Key: The KSIZE must be 32 for this version of the specification. For layout, see PCIe Base Specification. This field must be ignored if the QUERY_RSP response message from the port indicates CXL.cachemem IDE Key Generation Capable=0.
13h+KSIZE	12	Locally Generated CXL.cachemem IV: This field must be ignored if the QUERY_RSP response message from the port indicates CXL.cachemem IV Generation Capable=0. Byte Offsets 16h+KSIZE:13h+KSIZE carry the IV DWORD, IV[95:64]. Byte Offsets 20h+KSIZE:17h+KSIZE carry the IV DWORD, IV[63:32]. Byte Offsets 24h+KSIZE:21h+KSIZE carry the IV DWORD, IV[31:0].

Figure 11-23 illustrates various key states and their transitions. Note that this figure is not meant to be exhaustive and does not include several legal transition arrows for simplicity.

Figure 11-23. Active and Pending Key State Transitions



IMPLEMENTATION NOTE**Establishing CXL.cachemem IDE between a DSP and EP - Example**

In this example, host software plays the role of the CIKMA. The switch implementation is such that the USP implements the DOE capability on behalf of all the DSPs and the specific DSP that is involved here is referenced as Port 4. Further, it is also assumed that the desired mode of operation is Skid mode. Host Software reads and configures the CXL IDE capability registers on the DSP and on the EP. See [Section 8.2.4.21](#) for the definition of these registers and programming guidelines.

1. Host Software sets up independent SPDM secure sessions with the USP and the EP. This is accomplished by issuing SPDM key exchange messages over PCIe DOE.
2. All subsequent messages are secured as per DSP0277. The messages to/from the USP are secured using the SPDM session key established with the USP. The messages to/from the EP are secured using the session key established with the EP.
 - a. Host Software sends a CXL_QUERY (PortIndex=4) message to the USP DOE mailbox. The USP returns a CXL_QUERY_RESP. Host Software compares the CXL_QUERY_RESP contents against the CXL IDE Capability structure associated with the DSP. Host Software exits with an error if there is a mismatch or a timeout. In this example, the USP reports that it supports Locally generated CXL.cachemem IV and Locally generated CXL.cachemem IDE Key.
 - b. Host Software sends a CXL_QUERY (PortIndex=0) message to the EP DOE mailbox. The EP returns a CXL_QUERY_RESP. Host Software compares the CXL_QUERY_RESP contents against the CXL IDE Capability structure associated with the EP. Host Software exits with an error if there is a mismatch or a timeout. In this example, the EP reports that it supports Locally generated CXL.cachemem IV and Locally generated CXL.cachemem IDE Key.
 - c. Host Software issues a CXL_GETKEY request to the USP and saves the Locally generated CXL.cachemem IDE Key from the response as KEY2 and the Locally generated CXL.cachemem IV from the response as IV2. Host Software issues CXL_GETKEY request to the EP and saves the Locally generated a CXL.cachemem IDE Key from the response as KEY1 and the Locally generated Tx IV from the response as IV1.
 - d. Host Software programs the Rx pending key in the EP by sending a CXL_KEY_PROG(RxTxB=0, Use Default IV=0, KEY2, IV2) message to the EP DOE mailbox. Host Software programs the Tx pending keys in the DSP by sending a CXL_KEY_PROG(PortIndex=4, RxTxB=1, Use Default IV=0, KEY2, IV2) message to the USP DOE mailbox. Host Software programs the Rx pending keys in the DSP by sending a CXL_KEY_PROG(PortIndex=4, RxTxB=0, Use Default IV=0, KEY1, IV1) message to the USP DOE mailbox. Host Software programs the Tx pending key in the EP by sending a CXL_KEY_PROG(RxTxB=1, Use Default IV=0, KEY1, IV1) message to the EP DOE mailbox. These 4 steps can be performed in any order. Host Software exits with an error if the CXL_KP_ACK indicates an error or if there is a timeout.

IMPLEMENTATION NOTE**Continued**

- e. Host Software instructs DSP Rx to be ready by sending a CXL_K_SET_GO(PortIndex=4, Skid mode, RxTxB=0) to the USP DOE mailbox. Host Software instructs the EP Rx to be ready by sending a CXL_K_SET_GO(Skid mode, RxTxB=0) to the EP DOE mailbox. Host Software exits with an error if either CXL_K_SET_GO request times out.
- f. Host Software instructs DSP Tx to enable CXL.cachemem IDE by sending a CXL_K_SET_GO(PortIndex=4, Skid mode, RxTxB=1) to the USP DOE mailbox. DSP sends an IDE.Start Link Layer Control flit to EP and thus initiating IDE in one direction using KEY1 and Starting IV=IV1. Host Software exits with an error if the CXL_K_SET_GO request times out.
- g. Host Software instructs EP Tx to enable CXL.cachemem IDE by sending a CXL_K_SET_GO(Skid mode, RxTxB=1) to the EP DOE mailbox. EP sends an IDE.Start Link Layer Control flit to the DSP and thus initiates IDE in the other direction, using KEY2 and Starting IV=IV2. Host Software exits with an error if CXL_K_SET_GO request times out.
- h. At the end of these steps, all the CXL.cachemem protocol flits traveling between the DSP and EP are protected by IDE.

If both ports support Locally generated CXL.cachemem IDE Key and Locally generated CXL.cachemem IV, the following sequence will result in a programming error and should be avoided:

1. CIKMA issues CXL_GETKEY request to Port1 and saves the key, KEY1
2. CIKMA issues another CXL_GETKEY request to Port1 and saves the IV, IV1
3. CIKMA issues CXL_KEY_PROG request to Port1 Tx and passes in KEY1 and IV1.
4. Port1 returns a CXL_KP_ACK with Status=08h because the 2nd CXL_GETKEY request changed its locally CXL.cachemem IDE generated key to KEY2, which is not equal to KEY1.

11.5**CXL Trusted Execution Environments Security Protocol (TSP)****11.5.1****Overview**

Virtualization-based Trusted Execution Environments (TEE) are used to host confidential computing workloads that are isolated from hosting environments. This specification refers to such TEE as Trusted Execution Environment VMs (TVMs) to distinguish them from traditional virtual machines.

The PCI-SIG TEE Device Interface Security Protocol (TDISP) ECR specifies the architecture of a framework for trusted I/O virtualization to include PCIe devices within the TVM trust boundary. The CXL TEE Security Protocol (CXL-TSP), complements the PCI-SIG TDISP specification by specifying mechanisms to include direct attached CXL memory devices within the TVM trust boundary specifically for confidential computing scenarios.

11.5.2 Scope

This CXL security content scope focuses on features that are needed for confidential computing utilizing CXL Type 3 memory expander devices, referred to as targets in the TSP, directly connected to CXL Root Ports owned by the host which is an initiator in TSP. TSP defines the security objectives, capabilities, and interfaces, and the host, initiator, and target behaviors that are required to create a secure CXL memory hierarchy that meets the needs of confidential computing. The scope does not include details on initiator or target security implementation.

- This scope includes support for the following:
 - SPDM 1.2 or newer for authentication and attestation
 - Directly connected LDs, SLDs, and MH-SLDs
 - Dynamic Capacity devices
 - HDM-H memory
 - 256B flit format
 - Memory pooling — Multiple initiators accessing the same physical memory on a device but not sharing access to it
 - Comprehensive Trust security model
 - Selective Trust security model
 - Implicit 64B Cacheline TE State Access Control
 - Explicit TE State Access Control
- This scope does not include the following:
 - CXL switches
 - Devices connected via a CXL switch, including MLDs
 - UIO Direct Peer-to-peer (P2P) to HDM
 - Direct P2P using CXL.mem
 - Direct P2P using CXL.io
 - Type 1 and Type 2 accesses to Type 3 HDM memory
 - HDM-D or HDM-DB memory
 - PBR and 68B flit formats
 - Memory sharing — Multiple initiators accessing the same physical memory on a device and simultaneously sharing access to it

11.5.3 Threat Model

This version of TSP shall focus on providing confidential computing support for direct attached CXL memory. Direct attached memory shall be defined as using the CXL protocol to communicate with a memory device, or target and the CXL Root Ports of the host, without intermediaries in the middle of the two. Within the context of extending CXL for confidential computing, one of TSP's objectives is to minimize the Trusted Computing Base (TCB). The TSP supports both a selective trust and comprehensive trust security model.

11.5.3.1 Definitions

The following additional terms are utilized in this threat model section:

- **Attacker:** Entity that wants to extract information from a communication or influence a computation by modifying information that flows between two participants.
- **Confidential Computing:** Confidential Computing protects Data in Use, Data in Transit and Data at Rest. Data at Rest applies to the CXL memory device and Data in Transit applies to TSP Transport Security such as CXL IDE. Trusted Execution Environments (TEEs) prevent unauthorized access or modification of applications and data while in use, thereby increasing the security assurances for organizations that manage sensitive and regulated data.
- **Covert Channel:** Method for an accomplice inside a trusted entity to signal to an attacker outside a trusted entity.
- **Target:** Participant in the protocol that does not forward packets to other participants. The memory device.
- **Host:** Location in which multiple participants concurrently reside. The host is an initiator that contains CXL Root Ports.
- **Information:** Data or properties of the data exchanged between two participants that would allow the attacker to take or cause an adverse action. Examples include cryptographic keys, questions being considered, decisions of the TEE, results of a database query, etc.
- **Intermediary/switch:** Participant in the protocol that routes or forwards packets to targets. TSP initially focuses on direct attached confidential computing scenarios; thus, switch support in the threat model is beyond the scope of this specification.
- **Participant:** Initiator or target in a communication that utilizes a correct and error free implementation of the protocol.
- **Protocol Secrets:** Secrets that shall be protected, from users of the protocol and/or attackers, to maintain the TEE.
- **Side Channel:** Ability of an attacker to extract information without the knowledge of the participating parties.
- **Trusted Execution Environment (TEE):** Execution environment designed to provide secure separation between itself and any other computation. The environment may include or be extended to multiple devices.

11.5.3.2 Assumptions

The threat model described below is based on the following assumptions:

- CXL does not guarantee that messages arrive in order. It requires initiator ordering. If the initiator has two messages that must be ordered, Message A and Message B, the initiator shall wait until Message A is acknowledged before submitting Message B.
- CXL relies on industry-standard secure protocols: SPDM and PCIe.
- CXL relies on industry-standard capabilities: Secure boot, trusted boot, and attestation.
- There are no errors in the implementation of the protocol, regardless of whether implemented in hardware, software, or firmware.
- An implementation of the protocol shall not disclose protocol secrets to an attacker. The participants shall have a secure location in which to store and/or retain this information.
- For confidential computing, everything inside the TEE shall not be observable by an attacker outside the TEE.

- When data is securely delivered to an attached target, the target shall protect that data from attacks. The TSP protocol facilitates multiple means of protecting the data. How the device implements such protections is beyond the scope of the threat model and TSP.
- There are non-overlapping resources for distinct hosts.
- Hardware in the host is trusted to maintain protocol separation between TEEs and keep TEEs isolated from one another.
- Hardware in the host is trusted to maintain protocol separation and isolation between Root Ports. Root Ports shall not accept incorrectly formatted transactions. If a host has a single port through which multiple sessions flow, the host hardware shall keep the sessions isolated and reliably deliver the transactions to the session owners, as defined by the hardware configuration.
- A correct implementation of the protocol. This means that the attacker cannot be inside the protocol.
- The protocol cannot defend against an attacker from within the TEE. Host hardware shall be responsible for defending the TEE from an attacker inside a host. An attacker in an initiator or target can use the protocol but is constrained by the requirements of this threat model.
- The target is directly connected to the host; thus, there are no attackers in the intermediaries in the initial TSP threat model. Targets connected via CXL switches have not been evaluated and the presence of switches are considered to be outside the threat model. Fabric-attached memory may require initiator-based memory encryption to keep the intermediaries out of the TCB and shall be addressed in a future version of the TSP.
- TEEs that require confidentiality of the information flowing between the initiator and the target, shall enable a CXL-approved Transport Security such as CXL IDE.
- A target can concurrently hold data for a computation for multiple initiators, resulting in data from multiple participants residing concurrently on the target. The target shall be responsible for keeping each initiator's data or computations separate and isolated. If the target has multiple ports, the target hardware shall keep the ports isolated and independent.
- If a target has a single port through which multiple sessions flow, the target hardware shall keep the sessions isolated and reliably deliver the transactions to their respective session owners, as defined by the hardware configuration. The target determines the threats from which initiator data is protected.
- Initiators and targets shall utilize an SPDMM 1.2 or newer connection to authenticate and attest the target. For authenticated and trusted direct attached targets, multiple initiators can communicate with the same target without leaking information to other initiators.
- The protocol shall carry sufficient information to allow the target to maintain separation between initiators. Additionally, the protocol shall contain sufficient information to enable the target to enforce ciphertext hiding if needed.
- The protocol supports both initiator-based and target-based memory encryption so it shall carry sufficient information for the memory device to prevent access by non-TEEs to TEE memory.
- The protocol shall minimize the number of bits transmitted in the clear. These bits can be utilized as a covert channel if an application inside an initiator is compromised. PBR removes this exposure because all address bits can be encrypted; however, this is beyond the scope of the initial TSP.

11.5.3.3 Threats and Mitigations

Table 11-19 outlines the security threats considered as part of the threat model and how the threat is mitigated.

Table 11-19. Security Threats and Mitigations

Primary Threat of Attacker	Threat Mitigation
Extract protocol secrets	- Transport Security such as CXL IDE - TSP initiator-based or target-based memory encryption
Masquerade as a legitimate initiator or target	- SPDM attestation and authentication - SPDM mutual authentication
Insert itself between an initiator and target as a manipulator-in-the-middle	- Prevent physical attack - SPDM attestation and authentication - Transport Security such as CXL IDE
Derive actionable data or information from observed packets (side channel)	- Minimize the number of address bits transmitted in the clear (possibly 0) - Transport security such as CXL IDE
Insert data and/or requests/responses into an initiator target communication	Transport Security such as CXL IDE
Modify data and/or requests/responses exchanged in an initiator target communication	Transport Security such as CXL IDE
Remove data and/or requests/responses from an initiator target communication	Transport Security such as CXL IDE
Replay legitimate packets	Transport Security such as CXL IDE
A non-TEE reading or writing of TEE data	TSP TE State checking for access control
A TEE reading or writing unauthorized non-TEE data	TEE is correctly configured to allow access only to memory for which the TEE is authorized to access
One TEE reading or writing another TEE's data	- TEE is correctly configured to allow access only to memory for which the TEE is authorized to access - TSP initiator-based or target-based memory encryption to protect each TEE's data from other TEEs
A TEE on one host reading and/or writing resources that belong to a TEE of another host	- Hosts shall maintain separation and isolation between TEEs - TSP initiator-based or target-based memory encryption
A target on one port being able to send requests and/or responses on another port	Hardware within the host is trusted to maintain protocol separation and isolation between RPs. RPs shall not accept incorrectly formatted transactions. If a host has a single port through which multiple sessions flow, the host hardware shall keep the sessions isolated and reliably deliver the transactions to the session owners, as defined by the hardware's configuration.

11.5.4 Reference Architecture

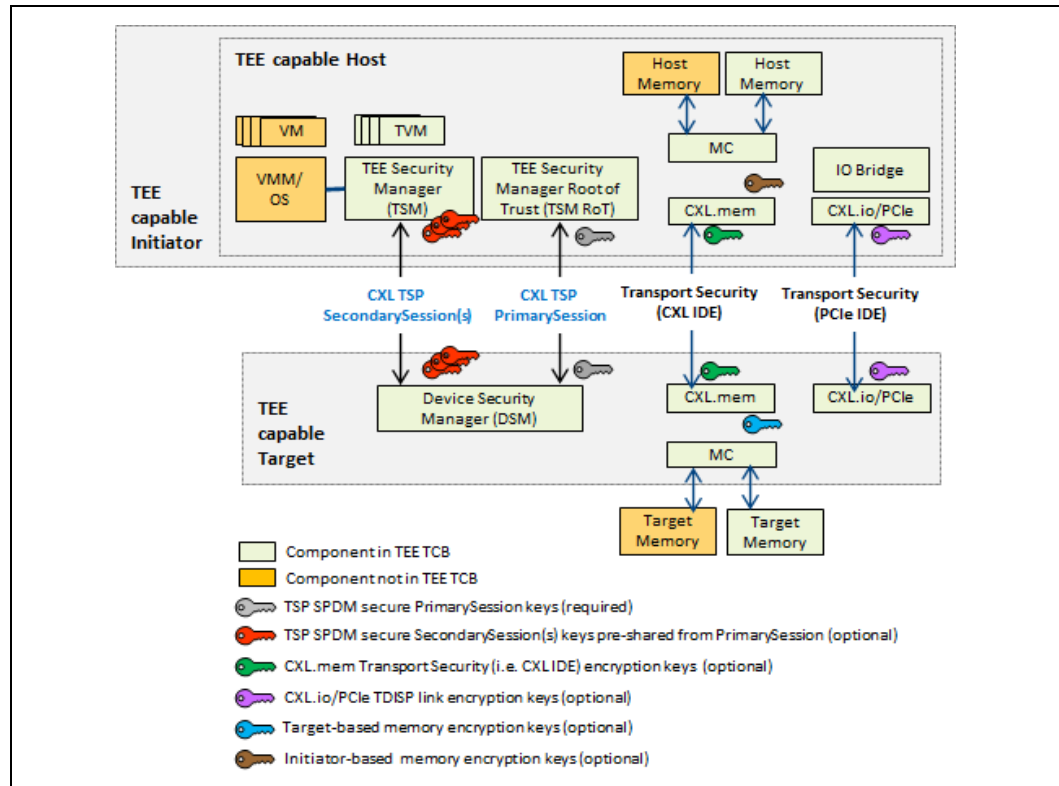
The reference architecture covers the security requirements and behaviors that are needed to support confidential computing use cases and covers the architectural scope, detecting TSP support, CMA/SPDM, attestation and authentication, memory encryption, transport security, access control, configuration, and Dynamic Capacity.

11.5.4.1 Architectural Scope

Figure 11-24 outlines the major components that the TSP considers to be inside the TCB or outside the TCB, the different connections between the TEE-capable initiator and TEE-capable target memory device, and those connections that are specified by the TSP. Hosts are the only initiators defined for this version of the TSP architecture for

support of direct attached confidential computing in the TSP architecture. Not all initiators are hosts because other CXL accelerators and PCIe P2P devices can function as initiators, the focus of future TSP additions.

Figure 11-24. Reference Architecture



For implementations that utilize initiator-based memory encryption or target-based memory encryption, it is recommended to enable Transport Security (such as CXL IDE) as discussed in [Section 11.5.4.7](#).

Securing CXL.io is optional from a TSP perspective.

11.5.4.2 Determining TSP Support

For targets that support the TSP, the DVSEC CXL Capability register TSP Capable bit (see [Section 8.1.3.1](#)) shall be set by the target to indicate support for the TSP requests and responses detailed in the following sections. This bit also indicates to the initiator that the target supports the MemRdFill memory request which is required for deadlock prevention with partial writes and initiator-based encryption.

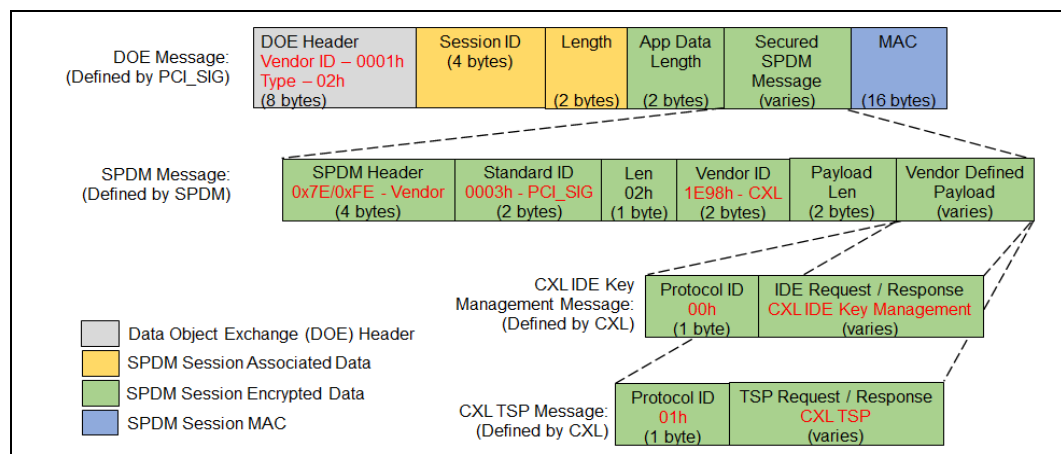
11.5.4.3 CMA/SPDM

CMA/SPDM 1.2 or later secure sessions are utilized with CXL Vendor defined payloads for all TSP request and response payloads defined herein. The Protocol ID in the first byte of the Vendor Defined Payload identifies TSP requests independently from IDE or other requests that may also be defined by CXL. [Figure 11-25](#) outlines the encapsulation of the TSP-defined payloads in a CMA/SPDM message, which is similar to those defined in the PCI-SIG TDISP, with the following changes to establish CXL control of the message interpretation:

- The DOE Data Object type shall report Vendor ID 0001h and Data Object Type 02h to point to Secured CMA/SPDM.
- The CMA/SPDM vendor defined message Standards ID shall utilize 0003h to indicate that PCI-SIG is the body that assigned the CMA/SPDM Message Vendor ID.
- The CMA/SPDM vendor defined message Vendor ID shall utilize 1E98h, indicating that the CXL Consortium assigned the interpretation of the CMA/SPDM vendor defined payload.
- The first byte in the CXL vendor defined payload is the Protocol ID. All CXL.cachemem IDE Key Management requests shall utilize Protocol ID = 00h. All TSP request and response messages shall utilize Protocol ID = 01h.

The encapsulation of TSP requests and responses inside an encrypted SPDM session is shown in [Figure 11-25](#).

Figure 11-25. CMA/SPDM, CXL IDE, and CXL TSP Message Relationship



The Session ID that precedes the CMA/SPDM payload contains the TSP session utilized for each request or response payload. The TSP specification utilizes two types of sessions:

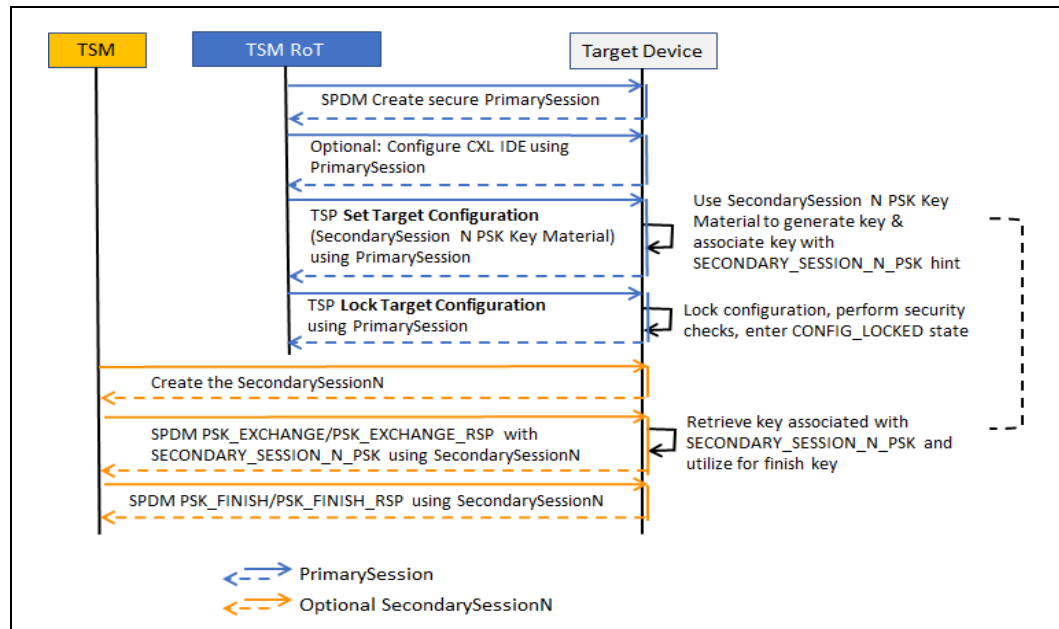
- **PrimarySession:** Required CMA/SPDM session that is established between the host and the target.
 - Utilized to configure and lock the target as defined by TSP.
 - For target-based memory encryption, this session may be utilized to set or clear memory encryption keys. The session utilized to set a key shall be the same session that is utilized to clear the same key.
 - PrimarySession is the CMA/SPDM session that is utilized to send the Set Target Configuration request. The target shall terminate any existing SecondarySession(s) anytime a new PrimarySession is established.
 - PrimarySession shall be utilized for all TSP and Transport Security (such as CXL IDE) requests and responses (see [Figure 11-25](#)).
 - Primary SPDM Session shall be utilized to provision PSK Key Material for establishing each Secondary SPDM Session(s)
- **SecondarySession(s):** Optional CMA/SPDM sessions that are generated from the PrimarySession by utilizing CMA/SPDM PSK_EXCHANGE between the host and the target.
 - For target-based memory encryption, this session may be utilized to set or clear memory encryption keys. Some host implementations may need to utilize a separate but related SPDM SecondarySession for setting and clearing

memory keys independently from the PrimarySession that is utilized to configure and lock the configuration. The session utilized to set a key shall be the same session that is utilized to clear the same key.

- Target advertises the number of SecondarySession(s) that it supports in Get Target Capabilities.
- Initiator can configure the number of SecondarySession(s) to utilize and the type of TEE opcode checking each will use through Set Target Configuration.
- PrimarySession is independent of these sessions. The termination or closing of any SecondarySession(s) shall have no effect on the PrimarySession.
- These sessions are independent of the PrimarySession. Termination or closing of the PrimarySession shall have no effect on these sessions. However, if a new PrimarySession is started, the target shall terminate these sessions since new secondary sessions will need to be generated based on the new primary session key material.
- CMA/SPDM session that successfully completes the CMA/SPDM PSK that utilizes the correct PSK hint shall be considered a valid SecondarySession.

Figure 11-26 outlines the high-level sequence for creating the PrimarySession and how TMVSession PSK Key Material is utilized by the target to generate keys for a secure CMA/SPDM SecondarySession(s).

Figure 11-26. CMA/SPDM Sessions Creation Sequence



11.5.4.4 Authentication and Attestation

Because the TSP interface requires requests and responses to utilize a CMA/SPDM 1.2 (or later) secure session, target attestation and authentication is accomplished using the CMA/SPDM-defined secure session setup sequence.

11.5.4.5 TE State Changes and Access Control

The TEE Exclusive State (TE State) of memory indicates whether the content of the memory is for TEE or non-TEE data.

Initiators that generate memory accesses shall determine the TEE status of each memory transaction, referred to as the TEE Intent. TEEs are permitted to access both exclusive and non-exclusive memory, while non-TEE entities are permitted to access only memory that is not intended for the exclusive use of a TEE.

Access control is outlined in the following sections and is defined as the verification of the TEE Intent against the TE State of the memory being accessed and the resulting target behavior when the verification fails. Access control is split in to Write Access Control and Read Access Control that can be supported by the target independently and enabled independently by the initiator.

Initiators shall not generate memory accesses with TEE Intent if those accesses do not arise within the execution context of a TEE. Initiators that generate memory accesses that originate within the execution context of a TEE shall understand the request's TEE Intent, based on the specific design of the TEE architecture, and shall express the correct TEE Intent. Because each request carries the correct TEE Intent, it is unnecessary for a request to indicate whether it originates from a TEE.

Initiators shall convey TEE Intent in a request by utilizing the TEE-specific M2S Req and M2S Rwd opcodes specified in the CXL Transaction Layer. Targets shall convey the TE state by utilizing the S2M NDR and S2M DRS response opcodes specified in the CXL Transaction Layer. The specific opcodes utilized are specified in the tables provided later in this section.

Hosts that support implicit and explicit TE State changes:

- May enable either mechanism individually or both mechanisms at the same time on the target. When enabling implicit and explicit in-band TE State changes simultaneously, the TE State granularity utilized for explicit in-band TE State changes shall be 64B.
- Shall account for interleaving and send a single TE State change request to each target for a given interleave set.

Targets that have TE State changes enabled:

- Shall change the TE State of memory at a 64B cacheline granularity for implicit changes and at a 64B or greater granularity for explicit changes.
- Shall support explicit in-band TE State changes with a granularity of 64B when supporting implicit TE State changes
- Shall utilize implicit and/or explicit TE State changes as enabled by the host.
- Shall support the TEUpdate memory transaction when implicit or explicit in-band TE State changes are enabled.
- Shall return the current TE State saved for the memory location being accessed

Targets that have TE State changes disabled and CKID-based memory encryption disabled:

- Shall return the TEE Intent from the memory request in the response opcode

Targets that have read and/or write access control enabled:

- Shall implement TE State changes
- Shall follow the rules defined below for implicit or explicit target behavior for updating TE State, verifying TE State, and responding to access control state verification

Targets optionally provide an event log entry of all dropped writes or failed reads that occur in response to failed TE State checks to aid in root-cause analysis of unexpected behavior by reporting a General Media or DRAM Event Record with a Memory Event Type of TE State Violation.

Targets that implement CKID-based target encryption shall perform CKID-type checks as described in [Section 11.5.4.6.2.1](#).

The granularity utilized for TE State changes shall be consistent with the interleave granularity being configured. For example, if the host utilizes a 4K TE State change granularity on each target that is part of a 16-way interleave set with a 256B interleave granularity, each target will utilize 256B of DPA space to change 4K of TE State.

11.5.4.5.1 TEUpdate Memory Transaction

The TEUpdate memory transaction shall utilize the flit's 3-bit SnpType field to provide a Length Index to preconfigured fixed granularities of TE State.

Length Index encodings 1 through 6 are configurable. Length Index encodings 0 and 7 are fixed, where 0 is defined as 64B and 7 is defined as the target's entire memory space.

Targets that support implicit TE State changes or in-band explicit TE State changes shall support this transaction with Length Index = 0 and may support a Length Index of 7 as reported in Get Target Capabilities. If the target only supports implicit TE State changes, then Length Index encodings 1 through 6 shall be reserved.

The HPA present in the TEUpdate transaction shall be decoded by the target to the correct HDM decoder and the starting HPA, HDM decoder Interleave Granularity (IG), and HDM decoder Interleave Ways (IW) are utilized by the target to change the TE State of those HPA ranges within the granularity determined from the SnpType field.

The CKID field is reserved for the TEUpdate transaction and shall be ignored by the target.

There is no mechanism for the target to reject an explicit TEUpdate transaction.

11.5.4.5.2 Implicit TE State Changes

Implicit state changes shall always occur on a cacheline write and shall not utilize Write Access Control.

When utilizing implicit TE State changes, the target shall also support the TEUpdate memory transaction with Length Index 0 to indicate a 64B length.

[Table 11-20](#) outlines the expected target behavior for implementing implicit TE State changes.

Table 11-20. Target Behavior for Implicit TE State Changes

Target's TE State Associated with Memory Access Address	TEE Intent of Memory Transaction Received by the Target	
	TEE Opcodes ¹	Non-TEE Opcodes ²
TE 0	- Writes: - Full cacheline write shall cause an implicit state change to TE=1 - S2M NDR TEE opcode shall be returned	- Writes: - No change to TE State - S2M NDR non-TEE opcode shall be returned
TE 1	- Writes: - No change to TE State - S2M NDR TEE opcode shall be returned	- Writes: - Full cacheline writes shall cause an implicit state change to TE=0 - S2M NDR non-TEE opcode shall be returned

1. MemWrTEE, MemWrPtTEE.

2. MemWr, MemWrPt.

11.5.4.5.2.1 Partial Write Handling with Implicit TE State Changes

Full cacheline writes are required to change the TE State implicitly.

Initiator-based memory encryption shall be handled as follows:

- A partial write shall be treated as an under fill read, merging of partial write data with under fill read data, followed by a full cacheline write. The under fill read shall utilize the same TEE intent as the full cacheline write that follows.

Target-based memory encryption shall be handled as follows:

- A partial write shall be treated as an under fill read, merging of partial write data with under fill read data, followed by a full cacheline write. The under fill read shall utilize the same TEE intent as the full cacheline write that follows.
- The under fill read shall follow the rules for reads and the full cacheline write shall follow the rules for writes.

11.5.4.5.3 Explicit TE State Changes

If explicit state changes are supported, the target shall support utilizing the TEUpdate memory transaction for in-band state changes and/or the CMA/SPDM secure session TSP request, Set Target TE State, for out-of-band changes. For explicit TE State changes > 64B, the target shall pre-allocate resources for a single explicit state change request to avoid head-of-line blocking.

The target shall be configured to enable explicit in-band TE State changes or explicit out-of-band TE State changes and either may be enabled individually or both may be enabled at the same time. If the host is utilizing both simultaneously, the host shall maintain coherency between them.

Explicit TE State changes shall be initiated from the host. The host shall ensure that memory affected by the TE state change is flushed from caches before initiating the explicit state change request.

The host shall be responsible for maintaining coherency for accesses to memory ranges that are also executing an explicit state change.

While the explicit TE State change request is executing, the target shall handle memory transactions as follows:

- The target shall continue to process unrelated memory transactions while the state change is executing
- For explicit TE State changes > 64B:
 - For writes to memory ranges that are undergoing the state change, the target shall drop the write and return the inverted TEE Intent in the write completion
 - For reads to memory ranges that are undergoing the state change, the target shall return all 1s and the inverted TEE Intent in the read completion

When executing out-of-band explicit TE State changes that cover a large amount of data, the target may require extra execution time and may utilize the Delayed Response to prevent request timeouts as described in [Section 11.5.5.9](#).

The target shall report optional support to sanitize the contents of memory with 0s anytime the explicit TE State change request is received. If enabled by the host, the target shall complete the overwrite of the affected range before the explicit state change is considered complete. This sanitize capability is reported in Get Target Capabilities and the host may enable its use with Set Target Configuration. Sanitizing large amounts of memory may require extra execution time and targets may utilize the Delayed Response to prevent request timeouts as described in [Section 11.5.5.9](#).

If in-band and out-of-band explicit state changes overlap, the host shall ensure that those requests have non-overlapping address ranges. Otherwise, an indeterminate result could occur. If the target can detect this overlap, the target should generate an appropriate event record to aid in debug.

For explicit updates > 64B: A single explicit state change shall be sent to every target in the same interleave set. Explicit state changes specify a starting address and length that cover the entire range to be changed. The target shall change the state for all portions of the range that land on its portion of the interleave.

11.5.4.5.3.1 Optional Explicit In-band TE State Change

For explicit in-band TE State changes:

- The in-band mechanism shall utilize the TEUpdate memory transaction.
- The association of length index in the SnpType field to a given granularity is configured by the initiator utilizing Set Target Configuration. This is done to minimize the size of the TEUpdate flit to a single slot to minimize Transaction Layer complications.
- Length Index value of 0 is reserved for 64B state changes.
- Length Index value of 7 is reserved for state changes affecting the entire memory space of the target.
- Length Index values 1-6 are host configurable to any supported length utilizing Set Target Configuration.
- If the in-band TE State change granularity is > 64B, the host shall only issue a single explicit in-band state change request at a time.
- If the in-band TE State change granularity is 64B, the host may issue multiple explicit in-band TE State change requests to non-overlapping address ranges and the target shall queue those requests waiting to execute.

Figure 11-27 outlines the association between the Explicit In-band TE State Granularity specified in Set Target Configuration and the Length Index specified in the TEUpdate transaction SnpType field.

Figure 11-27. Optional Explicit In-band TE State Change Architecture

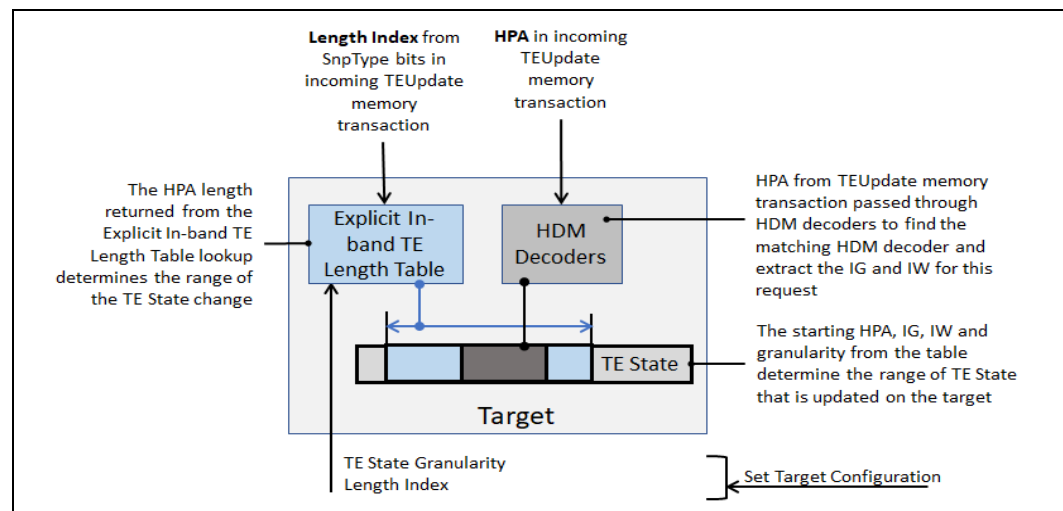


Table 11-21 outlines the expected target behavior for utilizing explicit in-band TE State changes.

Table 11-21. Target Behavior for Explicit In-band TE State Changes

Target's TE State Associated with Memory Access Address	TEE Intent of Memory Transaction Received by the Target
	TEE Opcodes ¹
TE 0	- TEUpdate(TE = 0): - No change in TE state - TEUpdate(TE = 1): - This shall cause an explicit state change to TE=1 for the affected memory granularity
TE 1	- TEUpdate(TE = 0): - This shall cause an explicit state change to TE=0 for the affected memory granularity - TEUpdate(TE = 1): - No change in TE state

1. TEUpdate.

11.5.4.5.3.2 Optional Explicit Out-of-Band TE State Change

For explicit out-of-band TE State changes:

- Out-of-band mechanism utilizes the Set TE State TSP request and supports a robust set of possible TE State change granularities reported in Get Target Capabilities that cannot be utilized with the limitations of the in-band mechanism
- Host shall only issue a single explicit out-of-band state change request at a time
- Target shall reject any request to set TE State when another TE State change request is already executing

11.5.4.5.4 Write Access Control

Table 11-22 outlines the required target behavior when Write Access Control is enabled on the target.

Table 11-22. Target Behavior for Write Access Control

Target's TE State Associated with Memory Access Address	TEE Intent of Memory Transaction Received by the Target	
	TEE Opcodes ¹	Non-TEE Opcodes ²
TE 0	- Writes: - Full/Partial cacheline write shall be dropped - S2M NDR non-TEE opcode shall be returned - Optionally log event	- Writes: - Full/Partial cacheline write allowed - S2M NDR non-TEE opcode shall be returned
TE 1	- Writes: - Full/Partial cacheline write allowed - S2M NDR TEE opcode shall be returned	- Writes: - Full/Partial cacheline write shall be dropped - S2M NDR TEE opcode shall be returned - Optionally log event

1. MemWrTEE, MemWrPtlTEE.

2. MemWr, MemWrPtl.

If Write Access Control is not enabled on the target, the target shall not check write requests for possible access control violations. See [Section 11.5.4.5.2](#) for implicit TE State changes and required target behavior.

If Write Access Control is enabled on the target, the target shall clear the TE State to 0 for all addressable memory in response to the Lock Target Configuration Request and before generating a Lock Target Configuration Response.

Write Access Control requires the target to also support explicit TE State changes. The target shall reject attempts to enable Write Access Control without one or more explicit TE State change mechanisms also being enabled.

The target shall not perform Write Access Control when updating MetaValue. See [Section 11.5.4.5.6](#) for requirements for handling MetaValue updates.

If enabled, the target shall perform Write Access Control when updating Extended Metadata (EMD). See [Section 11.5.4.5.7](#) for requirements for handling EMD updates.

11.5.4.5.4.1 Partial Write Handling with Write Access Control

Target-based memory encryption shall be handled as follows:

- A partial write shall be treated as an under fill read, merging of partial write data with under fill read data, followed by a full cacheline write.
- The under fill read shall follow the rules for reads and the full cacheline write shall follow the rules for writes. In case of a TEE mismatch between the TE State obtained in the under fill read and the TEE Intent of the request, the target shall drop the write.

11.5.4.5.5 Read Access Control

[Table 11-23](#) outlines the required target behavior when Read Access Control is enabled on the target.

Table 11-23. Target Behavior for Read Access Control

Target's TE State Associated with Memory Access Address	TEE Intent of Memory Transaction Received by the Target	
	TEE Opcodes ¹	Non-TEE Opcodes ²
TE 0	• Reads: — Reads shall return fixed data of all 1s — S2M DRS non-TEE opcode shall be returned — Optionally log event	• Reads: — Allowed — S2M DRS non-TEE opcode shall be returned
TE 1	• Reads: — Allowed — S2M DRS TEE opcode shall be returned	• Reads: — Reads shall return fixed data of all 1s — S2M DRS TEE opcode shall be returned — Optionally log event

1. MemRdTEE, MemRdDataTEE, MemRdFillTEE, and MemSpecRdTEE.

2. MemRd, MemRdData, MemRdFill, and MemSpecRd.

If Read Access Control is not enabled on the target, the target shall not check read requests for possible access control violations.

The target shall not perform Read Access Control when updating MetaValue. See [Section 11.5.4.5.6](#) for requirements for handling MetaValue updates.

If enabled, the target shall perform Read Access Control when updating Extended Metadata (EMD). See [Section 11.5.4.5.7](#) for requirements for handling EMD updates.

11.5.4.5.6 MetaValue Updates for HDM-H

MetaValue is a property of the memory address and unrelated to any of the data associated with that address. Consequently, the target shall not perform access control checks on MetaValue updates.

TEUpdate:

- TEUpdate uses the MetaValue to convey the TE State and does not update MetaValue

For all other (non-TEUpdate) transactions:

- There is no TE State associated with MetaValue
- Targets that implement access control shall ignore access control checks when updating the MetaValue and shall allow MetaValue updates, even if the associated read or write request fails access control checks.

Initiators are responsible for ensuring that changes to MetaValue do not negatively affect data coherency. How an initiator guarantees this is beyond the scope of CXL.

11.5.4.5.7 Extended Metadata Updates

Extended Metadata (EMD) is a property of the data and is updated using the same flows and transactions as data. Consequently, TE State and access control, if enabled, shall be utilized when updating EMD.

If Write Access Control checks fail, the target shall not update EMD.

If Read Access Control checks fail, the target shall return fixed data of all 1s for EMD.

11.5.4.6 Memory Encryption

Protecting data at rest in the target memory device is required for confidential computing and requires memory encryption. The TSP supports both initiator-based and target-based memory encryption capabilities, and an initiator shall utilize one or the other when adding a target to the TCB. If target-based memory encryption is not enabled by the initiator, it shall be the initiator's responsibility to utilize initiator-based memory encryption.

This version of the TSP specification is focused on direct attached CXL devices. In such a configuration, there is a single initiator for a given memory range and all transactions to that memory region shall flow through that initiator. When initiator-based encryption is utilized, it is the initiator's responsibility to ensure that the cryptographic keys are correct for all initiators and to maintain data coherency. Initiator-based encryption implementations that utilize partial writes require the initiator to perform a read of a complete cacheline, update the corresponding bytes, and write the complete cacheline back to the target. The target cannot modify any blocks that are encrypted by an initiator and thus cannot perform the RMW (read-modify-write) that is required to perform partial writes.

Target-based encryption requires Transaction Layer and Link Layer protocol changes to pass a CKID to the target so that the target can correctly choose the key that is utilized for the encryption/decryption of data associated with each transaction.

Both initiator-based and target-based memory encryption are optional. The target reports its supported memory encryption capabilities, and the initiator selects the memory encryption that it needs to utilize.

11.5.4.6.1 Initiator-based Memory Encryption

Initiator-based memory encryption may be utilized, independent of target-based memory encryption being enabled or disabled, as follows:

- Target shall support the MemRdFill memory request, which is required for deadlock prevention with partial writes and initiator-based encryption. Support for this request is a target requirement when indicating TSP Capable support in the DVSEC CXL Capability register.
- If target-based CKID memory encryption is not enabled, the CKID field in the memory transaction is reserved and ignored by the target.

11.5.4.6.2 Target-based Memory Encryption

Targets are not required to implement memory encryption. When memory encryption is implemented on the target, the following applies:

- Encryption shall be implemented using one of the Memory Encryption Algorithms Supported that are reported in Get Target Capabilities Response.
- TSP supports two target-based memory encryption mechanisms:
 - CKID-based encryption requires use of the CKID field in the Transaction Layer to identify a specific key to be utilized when encrypting/decrypting memory contents for a given transaction. This feature requires the use of TEE opcodes in the memory transaction to perform the proper CKID type checking.
 - Range-based encryption utilizes memory range registers that are configured to associate a particular encryption key with a specific memory range and does not rely on the CKID field in the transaction.
 - Host may enable CKID-based or range-based target memory encryption, but shall not enable both.
 - Target shall reject attempts to enable both target-based encryption methods.

11.5.4.6.2.1 CKID-based Memory Encryption

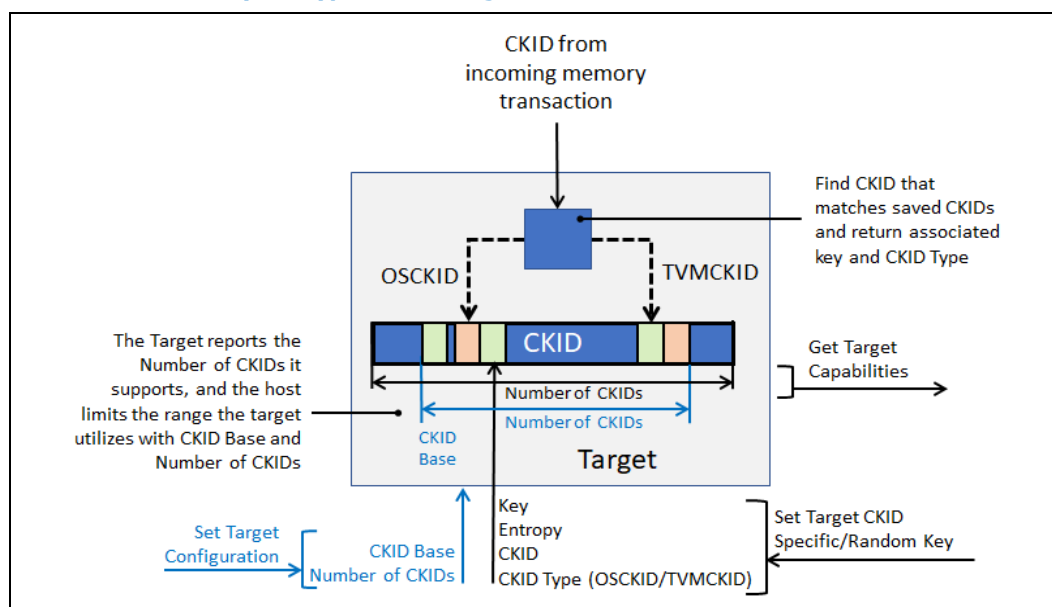
When CKID-based target memory encryption is implemented, the following applies:

- For CKID-based encryption, the initiator and target shall utilize the CKID field in the Transaction Layer for memory requests.
- Each host Root Port has a unique 13-bit CKID value to be utilized across all targets. A host may choose to share CKID values across Root Ports.
- Target shall accept the entire range of the 13-bit CKID field defined in the Transaction Layer.
- CKIDs that are configured on each target can start and end anywhere within the CKID space supported by the protocol.
- Target reports the Number of CKIDs that it supports in Get Target Capabilities. How the target maps the supported number of CKIDs to the 13-bit CKID field in the transaction is target implementation specific.
- Number of CKIDs that the target supports may be sparsely distributed across the 13-bit range, or optionally the target may require that the CKIDs be assigned a contiguous range starting at a specific CKID Base.
- If the target does not require a CKID Base:
 - CKID specified in Set Target CKID Specific Key, Set Target CKID Random Key, and Clear Target CKID Key shall not cause the target to utilize more than the Number of CKIDs that the target supports
- If the target requires a CKID Base to be utilized:
 - Target shall indicate that a range of CKIDs using a CKID Base and Number of CKIDs is required in Get Target Capabilities
 - Host shall configure a contiguous range of CKIDs on the target by specifying the CKID Base and Number of CKIDs that the target shall utilize with Set Target Configuration. The Number of CKIDs enabled by the host shall be $\leq$ Number of CKIDs reported by the target.
 - CKID in the Transaction Layer memory requests from the initiator shall be $\text{CKID Base} \leq \text{CKID} < \text{CKID Base} + \text{Number of CKIDs configured on the target}$. See [Table 11-24](#) for the target behavior if the CKID in the transaction is outside the configured CKID of the target.

- CKID specified by the host in Set Target CKID Specific Key, Set Target CKID Random Key, and Clear Target CKID Key shall be $\text{CKID Base} \leq \text{CKID} < \text{CKID Base} + \text{Number of CKIDs configured on the target}$.
- Initiator may configure a specific CKID to a specific initiator-supplied key by utilizing Set Target CKID Specific Key.
- Initiator may configure a specific CKID to a random target-generated key with optional initiator-supplied key entropy by utilizing Set Target CKID Random Key.
- Initiator may clear a previously configured key to allow the CKID to be recycled for another key by utilizing Clear Target CKID Key sent on the same session that was utilized to set the key. If the session utilized to set the key has terminated or closed then the target may need to be reset to break the memory encryption key to CKID association.
- Target implements the mapping of keys to CKIDs.
- Each CKID shall be a TVMCKID or OSCKID that is configured utilizing the CMA/SPDM PrimarySession or SecondarySession(s) (see Figure 11-28, which describes this partitioning). See the description below on how the target utilizes the CKID Type to verify memory transactions and the behavior of the target when the verification fails.
- TSM shall be responsible for setting TVMCKIDs.
- When utilizing target CKID-based memory encryption, the initiator's transactions that contain TVMCKIDs shall utilize TEE opcodes and those that contain OSCKIDs shall utilize non-TEE opcodes. The target shall verify that the opcodes are correct for accessing each CKID Type.
- If CKID target based encryption has not yet been enabled, the CKID field in the memory transaction is reserved and ignored by the target and no encryption shall be performed by the target until it is successfully enabled.

Figure 11-28 demonstrates the target mixing of TVMCKIDs and OSCKIDs in a single range of possible CKIDs that utilize the CKID Base and Number of CKIDs.

Figure 11-28. CKID-based Memory Encryption Utilizing CKID Base



The target stores the CKID Type along with the CKID and key. The target shall verify that each incoming memory transaction passes the following checks:

- The memory transaction CKID is within the valid configured range of the target
- The memory transaction has a non-TEE opcode and the CKID is an OSCKID
- The memory transaction has a TEE opcode and the CKID is a TVMCKID

If any of these checks fail, the target should provide an event log entry to aid in root-cause analysis of unexpected behavior by reporting a General Media or DRAM Event Record with Memory Event Type of CKID Violation.

Table 11-24 outlines the target behavior for memory transactions that request a CKID that is outside the configured range of the target.

Table 11-24. Target Behavior for Invalid CKID Ranges

CKID	Memory Transaction Received by the Target	
	Read Opcodes	Write Opcodes
Within Configured Range	• Allowed	• Allowed
Outside Configured Range	• Reads shall return fixed data of all 1's • Optionally log event	• Writes shall be dropped • Optionally log event

Table 11-25 outlines the target behavior for the CKID-type verification checks based on TEE Intent using TEE opcodes and non-TEE opcodes, which is described in Section 11.5.4.5.

Table 11-25. Target Behavior for Verifying CKID Type

Target's CKID type Associated with CKID Contained in Memory Transaction	TEE Intent of Memory Transaction Received by the Target	
	TEE Opcodes ¹	Non-TEE Opcodes ²
OSCKID	• Writes: — Writes shall be dropped — S2M NDR non-TEE opcode shall be returned — Optionally log event • Reads: — Reads shall return fixed data of all 1s — S2M DRS non-TEE opcode shall be returned — Optionally log event	• Writes: — Full/Partial cacheline write allowed — S2M NDR non-TEE opcode shall be returned • Reads: — Allowed — S2M DRS non-TEE opcode shall be returned
TVMCKID	• Writes: — Full/Partial cacheline write allowed — S2M NDR TEE opcode shall be returned • Reads: — Allowed — S2M DRS TEE opcode shall be returned	• Writes: — Writes shall be dropped — S2M NDR TEE opcode shall be returned — Optionally log event • Reads: — Reads shall return fixed data of all 1s — S2M DRS TEE opcode shall be returned — Optionally log event

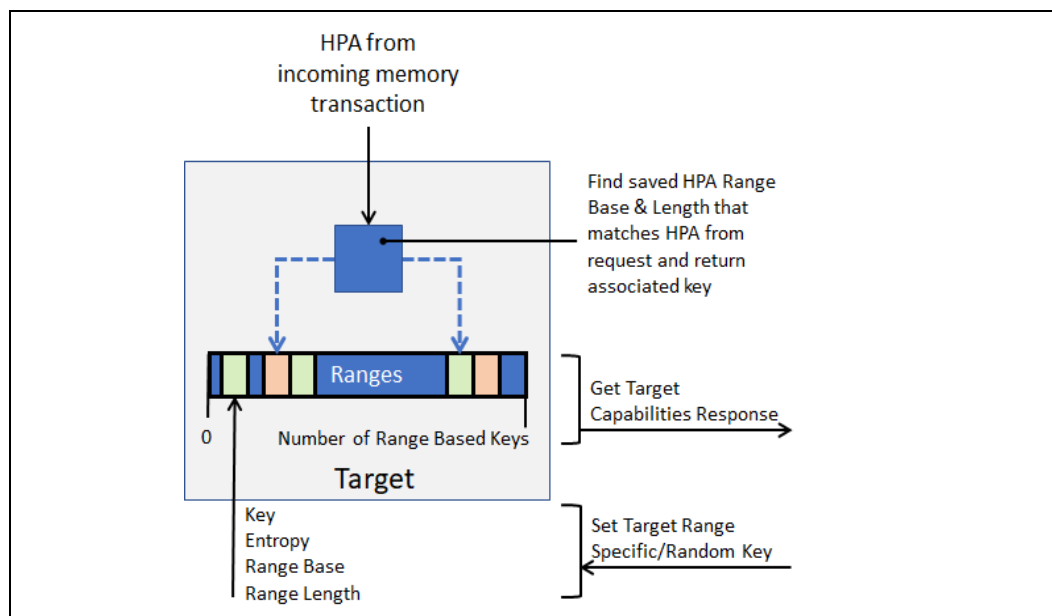
1. MemWrTEE, MemWrPtTEE, MemRdTEE, MemRdDataTEE, MemRdFillTEE, and MemSpecRdTEE.
2. MemWr, MemWrPt, MemRd, MemRdData, MemRdFill, and MemSpecRd.

11.5.4.6.2.2 Range-based Memory Encryption

- The initiator may configure a specific HPA memory range to use a specific initiator-supplied key that utilizes Set Target Range Specific Key.
- The initiator may configure a specific HPA memory range to use a random target-generated key with optional initiator-supplied key entropy that utilizes Set Target Range Random Key.
- The initiator may clear a previously configured key to allow the memory range to be recycled for another key by utilizing Clear Target Range Key. If the session utilized to set the key has terminated or closed then the target may need to be reset to break the memory encryption key to memory range association.
- The target advertises the Memory Encryption Number of Range Based Keys that it supports in Get Target Capabilities, and the initiator assigns a Range ID and HPA range at the time the key is programmed. The target shall verify that the HPA memory ranges and Range IDs do not overlap. The HPA contained within the memory request shall be compared to the programmed memory ranges by the target to retrieve the correct key to utilize in the encryption. The Range ID specified in Set Target Range Specific Key and Set Target Range Random Key shall be $0 \leq \text{Range ID} < \text{Memory Encryption Number of Range Based Keys}$.
- The PrimarySession or SecondarySession(s) shall be utilized for setting and clearing range based keys. The session utilized to set a range based key shall be the same session utilized to clear the same range based key.

Figure 11-29 outlines range-based memory encryption utilizing the HPA.

Figure 11-29. Range-based Memory Encryption



11.5.4.7 Transport Security

Transport security is optional for TSP. If supported and enabled, a CXL-approved Integrity and Data Encryption (IDE) mechanism shall be used. It is up to the TEE policy to decide whether targets that do not support transport security invalidates their use for confidential computing scenarios.

Currently hop-by-hop CXL IDE is the only defined CXL transport security mechanism, and determining the target's IDE capabilities or enabling IDE modes is done through CXL IDE-defined registers. There are no Transport Security-specific TSP interfaces defined at this time.

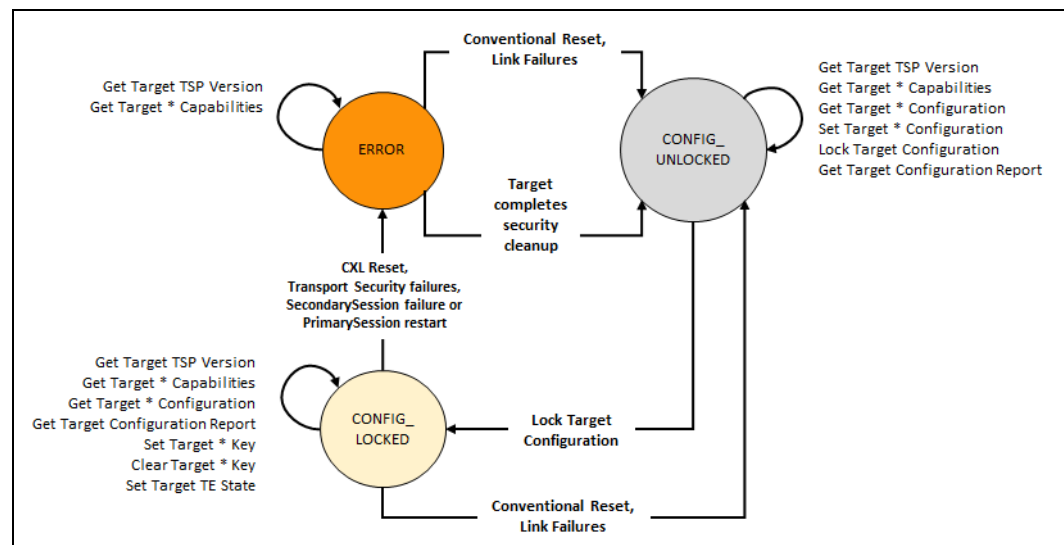
Warning: Although Transport Security is optional, disabling of Transport Security adds additional exposure to physical attacks such as device removal and manipulator-in-the-middle because the binding provided by Transport Security is not utilized. If these attacks are considered part of the threat model for the TEE and cannot be protected via other methods, then Transport Security shall be enabled for confidential computing scenarios.

11.5.4.8 Configuration

The PrimarySession shall be utilized to configure the security for each target in the CXL hierarchy that utilizes TSP, supported CXL Transport Security mechanisms, and other interfaces to setup the TCB. After the targets are correctly configured, the PrimarySession shall be utilized to lock the target to disable initiator access to registers that could cause data coherency issues, loss of data, reveal TVM data to an untrusted VM, and/or otherwise compromise the TCB. After the target is locked, trusted memory transactions are allowed, and the target or initiator shall perform access verification by utilizing the TE State.

Figure 11-30 outlines the defined target security states that are utilized by TSP.

Figure 11-30. Target TSP Security States



- **CONFIG_UNLOCKED**
 - Default security state after Conventional Reset (see [Section 11.5.4.8.3](#) for details).
 - TSP security configuration is performed in this state.
 - TEE opcode transactions are not allowed by the target in this state (see [Section 11.5.4.10.1](#)).
 - Non-TEE opcode transactions are allowed as discussed in [Section 11.5.4.5](#).
 - Transition to CONFIG_LOCKED state:
 - After successfully locking the target.

- CONFIG_LOCKED
 - Restrictions placed on register accesses, CCI commands to protect the TCB. See the Lock Target Configuration interface description (see [Section 11.5.5.6.7](#) and [Section 11.5.5.6.8](#)) for more details on target behavior after locking.
 - Target shall save TE State and if enabled, enforce TE State checking.
 - Assigning keys to CKIDs or memory ranges, clearing of keys allowed in this state.
 - Non-TEE opcode transactions are allowed as discussed in [Section 11.5.4.5](#).
 - TEE opcode transactions allowed as discussed in [Section 11.5.4.5](#).
 - Transition to ERROR state:
 - The following are treated as errors that shall transition the target to the ERROR state: Transport Security failures (e.g., CXL IDE becoming insecure), CXL Reset (which doesn't reset the entire device), and/or other cases in which the link can no longer be trusted or only a portion of the device is affected by the error (see [Section 11.5.4.8.3](#) for details).
 - Transition to CONFIG_UNLOCKED state:
 - Upon receipt of a Conventional Reset (see [Section 11.5.4.8.3](#) for details).
- ERROR
 - The target shall continue to protect all TVM data when in the error state.
 - After the target enters the error state, the target shall stop accepting all future TEE memory transactions. For transactions that were accepted prior to going to the error state, it is permissible to handle those transactions as normal. See [Section 11.5.4.8.3](#) for details.
 - Non-TEE opcode transactions are allowed as discussed in [Section 11.5.4.5](#).
 - Transition to CONFIG_UNLOCKED state:
 - The target shall automatically transition from the ERROR to the CONFIG_UNLOCKED state after it has cleaned up the current secure sessions and data (see [Section 11.5.4.8.3](#) for details).
 - Upon receipt of a Conventional Reset (see [Section 11.5.4.8.3](#) for details).

Use of these states and the target's behavior in each state are further detailed in [Section 11.5.5](#).

- The TSP architecture assumes that a single authority model shall be utilized for configuration and locking of the target:
 - The host owns all configuration policies
 - The host establishes and locks the security configuration for all targets within its domain
 - MH-SLDs implement separate target configurations for each host, thereby allowing each host to independently configure its SLDs

11.5.4.8.1 Locking the Target

As a pre-condition to performing the memory security checks, the DSM shall first lock the configuration to ensure that it cannot be modified during or after completion of the memory security checks. The mechanisms used by the DSM to lock the memory controller configuration, HDM decoder configuration, and other configuration context specific to the target micro-architecture are beyond the scope of this specification.

The target shall implement configuration and security checks that verify the locked configuration before successfully responding to the lock configuration request. Memory security checks are mechanisms that the target implements to verify whether its configuration is locked down and are acceptable to meet the confidential computing security objectives of protecting the TVM data. The TSM RoT relies on the DSM for these memory security checks.

- General Assumptions:
 - Any target operation mode that exposes internal data or allows data logging or tracing is disabled
 - Any register that can lead to data corruption shall be locked for writing.
 - Registers leading to reset events that are guaranteed to transition the DSM to ERROR or CONFIG_UNLOCKED state are not required to be locked for writing.
 - Reading of security-sensitive registers shall be blocked. Reading of non-security-sensitive registers do not need to be blocked.
 - Transport Security cannot be assumed to be enabled.
- As part of the memory security checks, the DSM shall ensure the following:
 - HDM decoders in the target are configured consistently and with no aliases. These decoders are specific to the target's implementation. An alias is present if two HPAs decode to the same DPA.
 - Memory controllers and other logic in the target are configured consistently to meet the confidential computing security objectives. Such configuration shall be specific to the target implementation. Examples of such configuration include the DIMM population registers, interleave configuration, error-detection capabilities, ECC mode configuration, debug capabilities such as error/pattern injection logic, target row refresh controls, etc.
 - HDM decoders of the LD are in a consistent state and do not have aliases or overlaps. The HDM decoders shall be locked with the Lock on Commit bit in the HDM Decoder Control register set to 1.
 - Addresses that are decoded by the HDM decoders shall not overlap with addresses that are decoded as PCIe memory space for MMIO. The target shall prevent the overlapping of PCIe BARs.
 - Error-detection capabilities required to ensure security of TVM data shall be enabled in the Uncorrectable Error Mask register of the CXL RAS Capability structure.
 - CXL link-specific registers for the enabled TSP Transport Security mechanism control registers are programmed with parameters that are validated to be safe to support the confidential computing security objectives.
 - If the target is configured to utilize Transport Security, all target CXL.cachemem links shall have Transport Security enabled to protect the security of the transport.
 - PCIe DVSEC for CXL target shall be locked by asserting CONFIG_LOCK.
 - PCIe DVSEC for Test Capability, if implemented, shall be locked by asserting TestLock and no test algorithms, test capabilities, and/or error injection methods are currently active.
 - No test algorithms, test capabilities, and/or error injection methods are currently active or configured through the compliance DOE mailbox.
 - Other target implementation-specific configuration checks as defined by the micro-architecture of the target.
 - CXL Capabilities are configured on the target to prevent the leaking of cypher text from the target after a CXL Reset, as discussed in [Section 11.5.4.8.3](#).

- Successfully locking the target should result in the following target behavior:
 - The TEE's memory range associated with the target is locked by making the HDM decoder configuration immutable after the target is locked.
 - The target shall limit supported TSP requests to the subset of requests that are allowed when the target is in the CONFIG_LOCKED state. See [Table 11.5.5.1](#) for the TSP requests that are allowed on a locked target.
 - Prevention of surprise changes to the target configuration that would allow unauthorized access to data that was written by a TVM, cause the target or initiator to break the data coherency model, and/or otherwise compromise TEE integrity. This may require the target to drop host writes to PCIe or CXL registers after the target is locked.
 - The Get Target Configuration Report request and response (see [Section 11.5.5.6.6](#)) shall allow the initiator to securely retrieve and verify the content of specific PCIe and CXL configuration registers, after the target is locked. This allows the TSM RoT to check the configuration, securely independent of standard insecure PCIe or CXL register accesses.
 - The following actions shall be prevented by making the associated registers immutable after the target is locked:
 - Memory aliasing
 - CXL HDM Decoder Global Control register: Changing the HDM Decoder Enable state at runtime could allow an attacker to change the address for a write transaction, potentially affecting the data coherency and/or the TE State maintained on the target.
 - CXL HDM Decoder [n] Low/High, Size Low/High, and DPA Skip Low/High Registers: Changing the HDM decoder programming at runtime could allow an attacker to change the address for a write transaction, potentially affecting the data coherency and/or the TE State maintained on the target.
 - The host shall lock all target HDM decoders that were programmed for interleave sets that are within the TEE, utilizing the Commit On Lock feature. Register writes to locked HDM decoders are dropped.
 - Writes to HDM decoders that are unlocked, not already programmed (size and skip = 0), and not considered part of the TEE, need to be allowed by the target. However, the target shall perform alias checking of those register writes to ensure that they do not alias address ranges that are already locked and considered to be in the TEE, independent of whether the host has set the Commit On Lock bit during decoder programming. The target shall treat aliased HDM decoders as a programming failure and shall behave the same way as if the Commit On Lock bit was set when the HDM decoder alias checks failed. The failed checks shall have no effect on the target, the TEE, and/or in-flight transactions between them.
 - Altering target behavior at runtime
 - CXL HDM Decoder Global Control register: Disabling the Poison On Decode Error Enable bit at runtime could cause an attacker to affect the data coherency of the target and allow the initiator to potentially consume invalid data and/or a fixed data pattern as valid data.
 - CXL Control register.
 - CXL Range 1 Base High/Low registers.
 - CXL Range 2 Base High/Low registers.
- Altering TEE memory interleave configuration at runtime

- CXL HDM Decoder [n] Control register(s): Altering IG, IW, Range Type, BI, UIO, UIG, UIW, and/or ISP at runtime could allow an attacker to steer transactions to another target and/or memory range, potentially affecting the data coherency and/or the memory integrity of the TE State stored in the target.
- Altering link or Transaction Layer behavior at runtime
 - PCIe DVSEC for Flex Bus Port registers.
 - CXL Timeout and Isolation Control register.
 - CXL IDE Control register (if utilizing CXL IDE for Transport Security): Altering PCRC Disable or IDE.StopEnable at runtime could allow the link to operate incorrectly if there is a mismatch with the initiator. This should result in a MAC error which should cause the IDE on the link to transition to Insecure State (if IDE is enabled) and is indicated in the CXL IDE Error Status register, bit[1] and bit[7]. When IDE transitions to Insecure State, the target shall transition to the ERROR state.
- Prevention of CCI mailbox commands that could allow untrusted access to data that was written by a TVM, cause the target or initiator to break the data coherency model, and/or otherwise compromise TEE integrity (see [Section 11.5.4.9](#) for details).
- TEE opcodes shall be allowed by the target as outlined in the Access Control section (see [Section 11.5.4.5](#)).
- Enforce Read and/or Write Access Control if enabled (see [Section 11.5.4.8](#)).
- Prevention of surprise changes to the target configuration for any target implementation-specific or vendor-implementation-specific registers or CCI mailbox commands that could allow untrusted access to data that was written by a TVM, cause the target or initiator to break the data coherency model, and/or otherwise compromise TEE integrity. This requires implementation-specific analysis and is beyond the scope of the TSP.

11.5.4.8.2 Considerations for Securing the Host

- The host is responsible for securing the Root Ports to secure the TEE environment, and such mechanism is beyond the scope of the TSP.
- The host should prevent surprise changes to the host configuration that would allow unauthorized access to data that was written by a TVM, cause the target and/or host to break the data coherency model, and/or otherwise compromise TEE integrity.
- Some PCIe and CXL Root Port registers that should be protected by the host are outlined here (this is not an exhaustive list, but an example of some of the registers that affect the initiator's link to the locked target):
 - Memory aliasing
 - CXL HDM Decoder Global Control register: Changing the HDM Decoder Enable state at runtime could allow an attacker to change the address for a write transaction, potentially affecting the data coherency and/or the TE State maintained on the target.
 - CXL HDM Decoder [n] Low/High, Size Low/High, and Target List Low/High registers: Changing the HDM decoder programming at runtime could allow an attacker to change the address for a write transaction, potentially affecting the data coherency and/or the TE State maintained on the target.
 - The host shall lock all HDM decoders that were programmed for interleaved sets that are within the TEE, utilizing the Commit On Lock feature. Register writes to locked HDM decoders are dropped.

- Writes to HDM decoders that are unlocked, not already programmed (size and skip = 0), and not considered part of the TEE need to be allowed by the initiator.
- Altering host behavior at runtime
 - CXL HDM Decoder [n] Global Control register: Disabling the Poison On Decode Error Enable bit at runtime could cause an attacker to affect the data coherency of the target and allow the host to potentially consume invalid data and/or a fixed data pattern as valid data.
 - CXL BI Decoder Control register: Toggling of BI Enable during runtime could affect data coherency of HDM-DB memory. If BI Decoder Commit is not toggled after reassigning bus numbers, data could be steered to the wrong location, thereby causing HDM-DB memory coherency issues.
 - CXL Cache ID RT Control register: If Cache ID RT Commit toggles at runtime and the Route Table contains invalid routing information, this could cause transactions to be steered to the wrong location, thereby causing data coherency issues.
 - CXL Cache ID Target register: Not toggling Valid and Port Number after a Route Table change could cause transactions to be steered to the wrong location, thereby causing data coherency issues.
- Altering memory interleave configuration at runtime
 - CXL HDM Decoder [n] Control register(s): Altering IG, IW, Range Type, BI, UIO, UIG, UIW, and/or ISP at runtime could allow an attacker to steer transactions to another target and/or memory range, potentially affecting the data coherency and/or the memory integrity of the TE State stored in the target.
- Altering link or Transaction Layer behavior at runtime
 - PCIe DVSEC for Flex Bus Port registers
 - CXL IDE Control register (if utilizing CXL IDE for Transport Security): Altering PCRC Disable or IDE.StopEnable at runtime could allow the link to operate incorrectly if there is a mismatch with the initiator. This should result in a MAC error which should cause the IDE on the link to transition to Insecure State (if IDE is enabled) and is indicated in the CXL IDE Error Status register, bit[1] and bit[7]. When IDE transitions to Insecure State, the target shall transition to the ERROR state.
- Configuring CXL Capabilities on the target to prevent the leaking of cypher text from the target after a CXL Reset, as discussed in [Section 11.5.4.8.3](#).

11.5.4.8.3 Reset and Error Handling Behavior of the Target

This section outlines the expected target TSP behavior to Conventional Reset, CXL Reset, Transport Security Failures, and changes in secure sessions. All other resets, including FLR, do not affect the secured link or Transport Security (i.e., CXL IDE) and therefore do not change the target's TSP security state.

11.5.4.8.3.1 Conventional Reset and Link Failures

Because a Conventional Reset or a Link Failure shall take down the CXL links to the target, all transactions shall immediately terminate. The target shall perform the following after receiving a Conventional Reset or on link failure:

- Terminate the CMA/SPDM SecondarySession(s) and PrimarySession
- Clear CMA/SPDM secure session keys to 0
- Target shall ensure that all TE State = 1 clear text data cannot be leaked
- If implementing target-based encryption:

- Clear association of CKID to encryption keys
- Clear association of HPA ranges to encryption keys
- Clear memory encryption keys
- Transition to CONFIG_UNLOCKED state

11.5.4.8.3.2 CXL Reset, Transport Security Failures, SecondarySession(s) Termination, and PrimarySession Restart

CXL Reset, Transport Security failures such as IDE going insecure, SecondarySession(s) terminating, and/or a new PrimarySession being started all affect the target but do not take down the link. For these conditions, the target shall support the following additional requirements:

- Transition to ERROR state
- Target shall stop accepting all future TEE memory transactions
 - **Writes:** TEE opcode writes shall be dropped and S2M NDR non-TEE opcode shall be returned for the write response.
 - **Reads:** TEE opcode reads return a fixed data pattern of all 1s. S2M DRS non-TEE opcode shall be returned for the read response.
- Terminate the CMA/SPDM SecondarySession(s) and PrimarySession
- Clear CMA/SPDM secure session keys to 0
- Target shall ensure that all TE State = 1 clear text data cannot be leaked
- If implementing target-based encryption:
 - Clear association of CKID to encryption keys
 - Clear association of HPA ranges to encryption keys
 - Clear memory encryption keys
- Transition to CONFIG_UNLOCKED state

Prevention of the leaking of cypher text from the target after a CXL Reset:

- Data on the target is encrypted using initiator-based or target-based memory encryption. Clear text data is protected; however, the cipher text stored on the target could be leaked after a CXL Reset, which should be avoided.
- Target should support clearing or randomizing of volatile HDM data on reset and indicate this capability by setting CXL Reset Mem Clr Capable in DVSEC CXL Capability to 1 and by setting Default Volatile HDM State after Cold Reset, Default Volatile HDM State after Warm Reset, and Default Volatile HDM State after Hot Reset to 1 in DVSEC CXL Capability3.
- Target should set Volatile HDM State after Hot Reset Configurability in DVSEC CXL Capability3 to 0 to indicate that the target is not capable of preserving volatile HDM state across Hot Reset.
- Target shall support resetting all TE State to 0 when enabling the ability to clear or randomize all data in response to a CXL Reset.
- If the target sets CXL Reset Mem Clr Capable to 1, the host should allow the target clear memory during CXL Reset by setting CXL Reset Mem Clr Enable in DVSEC CXL Status.

11.5.4.9 Component Command Interfaces

When the target is locked, Component Command Interface (CCI) commands that allow the target's configuration to be altered, new features to be set, maintenance operations to be executed, target memory partitioning changes, injecting and clearing of poison, sanitizing, and/or secure erasing shall be rejected by the target by returning Invalid Security State status.

Likewise, FM requests that can change the Dynamic Capacity configuration at runtime shall be rejected by the target by returning Invalid Security State status.

See [Section 8.2.9](#) for the following command-specific additions that are relevant to TSP support:

- Transfer FW, Activate FW
- Set Features
- Perform Maintenance
- Set Partition Info
- Inject Poison, Clear Poison
- Sanitize, Secure Erase, Passphrase Secure Erase, Security Send
- Release Dynamic Capacity

See [Section 7.6.7.6.3](#) for the following command-specific addition that is relevant to TSP support:

- Set DC Region Configuration

11.5.4.10 Dynamic Capacity

As described above, CXL security requires the device to implement configuration locking to prevent tampering with the trusted configuration at runtime. Dynamic Capacity allows for memory capacity to be added or released from one or more hosts at runtime but relies on a static maximum capacity configuration to have been configured at initialization time.

The following sections outline specific areas in which there are additional requirements for targets that implement both Dynamic Capacity and TSP.

11.5.4.10.1 TE State Changes

For Dynamic Capacity targets that implement TE State changes, there are additional responsibilities:

- Before a target adds any Dynamic Capacity to a host and before adding capacity to one host after releasing the capacity from a different host, the target shall overwrite or cryptographically clear the memory contents and reset the TE State to 0 for memory ranges described in extents that will be added to a host in response to an Add Dynamic Capacity command (see [Section 8.2.9.9.3](#)). By overwriting those ranges and resetting the TE State, the extents that will be added shall be reset to an untrusted state to prevent stale trusted data released from one host from being exposed to a trusted entity when adding that memory to another host.

11.5.4.10.2 Multiple Host Considerations

For Dynamic Capacity implementations that utilize multiple hosts, only a single host may generate transactions to a specific DPA range at any given time.

- Target shall maintain an association between the host in which the Dynamic Capacity is being added and the target that is providing the memory. The target

shall not allow other hosts to access the memory until the memory has been released from the current host, at which point the host to target association is removed and the TE State is reset as described above.

- Note that implementing Dynamic Capacity on a target already requires the target to maintain this host to target association and the target shall correctly reject transactions from hosts that do not currently own the Dynamic Capacity. This behavior is described in [Section 9.13.3](#).

11.5.5 TSP Requests and Responses

Each TSP Request sent over the secure CMA/SPDM session shall result in exactly one TSP Response, the Delayed Response if the request will take significant time to complete, or the Error Response if the request could not be executed.

11.5.5.1 TSP Request Overview

Table 11-26 outlines the TSP Request payloads, defined in the sections that follow.

Table 11-26. TSP Request Overview

TSP Request Message		Message Support ¹	Payload Size	Legal TSP State	TSP Usage
Opcode	Name	HDM-H Device			
81h	Get Target TSP Version	M	4	CONFIG_UNLOCKED CONFIG_LOCKED ERROR	Target config
82h	Get Target Capabilities		4		
83h	Set Target Configuration		C2h+	CONFIG_UNLOCKED	
84h	Get Target Configuration		4	CONFIG_UNLOCKED CONFIG_LOCKED	
85h	Get Target Configuration Report		8		
86h	Lock Target Configuration		4	CONFIG_UNLOCKED	Target config lock
87h	Set Target CKID Specific Key	O - CKID-based target memory encryption	10h+	CONFIG_LOCKED	Runtime CKID-based target memory encryption
88h	Set Target CKID Random Key		10h+		
89h	Clear Target CKID Key		8		
8Ah	Set Target Range Specific Key	O - Range-based target memory encryption	20h+		Runtime range-based target memory encryption
8Bh	Set Target Range Random Key		20h+		
8Ch	Clear Target Range Key		8		
8Dh	Set Target TE State	O - Explicit TE State changes	20h+		Runtime explicit state changes
8Eh	Check Target Delayed Completion	O - Delayed completion of a request	4		Checking for completion of a long executing request

1. M = Mandatory message, O = Optional message. Targets shall return a Error Response of Unsupported Request if the target does not support the request.

11.5.5.2 TSP Response Overview

Table 11-27 outlines the TSP Response payloads, defined in the sections that follow.

Table 11-27. TSP Response Overview

TSP Response Message		Message Support ¹	Payload Size
Opcode	Name	HDM-H Device	
01h	Get Target TSP Version Response	M	5+
02h	Get Target Capabilities Response		34h
03h	Set Target Configuration Response		4
04h	Get Target Configuration Response		C0h
05h	Get Target Configuration Report Response		8+
06h	Lock Target Configuration Response		4
07h	Set Target CKID Specific Key Response	O - CKID-based target memory encryption	4
08h	Set Target CKID Random Key Response		4
09h	Clear Target CKID Key Response		4
0Ah	Set Target Range Specific Key Response	O - Range-based target memory encryption	4
0Bh	Set Target Range Random Key Response		4
0Ch	Clear Target Range Key Response		4
0Dh	Set Target TE State Response	O - Explicit TE State changes	4
0Eh	Check Target Delayed Completion Response	O - Delayed completion of a request	4
7Eh	Delayed Response		8
7Fh	Error Response	M	0Ch+

1. M = Mandatory message, O = Optional message.

11.5.5.3 Request Response and CMA/SPDM Sessions

Table 11-28 outlines which TSP-defined Request and Response payloads are allowed on a given TSP-defined CMA/SPDM session and which are prohibited.

Table 11-28. TSP Request Response and CMA/SPDM Sessions

TSP Request Response Message	SPDM PrimarySession	SPDM SecondarySession(s)	Other non-TSP-related SPDM Session
Get Target TSP Version Get Target TSP Version Response	Allowed	Allowed	Allowed
Get Target Capabilities Get Target Capabilities Response		Allowed	Allowed
Set Target Configuration Set Target Configuration Response		Prohibited	Prohibited
Get Target Configuration Get Target Configuration Response		Allowed	Prohibited
Get Target Configuration Report Get Target Configuration Report Response		Allowed	Prohibited
Lock Target Configuration Lock Target Configuration Response		Prohibited	Prohibited
Set Target TE State Set Target TE State Response		Allowed	Prohibited
Check Target Delayed Completion Check Target Delayed Completion Response		Allowed	Prohibited
Set Target CKID Specific Key Set Target CKID Specific Key Response		Allowed	Prohibited
Set Target CKID Random Key Set Target CKID Random Key Response		Allowed	Prohibited
Clear Target CKID Key Clear Target CKID Key Response		Allowed	Prohibited
Set Target Range Specific Key Set Target Range Specific Key Response		Allowed	Prohibited
Set Target Range Random Key Set Target Range Random Key Response		Allowed	Prohibited
Clear Target Range Key Clear Target Range Key Response		Allowed	Prohibited
Delayed Response		Allowed	Allowed
Error Response		Allowed	Allowed

11.5.5.4 Version

11.5.5.4.1 TSP Version Negotiation

The PrimarySession shall be utilized to perform the following process to negotiate TSP version with TSP-capable targets:

- Initiator shall send Get Target TSP Version request with Major Version Number = 1h.
- DSM shall support Get Target TSP Version request with Major Version Number = 1h and shall return Get Target TSP Version Response with a list of all supported versions.
- Initiator shall select a common (typically highest) version supported and utilize this version number in all subsequent messages to the target.

- Initiator shall not issue requests to the target other than Get Target TSP Version until the initiator has received a successful Get Target TSP Version Response and selected a common version that is supported by both the initiator and the target.

11.5.5.4.2 Get Target TSP Version

The initiator shall utilize Get Target TSP Version to discover the TSP versions that the target supports.

Possible Error Response, Error Codes:

- None

Table 11-29. Get Target TSP Version

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target TSP Version = 81h.
02h	2	Reserved

11.5.5.4.3 Get Target TSP Version Response

If no error condition is detected, the DSM shall respond to the Get Target TSP Version with a Get Target TSP Version Response message.

Table 11-30. Get Target TSP Version Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target TSP Version Response = 01h.
02h	2	Reserved
04h	1	Version Number Entry Count: The number of version entries, N, that follow. Shall be > 0.
05h	N	Version Number Entry: 8-bit version entry formatted as: - Bits[7:4]: Major Version Number = 1 - Bits[3:0]: Minor Version Number = 0

11.5.5.5 Target Capabilities

The following request and response payload defines the TSP security features that the target supports.

11.5.5.5.1 Get Target Capabilities

Any SPDMM session may utilize the Get Target Capabilities request to discover the target's memory encryption, access control, and configuration capabilities.

Possible Error Response, Error Codes:

- Version Mismatch

Table 11-31. Get Target Capabilities

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target Capabilities = 82h.
02h	2	Reserved

11.5.5.5.2 Get Target Capabilities Response

If no error condition is detected, the DSM shall respond to the Get Target Capabilities request with a Get Target Capabilities Response message.

Table 11-32. Get Target Capabilities Response (Sheet 1 of 3)

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target Capabilities Response = 02h.
02h	2	Memory Encryption Features Supported: Memory encryption related features that the target supports. Zero or more bits may be set. 1 indicates supported, 0 indicates not supported. • Bit[0]: Encryption: When set, memory encryption for data at rest is implemented on the target. • Bit[1]: CKID-based Encryption: When set, the target supports the CKID-based TSP requests and responses for memory encryption, Encryption shall be set, and the CKID Base Required and Number of CKIDs fields shall be valid. When cleared, the target does not support the CKID field in Transaction Layer requests. • Bit[2]: Range-based Encryption: When set, the target supports the range-based TSP requests and responses, Encryption shall also be set, and Memory Encryption Number of Range Based Keys shall be valid. • Bit[3]: Initiator Supplied Entropy: The target supports initiator-supplied entropy when generating a random key. • Bit[4]: CKID Base Required: Valid only when CKID-based Encryption is set. When set, the target requires a CKID Base and Number of CKIDs to be programmed. When cleared, the target supports any CKID value within the 13-bit field. • Bits[15:5]: Reserved.
04h	4	Memory Encryption Algorithms Supported: Valid only if Encryption is set in Memory Encryption Features Supported. 1 indicates supported, 0 indicates not supported. If target memory encryption is supported, one or more bits shall be set. • Bit[0]: AES-XTS-128 • Bit[1]: AES-XTS-256 • Bits[30:2]: Reserved • Bit[31]: Vendor Specific Algorithm: When set, all other bits in this field are vendor specific
08h	2	Memory Encryption Number of Range Based Keys: Valid only if Range-based Encryption is set in Memory Encryption Features Supported. This is the maximum Range ID that can be utilized with the range-based memory encryption requests. Targets that do not support range-based memory encryption shall report 0.
0Ah	2	Reserved

Table 11-32. Get Target Capabilities Response (Sheet 2 of 3)

Byte Offset	Length in Bytes	Description
0Ch	2	TE State Change and Access Control Features Supported: The TE State change and access control features that the target supports. Zero or more bits may be set. 1 indicates supported, 0 indicates not supported. • Bit[0]: Write Access Control: When set, indicates that the target supports dropping writes that fail the verification of TEE Intent to stored TE State. When set, explicit state changes shall be supported and one or more of bits[4:3] shall also be set. • Bit[1]: Read Access Control: When set, indicates that the target supports returning all 1s for read data in response to reads that fail the verification of TEE Intent to stored TE State. When set, one or more of bits[4:2] shall also be set. • Bit[2]: Implicit TE State Change: When set, indicates that the target supports implicit TE State changes using a 64B granularity, Explicit In-band TE State Change shall be set, and Explicit In-band TE State Granularity support for 64B shall be set. • Bit[3]: Explicit Out-of-band TE State Change: When set, indicates that the target supports the CMA/SPDM out-of-band explicit Set Target TE State change message and the Supported Explicit Out-of-band TE State Granularity field shall be valid. Support is optional for targets that support implicit TE State changes or explicit in-band TE State changes. • Bit[4]: Explicit In-band TE State Change: When set, indicates that the target supports explicit TE State changes utilizing the TEUpdate memory transaction and the Supported Explicit In-band TE State Granularity field shall be valid. Support is required for targets that support implicit TE State changes and optional for targets that support explicit out-of-band TE State changes. • Bit[5]: Explicit TE State Change Sanitize: When set, indicates that the target supports overwriting data that is affected by the explicit state change with 0s when the explicit request is received and before the change is considered complete by the target. When set, one or more of bits[4:3] shall also be set. • Bits[15:6]: Reserved.
0Eh	2	Reserved
10h	4	Supported Explicit Out-of-band TE State Granularity: The granularity the target supports for explicit out-of-band TE State changes and verification in powers of 2, starting with 64B. Valid only if Explicit Out-of-band TE State Change is set in TE State Change and Access Control Features Supported. One or more bits shall be set. 1 indicates target support, 0 indicates no target support. • Bit[0]: 64B • ... • Bit[6]: 4K • ... • Bit[15]: 2MB • ... • Bit[24]: 1GB • ... • Bit[31]: 128GB
14h	4	Supported Explicit In-band TE State Granularity: The granularity the target supports for explicit in-band TE State changes and verification in powers of 2 starting with 64B. Valid only if Explicit In-band TE State Change is set in TE State Change and Access Control Features Supported. One or more bits shall be set. 1 indicates target support, 0 indicates no target support. • Bit[0]: 64B • Bit[1]: 128B • Bit[2]: 256B • Bit[3]: 512B • Bit[4]: 1KB • Bit[5]: 2KB • Bit[6]: 4KB • Bit[7]: 8KB • Bit[8]: 16KB • Bit[9]: 32KB • Bit[10]: 64KB • Bits[30:11]: Reserved • Bit[31]: The entire memory space of the device. When set, the target supports TEUpdate using Length Index 7 to change the TE State for the entire address range. When cleared, the target does not support use of Length Index value of 7.

Table 11-32. Get Target Capabilities Response (Sheet 3 of 3)

Byte Offset	Length in Bytes	Description
18h	2	Configuration Features Supported: The configuration features that the target supports. Zero or more bits may be set. 1 indicates supported, 0 indicates not supported. - Bit[0]: Locked Target FW Update: When set, the target supports FW updates after the target is locked. When cleared to 0, the target does not support FW updates after the target is locked. - Bit[1]: Target Supports Additional CMA/SPDM Sessions: The target supports using CMA/SPDM PSK to set up one or more SecondarySession(s). If this bit is set, then Number of Secondary Sessions shall be valid. When cleared, the target does not support secondary SPDM sessions. - Bits[15:2]: Reserved.
1Ah	2	Reserved
1Ch	4	Number of CKIDs: Total number of CKIDs that the target supports. Valid only if CKID-based Encryption is set in Memory Encryption Features Supported. Shall be ≥ 2 and $< 2^{13}$.
20h	1	Number of Secondary Sessions: Total number of additional SPDM SecondarySessions that the target supports. When valid, this shall be > 0 and ≤ 4 .
21h	13h	Reserved

11.5.5.6 Target Configuration

The following request and response payloads provide TSP configuration, locking, and register reporting of the target.

11.5.5.6.1 Set Target Configuration

The PrimarySession is utilized with the Set Target Configuration request to place the target in the preferred transport configuration. This includes providing SecondarySession CMA/SPDM PSK Key Material that shall be utilized by the Target to generate random keys for this additional session.

Possible Error Response, Error Codes:

- Version Mismatch
- Invalid Request
 - Entropy was not supplied
 - Number of CKIDs being enabled is $>$ Number of CKIDs the target reported in Get Target Configuration
 - CKID Base is $\geq 2^{13}$
 - CKID Base + Number of CKIDs $\geq 2^{13}$
 - TE State Granularity specified is not supported by the target
 - Length Indexes are not unique
 - Length Index 0 or 7 was specified but the TE State Granularity was not 0
 - Implicit TE State Change is enabled and Explicit In-band TE State Change is not enabled or no Explicit In-band TE State Granularity Entries enable Length Index 0
 - CKID-based Encryption and Range-based Encryption are both enabled
- No Privilege
 - Request was not received on the PrimarySession
- Invalid Security State
- Target not in CONFIG_UNLOCKED state

The following structure shall be utilized for associating explicit TE State lengths to specific indexes that are utilized with the TEUpdate memory transaction. Implementations that do not utilize explicit in-band TE State changes do not need to include valid information in the response payload for these entries.

Table 11-33. Explicit In-band TE State Granularity Entry

Byte Offset	Length in Bytes	Description
00h	8	TE State Granularity: The number of bytes of contiguous HPA space to which the explicit in-band TE state change will apply when the TEUpdate memory transaction is received by the target. Shall be one of the values reported by the target in Supported Explicit In-band TE State Granularity reported in Get Target Capabilities. When specifying Length Index 0 or 7, this field shall be 0 because the length is predefined. This field is ignored when Length Index is FFh.
08h	1	Length Index: The 3-bit length index that shall be utilized to represent the TE State Length in the SnpType portion of the TEUpdate memory transaction. Value shall be ≥ 0 and ≤ 7 for valid Explicit In-band TE State Granularity Entries. Each length entry specified shall utilize a unique Length Index. Value FFh is reported for unused Explicit In-band TE State Granularity Entries.
09h	7	Reserved

Table 11-34. Set Target Configuration (Sheet 1 of 4)

Byte Offset	Length in Bytes	Description
000h	1	TSP Version: V1.0 = 10h.
001h	1	Opcode: Set Target Configuration = 83h.
002h	2	Memory Encryption Features Enable: Enable the memory encryption features for the target. Zero or more bits may be set. 1 indicates to enable, 0 indicates to disable. - Bit[0]: Encryption: When set, memory encryption for data at rest shall be enabled on the target. When cleared, target memory encryption shall be disabled. - Bit[1]: CKID-based Encryption: When set, CKID-based encryption shall be enabled on the target, Encryption shall be set, and the CKID Base Required field shall be valid. When set, Range-based Encryption shall be cleared. When cleared, the target shall disable use of the CKID field in the Transaction Layer requests. - Bit[2]: Range-based Encryption: When set, range-based encryption shall be enabled on the target and Encryption shall also be set. When set, CKID-based Encryption shall be cleared. When cleared, the target shall disable use of range-based target memory encryption. - Bit[3]: CKID Base Required: Valid only when CKID-based Encryption is set. When set, the target shall enable a valid CKID range and CKID Base and Number of CKIDs fields shall be valid. When cleared, the target shall enable any CKID value within the 13-bit field. - Bits[15:4]: Reserved.
004h	4	Memory Encryption Algorithm Select: Valid only if Encryption is set in Memory Encryption Features Enable. Select the target memory encryption algorithm to utilize. Only one bit shall be set. 1 indicates selected, 0 indicates not selected. - Bit[0]: AES-XTS-128 - Bit[1]: AES-XTS-256 - Bits[30:2]: Reserved - Bit[31]: Vendor Specific Algorithm: When set, all other bits in this field are vendor specific
008h	4	Reserved

Table 11-34. Set Target Configuration (Sheet 2 of 4)

Byte Offset	Length in Bytes	Description
00Ch	2	TE State Change and Access Control Features Enable: Enable the TE State change and access control features for the target. Zero or more bits may be set. 1 indicates to enable, 0 indicates to disable. - Bit[0]: Write Access Control: When set, the target shall enable dropping writes that fail the verification of TEE Intent to stored TE State. When set, explicit state changes shall be enabled and one or more of bits[4:3] shall also be set. - Bit[1]: Read Access Control: When set, the target shall enable returning all 1s for read data in response to reads that fail the verification of TEE Intent to stored TE State. When set, one or more of bits[4:2] shall also be set. - Bit[2]: Implicit TE State Change: When set, implicit TE State changes shall be enabled on the target using 64B granularity, Explicit In-band TE State Change shall be set, and at least one Explicit In-band TE State Granularity Entry with Length Index 0 shall be enabled. When cleared, implicit TE State changes shall be disabled on the target. - Bit[3]: Explicit Out-of-band TE State Change: When set, the target shall be enabled to utilize the explicit CMA/SPDM out-of-band explicit Set Target TE State change request and Explicit Out-of-band TE State Change Granularity shall be valid. - Bit[4]: Explicit In-band TE State Change: When set, the target shall be enabled for explicit in-band TE State changes utilizing the TEUpdate memory transaction and Explicit In-band TE State Change Granularity Entries shall be valid. - Bit[5]: Explicit TE State Change Sanitize: When set, the target shall be enabled to overwrite data that is affected by the explicit state change with 0s when the explicit request is received and before the change is considered complete by the target. When set, one or more of bits[4:3] shall also be set. - Bits[15:6]: Reserved.
00Eh	2	Reserved
010h	4	Explicit Out-of-band TE State Granularity: The granularity that the initiator is requesting the target to utilize for explicit out-of-band TE State changes in powers of 2, starting with 64B. Valid only if Explicit Out-of-band TE State Change is set in TE State Change and Access Control Features Enable. Only one bit shall be set. 1 indicates target shall enable, 0 indicates target shall disable. - Bit[0]: 64B ... - Bit[6]: 4 KB ... - Bit[15]: 2 MB ... - Bit[24]: 1 GB ... - Bit[31]: 128 GB
014h	4	Reserved
018h	2	Configuration Features Enable: Enable the configuration features for the target. Zero or more bits may be set. 1 indicates to enable, 0 indicates to disable. - Bit[0]: Locked Target FW Update: When set, the target shall enable FW updates after the target is locked. When cleared, the target shall disable FW updates after the target is locked. - Bit[1]: Special Purpose Memory: When set, memory reported to the initiator should not be treated as general-purpose memory. Accelerator devices shall set this to indicate the exposed memory capacity cannot be utilized as general-purpose memory by the initiator. The target should also report EFI Memory Type and Attribute of EfiConventionalMemory EFI_MEMORY_SP for the corresponding memory ranges in the CDAT DSEMTS. When cleared, the initiator may utilize the memory capacity as general-purpose memory. - Bits[15:2]: Reserved.
01Ah	2	Reserved
01Ch	4	CKID Base: The lowest CKID that the target shall enable. Valid only if CKID Base Required is set in Memory Encryption Features Enable. Shall be $< 2^{13}$.
020h	4	Number of CKIDs: Number of contiguous CKID that the target shall enable starting at the CKID Base. Valid only if CKID Base Required is set in Memory Encryption Features Enable. Shall be $\leq$ Number of CKIDs reported by the target in Get Target Capabilities. CKID Base + Number of CKIDs shall be $< 2^{13}$.
024h	0Ch	Reserved
030h	10h	Explicit In-band TE State Granularity Entry 0
040h	10h	Explicit In-band TE State Granularity Entry 1

Table 11-34. Set Target Configuration (Sheet 3 of 4)

Byte Offset	Length in Bytes	Description
050h	10h	Explicit In-band TE State Granularity Entry 2
060h	10h	Explicit In-band TE State Granularity Entry 3
070h	10h	Explicit In-band TE State Granularity Entry 4
080h	10h	Explicit In-band TE State Granularity Entry 5
090h	10h	Explicit In-band TE State Granularity Entry 6
0A0h	10h	Explicit In-band TE State Granularity Entry 7
0B0h	10h	Reserved
0C0h	2	Configuration Validity Flags: Indicators of which fields are valid in the remaining portion of this request. Zero or more bits may be set. - Bit[0]: Secondary Session 0: When set, the Secondary Session 0 PSK Key Material field and Secondary Session 0 CKID Type field shall both be valid and shall be utilized by the target to generate a key for use when a SecondarySession0 is created utilizing CMA/SPDM PSK. When cleared, no SecondarySession0 is allowed by the target. This bit shall be set only if Get Target Capabilities, Number of Secondary Sessions is > 0. - Bit[1]: Secondary Session 1: When set, the Secondary Session 1 PSK Key Material field and Secondary Session 1 CKID Type field shall both be valid and shall be utilized by the target to generate a key for use when a SecondarySession1 is created utilizing CMA/SPDM PSK. When cleared, no SecondarySession1 is allowed by the target. This bit shall be set only if Get Target Capabilities, Number of Secondary Sessions is > 1. - Bit[2]: Secondary Session 2: When set, the Secondary Session 2 PSK Key Material field and Secondary Session 2 CKID Type field shall both be valid and shall be utilized by the target to generate a key for use when a SecondarySession2 is created utilizing CMA/SPDM PSK. When cleared, no SecondarySession2 is allowed by the target. This bit shall be set only if Get Target Capabilities, Number of Secondary Sessions is > 2. - Bit[3]: Secondary Session 3: When set, the Secondary Session 3 PSK Key Material field and Secondary Session 3 CKID Type field shall both be valid and shall be utilized by the target to generate a key for use when a SecondarySession3 is created utilizing CMA/SPDM PSK. When cleared, no SecondarySession3 is allowed by the target. This bit shall be set only if Get Target Capabilities, Number of Secondary Sessions is > 3. - Bits[15:4]: Reserved.
0C2h	0Eh	Reserved
0D0h	1	Secondary Session CKID Type: The CKID Type to assign to a SecondarySession. - Bit[0]: Secondary Session 0 CKID Type: When set, the CKID shall be considered a TVMCKID. When cleared, the CKID shall be considered an OSCKID. This field shall be valid if the Validity Flags, Secondary Session 0 is set. - Bit[1]: Secondary Session 1 CKID Type: When set, the CKID shall be considered a TVMCKID. When cleared, the CKID shall be considered an OSCKID. This field shall be valid if the Validity Flags, Secondary Session 1 is set. - Bit[2]: Secondary Session 2 CKID Type: When set, the CKID shall be considered a TVMCKID. When cleared, the CKID shall be considered an OSCKID. This field shall be valid if the Validity Flags, Secondary Session 2 is set. - Bit[3]: Secondary Session 3 CKID Type: When set, the CKID shall be considered a TVMCKID. When cleared, the CKID shall be considered an OSCKID. This field shall be valid if the Validity Flags, Secondary Session 3 is set. - Bits[7:4]: Reserved.
0D1h	0Fh	Reserved
0E0h	20h	Secondary Session 0 PSK Key Material: The CMA/SPDM PSK key material that the target shall utilize for key derivation for use when the SecondarySession0 CMA/SPDM secure session is created. This field shall be valid if the Validity Flags, Secondary Session 0 is set. When this PSK key material is used to set up the CMA/SPDM SecondarySession0, the associated CMA/SPDM PSK Hint entry in the CMA/SPDM PSK_EXCHANGE request shall be "SECONDARY_SESSION_0_PSK" string with NUL terminator.
100h	20h	Secondary Session 1 PSK Key Material: The CMA/SPDM PSK key material that the target shall utilize for key derivation for use when the SecondarySession1 CMA/SPDM secure session is created. This field shall be valid if the Validity Flags, Secondary Session 1 is set. When this PSK key material is used to set up the CMA/SPDM SecondarySession1, the associated CMA/SPDM PSK Hint entry in the CMA/SPDM PSK_EXCHANGE request shall be "SECONDARY_SESSION_1_PSK" string with NUL terminator.

Table 11-34. Set Target Configuration (Sheet 4 of 4)

Byte Offset	Length in Bytes	Description
120h	20h	Secondary Session 2 PSK Key Material: The CMA/SPDM PSK key material that the target shall utilize for key derivation for use when the SecondarySession2 CMA/SPDM secure session is created. This field shall be valid if the Validity Flags, Secondary Session 2 is set. When this PSK key material is used to set up the CMA/SPDM SecondarySession2, the associated CMA/SPDM PSK Hint entry in the CMA/SPDM PSK_EXCHANGE request shall be "SECONDARY_SESSION_2_PSK" string with NUL terminator.
140h	20h	Secondary Session 3 PSK Key Material: The CMA/SPDM PSK key material that the target shall utilize for key derivation for use when the SecondarySession3 CMA/SPDM secure session is created. This field shall be valid if the Validity Flags, Secondary Session 3 is set. When this PSK key material is used to set up the CMA/SPDM SecondarySession3, the associated CMA/SPDM PSK Hint entry in the CMA/SPDM PSK_EXCHANGE request shall be "SECONDARY_SESSION_3_PSK" string with NUL terminator.

11.5.5.6.2 Set Target Configuration Response

If no error condition is detected, the DSM shall respond to the Set Target Configuration request with a Set Target Configuration Response message.

Table 11-35. Set Target Configuration Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target Configuration Response = 03h.
02h	2	Reserved

11.5.5.6.3 Get Target Configuration

The PrimarySession or SecondarySession(s) shall utilize the Get Target Configuration request to verify that the target is in the correct security mode after the target is locked. While it is possible to report the configuration with this request before the target is locked, the content cannot be trusted to be immutable until this request is executed after the target is successfully locked.

Possible Error Response, Error Codes:

- Version Mismatch
- No Privilege
 - The request was not received on the PrimarySession or SecondarySession(s)
- Invalid Security State
 - Target not in CONFIG_LOCKED or CONFIG_UNLOCKED state

Table 11-36. Get Target Configuration

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target Configuration = 84h.
02h	2	Reserved

11.5.5.6.4 Get Target Configuration Response

If no error condition is detected, the DSM shall respond to the Get Target Configuration request with a Get Target Configuration Response message.

Table 11-37. Get Target Configuration Response (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target Configuration Response = 04h.
02h	2	Memory Encryption Features Enabled: The memory encryption features that are enabled for the locked target. Zero or more bits may be set. 1 indicates to enable, 0 indicates to disable. • Bit[0]: Encryption: When set, memory encryption for data at rest is enabled on the target. • Bit[1]: CKID-based Encryption: When set, CKID-based encryption is enabled on the target, Encryption shall be set, and the CKID Base Required field shall be valid. When set, Range-based Encryption shall be cleared. When cleared, the target has disabled use of the CKID field in the Transaction Layer requests. • Bit[2]: Range-based Encryption: When set, range-based encryption is enabled on the target and Encryption shall also be set. When set, CKID-based Encryption shall be cleared. When cleared, the target has disabled use of range-based target memory encryption. • Bit[3]: CKID Base Required: Valid only when CKID-based Encryption is set. When set, the target has been enabled for a valid CKID range and CKID Base and Number of CKIDs fields shall be valid. When cleared, the target has enabled any CKID value within the 13-bit field. • Bits[15:4]: Reserved.
04h	4	Memory Encryption Algorithm Selected: Valid only if Encryption is set in Memory Encryption Features Enabled. The target memory encryption algorithm that is selected. Only one bit may be set. 1 indicates selected, 0 indicates not selected. • Bit[0]: AES-XTS-128 • Bit[1]: AES-XTS-256 • Bits[30:2]: Reserved • Bit[31]: Vendor Specific Algorithm: When set, all other bits in this field are vendor specific
08h	4	Reserved
0Ch	2	TE State Change and Access Control Features Enabled: The TE State change and access control features that are enabled for the locked target. Zero or more bits may be set. 1 indicates enabled, 0 indicates disabled. • Bit[0]: Write Access Control: When set, dropping writes that fail the verification of TEE Intent to stored TE State is enabled on the target. When set, explicit state changes shall be enabled and one or more of bits[4:3] shall also be set. • Bit[1]: Read Access Control: When set, returning all 1s for read data in response to reads that fail the verification of TEE Intent to stored TE State is enabled on the target. When set, one or more of bits[4:2] shall also be set. • Bit[2]: Implicit TE State Change: When set, the implicit TE State change feature has been enabled on the target, Explicit In-band TE State Change shall be enabled, and at least one Explicit In-band TE State Granularity Entry with Length Index 0 shall be enabled. When cleared, the implicit TE State change feature has been disabled on the target. • Bit[3]: Explicit Out-of-band TE State Change: When set, use of the explicit CMA/SPDM out-of-band Set Target TE State change request is enabled on the target and Explicit Out-of-Band TE State Change Granularity shall be valid. • Bit[4]: Explicit In-band TE State Change: When set, explicit in-band TE State changes utilizing the TEUpdate memory transaction is enabled on the target and Explicit In-band TE State Change Granularity Entries shall be valid. • Bit[5]: Explicit TE State Change Sanitize: When set, the target is enabled to overwrite data that is affected by the explicit state change with 0s when the explicit request is received and before the change is considered complete by the target. When set, one or more of bits[4:3] shall also be set. • Bits[15:6]: Reserved.
0Eh	2	Reserved

Table 11-37. Get Target Configuration Response (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
10h	4	Explicit Out-of-band TE State Granularity Enabled: The granularity that has been enabled on the target to utilize for explicit TE State changes in powers of 2, starting with 64B. Valid only if Explicit Out-of-band TE State Change is set in TE State Change and Access Control Features Enabled. Only one bit shall be set. 1 indicates selected, 0 indicates not selected. • Bit[0]: 64B • ... • Bit[6]: 4 KB • ... • Bit[15]: 2 MB • ... • Bit[24]: 1 GB • ... • Bit[31]: 128 GB
14h	4	Reserved
18h	2	Configuration Features Enabled: The configuration features that are enabled for the locked target. Zero or more bits may be set. 1 indicates to enable, 0 indicates to disable. • Bit[0]: Locked Target FW Update: When set, FW updates after the target is locked are enabled on the target. When cleared, FW updates after the target is locked are disabled on the target. • Bits[15:1]: Reserved.
1Ah	2	Reserved
1Ch	4	CKID Base: The lowest CKID that the target enabled. Valid only if CKID Base Required is set in Memory Encryption Features Enabled. Shall be $< 2^{13}$.
20h	4	Number of CKIDs: Number of contiguous CKID that the target enabled starting at the CKID Base. Valid only if CKID Base Required is set in Memory Encryption Features Enabled. CKID Base + Number of CKIDs shall be $< 2^{13}$.
24h	1	Current TSP State • 00h = CONFIG_UNLOCKED • 01h = CONFIG_LOCKED • 02h = ERROR • All other encodings are reserved
25h	0Bh	Reserved
030h	10h	Explicit In-band TE State Granularity Entry 0
040h	10h	Explicit In-band TE State Granularity Entry 1
050h	10h	Explicit In-band TE State Granularity Entry 2
060h	10h	Explicit In-band TE State Granularity Entry 3
070h	10h	Explicit In-band TE State Granularity Entry 4
080h	10h	Explicit In-band TE State Granularity Entry 5
090h	10h	Explicit In-band TE State Granularity Entry 6
0A0h	10h	Explicit In-band TE State Granularity Entry 7
0B0h	10h	Reserved

11.5.5.6.5 Get Target Configuration Report

The PrimarySession shall be utilized with the Get Target Configuration Report request to return specific CXL.mem configuration register TSP Report content that is utilized to verify the locked target's configuration. [Section 11.5.4.8](#) describes the checks that can be conducted on the TSP Report response payload that is returned from this request.

This request allows select CXL configuration register contents on the endpoint target to be returned for verification through the secure PrimarySession, and is modeled after the TDISP Get Interface Report request.

While it is possible to read the configuration with this request, before the target is locked, the content cannot be trusted to be immutable until this request is executed after the target is successfully locked.

Possible Error Response, Error Codes:

- Version Mismatch
- No Privilege
 - Request was not received on the PrimarySession or SecondarySession(s)
- Invalid Security State
 - Target not in CONFIG_LOCKED or CONFIG_UNLOCKED state

Table 11-38. Get Target Configuration Report

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target Configuration Report = 85h.
02h	2	Reserved
04h	2	Offset: Offset in bytes from the start of the report to where this request message begins. For the first Get Target Configuration Report request, the initiator shall clear this field to all 0s. For non-first requests, the offset is the sum of Portion Length values reported in all the previous Get Target Configuration Report Responses.
06h	2	Length: The length of the report in bytes to be returned in the corresponding response. Length is an unsigned 16-bit integer. This value is the smaller of the following values: • Capacity of the initiator's internal buffer for receiving the target's report • Remainder Length of the preceding Get Target Configuration Report response If the Length is > Remainder Length, the target shall transfer the remaining Report Data, Portion Length, and a Remainder Length of 0 in the response.

11.5.5.6.6 Get Target Configuration Report Response

If no error condition is detected, the DSM shall respond to the Get Target Configuration Report request with a Get Target Configuration Report Response message.

Table 11-39. Get Target Configuration Report Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Get Target Configuration Report Response = 05h.
02h	2	Reserved
04h	2	Portion Length: Number of bytes of this portion of the TSP Report. This shall be less than or equal to Length received as part of the request. The target is permitted to set this field to a value that is less than the Length received in the request due to limitations on the target's internal buffer.
06h	2	Remainder Length: Number of bytes of the TSP Report that have not been sent yet after the current response. For the last response, the target shall clear this field to all 0s as an indication to the initiator that the entire TSP Report has been sent.
08h	Portion Length	Report Data: Requested contents of TSP Report.

Table 11-40 presents the TSP Report structure.

Table 11-40. TSP Report

Byte Offset	Length in Bytes	Description
00h	1	Valid TSP Report Fields: More than one bit may be set. - Bit[0]: CXL IDE Capability Structure Valid: When set, the TSP Report contains a valid CXL IDE Capability Structure - Bits[7:1]: Reserved
01h	3	Reserved
04h	3Ch	PCIe DVSEC for CXL Devices: See Section 8.1 .
40h	20h	PCIe DVSEC for Flex Bus Port: See Section 8.2 .
60h	38h	CXL Link Capability Structure: See Section 8.2.4.19 .
98h	10h	CXL Timeout and Isolation Capability Structure: See Section 8.2.4.24 .
A8h	10h + n*1Fh	CXL HDM Decoder Capability Structure: The number of HDM decoders (n) is specified in the Decoder Count field in the CXL HDM Decoder Capability register. See Section 8.2.4.20 .
B8h + n*1Fh	24h	CXL IDE Capability Structure: Structure for optionally reporting the CXL IDE Transport Security configuration. Valid if CXL IDE Capability Structure Valid is set.

11.5.5.6.7 Lock Target Configuration

The PrimarySession shall be utilized with the Lock Target Configuration request to lock the target configuration that is relevant to protecting the TEE configuration and TVM data, and perform memory security checks before responding to this request. The locked configuration includes the security configuration set, utilizing the Set Target Configuration request in addition to CXL and PCIe target registers that need to be made immutable to protect TEE integrity. This request does not lock configuration registers and other registers that are not part of protecting the TEE configuration and TVM data.

Once locked, the configuration shall be immutable until a subsequent Conventional Reset of the target.

The target shall reject requests to lock when already in the CONFIG_LOCKED state.

If Write Access Control is enabled on the target, the target shall clear the TE State to 0 for all addressable memory in response to the Lock Target Configuration Request and before generating a Lock Target Configuration Response. The target may require extra execution time to clear the initial TE State and may utilize the Delayed Response to prevent request timeouts, as described in [Section 11.5.5.9](#).

See [Section 11.5.4.8.1](#) for expected target locking behavior before successfully responding to this request.

Possible Error Response, Error Codes:

- Version Mismatch
- Invalid Security State
 - Target not in CONFIG_UNLOCKED state
- No Privilege
 - Request was not received on the PrimarySession
- Invalid Security Configuration
 - Locking security configuration failed due to memory security check failures
- Already Locked

— The target is already in CONFIG_LOCKED state

Table 11-41. Lock Target Configuration

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Lock Target Configuration = 86h.
02h	2	Reserved

11.5.5.6.8 Lock Target Configuration Response

The DSM responds with a Lock Target Configuration Response in response to Lock Target Configuration request if the target security checks and lock operations were successful, and the target transitioned to the CONFIG_LOCKED state. An Error Response shall be returned if the memory security checks failed, the configuration could not be locked, and/or other errors occurred.

See [Section 11.5.4.8.1](#) for expected target behavior before sending this successful response to the lock request.

Table 11-42. Lock Target Configuration Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Lock Target Configuration Response = 06h.
02h	2	Reserved

11.5.5.7 Optional Explicit TE State Change Requests and Responses

If the target supports explicit TE State changes, it shall support receiving a Set Target TE State request out-of-band that utilizes the CMA/SPDM secure PrimarySession or SecondarySession(s). The target reports its supported explicit TE State change mechanisms in Get Target Capabilities. The host that supports explicit TE State changes shall enable explicit changes, utilizing Set Target Configuration. The explicit TE State change request shall be sent by the host to the target before the memory range is to be accessed, ensuring that the target has the correct TE State to perform the access checks before receiving the memory transactions that it will verify.

11.5.5.7.1 Set Target TE State (Out-of-band)

Targets that utilize explicit TE State change notifications from the host shall implement the following request and response for updating the TE State based on memory range. The target shall change the TE State as specified for all memory locations covered by the Starting Address and Length that are relevant to the target for the interleave set configured.

If the target is enabled to sanitize memory affected by a state change, the target shall overwrite all data affected by this state change with 0s before generating the response.

This request could take a significant amount of time to complete if sanitization of a large amount of memory is required and is handled as follows:

- If the request can be completed without an excessive delay that could cause an SPDMA timeout, the target shall respond with Set Target TE State Response, once the request is complete.

- If the target is not capable of executing the request due to the execution time required or does not support delaying the request's completion, it may fail this request with a Long Execution Time Error Response.
- Otherwise, if the target is capable of executing the request and the request will take a significant amount of time to complete, the target shall respond with Delayed Response with a nonzero Delay Time in microseconds (us). The host should wait the prescribed amount of time and issue the Check Target Delayed Completion request to verify the state change is complete.

If the target is already executing a state change request and another state change request is received, it shall fail the new request with Busy for Error Response.

Possible Error Response, Error Codes:

- Version Mismatch
- Invalid Request
 - Number of Memory Ranges is 0
 - One or more Memory Range Starting Address and Length is invalid for the target
 - One or more Memory Range Starting Address is not aligned to Explicit Out-of-band TE State Granularity Selected reported in Get Target Configuration Response
 - One or more Memory Range Length is not an exact multiple of the Explicit Out-of-band TE State Granularity Selected reported in Get Target Configuration Response
 - One or more Memory Range Starting Address and Length spans TEE and non-TEE ranges
- No Privilege
 - Request was not received on the PrimarySession or SecondarySession(s)
- Invalid Security State
 - Target not in CONFIG_LOCKED state
- Busy
 - Target is currently executing another state change.
- Long Execution Time
 - Target cannot execute the request due to the amount of execution time required

Table 11-43. Memory Range

Byte Offset	Length in Bytes	Description
00h	8	Starting Address: HPA to start re-initializing the TE State. This address shall be aligned to the Access Control Granularity Selected reported in Get Target Configuration Response.
08h	8	Length: The length of the memory range, in bytes, at which to re-initialize the TE State. This number shall be an exact multiple of the Access Control Granularity Selected reported in Get Target Configuration Response.

Table 11-44. Set Target TE State

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target TE State = 8Dh.
02h	1	TE State: The new target TE State to set for the included memory ranges. Only one bit shall be set. - Bit[0]: TE State: When set to 1, the target shall set the TE State for all included memory ranges to TEE Exclusive. When cleared to 0, the target shall set the TE State for all included memory ranges to non-TEE Exclusive. - Bits[7:1]: Reserved.
03h	1	Number of Memory Ranges: The number of Memory Range structures (N) that are included in this payload. This shall be ≤ 32 and > 0 .
04h	Ch	Reserved
10h	10h * N	List of Memory Range structures.

11.5.5.7.2 Set Target TE State Response (Out-of-band)

If no error condition is detected and the state change request has completed execution on the target, the DSM shall respond to the Set Target TE State request with a Set Target TE State Response message.

Table 11-45. Set Target TE State Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target TE State Response = 0Dh.
02h	2	Reserved

11.5.5.8 Optional Target-based Memory Encryption Requests and Responses

The following interfaces are optionally supported by the target for use when target memory encryption is enabled. The target reports support for these interfaces in the Memory Encryption Features Supported reported in the Get Target Capabilities Response.

11.5.5.8.1 Set Target CKID Specific Key

The PrimarySession or SecondarySession(s) shall be utilized with the Set Target CKID Specific Key request to define a specific CKID as a Memory Encryption Features Supported reported and associate that CKID with specific key material. This request is utilized with CKID-based target memory encryption. Once set, the association between an initiator CKID and the target's keys are immutable and attempts to set a new key for the CKID shall fail. To change the association, the CKID shall be explicitly cleared by the initiator, utilizing Clear Target CKID Key before the CKID can be set for new keys using this request. The Attributes CKID Type shall be utilized by the target to ensure that each memory transaction TEE Intent matches the CKID Type (TVMCKID or OSCKID) as described in [Section 11.5.4.6.2.1](#).

Possible Error Response, Error Codes:

- Version Mismatch
- Unsupported Request
 - Target does not support CKID-based memory encryption

- Invalid Security State
 - Target not in CONFIG_LOCKED state
- No Privilege
 - Request was not received on the PrimarySession or SecondarySession(s).
- Invalid CKID
 - More CKIDs have been assigned to the target than the Number of CKIDs reported in Get Target Capabilities.
 - Requested CKID is outside the range of CKID Base to CKID Base + Number of CKIDs configured in Set Target Configuration.
 - CKID already has a key associated with it. Clear Target CKID Key shall be utilized to reset the CKID association before the CKID can be set again.

Table 11-46. Set Target CKID Specific Key

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target CKID Specific Key = 87h.
02h	2	Reserved
04h	4	CKID: The CKID assigned by the initiator to this encryption key. The attribute CKID Type defines whether the CKID is an OSCKID or TVMCKID.
08h	1	CKID Type: The type of CKID to utilize: • 00h = TVMCKID • 01h = OSCKID • All other encodings are reserved
09h	6	Reserved
0Fh	1	Validity Flags: Indicators of which fields are valid in the remaining portion of this request. More than one bit may be set. • Bit[0]: When set, the Data Encryption Key field is valid • Bit[1]: When set, the Tweak Key field is valid • Bits[7:2]: Reserved
10h	20h	Data Encryption Key: The memory encryption key to utilize with the CKID.
30h	20h	Tweak Key: The memory encryption tweak key to utilize with the CKID. If the configured encryption algorithm does not require a tweak key, then this field shall be ignored.

11.5.5.8.2 Set Target CKID Specific Key Response

If no error condition is detected, the DSM shall respond to the Set Target CKID Specific Key request with a Set Target CKID Specific Key Response message.

Table 11-47. Set Target CKID Specific Key Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target CKID Specific Key Response= 07h.
02h	2	Reserved

11.5.5.8.3 Set Target CKID Random Key

The PrimarySession or SecondarySession(s) shall be utilized with the Set Target CKID Random Key request to define a specific CKID as an OSCKID or TVMCKID and associate the CKID with a random target-generated key utilizing optional initiator-generated entropy. This request is utilized with CKID-based target memory encryption. Once set, the association between an initiator CKID and the target's keys are immutable and attempts to set a new key for the CKID shall fail. To change the association, the CKID shall be explicitly cleared by the initiator, utilizing Clear Target CKID Key before the CKID can be set for new keys using this request. The Attributes CKID Type shall be utilized by the target to ensure that each memory transaction TEE Intent matches the CKID Type (OSCKID or TVMCKID) as described in [Section 11.5.4.6.2.1](#).

When the target is utilizing the initiator-generated entropy, the target should generate a unique key, even if the initiator-supplied entropy is equivalent to the entropy provided in a previous request.

Possible Error Response, Error Codes:

- Version Mismatch
- Unsupported Request
 - Target does not support CKID-based memory encryption.
- Invalid Security State
 - Target not in CONFIG_LOCKED state.
- No Privilege
 - Request was not received on the PrimarySession or SecondarySession(s).
- Invalid CKID
 - More CKIDs have been assigned to the target than the Number of CKIDs reported in Get Target Capabilities.
 - Requested CKID is outside the range of CKID Base to CKID Base + Number of CKIDs configured in Set Target Configuration.
 - CKID already has a key associated with it. Clear Target CKID Key shall be utilized to reset the CKID association before the CKID can be set again.
- No Entropy
 - The target was not able to obtain enough entropy to generate a random key.

Table 11-48. Set Target CKID Random Key (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target CKID Random Key = 88h.
02h	2	Reserved
04h	4	CKID: The CKID assigned by the initiator to this encryption key. The attribute CKID Type defines whether the CKID is an OSCKID or TVMCKID.
08h	1	Attributes: Additional attributes of the CKID. Only one bit shall be set. • Bit[0]: CKID Type: When set, the CKID is considered a TVMCKID. If cleared, the CKID is considered an OSCKID. • Bits[7:1]: Reserved.
09h	6	Reserved

Table 11-48. Set Target CKID Random Key (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
0Fh	1	Validity Flags: Indicators of which fields are valid in the remaining portion of this request. More than one bit may be set. - Bit[0]: When set, the Data Encryption Key Entropy field is valid - Bit[1]: When set, the Tweak Key Entropy field is valid - Bits[7:2]: Reserved
10h	20h	Data Encryption Key Entropy: Optional initiator-supplied memory encryption key entropy for the target to utilize when generating a random data encryption key for the CKID. This field shall be ignored if Initiator Supplied Entropy is not set in Memory Encryption Features Supported reported in Get Target Capabilities Response.
30h	20h	Tweak Key Entropy: Optional initiator-supplied memory encryption tweak key entropy for the target to utilize when generating a random tweak key for the CKID. This field shall be ignored if Initiator Supplied Entropy is not set in Memory Encryption Features Supported reported in Get Target Capabilities Response.

11.5.5.8.4 Set Target CKID Random Key Response

If no error condition is detected, the DSM shall respond to the Set Target CKID Random Key request with a Set Target CKID Random Key Response message.

Table 11-49. Set Target CKID Random Key Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target CKID Random Key Response= 08h.
02h	2	Reserved

11.5.5.8.5 Clear Target CKID Key

The PrimarySession or SecondarySession(s) shall be utilized with the Clear Target CKID Key request to clear any association on the target between the previously set CKID and random or specific keys that may have been programmed. This request is utilized with CKID-based target memory encryption. This request is utilized by the initiator to clear the association between an initiator's CKID and the target's keys and allows a CKID to be utilized with a set of new keys. Once a CKID has been cleared the CKID Type is no longer a TVMCKID and encryption utilizing that CKID is bypassed.

The target shall break the association of CKID to key and shall clear the associated key to 0.

The same SPDm session that was utilized to set the key for the CKID shall be the same session that is utilized to clear the CKID. If the SPDm session utilized to set the key has been terminated or closed, then a Conventional Reset or CXL Reset shall be utilized to clear the CKID association with the key material.

Possible Error Response, Error Codes:

- Version Mismatch
- Unsupported Request
 - Target does not support CKID-based memory encryption
- Invalid Security State
 - Target not in CONFIG_LOCKED state
- No Privilege

- Request was not received on the PrimarySession or SecondarySession(s)
- CKID was not set on the same SPDH session
- Invalid CKID
 - Requested CKID is outside the range of CKID Base to CKID Base + Number of CKIDs configured in Set Target Configuration
 - CKID is not currently programmed in the target

Table 11-50. Clear Target CKID Key

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Clear Target CKID Key = 89h.
02h	2	Reserved
04h	4	CKID: The CKID for which to clear the encryption key.

11.5.5.8.6 Clear Target CKID Key Response

If no error condition is detected, the DSM shall respond to the Clear Target CKID Key request with a Clear Target CKID Key Response message.

Table 11-51. Clear Target CKID Key Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Clear Target CKID Key Response = 09h.
02h	2	Reserved

11.5.5.8.7 Set Target Range Specific Key

The PrimarySession or SecondarySession(s) shall be utilized with the Set Target Range Specific Key request to associate a specific memory range with initiator-specified key material. This request is utilized with range-based target memory encryption. Once set, the association between an initiator HPA memory range and the target's keys are immutable and attempts to set a new key for the same or subset of the same HPA range shall fail. To change the association, the memory range shall be explicitly cleared by the initiator, utilizing Clear Target Range Key before the memory range can be set for new keys using this request.

Possible Error Response, Error Codes:

- Version Mismatch
- Invalid Request
 - The memory Range Base and Range Size is invalid for the target.
 - The memory Range Base is not aligned to the Range Size.
 - The Range Size is not a power of 2.
 - The Range ID is $\geq$ Memory Encryption Number of Range Based Keys reported in Get Target Capabilities.
 - The memory range register specified by the Range ID requested is already associated with keys.

- The request attempted to change a memory range or subset of the range already configured. Clear Target Range Key shall be utilized to reset the memory range association.
- Unsupported Request
 - The target does not support range-based memory encryption.
- Invalid Security State
 - Target not in CONFIG_LOCKED state.
- No Privilege
 - The request was not received on the PrimarySession or SecondarySession(s).

Table 11-52. Set Target Range Specific Key

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target Range Specific Key = 8Ah.
02h	2	Reserved
04h	4	Range ID: The range ID assigned to this encryption key. Shall be within the range specified in Get Target Capabilities Response, Memory Encryption Number of Range Based Keys.
08h	8	Range Start: HPA of the first 4-KB-aligned block within the range. • Bits[11:0]: Reserved • Bits[63:12]: Start HPA for the range, HPA[63:12]
10h	8	Range End: HPA of the last 4-KB-aligned block within the range. • Bits[11:0]: Reserved • Bits[63:12]: End HPA for the range, HPA[63:12]
18h	7	Reserved
1Fh	1	Validity Flags: Indicators of which fields are valid in the remaining portion of this request. More than one bit may be set. • Bit[0]: When set, the Data Encryption Key field is valid • Bit[1]: When set, the Tweak Key field is valid • Bits[7:2]: Reserved
20h	20h	Data Encryption Key: The memory encryption key to utilize with the range.
40h	20h	Tweak Key: The memory encryption tweak key to utilize with the range. If the configured encryption algorithm does not require a tweak key, then this field shall be ignored.

11.5.5.8.8 Set Target Range Specific Key Response

If no error condition is detected, the DSM shall respond to the Set Target Range Specific Key request with a Set Target Range Specific Key Response message.

Table 11-53. Set Target Range Specific Key Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target Range Specific Key Response= 0Ah.
02h	2	Reserved

11.5.5.8.9 Set Target Range Random Key

The PrimarySession or SecondarySession(s) shall be utilized with the Set Target Range Random Key request to associate a specific memory range with initiator-specified entropy material. This request is utilized with range-based target memory encryption. Once set, the association between an initiator HPA memory range and the target's keys are immutable and attempts to set a new key for the same or subset of the same HPA range shall fail. To change the association, the memory range shall be explicitly cleared by the initiator, utilizing Clear Target Range Key before the memory range can be set for new keys using this request.

Possible Error Response, Error Codes:

- Version Mismatch
- Invalid Request
 - Memory Range Base and Range Size is invalid for the target.
 - Memory Range Base is not aligned to the Range Size.
 - Range Size is not a power of 2.
 - Range ID is $\geq$ Memory Encryption Number of Range Based Keys reported in Get Target Capabilities.
 - Memory range register specified by the Range ID is already associated with keys.
 - Request attempted to change a memory range or subset of the range already configured. Clear Target Range Key shall be utilized to reset the memory range association.
- Unsupported Request
 - Target does not support range-based memory encryption.
- Invalid Security State
 - Target not in CONFIG_LOCKED state.
- No Privilege
 - Request was not received on the PrimarySession or SecondarySession(s).

Table 11-54. Set Target Range Random Key (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target Range Random Key = 8Bh.
02h	2	Reserved
04h	4	Range ID: The range ID assigned to this encryption key. Shall be in the range specified in Get Target Capabilities Response, Memory Encryption Number of Range Based Keys.
08h	8	Range Start: HPA of the first 4-KB-aligned block within the range. • Bits[11:0]: Reserved • Bits[63:12]: Start HPA for the range, HPA[63:12]
10h	8	Range End: HPA of the last 4-KB-aligned block within the range. • Bits[11:0]: Reserved • Bits[63:12]: End HPA for the range, HPA[63:12]
18h	7	Reserved

Table 11-54. Set Target Range Random Key (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
1Fh	1	Validity Flags: Indicators of which fields are valid in the remaining portion of this request. More than one bit may be set. • Bit[0]: When set, the Data Encryption Key Entropy field is valid • Bit[1]: When set, the Tweak Key Entropy field is valid • Bits[7:2]: Reserved
20h	20h	Data Key Entropy: Optional initiator-supplied data key entropy that is utilized by the target when generating an encryption key for the range.
40h	20h	Tweak Key Entropy: Optional initiator-supplied memory encryption tweak key entropy that is utilized by the target when generating a tweak key for the range. If the configured encryption algorithm does not require a tweak key, then this field shall be ignored.

11.5.5.8.10 Set Target Range Random Key Response

If no error condition is detected, the DSM shall respond to the Set Target Range Random Key request with a Set Target Range Random Key Response message.

Table 11-55. Set Target Range Random Key Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Set Target Range Random Key Response = 0Bh.
02h	2	Reserved

11.5.5.8.11 Clear Target Range Key

The PrimarySession or SecondarySession(s) shall be utilized with the Clear Target Range Key request to clear any association between a previously set HPA memory range and random or specific keys that may have been programmed. This request is utilized with range-based target memory encryption. This request is utilized by the initiator to clear the association between an initiator's memory range and the target's keys and allows a memory range to be utilized with a new set of keys. The target shall break the association of HPA memory range to key, shall clear the associated key to 0 and memory encryption utilizing the cleared memory range shall be bypassed.

The same SPDM session that was utilized to set the key for the range shall be the same session that is utilized to clear the range. If the SPDM session utilized to set the key has been terminated or closed, then a Conventional Reset or CXL Reset shall be utilized to clear the memory range association with the key material.

Possible Error Response, Error Codes:

- Version Mismatch
- Unsupported Request
 - The target does not support range-based memory encryption
- Invalid Security State
 - Target not in CONFIG_LOCKED state
- No Privilege
 - The request was not received on the PrimarySession or SecondarySession(s)
 - The range was not set on the same SPDM session
- Invalid Request

- Range-based encryption is not supported by the target
- The range specified is not currently set

Table 11-56. Clear Target Range Key

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Clear Target Range Key = 8Ch.
02h	2	Reserved
04h	4	Range ID: The range ID assigned to this encryption key. Shall be within the range specified in Get Target Capabilities Response, Memory Encryption Number of Range Based Keys.

11.5.5.8.12 Clear Target Range Key Response

If no error condition is detected, the DSM shall respond to the Clear Target Range Key request with a Clear Target Range Key Response message.

Table 11-57. Clear Target Range Key Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Clear Target Range Key Response = 0Ch.
02h	2	Reserved

11.5.5.9 Optional Delayed Completion Requests and Responses

TSP provides a simple polling mechanism for requests that may take a significant amount of time to complete. If the time it takes for the target to execute the request might cause an SPDMA timeout, the target may return a Delayed Response. This will notify the initiator that the request passed all the syntax checks, has started execution on the target, and will take additional time to complete.

The initiator utilizes the returned Delay Time to check on the completion of the request by sending the Check Target Delayed Completion request after waiting the prescribed amount of time. If the request has completed the target shall send a Check Target Delayed Completion Response. If the request is still executing the target shall respond with another Delayed Response with the updated time the initiator is expected to wait.

11.5.5.9.1 Delayed Response

The response that the target has started executing a request that could take a significant amount of time to complete. The target shall return the number of microseconds (us) that it expects the initiator to wait before checking the completion of the request.

Table 11-58. Delayed Response (Sheet 1 of 2)

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Delayed Response = 7Eh.

Table 11-58. Delayed Response (Sheet 2 of 2)

Byte Offset	Length in Bytes	Description
02h	2	Reserved
04h	4	Delay Time: Estimated number of microseconds (us) that the initiator should delay before checking for the completion of the long executing command. Shall be > 0.

11.5.5.9.2 Check Target Delayed Completion

This request shall only be utilized by the initiator if a request returns a Delayed Response. It is utilized to verify the completion of a long executing request. If the request has completed execution, the target shall return a Check Target Delayed Completion Response. If the request is still executing, the target shall respond with Delayed Response with the new Delay Time that the initiator should wait before issuing this request again. If Delayed Response was not returned for a request, there is no delayed completion, and this request shall be failed by the target.

Possible Error Response, Error Codes:

- Version Mismatch
- Invalid Request
 - No request is outstanding that would result in a delayed completion

Table 11-59. Check Target Delayed Completion

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Check Target Delayed Completion = 8Eh.
02h	2	Reserved

11.5.5.9.3 Check Target Delayed Completion Response

If no error condition is detected and the execution of the long executing request has completed, the DSM shall respond to the Check Target Delayed Completion request with a Check Target Delayed Completion Response message. This indicates that the previously delayed completion of the request is now complete.

Table 11-60. Get Target TE State Change Completion Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Check Target Delayed Completion Response = 0Eh.
02h	2	Reserved

11.5.5.10 Error Response

The Error Response is permitted to be used by the target to complete any of the requests issued to the target.

Table 11-61. Error Response

Byte Offset	Length in Bytes	Description
00h	1	TSP Version: V1.0 = 10h.
01h	1	Opcode: Error Response = 7Fh.
02h	2	Reserved
04h	4	Error Code: See Table 11-62.
08h	4	Error Data: See Table 11-62.
0Ch	Varies	Extended Error Data: See Table 11-62.

Table 11-62. Error Response — Error Code, Error Data, Extended Error Data

Error Code	Definition	Error Data	Extended Error Data
0000h	Reserved	0000h	None
0001h	Invalid Request: One or more fields in the request are invalid.	0000h	None
0002h	Busy: The target could not process the request but the target may be able to process the request if it is sent again in the future.	0000h	None
0003h	Unspecified: An unspecified error occurred.	0000h	None
0004h	Unsupported Request: The Message Type in the command is unsupported.	Unsupported Message Type	None
0005h	Version Mismatch: The version in the request is not supported.	Highest TSP version number that the target supports	None
0006h	Vendor Specific Error: A vendor defined error occurred.	Length of Extended Error Data	Vendor specific error data
0007h	No Privilege: The requested session ID has no privilege to generate the request.	SPDM Session ID	None
0008h	No Entropy: Target failed to generate random numbers to execute the request.	0000h	None
0009h	Invalid CKID: The request contains an invalid CKID.	0000h	None
000Ah	Invalid Security Configuration: Target security checks failed due to an invalid configuration.	Length of Extended Error Data	Vendor specific error data
000Bh	Invalid Security State: The target was not in the correct TSP state to execute the request.	0000h	None
000Ch	Long Execution Time: The target did not start the execution of the request as it may cause SMTP timeouts waiting for the completion.	0000h	None
000Dh	Already Locked: An initiator is attempting to lock an already locked target.	0000h	None

§ §

12.0 Reliability, Availability, and Serviceability

CXL RAS capabilities are built on top of PCIe*. Additional capabilities are introduced to address cache coherency and memory as listed below.

12.1 Supported RAS Features

Table 12-1 lists the RAS features supported by CXL and their applicability to CXL.io vs. CXL.cache and CXL.mem.

Table 12-1. CXL RAS Features

Feature	CXL.io	CXL.cache and CXL.mem
Link CRC and Retry	Required	Required
Link Retraining and Recovery	Required	Required
eDPC	Optional	Leverage CXL.io capability CXL.cache or CXL.mem errors may be signaled via ERR_FATAL or ERR_NONFATAL and may trigger eDPC
ECRC	Optional	N/A
Hot-Plug	Not Supported in RCD mode Managed Hot-Plug is supported in CXL VH mode	Same as CXL.io
Data Poisoning	Required	Required
CXL Isolation	N/A	Optional (see Section 12.3)
Vital	N/A	Required (see Section 12.4)

12.2 CXL Error Handling

CXL Error handling can be subdivided into two parts:

- Link and Protocol Errors, which apply to the CXL component-to-component communication mechanism. These include errors detected by CXL.cache and CXL.mem protocol logic. This is further described in [Section 12.2.1](#) and [Section 12.2.2](#).
- Device Errors, which apply exclusively to the device itself. This is further described in [Section 12.2.3](#).

12.2.1 Protocol and Link Layer Error Reporting

Protocol and Link errors are detected and communicated to the Host where the errors can be exposed and handled. Errors may also be reflected to Platform software if so configured. There are no error pins that connect CXL devices to the Host. Errors are communicated between the Host and the CXL device via messages over CXL.io.

CXL Protocol and Link errors detected by components that are part of a CXL VH are escalated and reported using standard PCIe error reporting mechanisms over CXL.io as UIEs and/or CIEs. See PCIe Base Specification for details.

Reporting and logging of CXL Protocol and Link errors in RCD mode is described in this section.

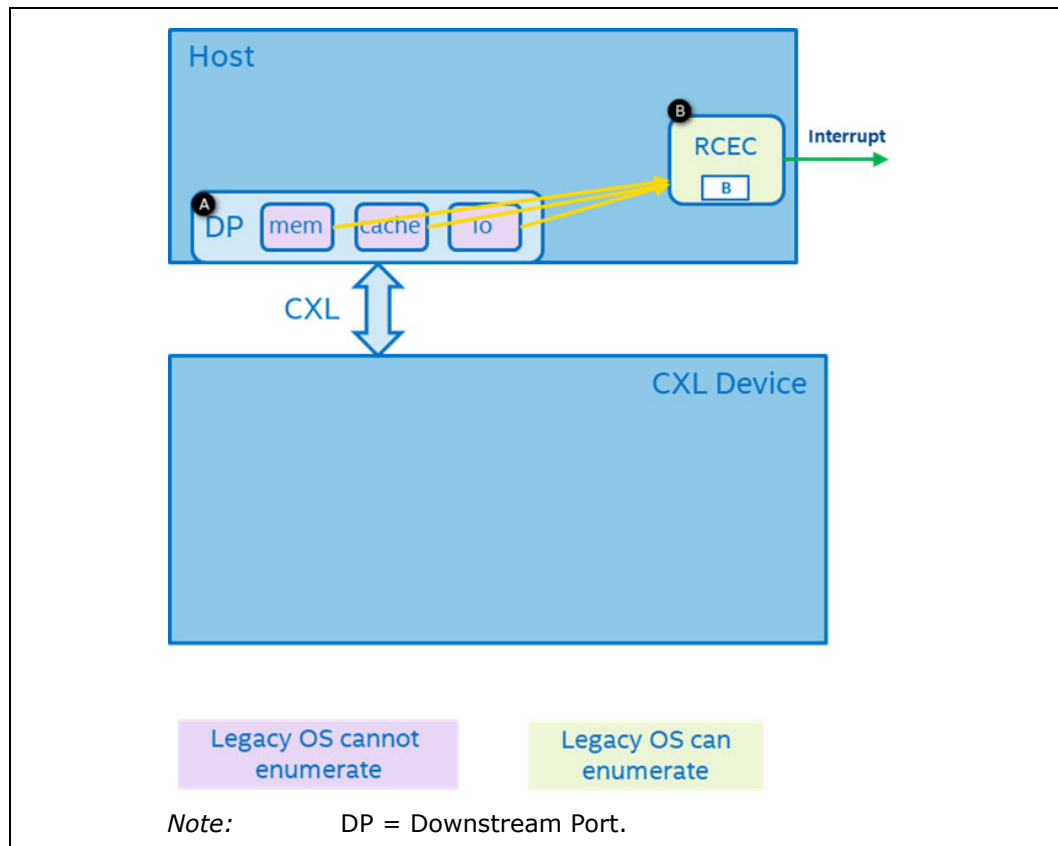
12.2.1.1 RCH Downstream Port-detected Errors

RCH Downstream Port-detected CXL Protocol errors are escalated and reported via the Root Complex error-reporting mechanisms as UIEs and/or CIEs. The various signaling and logging steps are listed below and illustrated in [Figure 12-1](#).

1. DP_A CXL.io-detected errors are logged in the local AER Extended Capability in DP_A RCRB. Software must ensure that the Root Port Control register in the DP_A AER Extended Capability is not configured to generate interrupts.
2. DP_A CXL.cache and CXL.mem log errors in the CXL RAS Capability (see [Section 8.2.4.17](#)).
3. DP_A CXL.cache, CXL.mem, or CXL.io sends error message(s) to RCEC.
4. RCEC logs UIEs and/or CIEs. The RCEC Error Source Identification register shall log the RCEC's Bus, Device, and Function Numbers because the RCH Downstream Port is not associated with one.
5. RCEC generates an MSI/MSI-X, if enabled.

The OS error handler may begin by inspecting the RCEC AER Extended Capability and following PCIe rules to discover the error source. The RCEC Error Source Identification register is insufficient for identifying the error source. The OS error handler may rely on RDPAS structures (see [Section 9.18.1.5](#)), if present, to identify such Downstream Port(s). The Platform Software Error Handler may interrogate the Platform-specific error logs in addition to the error logs defined in PCIe Base Specification and this specification.

Figure 12-1. RCH Downstream Port Detects Error



12.2.1.2 RCD Upstream Port-detected Errors

RCD Upstream Port-detected CXL protocol errors are also escalated and reported via the RCEC. The various signaling and logging steps are listed below and illustrated in Figure 12-2.

1. If a CXL.cache or CXL.mem logic block in UP_Z detects a protocol or link error, the block shall log the error in the CXL RAS Capability (see Section 8.2.4.17).
2. Upstream Port RCRB shall not implement the AER Extended Capability.
3. UP_Z sends an error message to all CXL.io Functions that are affected by this error. (This example shows a device with a single function. The message must include all the details that the CXL.io function needs for constructing an AER record.)
4. CXL.io Functions log the received message in their respective AER Extended Capability.
5. Each affected CXL.io Function sends an ERR_ message to UP_Z with its own Requester ID.
6. UP_Z forwards this Error message across the Link without logging.
7. DP_A forwards the Error message to the RCEC.
8. RCEC logs the error in the Root Error Status register and then signals an interrupt, if enabled, in accordance with PCIe Base Specification. The Error Source Identification register in the RCEC shall point to the CXL.io Function that sent the ERR_ message.

12.2.1.3 RCD RCiEP-detected Errors

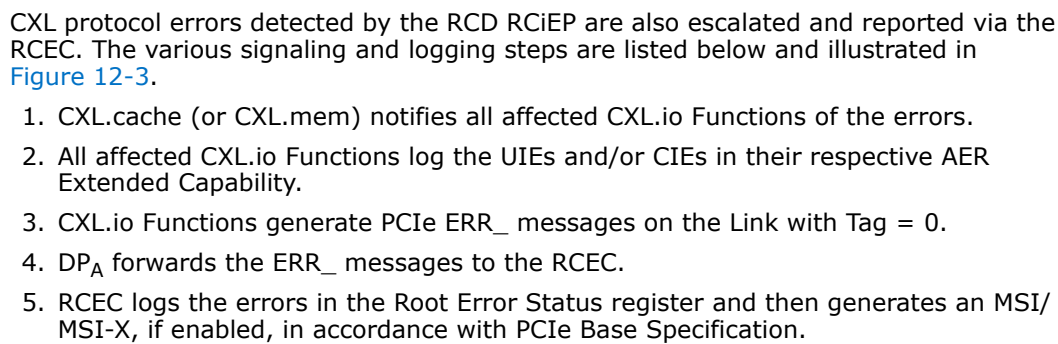
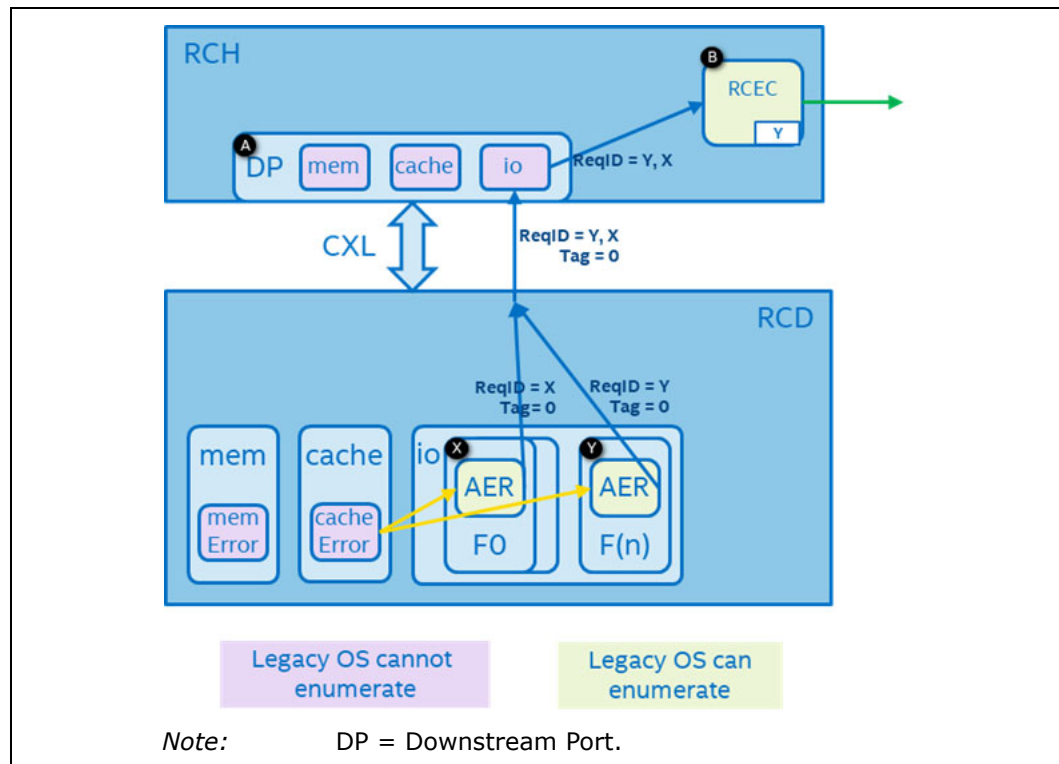


Figure 12-3. RCD RCiEP Detects Error



12.2.1.4 Header Log and Handling of Multiple Errors

Unmasked CXL protocol and link errors are captured in the Uncorrectable Error Status register and the Correctable Error Status register (see [Section 8.2.4.17.1](#) and [Section 8.2.4.17.4](#), respectively). In the scenarios where multiple bits are set in the Uncorrectable Error Status register, the First Error Pointer field in the Error Capabilities and Control register (see [Section 8.2.4.17.6](#)), if valid, points to the first uncorrectable error that was captured. The First Error Pointer is valid if the associated bit of the Uncorrectable Error Status register is set. Otherwise, it is invalid. By definition, First Error Pointer is considered invalid if bit 5 of that field is set to 1. For certain uncorrectable errors, the specification requires that the component capture part of the message header, called Error Header, in the Header Log register. [Section 8.2.4.17.1](#) defines the format of the Error Header for each error.

If the Multiple_Header_Recording_Capability bit in the Error Capabilities and Control register (see [Section 8.2.4.17.6](#)) is set, the component is capable of recording multiple Error Headers in the order in which they are detected. If header logging resources are unavailable when an unmasked uncorrectable error is detected, the corresponding error status bit is set to 1; however, the Error Header is not recorded in the Header Log register. After software has consumed the error to which the First Error Pointer points, software writes 1 to the corresponding error status bit to indicate that. The error status bit may remain set if there was another occurrence of the same error. If any bit in the Uncorrectable Error Status register remains set after this software action, the component must atomically update the Header Log register and the First Error Pointer to point to the next recorded error. If no other error is recorded, the component shall update the First Error Pointer to an invalid value. If Multiple_Header_Recording_Capability=1, it is recommended that software not clear

the Status bit other than the one pointed to by the First Error Pointer. If software violates this condition, the state of the Header Log register in the presence of other recorded errors is undefined.

12.2.2 CXL Root Ports, Downstream Switch Ports, and Upstream Switch Ports

CXL protocol errors detected by CXL root ports, DSPs, and USPs are escalated and reported using PCIe error-reporting mechanisms as UIEs and/or CIEs. It is strongly recommended that CXL.cachemem protocol errors that are detected by a CXL root port be logged as CIEs or UIEs in the root port's AER Extended Capability. The Error Source Identification register logs the Bus, Device, and Function Numbers of the Root Port itself. If the CXL.cachemem protocol errors detected by a CXL root port are logged as CIEs or UIEs in an RCEC's AER Extended Capability, it is recommended that the System Firmware populate an RDPAS record (see [Section 9.18.1.5](#)) to establish the association between the RCEC and the root port.

The OS error handler may begin by inspecting the Root Port AER Extended Capability and follow PCIe rules to discover the error source. The Platform Software Error Handler may interrogate the Platform-specific error logs in addition to the error logs defined in PCIe Base Specification and this specification.

If the CXL.cachemem errors are logged in an RCEC and the CEDT includes RDPAS structures (see [Section 9.18.1.5](#)) that reference the RCEC, the OS handler may consult those RDPAS structures to locate the CXL root port that is the error source.

12.2.3 CXL Device Error Handling

Whenever a CXL device returns data that is either known to be bad or suspect, the device must ensure that the consumer of the data is made aware of the nature of the data, either at the time of consumption or prior to data consumption. This allows the consumer to take appropriate containment action.

CXL defines two containment mechanisms - poison and viral:

- **Poison:** Return data on CXL.io and CXL.cachemem may be tagged as poisoned.
- **Viral:** CXL.cachemem supports viral, which is mainly used to indicate more-severe error conditions at the device (see [Section 12.4](#)). Any data returned by a device on CXL.cachemem after the device has communicated Viral is considered suspect, even if the data is not explicitly poisoned.

A device must set the MetaField to No-Op in the CXL.cachemem return response when the Metadata is suspect.

If a CXL component is not in the Viral condition, the component shall poison the data message on the CXL interface whenever the data being included is known to be bad or suspect.

If Viral is enabled and a CXL component is in the Viral condition, it is recommended that the component not poison the subsequent data responses on the CXL.cachemem interface to avoid error pollution.

The Host may send poisoned data to the CXL-connected device. How the CXL device responds to Poison is device specific but must follow PCIe guidelines. The device must consciously make a decision about how to handle poisoned data. In some cases, simply ignoring poisoned data may lead to Silent Data Corruption (SDC). A CXL device is required to keep track of any poison data that the device receives on a 64-byte granularity.

Any device errors that cannot be handled with Poison indication shall be signaled by the device back to the Host as messages since there are no error pins. To that end, [Table 12-2](#) shows a summary of the error types and their mappings, and error reporting guidelines for devices that do not implement Memory Error Logging and Signaling Enhancements (see [Section 12.2.3.2](#)).

For devices that implement Memory Error Logging and Signaling Enhancements, [Section 12.2.3.2](#) describes how memory errors are logged and signaled. Such devices should follow [Table 12-2](#) for dealing with all non-memory errors.

Table 12-2. Device-specific Error Reporting and Nomenclature Guidelines

Error Severity	Definition/Example	Signaling Options (SW picks one)	Logging ¹	Host HW/FW/SW Response
Corrected	Memory single bit error corrected via ECC	MSI or MSI-X to Device driver	Device-specific registers	Device-specific flow in Device driver
Uncorrected Recoverable	UC errors from which the Device can recover, with minimal or no software help (e.g., error localized to single computation)	MSI or MSI-X to driver	Device-specific registers	Device-specific flow in driver (e.g., discard results of suspect computation)
Uncorrected NonFatal	Equivalent to PCIe UCNF, contained by the device (e.g., write failed, memory error that affects many computations)	MSI or MSI-X to Device Driver	Device-specific registers	Device-specific (e.g., reset affected device) flow in driver. Driver can escalate through software.
		PCIe AER Internal Error	Device-specific registers + PCIe AER	System FW/SW AER flow, ends in reset
Uncorrected Fatal	Equivalent to PCIe UCF, poses containment risk (e.g., command/parity error, Power management Unit ROM error)	PCIe AER Internal error	Device-specific registers + PCIe AER	System FW/SW AER flow, ends in reset
		AER + Viral		System FW/SW Viral flow

1. For CXL devices that implement memory error logging and signaling enhancements (see [Section 12.2.3.2](#)), the memory error logging and signaling mechanisms are defined by the CXL specification.

In keeping with the standard error logging requirements, all error logs should be sticky.

12.2.3.1 CXL.cache and CXL.mem Errors

If demand accesses to memory result in an uncorrected data error, the CXL device must return data with poison. The requester (processor core or a peer device) is responsible for dealing with the poison indication. The CXL device should not signal an uncorrected error along with the poison. If the processor core consumes the poison, the error will be logged and signaled by the Host.

Any non-demand uncorrected errors detected by a device (e.g., memory scrub logic in CXL device memory controller) that does not support the Memory Error Logging and Signaling Enhancements (see [Section 12.2.3.2](#)) will be signaled to the device driver via a device MSI or MSI-X. Any corrected memory errors will be signaled to the device driver via a device MSI or MSI-X. The driver may choose to deallocate memory pages that have repeated errors. Neither the platform firmware nor the OS directly deal with these errors. An eRCD may implement the capabilities described in [Section 12.2.3.2](#), in which case a device driver is not required.

If a CXL component is unable to positively decode a CXL.mem address, the handling is described in [Section 8.2.4.20.2](#). If a component does not implement HDM Decoders (see [Section 8.2.4.20](#)), the component shall drop such a write transaction and return all 1s in response to such a read transaction.

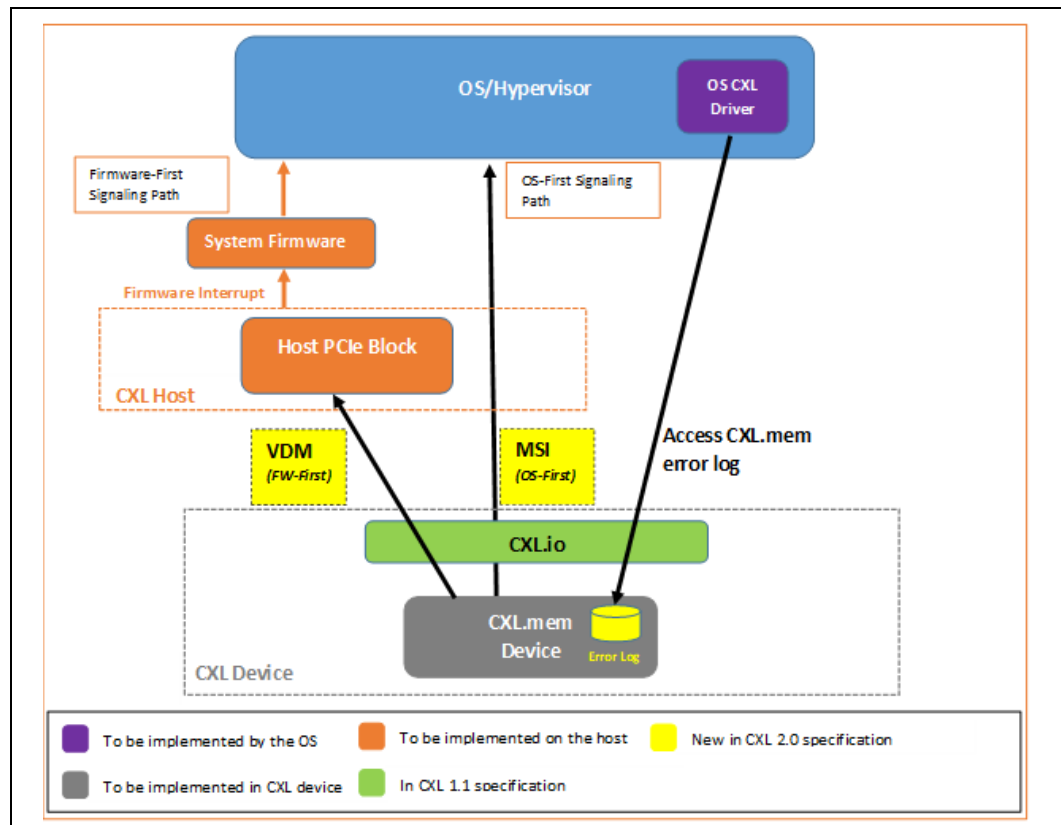
12.2.3.2 Memory Error Logging and Signaling Enhancements

Errors in memory may be encountered during a demand access or independent of any request issued to the memory. It is important to log sufficient data about such errors to enable the use of host platform-level RAS features, such as page retirement, without dependence on a driver.

In addition, general device events that are unrelated to the media, including changes in the device's health or environmental conditions detected by the device, need to be reported using the same general-event logging facility.

Figure 12-4 illustrates a use case where the two methods of signaling supported by a CXL.mem device - VDM and MSI/MSI-X – are used by a host to implement Firmware-first and OS-first error handling.

Figure 12-4. CXL Memory Error Reporting Enhancements



A CXL device that supports the Memory Error Logging and Signaling Enhancements capability must log such errors locally and expose the error log to system software via the MMIO Mailbox (see [Section 8.2.8.4.3](#)). Reading an error record from the mailbox will not automatically result in deletion of the error record on the device. An explicit clear operation is required to delete an error record from the device. To support error record access and deletion, the device shall implement the Get Event Records and Clear Event Records commands.

Both operations must execute atomically. Furthermore, all writes or updates to the error records by the CXL.mem device must also execute atomically.

Using these two operations, a host can retrieve an error record as follows:

1. The host reads a number of event records using the Get Event Records command.
2. When complete, the host clears the event records from the device with the Clear Event Records command, supplying one or more event record handles to clear.

The error records will be owned by the host firmware or OS so that all logged errors are made available to the host to support platform-level RAS features.

Error records stored on the CXL device must be sticky across device resets. The records must not be initialized or modified by a hot reset, an FLR, or CXL Reset (see [Section 9.7](#)). Devices that consume auxiliary power must preserve the error records when auxiliary power consumption is enabled. In these cases, the error records are neither initialized nor modified by hot reset, warm reset, or cold reset.

12.2.3.3 CXL Device Error Handling Flows

RCD errors may be sourced from a Root Port (RP) or Endpoint (RCiEP). For the purpose of differentiation, RCiEP-sourced errors shall use a tag value of 0, whereas RP-sourced errors shall use a tag of nonzero value.

Errors detected by the CXL device shall be communicated to the host via PCIe Error messages across the CXL.io link. Errors that are not related to a specific Function within the device (Non-Function errors) and not reported via an MSI/MSI-X are reported to the Host via PCIe error messages where the errors can be escalated to the platform.

The Upstream Port reports non-function errors to all EPs/RCiEPs where they are logged. Each EP/RCiEP reports the non-function-specific errors to the host via PCIe error messages. Software should be aware that although an RCiEP does not have a software-visible link, the RCiEP may still log link-related errors.

At most, one error message of a given severity is generated for a multi-function device. The error message must include the Requester ID of a function that is enabled to send the error message. Error messages with the same Requester ID may be merged for different errors with the same severity. No error message is sent if no function is enabled to do so. If different functions are enabled to send error messages of different severity, at most one error of each severity level is sent.

Errors generated by the RCD RCiEP will be sent to the corresponding RCEC. Each RCiEP must be associated with no more than one RCEC. Errors generated by a CXL component that is part of a CXL VH shall be logged in the CXL Root Port.

12.3 Isolation on CXL.cache and CXL.mem

Isolation on CXL.cache and CXL.mem is an optional normative capability of a CXL Root Port. Such isolation halts traffic on the respective protocol. Further, once triggered, the Root Port synthesizes the response for all pending and subsequent transactions on that protocol. This is further described in [Section 12.3.1](#) and [Section 12.3.2](#), respectively.

The specification defines two trigger mechanisms:

- Link Down – If a Root Port supports CXL.cache isolation and software enables CXL.cache isolation, a Link Down condition shall unconditionally trigger CXL.cache isolation. If a Root Port supports CXL.mem isolation and software enables CXL.mem isolation, a Link Down condition shall unconditionally trigger CXL.mem isolation.
- Transaction timeout – A Root Port that supports CXL.cache isolation may be capable of being configured in such a way that a CXL.cache timeout triggers CXL.cache isolation. A Root Port that supports CXL.mem isolation may be capable of being configured in such a way that a CXL.mem timeout triggers CXL.mem isolation.

Note: Transaction Timeout Value settings for CXL.cache and CXL.mem: The system needs to ensure that timeouts are appropriately set up. For example, a timeout should not be so short that isolation is triggered due to a non-erroneous, long-latency access to a CXL device. Software may need to temporarily disable the triggering of isolation upon timeout if one or more devices are being transitioned to a state (e.g., firmware update) where the device may violate the timeout.

The primary purpose of the isolation action is to complete pending and subsequent transactions that are associated with the isolated root port quickly, with architected semantics, after isolation is triggered. Since system memory and system caches must generally be assumed to be corrupted, software recovery generally relies on software to identify all software threads, VMs, containers, etc., whose system state might be corrupted, and then terminating them. Other software recovery mechanisms are also possible, and they are beyond the scope of this specification.

A Root Port indicates support for Isolation by implementing the CXL Timeout and Isolation Capability structure (see [Section 8.2.4.24](#)). The structure contains the capability, control, and status bits for both Transaction Timeout and Isolation on both CXL.cache and CXL.mem. Both Timeout and Isolation are disabled by default and must be explicitly and individually enabled by software for each protocol before they can be triggered. When Isolation is enabled for either CXL.cache or CXL.mem, software can optionally configure the Root Port to force a Link Down condition if the respective protocol enters Isolation.

When Isolation is entered, the Root Port, if capable, signals an MSI/MSI-X or send an ERR_COR Message if enabled. Software may also choose to rely only on mandatory synchronous exception handling (see [Section 12.3.1](#) and [Section 12.3.2](#)). Software may read the CXL Timeout and Isolation Status register to determine whether a Timeout or Isolation has occurred on CXL.cache and/or CXL.mem and if the Isolation was triggered due to a Timeout or due to a Link Down condition. The software must explicitly clear the corresponding Isolation status bits (see [Section 8.2.4.24.3](#)) for the root port to exit Isolation. The link must transition through the Link Down state before software can attempt re-enumeration and device recovery.

12.3.1 CXL.cache Transaction Layer Behavior during Isolation

This section specifies the CXL.cache Transaction Layer's behavior while the Root Port is in Isolation.

The Root Port shall handle host requests that would ordinarily be mapped to (H2D) CXL.cache messages in the following manner.

For each host snoop that would ordinarily be mapped to (H2D) CXL.cache request messages:

- If the host is tracking the device as a possible exclusive owner of the line, then data is treated as poison.
- Else if the host knows the device can only have a Shared or Invalid state for the line, then the device cache is considered Invalid (no data poisoning is needed).

IMPLEMENTATION NOTE

Exclusive vs. Shared/Invalid may be known based on an internal state within the host.

The Root Port timeout detection logic shall account for partial responses. For example, if the Root Port observes that the data is returned on the D2H Data channel in a timely manner, but no D2H Rsp was observed for a sufficient length of time, the Root Port shall treat it as a CXL.cache timeout.

For each pending Pull that is mapped to H2D CXL.cache Response of type *WritePull* which expects a data return, the Root Port must treat the returned data as poison.

12.3.2 CXL.mem Transaction Layer Behavior during Isolation

This section specifies the CXL.mem Transaction Layer's behavior while the CXL Root Port is in Isolation.

The Root Port shall handle host requests that it would ordinarily map to (M2S) CXL.mem messages in the following manner:

- For each host request that would ordinarily be mapped to CXL.mem Req and Rwd:
 - For Read transactions, the CXL Root Port synthesizes a synchronous exception response. The specific mechanism of synchronous exception response is CXL Root Port implementation specific. An example of a synchronous exception response would be returning data with Poison.
 - For non-read transactions, the CXL Root Port synthesizes a response as appropriate. The specific mechanism of the synthesized response is implementation specific. An example would be returning a completion (NDR) for a write (Rwd) transaction.

12.4 CXL Viral Handling

CXL links and CXL devices are expected to be Viral compliant. Viral is an error-containment mechanism. A platform must choose to enable Viral at boot. The Host implementation of Viral allows the platform to enable the Viral feature by writing into a register. Similarly, a BIOS-accessible control register on the device is written to enable Viral behavior (both receiving and sending) on the device. Viral support capability and control for enabling are reflected in the DVSEC.

When enabled, a Viral indication is generated whenever an Uncorrected_Fatal error is detected. Viral is not a replacement for existing error-reporting mechanisms. Instead, its purpose is an additional error-containment mechanism. The detector of the error is responsible for reporting the error through AER and generating a Viral indication. Any entity that is capable of reporting Uncorrected_Fatal errors must also be capable of generating a Viral indication.

CXL.cache and CXL.mem are pre-enabled with the Viral concept. Viral needs to be communicated in both directions. When Viral is enabled and the Host runs into a Viral condition, the Host shall communicate Viral across CXL.cache and/or CXL.mem to all downstream components. The Viral indication must arrive before any data that may have been affected by the error (general Viral requirement). If the host receives a Viral indication from any CXL components, the Host shall propagate Viral to all downstream components.

All types of Conventional Resets shall clear the viral condition. CXL Resets and FLRs shall have no effect on the viral condition.

12.4.1 Switch Considerations

Viral is enabled on a per-vPPB basis and the expectation is that if Viral is enabled on one or more DSPs, then Viral will also be enabled on the USP within a VCS.

A Viral indication received on any port transitions that VCS into the Viral state, but does not trigger a new uncorrected fatal error inside the switch. A Viral indication in one VCS has no effect on other VCSs within the switch component. The switch continues to process all CXL.io traffic targeting the switch and forward all traffic. All CXL.cache and CXL.mem traffic sent to all ports within the VCS is considered to have the Viral bit set. The Viral indication shall propagate from an input port to all output ports in the VCS

faster than any subsequent CXL.cache or CXL.mem transaction. The Viral bit is propagated across upstream links and links connected to SLDs with the Viral LD-ID Vector (see [Table 4-10](#)) set to 0 for compatibility with the CXL 1.1 specification.

If the switch detects an uncorrected fatal error, the switch must determine whether that error affects one or multiple VCSs. Any affected VCS enters the Viral state, sets the Viral_Status bit (see [Section 8.1.3.3](#)) to indicate that a Viral condition has occurred, asserts the Viral bit in all CXL.cache and CXL.mem traffic sent to all ports within the VCS, and then sends an AER message. The affected VCS continues to forward all CXL traffic.

Hot-remove and hot-add of devices below DSPs have no effect on the Viral state of the VCS within the switch.

If the switch has configured and enabled MLD ports, then there are additional considerations. When a VCS with an MLD port enters the Viral state, the VCS propagates the Viral indication to LDs within the MLD Component by setting the Viral bit in the Viral LD-ID Vector (see [Table 4-10](#)) for the LDs in that VCS. If an uncorrected fatal error causes one or more VCSs to enter the Viral state, then the corresponding bits in the Viral LD-ID Vector shall be set. An LD within an MLD component that has entered the Viral state sets the Viral bit in CXL.mem traffic with the Viral LD-ID Vector mask set to identify all the LD-IDs associated with all the affected VCSs. The indication from each LD-ID propagates the Viral state to all associated VCSs that have Viral containment enabled.

12.4.2 Device Considerations

Although the device's reaction to Viral is device specific, the device is expected to take error-containment actions that are consistent with Viral requirements. Mainly, the device must prevent bad data from being committed to permanent storage. If the device is connected to any permanent storage or to an external interface that may be connected to permanent storage, then the device is required to self-isolate to be Viral compliant. This means that the device has to take containment actions without depending on help from the Host.

The containment actions taken by the device must not prevent the Host from making forward progress. This is important for diagnostic purposes as well as for avoiding error pollution (e.g., withholding data for read transactions to device memory may cause cascading timeouts in the Hosts). Therefore, on Viral detection, in addition to the containment requirements, the device shall:

- Drop writes to the persistent HDM ranges on the device or connected to the device.
- Always return a Completion response.
- Set MetaField to No-Op in all responses that carry MetaField.
- Fail the Set Shutdown State command (defined in [Section 8.2.9.9.3.5](#)) with an Internal Error when attempting to change the state from "dirty" to "clean".
- Not transition the Shutdown State to "clean" after a GPF flow.
- Commit to the persistent HDM ranges any writes that were completed over the CXL interface before receipt of the viral condition.
- Keep responding to snoops.
- Complete pending writes to Host memory.
- Complete all reads and writes to Device volatile memory.

When the device itself runs into a Viral condition and Viral is enabled, the device shall:

- Set the Viral Status bit to indicate that a Viral condition has occurred

- Containment – Take steps to contain the error within the device (or logical device in an MLD component) and follow the Viral containment steps listed above.
- Communicate the Viral condition back up to CXL.cache and CXL.mem, toward the Host.
 - Viral propagates to all devices in the Virtual Hierarchy, including to the host.

Viral Control and Status bits are defined in the DVSEC (see [Chapter 3.0](#) for details).

12.5 Maintenance

Maintenance operations may include media maintenance, media testing, module testing, etc. A maintenance operation is identified by a Maintenance Operation Class and a Maintenance Operation Subclass. A Device may support one or more Maintenance Operation Subclasses related to a Maintenance Operation Class. See [Table 8-102](#).

The Device may use Event Records to notify the System Software or System Firmware about needing a maintenance operation. When the Device requires maintenance, the Maintenance Needed bit in the Event Record Flags is set to 1, while the class of recommended maintenance operation is indicated by the Maintenance Operation Class field. See [Table 8-43](#).

The Perform Maintenance command (see [Section 8.2.9.7.1](#)) initiates a maintenance operation. The maintenance operation to be executed is specified in the input payload by the Maintenance Operation Class field and the Maintenance Operation Subclass field.

12.6 CXL Error Injection

The major aim of error-injection mechanisms is to allow system validation and system firmware/software development, etc., the means to create error scenarios and error-handling flows. To this end, a CXL Upstream Port and Downstream Port are recommended to implement the following error injection hooks to a specified address (where applicable):

- One type of CXL.io UC error (optional - similar to PCIe)
 - CXL.io is always present in any CXL connection
- One type of CXL.cache UC error (if applicable)
- One type of CXL.mem UC error (if applicable)
- Link Correctable errors
 - Transient errors and
 - Persistent errors
- Returning Poison on a read to a specified address (CXL.mem only)

Error injection interfaces are documented in [Chapter 14.0](#).

§ §

13.0 Performance Considerations

CXL provides a low-latency, high-bandwidth path for an accelerator to access the system. Performance on CXL is dependent on a variety of factors. [Table 13-1](#) captures the main CXL performance attributes.

Table 13-1. CXL Performance Attributes

Characteristic	CXL via Flex Bus (if PCIe Gen 4)	CXL via Flex Bus (if PCIe Gen 5)	CXL via Flex Bus (if PCIe Gen 6)
Width	16 Lanes		
Link Speed	16 GT/s	32 GT/s	64 GT/s
Total Bandwidth per link ¹	32 GB/s	64 GB/s	128 GB/s

1. Achieved bandwidth depends on protocol and payload size. Expect 60-90% efficiency on CXL.cache and CXL.mem. Efficiency similar to PCIe* on CXL.io.

In general, it is expected that the Upstream Ports and Downstream Ports are rate-matched. However, if the implementations are not rate-matched, it would require the faster of the implementations to limit the rate of its protocol traffic to match the slower (including bursts) whenever there is no explicit flow-control loop.

CXL allows accelerators/devices to coherently access host memory, and allows memory attached to an accelerator/device to be mapped into the system address map and to be accessed directly by the host as writeback memory. To support this, it supports Coherency models as described in [Section 2.2.1](#) and [Section 2.2.2](#).

13.1 Performance Recommendations

To minimize buffering requirements and provide good responsiveness, CXL components need to strive for low latency. Specific transaction flows merit special attention, depending on component type, to ensure the system performance is not negatively impacted.

It is recommended that components meet the latency targets listed in [Table 13-2](#). These targets are measured at the component pins in an otherwise idle system. Measurements are for average idle response times, meaning that a single message is transmitted to the component and the response is received from the component before another message is transmitted to the component. Messages used for average measurements should have their addresses randomly distributed. All the targets listed in [Table 13-2](#) assume a x16 link at full speed (64 GT/s) with IDE disabled.

For CXL devices and switches, CDAT (see [Section 8.1.11](#)) provides a mechanism for

Table 13-2. Recommended Latency Targets for Selected CXL Transactions

Case ¹	Component	Protocol	Received Message	Transmitted Message	Latency Target
1	Type 1/Type 2	CXL.cache	H2D Req Snoop (Miss Case)	D2H Resp Snoop Response	90-150 ns
2			H2D Resp WritePull	D2H Data	65 ns
3	Type 3 (DDR)	CXL.mem	M2S Req MemRd	S2M DRS MemData	80 ns
4			M2S Rwd MemWr	S2M NDR Cmp	40 ns
5	Host		S2M BISnp	M2S BIRsp	90 ns

1. **Case 1:** The range provided accounts for implementation trade-offs, with a dedicated CXL interface snoop filter providing the lowest latency, and snoop filters embedded in the Device cache hierarchy resulting in higher latency.

Case 2: Assumes write data is ready to transmit in a CXL output buffer.

Cases 3 and 4: Applies to Type 3 Devices using DRAM media intended to provide system-level performance comparable to DDR DIMMs. Such Devices are assumed to be relatively simple, small, and low-power, and not complex, multi-ported, or pooled-memory Devices. Noting that Case 3 is more aggressive than the targets in [Table 13-3](#), the [Table 13-3](#) targets will be lower than the Case 3 target in these Devices. Memory Devices that use slower media, such as some persistent-memory types, will have longer latencies that must be considered by system designers.

Case 5: The BISnp will cause the host to resolve coherence for an HDM-DB memory address owned by a device that sent the BISnp. The target latency for resolving coherence is for the simple case in which the host does not have the cacheline in any host-managed cache (this scope includes peer CXL devices that are using CXL.cache).

reporting both lowest latency and highest bandwidth to Host software (see ACPI Specification for the definition of the System Locality Latency and Bandwidth Information Structure). That reporting structure, however, only assists Host software. Hardware and system designers must consider the topologies and components of interest at design time. Hardware designers should consider the maximum latency that their component needs to tolerate while still maintaining high-link bandwidth, sizing outstanding requests, and data buffers accordingly.

The QoS Telemetry mechanism (see [Section 3.3.4](#)) defines a mechanism by which Hosts can dynamically adjust their request rate to avoid overloading memory devices. This mechanism is particularly important for MLD components that are shared among multiple Hosts.

At the Link Layer, the maximum loaded latencies listed in [Table 13-3](#) are recommended. If not adhered to, the link between two ports risks being throttled to less than the line rate. These recommendations apply to all CXL ports and both the CXL.cache and CXL.mem interfaces. The targets assume a x16 link with IDE disabled.

Table 13-3. Recommended Maximum Link Layer Latency Targets

Case ¹	Condition	Latency Target
1	Message received to Ack Transmitted	65 ns
2	Credit Received to Flit transmitted	50 ns

1. **Case 1:** Accounts for a sequence of 8 back-to-back flits with a clean CRC that needs to accumulate before the Ack can be transmitted. Applies only to links operating in 68B Flit mode and thus assumes a full link speed of 32 GT/s. Links operating in 256B Flit mode share Ack/Retry logic with PCIe and fall under any guidelines or requirements provided in PCIe Base Specification.

Case 2: In this case, the port lacks the Link Layer credits that are needed to transmit a message and then receives a credit update that enables transmission of the message. Assumes full link speed of 64 GT/s.

13.2 Performance Monitoring

Performance tuning and performance debug activities rely on performance counters that are located within different system components. This section introduces the Performance Monitoring infrastructure for CXL components.

The base hardware unit is called the CXL Performance Monitoring Unit (CPMU). A CXL component may have zero or more CPMU instances. Each CPMU instance includes one or more Counter Units. The registers associated with a CPMU are located by following the CXL Register Locator DVSEC with Register Block Identifier=4. Each CPMU instance in a Switch shall count events that are associated with a single Port. The CPMU instance associated with the Port can be identified by following the CXL Register Locator DVSEC in the Configuration Space of that Port. If a CXL multi-function device implements one or more CPMU instances, the Register Locator DVSEC that is associated with Function 0 shall point to them.

A CXL Event is defined as the occurrence of a specific component activity that is relevant to the operation of the CXL component. Events are grouped into Event Groups based on the type of activity that the Events represent. The pair <Event Vendor ID, Event Group ID> identifies an Event Group. Each Event Group can have up to 32 different types of events and each event is identified by a 5-bit Event ID. The tuple <Event Vendor ID, Event Group ID, Event ID> uniquely identifies the type of the Event. See [Table 13-5](#) for the list of CXL Events that are defined by this specification. The **Filter** column lists all the Filter ID values that are applicable to the Event Group. The Multiple Event Counting (**MEC**) column defines the Counter Unit behavior when it is configured to simultaneously count more than one event within the Event Group by setting multiple bits. If the **MEC** column indicates ADD, the Counter Unit shall *add* the occurrences of all the enabled events every clock, which may result in the Counter Data being incremented by a value of more than one within a single clock. If the **MEC** column indicates OR, the Counter Unit shall *logically or* the occurrences of all the enabled events every clock and the Counter Data shall never increment by more than one within any single clock.

None of the events defined in this specification are capable of incrementing the Counter Data by more than one per cycle. As such, software must set the Threshold field in the Counter Configuration register (see [Section 8.2.7.2](#)) to 1 when counting any event specified here.

A Counter Unit is capable of counting the occurrence of one or more events. Counter Units are capable of being configured to count a subset of the Events that the Counter Unit is capable of counting. A Counter Unit may be capable of being configured to take certain predefined actions when the count overflows. [Table 13-4](#) describes the three types of Counter Units.

Table 13-4. CPMU Counter Units

Counter Unit Type	Description
Fixed Function	Capable of counting a fixed set of one or more events within a single Event Group. Counting is halted when the Counter Unit is Frozen or Counter Enable=0.
Free-running	Capable of counting a fixed set of one or more events within a single Event Group. Not affected by freeze. The Counter Enabled bit is RO and always returns 1.
Configurable	Capable of counting any Events that are identified via CPMU Event Capabilities register (see Section 8.2.7.1.4), and may be configured by software to count a specific set of events in a specific manner. Counting is halted when the Counter Unit is Frozen or Counter Enable=0.

The CPMU register interface is defined in [Section 8.2.7](#). These registers are for Host Software or System Firmware consumption. The component must not expose CPMU registers or the underlying resources to an out-of-band agent if such an access may

interfere with the Host Software actions. Although a component may choose to implement a separate set of counters for out-of-band usage, use of such a mechanism is beyond the scope of this specification.

Table 13-5. Events under CXL Vendor ID (Sheet 1 of 3)

Event Group	Event ID	Mnemonic	Event Description	Filters	MEC ¹
00h (Base)	00h	Clock Ticks	Count the clock ticks of the always-on fixed-frequency clock that is used to increment the Counters. Every CPMU must allow counting of this event, either via a Fixed Function Counter Unit or via a Configurable Counter Unit.	None	N/A
01h-0Fh	00h - 1Fh	Reserved	Reserved	N/A	N/A
10h (CXL.cache D2H Req Channel)	D2H Req Opcode encoding	D2HReq.Opcode mnemonic	Counts the number of messages in the D2H Req message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The D2H Req message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-22. All Event ID values corresponding to unused D2H Req message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=10h, Event ID=01h is D2HReq.RdCurr and counts the number of RdCurr requests.	None	ADD
11h (CXL.cache D2H Rsp Channel)	D2H Rsp Opcode encoding	D2HRsp.Opcode mnemonic	Counts the number of messages in the D2H Rsp message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. D2H Rsp message class Opcode mnemonics and Opcode encodings are enumerated in Table 3-25. All Event ID values corresponding to unused D2H Rsp message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=11h, Event ID=05h is D2HRsp.RspIHitSE and counts the number of RspIHitSE messages.	None	ADD
12h (CXL.cache H2D Req Channel)	H2D Req Opcode encoding	H2DReq.Opcode mnemonic	Counts the number of messages in the H2D Req message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The H2D Req message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-26. All Event ID values corresponding to unused H2D Req message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=12h, Event ID=02h is H2DReq.SnpInv and counts the number of SnpInv requests.	None	ADD
13h (CXL.cache H2D Rsp Channel)	H2D Rsp Opcode encoding	H2DRsp.Opcode mnemonic	Counts the number of messages in the H2D Rsp message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. H2D Rsp message class Opcode mnemonics and Opcode encodings are enumerated in Table 3-27. All Event ID values corresponding to unused D2H Rsp message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=13h, Event ID=01h is H2DRsp.WritePull and counts the number of WritePull messages.	None	ADD
14h (CXL.cache Data)	00h	D2H Data	Counts the number of D2H Data messages that were initiated, or forwarded, or received by the component.	None	ADD
	01h	H2D Data	Counts the number of H2D Data messages that were initiated, or forwarded, or received by the component.	None	
	02h - 1Fh	Reserved	Reserved	N/A	N/A
15h-1Fh	00h - 1Fh	Reserved	Reserved	N/A	N/A

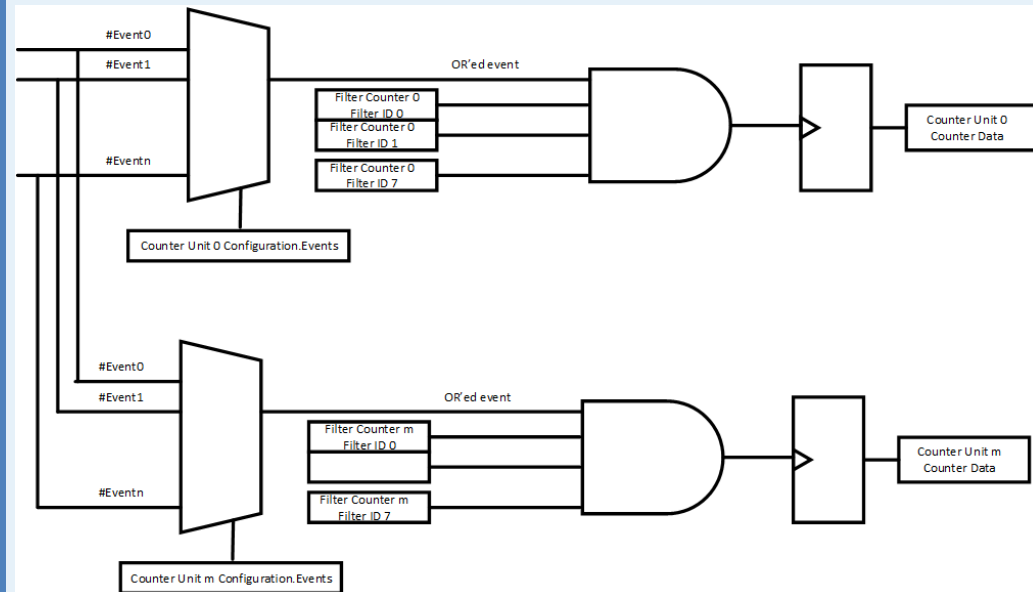
Table 13-5. Events under CXL Vendor ID (Sheet 2 of 3)

Event Group	Event ID	Mnemonic	Event Description	Filters	MEC ¹
20h (CXL.mem M2S Req Channel)	M2S Req Opcode encoding	M2SReq.Opcode mnemonic	Counts the number of messages in the M2S Req message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The M2S Req message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-35. All Event ID values corresponding to unused M2S Req message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=20h, Event ID=00h is M2SReq.MemInv and counts the number of MemInv requests.	Filter ID=0	ADD
21h (CXL.mem M2S Rwd Channel)	M2S Rwd Opcode encoding	M2SRwd.Opcode mnemonic	Counts the number of messages in the M2S Rwd message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The M2S Rwd message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-41. All Event ID values corresponding to unused M2S Req message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=21h, Event ID=02h is M2SRwd.MemWrPtl and counts the number of MemWrPtl requests.	Filter ID=0	ADD
22h (CXL.mem M2S BIRsp Channel)	M2S BIRsp Opcode encoding	M2SBIRsp.Opcode mnemonic	Counts the number of messages in the M2S BIRsp message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The M2S BIRsp message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-45. All Event ID values corresponding to unused M2S BIRsp message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=22h, Event ID=00h is M2SBIRsp.BIRspI and counts the number of BIRspI messages.	Filter ID=0	ADD
23h (CXL.mem S2M BISnp Channel)	S2M BISnp Opcode encoding	S2MBISnp.Opcode mnemonic	Counts the number of messages in the S2M BISnp message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The S2M BISnp message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-47. All Event ID values corresponding to unused S2M BISnp message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=23h, Event ID=00h is S2MBISnp.BISnpCur and counts the number of BISnpCur requests.	Filter ID=0	ADD
24h (CXL.mem S2M NDR Channel)	S2M NDR Opcode encoding	S2MNDR.Opcode mnemonic	Counts the number of messages in the S2M NDR message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The S2M NDR message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-50. All Event ID values corresponding to unused S2M NDR message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=24h, Event ID=02h is S2MNDR.Cmp-E and counts the number of Cmp-E messages.	Filter ID=0	ADD
25h (CXL.mem S2M DRS Channel)	S2M DRS Opcode encoding	S2MDRS.Opcode mnemonic	Counts the number of messages in the S2M DRS message class with Opcode=Event ID that were initiated, or forwarded, or received by the component. The S2M DRS message class Opcode mnemonics and the Opcode encodings are enumerated in Table 3-53. All Event ID values corresponding to unused S2M DRS message class Opcode encodings are reserved. For example, the mnemonic associated with the Event Group=25h, Event ID=00h is S2MDRS.MemData and counts the number of MemData messages.	Filter ID=0	ADD
26h - 7FFFh	00h-1Fh	Reserved	Reserved	N/A	N/A

Table 13-5. Events under CXL Vendor ID (Sheet 3 of 3)

Event Group	Event ID	Mnemonic	Event Description	Filters	MEC ¹
8000h (DDR Interface) ²	00h	ACT Count	Counts the number of DRAM Activate commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	ADD
	01h	PRE Count	Counts the number of DRAM Precharge commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	
	02h	CAS Rd	Counts the number of DRAM Column Address Strobe read commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	
	03h	CAS Wr	Counts the number of DRAM Column Address Strobe write commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	
	04h	Refresh	Counts the number of DRAM Refresh commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	
	05h	Self Refresh Entry	Counts the number of Self Refresh Entry commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	
	06h	RFM	Counts the number of Refresh Management (RFM) commands that were issued by the Memory Controller associated with this CPMU.	Filter ID=1	
	07h - 1Fh	Reserved	Reserved	N/A	N/A
8001h - FFFFh	Reserved	Reserved	Reserved	N/A	N/A

1. In the **MEC** column, ADD indicates that the Counter Unit shall add the occurrences of all the enabled events every clock, which may result in the Counter Data being incremented by a value of more than one within a single clock. In the **MEC** column, OR indicates that the Counter Unit shall logically OR the occurrences of all the enabled events every clock and the Counter Data shall never increment by more than one within any single clock.
2. See JEDEC DDR5 Specification for the definition of the specific commands that are referenced in the **Event Description** column.

IMPLEMENTATION NOTE**Event Selection and Counting Summary**

This diagram pictorially represents how a simple CPMU that supports a single Event Group and two Configurable Counters counts events.

1. Every clock, the events selected via the Events field in the Counter Configuration register are OR'ed together.
2. The output of step 1, labeled "OR'ed event" is subjected to various configured filters.
3. The output of step 2 is added to the Counter Data.

Note: For readability, the Threshold, Edge, and Invert controls are not shown.

14.0 CXL Compliance Testing

14.1 Applicable Devices under Test (DUTs)

The tests outlined in this chapter are applicable to all devices that support alternate protocol negotiation and are capable of CXL only or CXL and PCIe* protocols. The tests are broken into the different categories corresponding to the different chapters of CXL specification, starting with [Chapter 3.0](#).

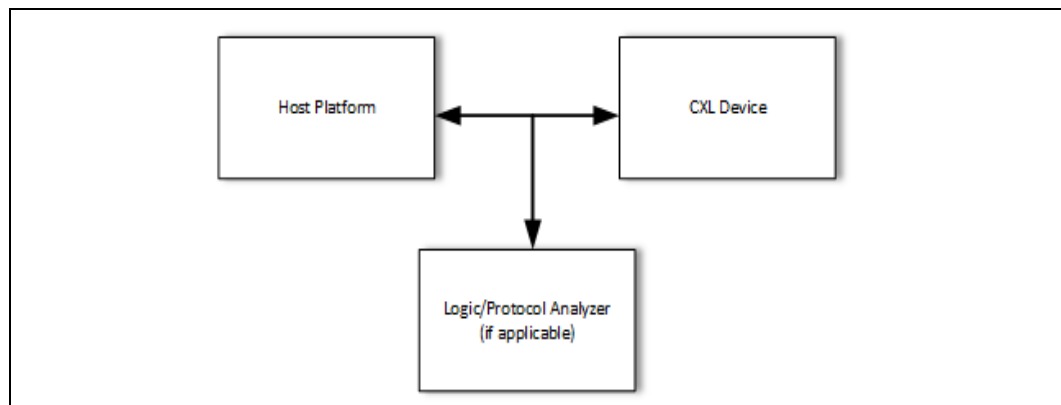
14.2 Starting Configuration/Topology (Common for All Tests)

In most tests, the initial conditions assumed are as follows (deviations from these conditions are pointed out in specific tests, if applicable): System is powered on, running in test environment OS, device-specific drivers have loaded on device, and link has trained to supported CXL modes. All error status registers should be cleared on the DUT.

Some tests make assumptions about only one CXL device being present in the system – this is identified in relevant tests. If nothing is mentioned, there is no limit on the number of CXL devices present in the system; however, the number of DUTs is limited to what the test software can support.

Certain tests may also require the presence of a protocol analyzer to monitor flits on the physical link for determining Pass or Fail results.

Figure 14-1. Example Test Topology



Each category of tests has certain device capability requirements to exercise the test patterns. The associated registers and programming is defined in the following sections.

See [Section 14.16](#) for the registers that are applicable to the tests in the following sections.

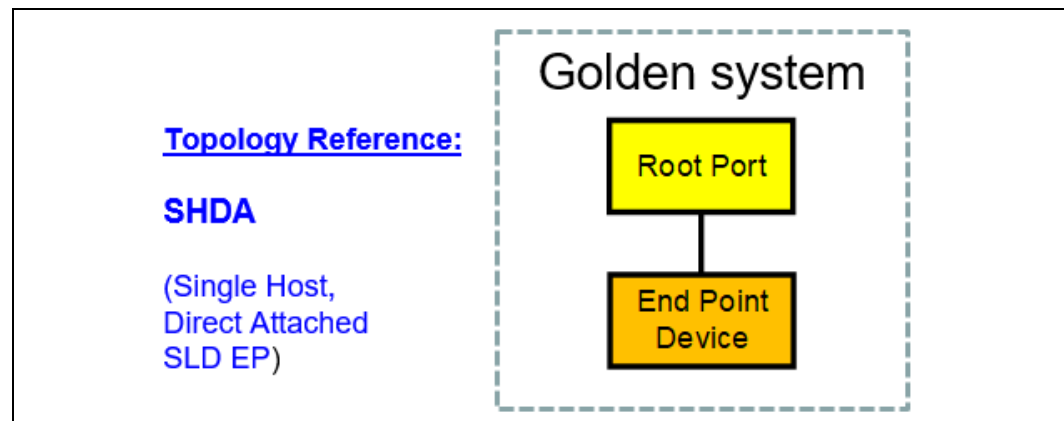
14.2.1 Test Topologies

Some tests may require a specific topology to achieve the desired requirements. Throughout this chapter there will be references to these topologies as required. This section of the document will describe these topologies at a high level to provide context for the intended test configuration.

14.2.1.1 Single Host, Direct Attached SLD EP (SHDA)

Figure 14-2 is the most direct connected topology between a root port and an endpoint device.

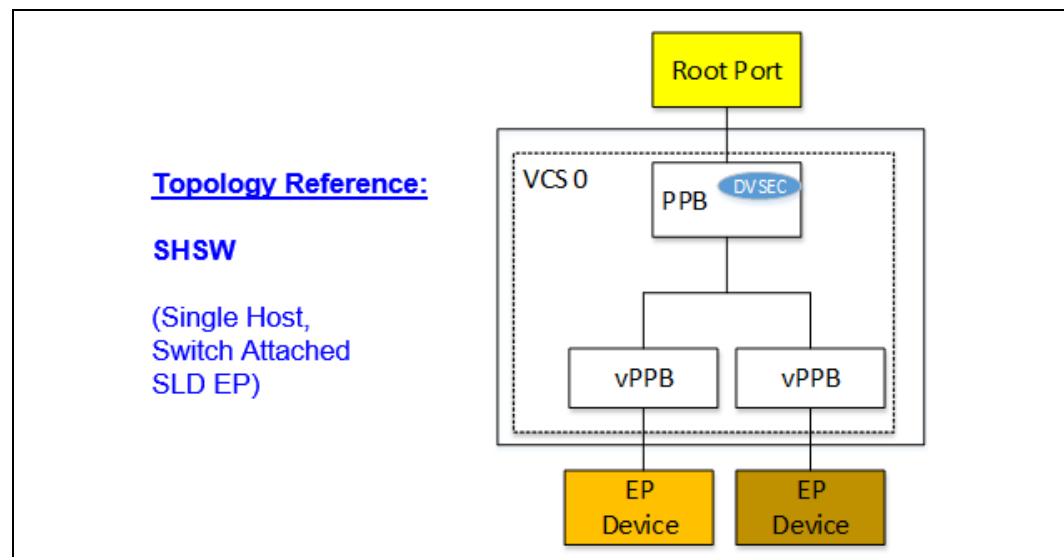
Figure 14-2. Example SHDA Topology



14.2.1.2 Single Host, Switch Attached SLD EP (SHSW)

Figure 14-3 is the initial configuration for using a CXL-capable switch in the test configurations.

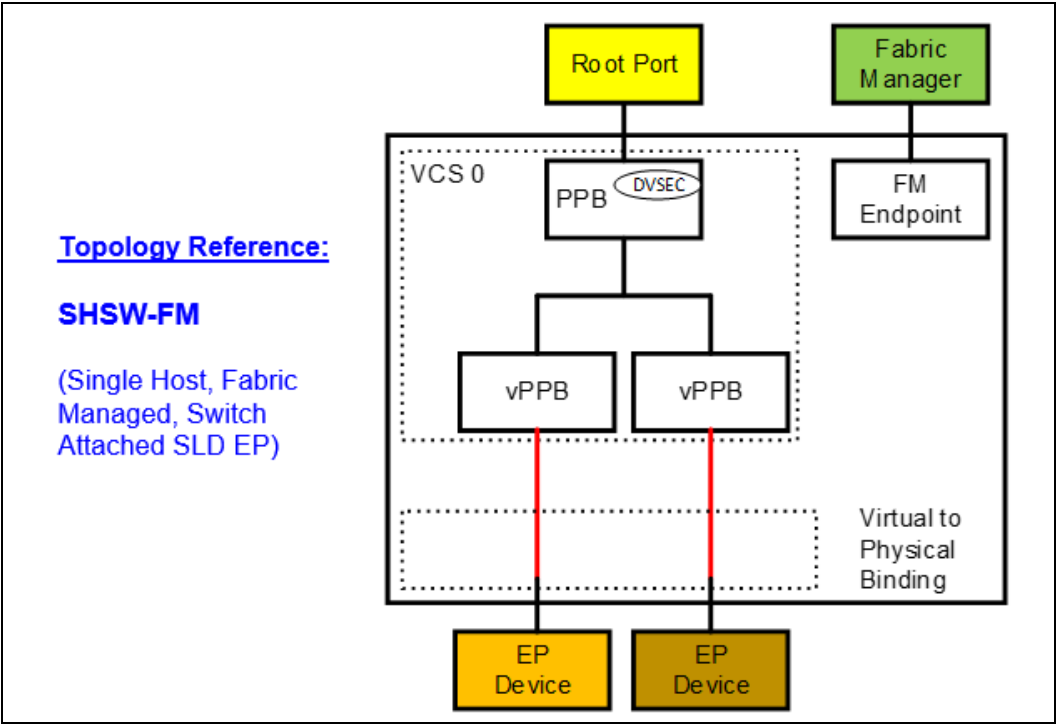
Figure 14-3. Example Single Host, Switch Attached, SLD EP (SHSW) Topology



14.2.1.3 Single Host, Fabric Managed, Switch Attached SLD EP (SHSW-FM)

Figure 14-4 shows the configuration which will use the Fabric Manager as part of the test configuration.

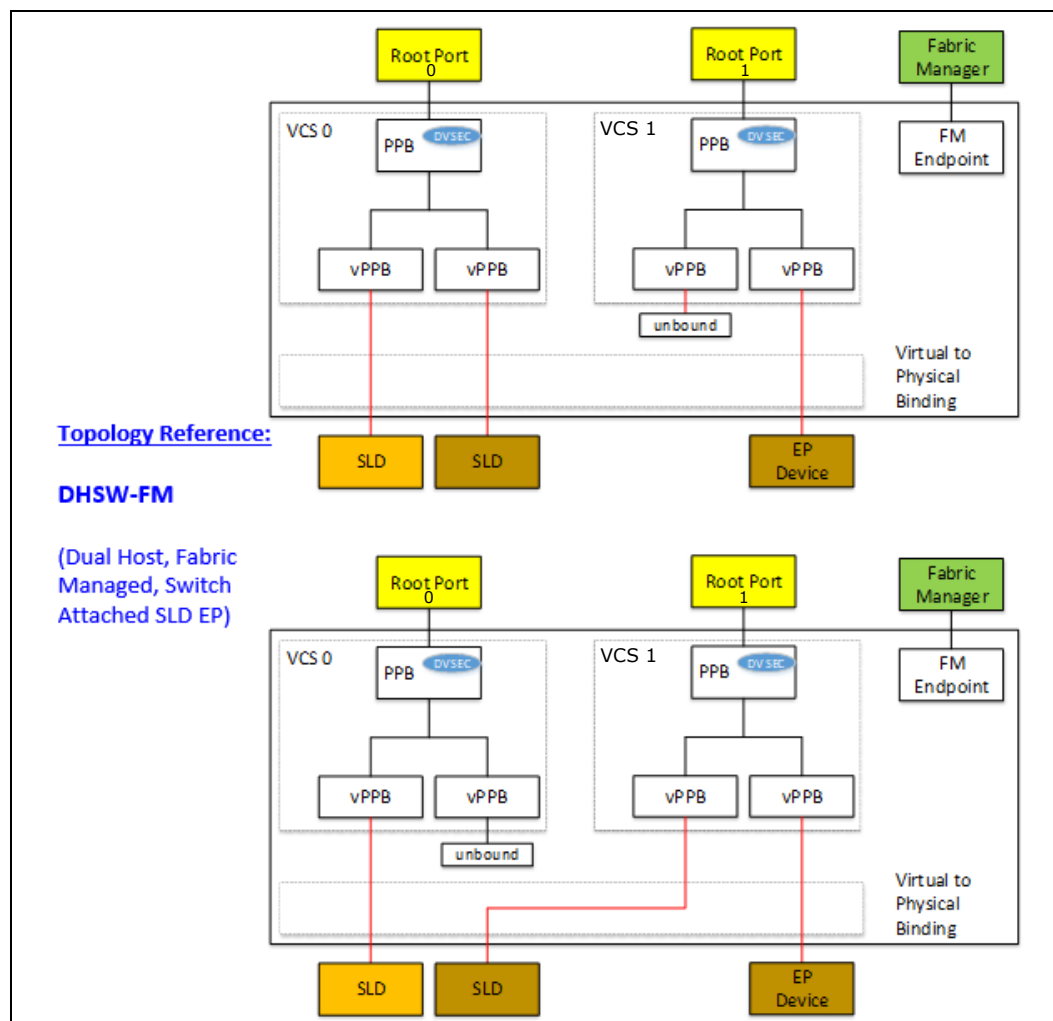
Figure 14-4. Example SHSW-FM Topology



14.2.1.4 Dual Host, Fabric Managed, Switch Attached SLD EP (DHSW-FM)

Figure 14-5 shows an example configuration topology for having dual hosts during a test.

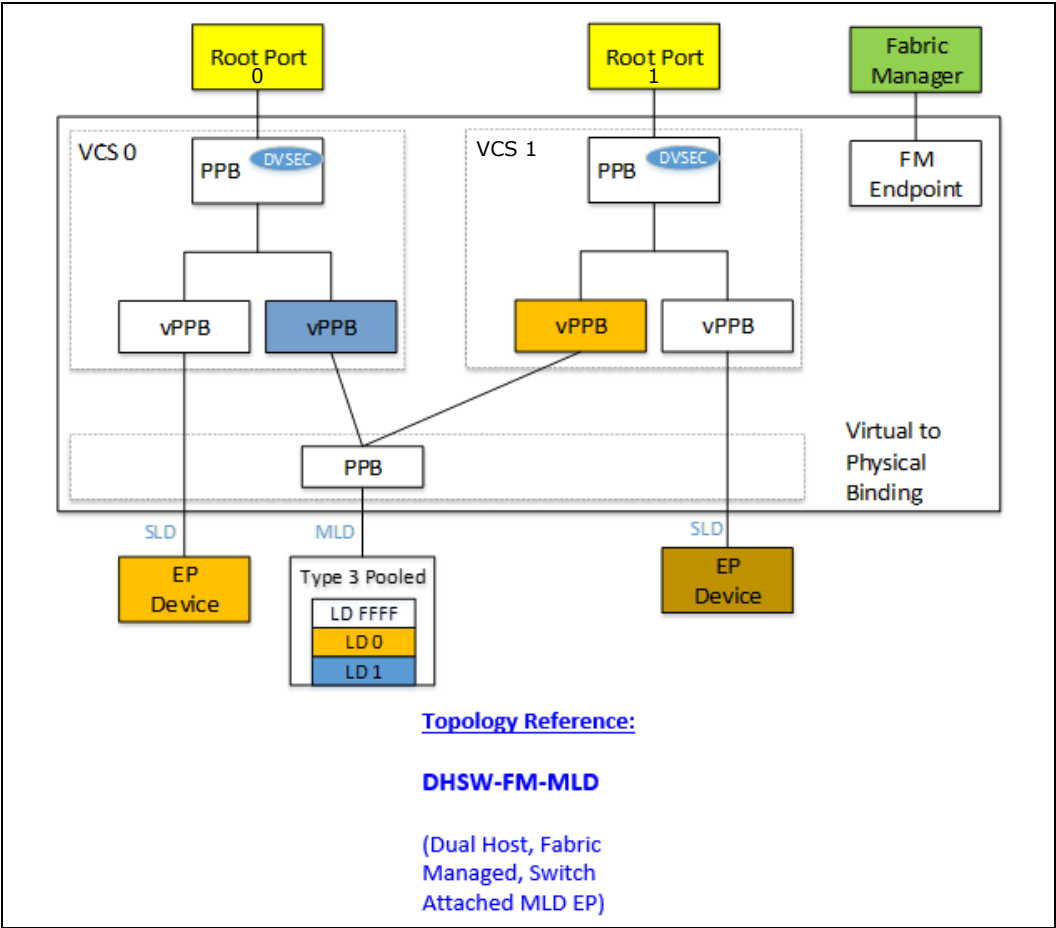
Figure 14-5. Example DHSW-FM Topology



14.2.1.5 Dual Host, Fabric Managed, Switch Attached MLD EP (DHSW-FM-MLD)

Figure 14-6 shows the topology for having dual hosts in a managed environment with multiple logical devices.

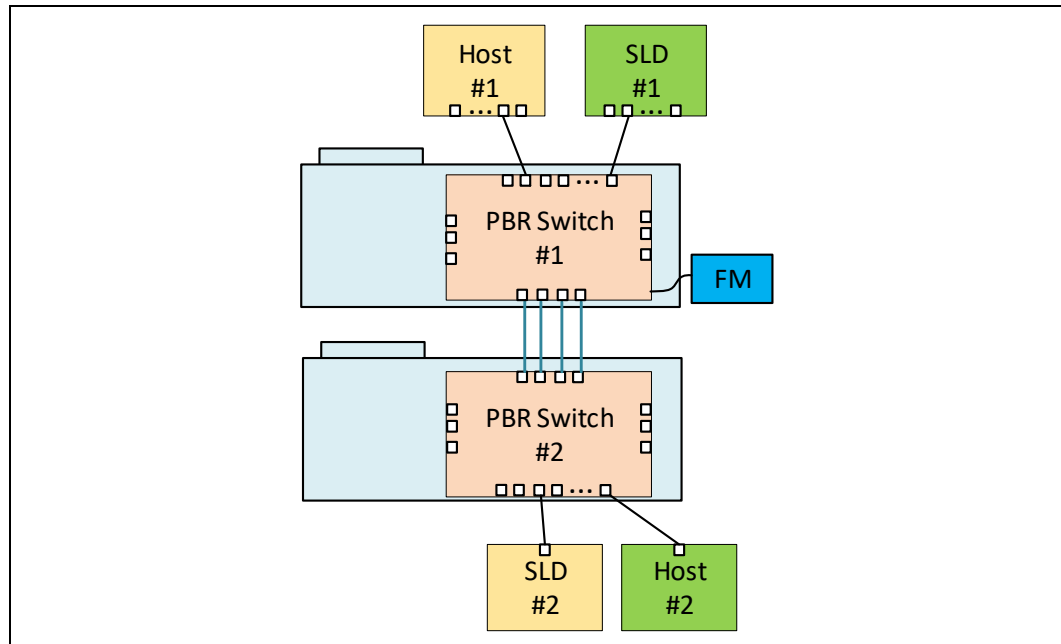
Figure 14-6. Example DHSW-FM-MLD Topology



14.2.1.6 Cascaded Switch Topologies

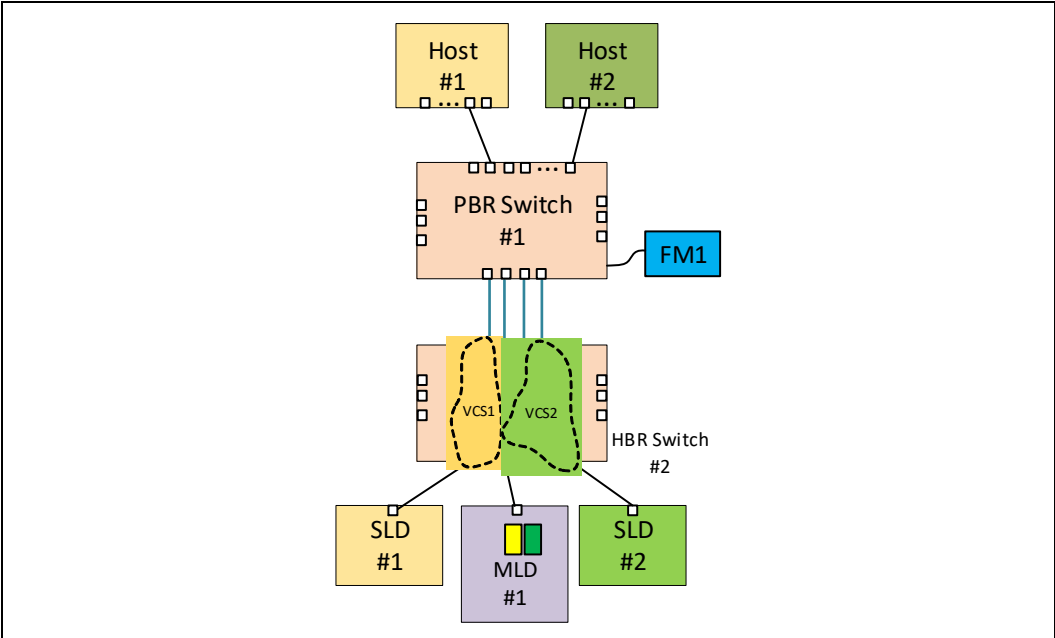
PBR switches enable cascaded and mesh topologies. Figure 14-7 shows a cascaded switch topology that is supported by PBR switches. PBR switches use PBR flits for Inter-switch links. A Fabric Manager is required to configure the fabric port routing. HBR switches may be attached to a PBR switch fabric.

Figure 14-7. Example Topology for Two PBR Switches



In a topology that has a single PBR switch and a single HBR switch (see [Figure 14-8](#)), the host devices are connected to the PBR switch and the HBR switch's Upstream Switch Ports (USPs) are connected to the PBR switch, to allow for multiple-host routing. The HBR switch configures a unique VCS for each host.

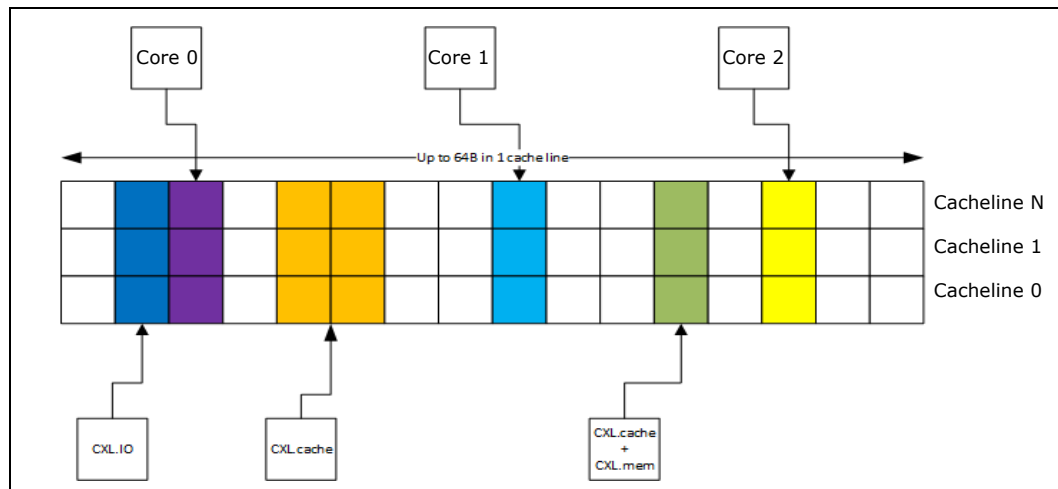
Figure 14-8. Example Topology for a PBR Switch and an HBR Switch



14.3 CXL.io and CXL.cache Application Layer/Transaction Layer Testing

14.3.1 General Testing Overview

Standard practices of testing coherency rely on “false sharing” of cachelines. Different agents in the system (e.g., cores, I/O, etc.) are assigned one or more fixed-byte locations within a shared set of cachelines. Each agent continuously executes an assigned Algorithm independently. Since multiple agents are sharing the same cacheline, stressful conflict scenarios can be exercised. [Figure 14-9](#) illustrates the concept of false sharing. This can be used for CXL.io (Load/Store semantics) or CXL.cache (caching semantics) or (CXL.cache + CXL.mem) devices (Type 2 devices).

Figure 14-9. Representation of False Sharing between Cores (on Host) and CXL Devices

This document outlines three Algorithms that enable stressing the system with false sharing tests. In addition, this document specifies the prerequisites that are needed to execute, verify, and debug runs for the Algorithms. All the Algorithms are applicable for CXL.io and CXL.cache (protocols that originate requests to the host). Devices are permitted to be self-checking. Self-checking devices must have a way to disable the checking Algorithm independent of executing the Algorithm. All devices must support the non-self-checking flow in the Algorithms outlined below. The algorithms presented for false sharing require coordination with the cache on the device (if present). Hence, it may add certain responsibility on the application layer if the cache resides there.

14.3.2 Algorithms

14.3.3 Algorithm 1a: Multiple Write Streaming

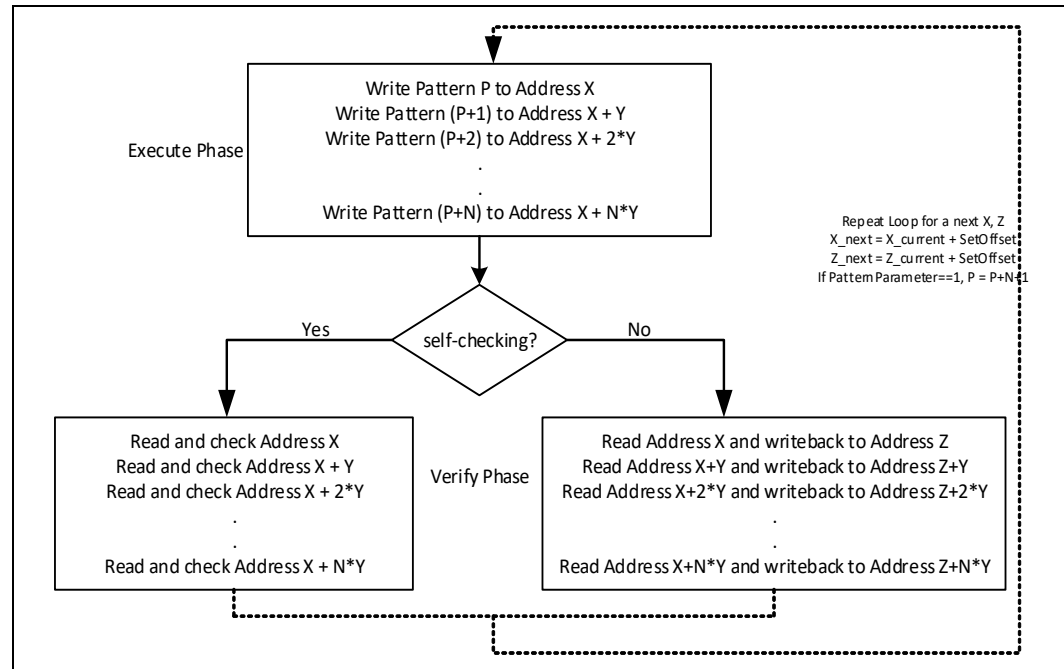
In this Algorithm, the device is setup to stream an incrementing pattern of writes to different sets of cachelines. Each set of cacheline is defined by a base address "X", and an increment address "Y". Increments are in multiples of 64B. The number of increments "N" dictates the size of the set beginning from base address X. The base address includes the byte offset within the cacheline. A pattern P (of variable length in bytes) determines the starting pattern to be written. Subsequent writes in the same set increment P. A device is required to provide a byte mask configuration capability that can be programmed to replicate pattern P in different parts of the cacheline. The programmed byte masks must be consistent with the base address.

Different sets of cachelines are defined by different base addresses (so a device may support a set like "X₁, X₂, X₃"). "X₁" is programmed by software in the base address register, X₂ is obtained by adding a fixed offset to X₁ (offset is programmed by software in a different register). X₃ is obtained by adding the same offset to X₂ and so on. Minimum support of 2 sets is required by the device. Figure 14-10 illustrates the flow of this Algorithm as implemented on the device. Address Z is the write back address where system software can poll to verify the expected pattern associated with this device, in cases where self-checking on the device is disabled. There is 1:1 correspondence between X and Z. It is the responsibility of the device to ensure that the writes in the execute phase are globally observable before beginning the verify phase. Depending on the write semantics used, this may imply additional fencing mechanism on the device to ensure the writes are globally visible before the verify phase can begin. When beginning a new set iteration, devices must also give an option

to use “P” again for the new set, or continue incrementing “P” for the next set. The select is programmed by software in “PatternParameter” field described in the register section.

Open: PatternParameter was in Table 14-41, which was removed in r3.0, v0.7. Please search the PDF for this term and determine how it and surrounding text should be revised. (Also appears in Figure 14-10 and Figure 14-11.)

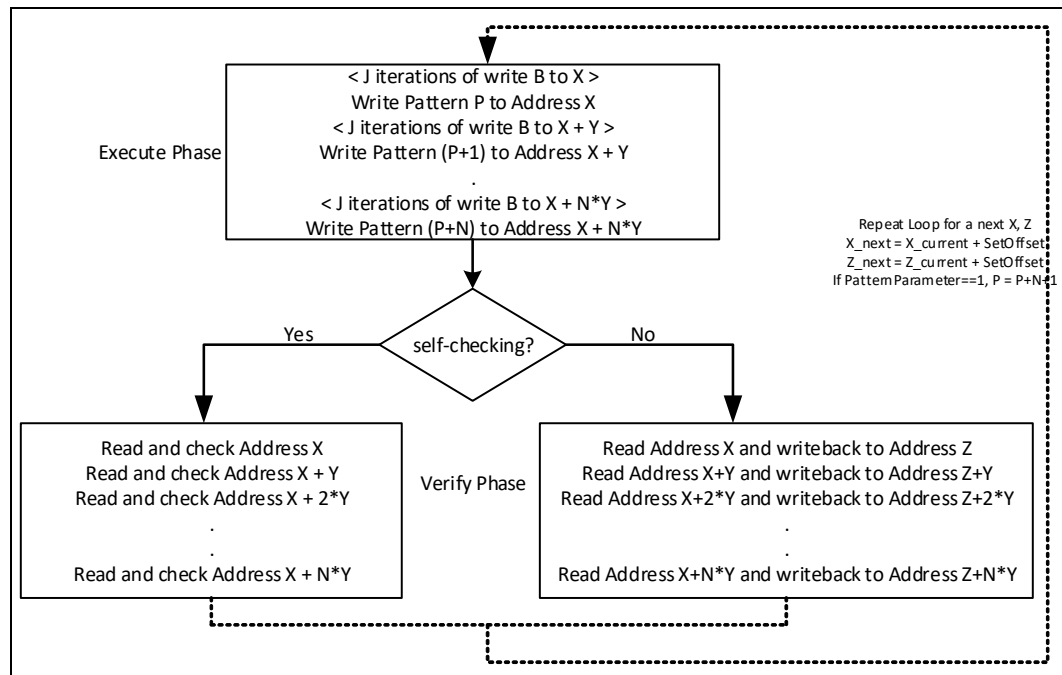
Figure 14-10. Flow Chart of Algorithm 1a



14.3.4 Algorithm 1b: Multiple Write Streaming with Bogus Writes

This Algorithm is a variation on Algorithm 1a, except that before writing the expected pattern to an address, the device does “J” iterations of writing a bogus pattern “B” to that address. Figure 14-11 illustrates this Algorithm. In this case, if a pattern “B” is ever seen in the cacheline during the Verify phase, it is a Fail condition. The bogus writes help give a longer duration of conflicts in the system. It is the responsibility of the device to ensure that the writes in the execute phase are globally observable before beginning the verify phase. Depending on the write semantics used, this may imply additional fencing mechanism on the device to ensure the writes are globally visible before the verify phase can begin. When beginning a new set iteration, devices must also give an option to use “P” again for the new set, or continue incrementing “P” for the next set. The select is programmed by software in “PatternParameter” field described in the register section.

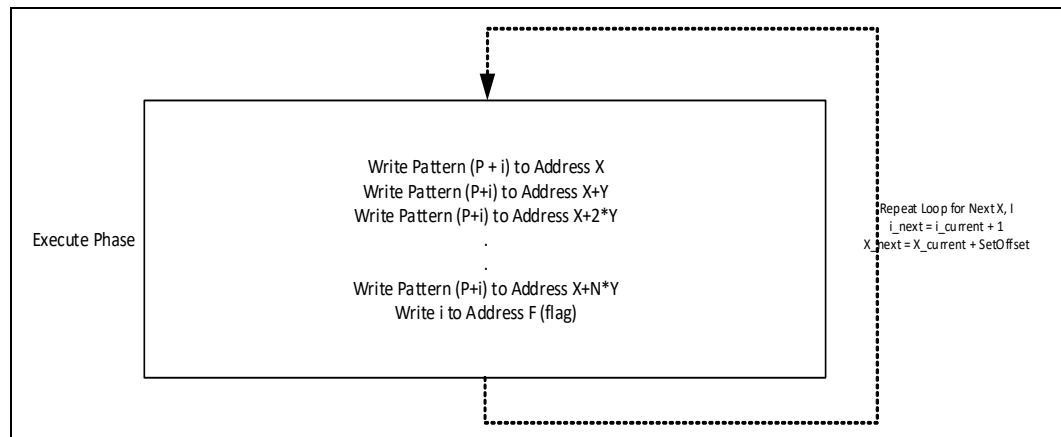
Figure 14-11. Flow Chart of Algorithm 1b



14.3.5 Algorithm 2: Producer Consumer Test

This Algorithm tests the scenario in which a Device is a producer and the CPU is a consumer. The Device simply executes a predetermined Algorithm of writing known patterns to a data location, followed by a flag update write. Threads on the CPU poll the flag, followed by reading the data patterns, followed by repolling the flag. This is a simple way of ensuring that the ordering rules of Producer-Consumer workloads are being followed through the stack. Device only participates in the execute phase of this Algorithm. Figure 14-12 illustrates the device execute phase. The Verify phase is run on the CPU, software reads addresses in the following order [F, X, (X+Y)...(X+N*Y), F]. Knowing the value of the flag at two ends, the checker knows the range within which [X, (X+Y)...(X+N*Y)] have to be. For example, if P=0, the first read of F returns a value of 3 and the next read of F returns a value of 4, then checker knows that all intermediate values have to be either 3 or 4. Moreover, if the device is using strongly ordered semantics, then the checker should never see a transition of values from 3 to 4 (implying monotonically decreasing values for the non-flag addresses). If using CXL.cache protocol, device must ensure global observability of previous "data" writes before updating the flag. When using strongly ordered semantics, each update must be globally visible before the next write. Depending on the flow used for dirty evicts, this can be implementation specific. It is the responsibility of the device to ensure that the writes in the execute phase are globally observable before updating the flag "F". The "PatternParameter" field is not relevant for this Algorithm. The Flag "F" should be written to Register 2: "WriteBackAddress1" in the Device Capabilities to support the Test Algorithms.

Figure 14-12. Execute Phase for Algorithm 2



14.3.6 Test Descriptions

Unless specified otherwise, the tests in this section are applicable to both 68B Flit mode and 256B Flit mode.

14.3.6.1 Application Layer/Transaction Layer Tests

The Transaction Layer Tests implicitly give coverage for Link Layer functionality. Specific error injection cases for the Link Layer are covered in [Section 14.12](#).

14.3.6.1.1 CXL.io Load/Store Test

For CXL.io, this test and associated capabilities are optional but strongly recommended. This test sets up the device to execute Algorithms 1a, 1b, and 2 in succession to stress the data path for CXL.io transactions. Configuration details are determined by the host platform testing the device. See [Section 14.16](#) for the configuration registers and device capabilities. Each run includes execute/verify phases as described in [Section 14.3.1](#).

Prerequisites:

- Hardware and configuration support for Algorithms 1a, 1b, and 2 described in [Section 14.3.1](#) and [Section 14.16](#)
- If the device supports self-checking, it must escalate a fatal system error if the Verify phase fails (see [Section 12.2](#) for specific error-escalation mechanisms)
- Device is permitted to log failing address, iteration number, and/or expected data vs. received data

Test Steps:

1. Host software will set up the device for Algorithm 1a: Multiple Write Streaming.
2. If the device supports self-checking, enable it.
3. Host software decides the test runtime and runs the test for that period of time. (The software details of this are host-platform specific, but will be compliant with the flows mentioned in [Section 14.3.1](#) and follow the configurations outlined in [Section 14.16](#).)
4. Set up the device for Algorithm 1b: Multiple Write Streaming with Bogus writes.
5. If the device supports self-checking, enable it.

6. Host software decides the test runtime and runs the test for that period of time.
7. Set up the device for Algorithm 2: Producer Consumer Test.
8. Host software decides the test runtime and runs the test for that period of time.

Pass Criteria:

- No data corruptions or system errors are reported

Fail Conditions:

- Data corruptions or system errors are reported

14.3.6.1.2 CXL.cache Coherency Test

This test sets up the device and the host to execute Algorithms 1a, 1b, and 2 in succession to stress the data path for CXL.cache transactions. This test should only be run if the device and the host support CXL.cache or CXL.cache + CXL.mem protocols. Configuration details are determined by the host platform testing the device. See [Section 14.16](#) for the configuration registers and device capabilities. Each run includes execute/verify phases as described in [Section 14.3.1](#).

Prerequisites:

- Device is CXL.cache capable
- Hardware and configuration support for Algorithms 1a, 1b, and 2 described in [Section 14.3.1](#) and [Section 14.16](#)
- If a Device supports self-checking, it must escalate a fatal system error if the Verify phase fails (see [Section 12.2](#) for specific error-escalation mechanisms)
- Device is permitted to log failing address, iteration number, and/or expected data vs. received data

Test Steps:

1. Host software will set up the device and the host for Algorithm 1a: Multiple Write Streaming. An equivalent version of the algorithm is setup to be executed by host software so as to enable false sharing of the cachelines.
2. If the device supports self-checking, enable it.
3. Host software decides the test runtime and runs the test for that period of time. (The software details of this are host-platform specific, but will be compliant with the flows mentioned in [Section 14.3.1](#) and follow the configurations outlined in [Section 14.16](#).)
4. Set up the device for Algorithm 1b: Multiple Write Streaming with Bogus writes.
5. If the device supports self-checking, enable it.
6. Host software decides the test runtime and runs the test for that period of time.
7. Set up the device for Algorithm 2: Producer Consumer Test.
8. Host software decides the test runtime and runs the test for that period of time.

Pass Criteria:

- No data corruptions or system errors are reported

Fail Conditions:

- Data corruptions or system errors are reported

14.3.6.1.3 CXL Test for Receiving GO-ERR

This test is applicable only for devices that support CXL.cache protocols. This test sets up the device to execute Algorithm 1a while mapping one of the sets of the address to a memory range that is not accessible by the device. Test system software and configuration details are determined by the host platform and are system specific.

Prerequisites:

- Device is CXL.cache capable
- Support for Algorithm 1a

Test Steps:

1. Configure device for Algorithm 1a, and set up one of the base addresses to be an address not accessible by the DUT.
2. Disable self-checking in the DUT.
3. Host software decides test runtime and runs test for that period of time.

Pass Criteria:

- No data corruptions or system errors are reported
- No fatal device errors on receiving GO-ERR
- Inaccessible memory range has not been modified by the device

Fail Conditions:

- Data corruptions or system errors reported
- Fatal device errors on receiving GO-ERR
- Inaccessible memory range modified by the device (host error)

14.3.6.1.4 CXL.mem Test

This test sets up the **host** and the device to execute Algorithms 1a, 1b, and 2 in succession to stress the data path for CXL.mem transactions. An equivalent version of the algorithm is setup to be executed by host software so as to enable false sharing of the cachelines. Test system software and configuration details are determined by the host platform and are system specific.

Prerequisites:

- Device is CXL.mem capable

Test Steps:

1. Map the device-attached memory to a test-memory range that is accessible by the host.
2. Run the equivalent of Algorithms 1a, 1b, and 2 on the host and the device targeting device-attached memory.

Pass Criteria:

- No data corruptions or system errors are reported

Fail Conditions:

- Data corruptions or system errors are reported

14.3.6.1.5 Egress Port Backpressure Test

This test applies to an MLD that supports FM API or an SLD that supports the Memory Device command set. This test sets up the device to execute Algorithms 1a, 1b, and 2 in succession to stress the data path for CXL.mem transactions. An equivalent version of the algorithm is setup to be executed by **host** software so as to enable false sharing of the cachelines. Test system software and configuration details are determined by the host platform and are system specific. NUMBER_OF_QOS_TEST_LOOPS, NUMBER_OF_CHECK_AVERAGE, and BackpressureSample Interval setting in the test steps below is decided upon by the testing platform/software.

Prerequisites:

- Device is CXL.mem capable

Test Steps:

For an MLD:

1. Through the FM API, check if Egress Port Congestion Supported is set by issuing a Get LD Info command.
2. If Egress Port Congestion Supported is enabled:
Repeat for NUMBER_OF_QOS_TEST_LOOPS:
 - a. Set the BackpressureSample Interval setting to a value between 1 -31 through the Set QoS Control command.
 - b. Set the Egress Port Congestion Enable bit through the Set QoS Control command.
 - c. Check that the Egress Port Congestion Enable bit was set successfully in the Get QoS Control Response.
 - d. Run the equivalent of Algorithms 1a, 1b, and 2 in succession on the host and the device targeting device-attached memory.
 - e. While Algorithms 1a, 1b, and 2 are running: Check the reported Backpressure Average Percentage through the Get QoS Status command and response. It should report values within the valid range which is 0 – 100. Repeat this step NUMBER_OF_CHECK_AVERAGE times at a certain interval.

For an SLD:

1. Check if Egress Port Congestion Supported is set by issuing an Identify Memory Device, and checking the corresponding Identify Memory Device Output Payload.
2. If Egress Port Congestion Supported is enabled, repeat for
NUMBER_OF_QOS_TEST_LOOPS:
 - a. Set the BackpressureSample Interval setting to a value between 1 - 31 through the Set SLD QoS Control Request command.
 - b. Set the Egress Port Congestion Enable bit through the Set SLD QoS Control Request.
 - c. Check that the Egress Port Congestion Enable bit was set successfully in the Get SLD QoS Control Response.
 - d. Check the reported Backpressure Average Percentage through the Get QoS Status command and response.
 - e. Run the equivalent of Algorithms 1a, 1b, and 2 in succession on the host and the device targeting device-attached memory.
 - f. While Algorithms 1a, 1b, and 2 are running: Check the reported Backpressure Average Percentage through the Get SLD QoS Status command and response.

It should report values within the valid range which is 0 – 100. Repeat this step NUMBER_OF_CHECK_AVERAGE times at a certain interval.

Pass Criteria:

- Egress Port Congestion Enable is set after enabling it
- Backpressure Average Percentage reports valid values within 0-100.
- No data corruptions or system errors are reported while executing Algorithms 1a, 1b, and 2

Fail Conditions:

- Egress Port Congestion Enable is not set after enabling it
- Backpressure Average Percentage reports any value outside the valid 0-100 range
- Data corruptions or system errors reported while executing Algorithms 1a, 1b, and 2

14.3.6.1.6 Temporary Throughput Reduction Test

This test applies to an MLD that supports FM API or an SLD that supports the Memory Device Command set. This test sets up the device to execute Algorithms 1a, 1b, and 2 in succession to stress the data path for CXL.mem transactions. For Type 3 (MLD or SLD), it is the responsibility of the host to take care of running the algorithms as appropriate. An equivalent version of the algorithm is setup to be executed by **Host** software so as to enable false sharing of the cachelines. Test system software and configuration details are determined by the host platform and are system specific. NUMBER_OF_QOS_TEST_LOOPS in the test steps is decided upon by the testing platform/software.

Prerequisites:

- Device is CXL.mem capable

Test Steps:

For an MLD:

1. Through the FM API, check if Temporary Throughput Reduction Supported is set by issuing a Get LD Info command.
2. If Temporary Throughput Reduction Supported is enabled, repeat for NUMBER_OF_QOS_TEST_LOOPS:
 - a. Set the Temporary Throughput Reduction Enable bit by issuing the Set QoS Control command.
 - b. Check that the Temporary Throughput Reduction Enable bit was set successfully in the Get QoS Control Response.
 - c. Run the equivalent of Algorithms 1a, 1b, and 2 in succession on the host and the device targeting device-attached memory.

For an SLD:

1. Through the Memory Device Command set, check if Temporary Throughput Reduction Supported is set by issuing an Identify Memory Device, and checking corresponding Identify Memory Device Output Payload.
2. If Temporary Throughput Reduction Supported is enabled, repeat for NUMBER_OF_QOS_TEST_LOOPS:
 - a. Set the Temporary Throughput Reduction Enable bit through the Set SLD QoS Control Request.

- b. Check that the Temporary Throughput Reduction Enable bit was set successfully in the Get SLD QoS Control Response.
- c. Run the equivalent of Algorithms 1a, 1b, and 2 in succession on the host and the device targeting device-attached memory.

Pass Criteria:

- Temporary Throughput Reduction Enable is set after enabling it
- No data corruptions or system errors are reported while executing Algorithms 1a, 1b, and 2

Fail Conditions:

- Temporary Throughput Reduction Enable is not set after enabling it
- Data corruptions or system errors reported while executing Algorithms 1a, 1b, and 2

14.4 Link Layer Testing**14.4.1 RSVD Field Testing CXL.cachemem****Test Equipment:**

- Exerciser

Prerequisites:

- Applicable for 68B and 256B Flit modes
- Device is CXL.cachemem capable
- CXL link is up

14.4.1.1 Device Test**Test Steps:**

1. Send from host Link Layer Control.INIT.Param with all RSVD fields set to 1.
2. Wait for Control-INIT.Param from the device.
3. Wait for the Link to reach L0 state and the device is in a configured state.

Pass Criteria:

- CXL Link Layer Control and Status Register INIT_State is 11b
- Link Layer initialization is successful and Reserved fields are ignored

Fail Conditions:

- Pass criteria is not met

14.4.1.2 Host Test**Test Steps:**

1. Send from device Link Layer Control.INIT.Param with all RSVD fields set to 1.
2. Wait for Link to reach L0 state.

Pass Criteria:

- CXL Link Layer Control and Status Register INIT_State is 11b
- Link Layer initialization is successful and Reserved fields are ignored

Fail Conditions:

- Pass criteria is not met

14.4.2 CRC Error Injection RETRY_PHY_REINIT**Test Equipment:**

- Protocol Analyzer
- Protocol Exerciser

Prerequisites:

- Applicable for 68B Flit mode only
- CXL Host must support Algorithm 1a
- CXL Host must support Link Layer Error Injection capabilities for CXL.cache

Test Steps:

1. Setup is the same as Test 14.3.6.1.2.
2. While a test is running, software will insert the following error injection. The Protocol Exerciser will retry the flit for at least MAX_NUM_RETRY times upon detecting a CRC error.

Table 14-1. CRC Error Injection RETRY_PHY_REINIT: Cache CRC Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	7, CRC Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	2
Dh	1	Num Bits Flipped	1
Eh	1	Num Flits Injected	1

Pass Criteria:

- Same as Test 14.3.6.1.2
- Monitor and verify that CRC errors are injected (using the Protocol Analyzer), and that Retries are triggered as a result
- Five RETRY.Frame Flits are sent before RETRY.Req and RETRY.Ack (protocol analyzer)
- Check that link enters RETRY_PHY_REINIT
- Means value of NUM_Phy_Reinit_Received: Num_Phy_Reinit value reflected in the last RETRY.Req message received in CXL Link Layer Capability register is greater than 1

Fail Conditions:

- Same as Test 14.3.6.1.2
- Link does not reach RETRY_PHY_REINIT

14.4.3 CRC Error Injection RETRY_ABORT**Test Equipment:**

- Protocol Analyzer
- Protocol Exerciser

Prerequisites:

- Applicable for 68B Flit mode only
- CXL device must support Algorithm 1a
- CXL device must support Link Layer Error Injection capabilities for CXL.cache

Test Steps:

1. Set up is the same as Test 14.3.6.1.2.
2. While a test is running, software will insert the following error injection. The Protocol Exerciser will retry the flit for at least (**MAX_NUM_RETRY** x **MAX_NUM_PHY_REINIT**) times upon detecting a CRC error:

Table 14-2. CRC Error Injection RETRY_ABORT: Cache CRC Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	7, CRC Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	2
Dh	1	Num Bits Flipped	1
Eh	1	Num Flits Injected	1

Pass Criteria:

- Same as Test 14.3.6.1.2
- Monitor and verify that CRC errors are injected (using the Protocol Analyzer), and that Retries are triggered as a result
- Five RETRY.Frame Flits are sent before RETRY.Reg and RETRY.Ack (protocol analyzer)
- Link retrains for MAX_NUM_PHY_REINIT number of times and fails to recover

Fail Conditions:

- Same as Test 14.3.6.1.2
- Link does not reach RETRY_PHY_REINIT
- Link does not reach RETRY_ABORT

14.5 ARB/MUX

14.5.1 Reset to Active Transition

Test Equipment:

- Protocol Analyzer

Prerequisites:

- Applicable for 68B Flit mode, 256B Flit mode, and Latency-Optimized 256B Flit mode
- CXL link is not assumed to be up
- Device drivers are not assumed to have been loaded

Test Steps:

1. With the link in Reset state, Link layer sends a Request to enter Active.
2. ARB/MUX waits to receive indication of Active from Physical Layer.

Pass Criteria:

- ALMP Status sync exchange completes before ALMP Request{Active} sent by Local ARB/MUX (if applicable)
- Local ARB/MUX sends ALMP Request{Active} to the remote ARB/MUX
- Validate the first ALMP on the initial bring up is from the Downstream Port to the Upstream Port
- Local ARB/MUX waits for ALMP Status{Active} and ALMP Request{Active} from remote ARB/MUX
- Local ARB/MUX sends ALMP Status{Active} in response to Request
- Link transitions to Active after the ALMP handshake completes
- Link successfully enters Active state with no errors

Fail Conditions:

- Link hangs and does not enter Active state
- Any error occurs before transition to Active state

14.5.2 ARB/MUX Multiplexing

Test Equipment:

- Protocol Analyzer (used to ensure that traffic is sent simultaneously on both CXL.io and CXL.cachemem)

Prerequisites:

- Applicable for 68B Flit mode, 256B Flit mode, and Latency-Optimized 256B Flit mode
- Device is CXL.cache and/or CXL.mem capable
- Host-generated traffic or Device-generated traffic
- Support for Algorithm 1a, 1b, or 2

Test Steps:

1. Bring the link up into CXL mode with CXL.io and CXL.cache and/or CXL.mem enabled.
2. Ensure the arbitration weight is a nonzero value for both interfaces.
3. Send continuous traffic on both CXL.io and CXL.cache and/or CXL.mem using Algorithm 1a, 1b, or 2.
4. Allow time for traffic transmission while snooping the bus.

Pass Criteria:

- Data from both CXL.io and CXL.cache and/or CXL.mem are sent across the link by the ARB/MUX

Fail Conditions:

- Data on the link is only CXL.io
- Data on the link is only CXL.cache or CXL.mem (CXL.cache and CXL.mem share a single Protocol ID; see [Table 6-2](#))

Test Steps (256B Flit Mode):

1. Upstream Port sends PM state Request ALMP.
2. Wait for an ALMP Request for entry to a PM state.
3. Downstream Port rejects the request by responding Active.PMNAK Status ALMP.
4. On receiving Active.PMNAK Status ALMP, the Upstream Port must transition the corresponding vLSM to Active.PMNAK state.
5. After Active.PMNAK is observed, the Link Layer must request Active to the ARB/MUX and then wait for the vLSM to transition to Active before transmitting flits.

Pass Criteria:

- Upstream Port must continue to receive and process flits while the vLSM state is Active or Active.PMNAK
- Upstream Port must transition back to Active state
- For Upstream Ports, after the Link Layer has requested PM entry, the Link Layer must not change this request until it observes the vLSM status change to either the requested state or to Active.PMNAK or to one of the non-virtual states (LinkError, LinkReset, LinkDisable, or Reset)

Fail Conditions:

- Any system error

14.5.3**Active to L1.x Transition (If Applicable)****Test Equipment:**

- Protocol Analyzer

Prerequisites:

- Applicable for 68B Flit mode, 256B Flit mode, and Latency-Optimized 256B Flit mode
- Support for ASPM L1

Test Steps:

1. Force the remote and local link layer to send a request to the ARB/MUX for L1.x state.
2. This test should be run separately for each Link Layer independently (to test one Link Layer's L1 entry while the other Link Layer is in ACTIVE), as well as both Link Layers concurrently requesting L1 entry.

Pass Criteria:

- Upstream Port ARB/MUX sends ALMP Request{L1.x}
- Downstream Port ARB/MUX sends ALMP Status{L1.x} in response
- L1.x is entered after the local ARB/MUX receives ALMP Status
- State transition doesn't occur until ALMP handshake is complete
- LogPHY enters L1 ONLY after both Link Layers enter L1 (applies to CXL mode only)

Fail Conditions:

- Error in ALMP handshake
- Protocol layer packets sent after ALMP L1.x handshake is complete (requires Protocol Analyzer)
- State transition occurs before ALMP handshake completed

14.5.4**L1.x State Resolution (If Applicable)****Test Equipment:**

- Protocol Analyzer

Prerequisites:

- Applicable for 68B Flit mode, 256B Flit mode, and Latency-Optimized 256B Flit mode
- Support for ASPM L1

Test Steps:

1. Force the remote and local link layer to send a request to the ARB/MUX for **different** L1.x states.

Pass Criteria:

- Upstream Port ARB/MUX sends ALMP Request{L1.x} according to what the link layer requested
- Upstream Port ARB/MUX sends ALMP Status{L1.y} response
- The state in the Status ALMP is the more-shallow L1.y state
- L1.y is entered after the local ARB/MUX receives ALMP Status
- State transition doesn't occur until the ALMP handshake is complete
- LogPHY enters L1 ONLY after both protocols enter L1 (applies to CXL mode only)

Fail Conditions:

- Error in ALMP handshake